Digital Twin as a Service (DTaaS)

DTaaS Development Team

Table of contents

1. What is the DTaaS Platform?

The Digital Twin as a Service (DTaaS) software platform is designed to **Build, Use and Share** digital twins (DTs).

🦾 **Build**: DTs are constructed on DTaaS using reusable DT assets available on the platform.

👩‍💻 👨‍✈️ **Use**: DTs can be executed on the DTaaS platform.

🤝 **Share**: Ready-to-use DTs can be shared with other users. It is also possible to share the services offered by one DT with other users.

Here is an overview of the DTaaS platform available in the form of slides, video, and feature walkthrough.

1.1 License

This software is owned by The INTO-CPS Association and is available under the INTO-CPS License.

The DTaaS platform uses third-party open-source software. These software components have their own licenses.

2. Video Playlist

This page lists all video resources currently referenced in the DTaaS documentation. Each entry includes a brief description and a direct link.

Video	Description	Referenced From
DTaaS short introduction	High-level introduction to the DTaaS platform and scope.	index.md
DTaaS feature walkthrough	Guided walkthrough of DTaaS v0.7 features.	index.md
User presentation	User-oriented overview of platform usage patterns and capabilities.	user/motivation.md
Developer presentation	Developer-oriented overview of architecture, workflow, and contribution context.	developer/index.md
Examples overview	Overview of the digital twin examples available in DTaaS.	user/examples/index.md
CP-SENS project demo	Demonstration of DTaaS usage in the CP-SENS project context.	user/examples/index.md
Wind turbine blade testing demo	Demonstration of wind turbine blade testing in a workspace-driven flow.	user/examples/index.md
Python package demo (CP-SENS)	Demonstration associated with the CP-SENS Python package release.	user/examples/index.md
DAQ part 1	Data acquisition workflow demonstration, part 1.	user/examples/index.md
DAQ part 2	Data acquisition workflow demonstration, part 2.	user/examples/index.md
Operational model analysis	Demonstration of operational model analysis workflow.	user/examples/index.md
Model update demo	Demonstration of model update workflow in a digital twin context.	user/examples/index.md
Incubator demo	Demonstration of the incubator digital twin example.	user/examples/index.md
MATLAB Simulink screencast	Demonstrates use of MATLAB Simulink within a DTaaS workspace.	FAQ.md

3. User

3.1 DTaaS for Users

3.1.1 User Guide

This guide is intended for users of the DTaaS platform. Access to a live installation of the DTaaS platform is required. The simplest option is the workspace localhost installation scenario.

The following user-specific Slides and Video provide the conceptual framework behind composable digital twins in the DTaaS platform.

3.1.2 Motivation

A central question in Digital Twin (DT) software platforms is how to enable collaborative activities:

- Building digital twins (DTs)
- Utilizing DTs independently
- Sharing DTs with other users
- Providing existing DTs as a service to other users

Additionally, DT software platforms must address:

- Support for DT lifecycle management
- Scalability through flexible convention over configuration

3.1.3 Existing Approaches

Several solutions have been proposed in recent literature to address these challenges. Notable approaches include:

- Focus on data from Physical Twins (PTs) for analysis, diagnosis, and planning
- Sharing DT assets across upstream and downstream stakeholders
- Evaluating different models of PT
- DevOps methodologies for Cyber Physical Systems (CPS)
- Scaling DT execution and ensemble management of related DTs
- Support for PT product lifecycle

3.1.4 Our Approach

The DTaaS platform adopts the following principles[1]:

- Support for transition from existing workflows to DT frameworks
- Creation of DTs from reusable assets
- Enabling users to share DT assets
- Offering DTs as a Service
- Integration of DTs with external software systems
- Separation of configurations for independent DT components

3.1.5 References

[1]: Talasila, Prasad, et al. "Composable digital twins on Digital Twin as a Service platform." Simulation 101.3 (2025): 287-311.

3.2 Overview

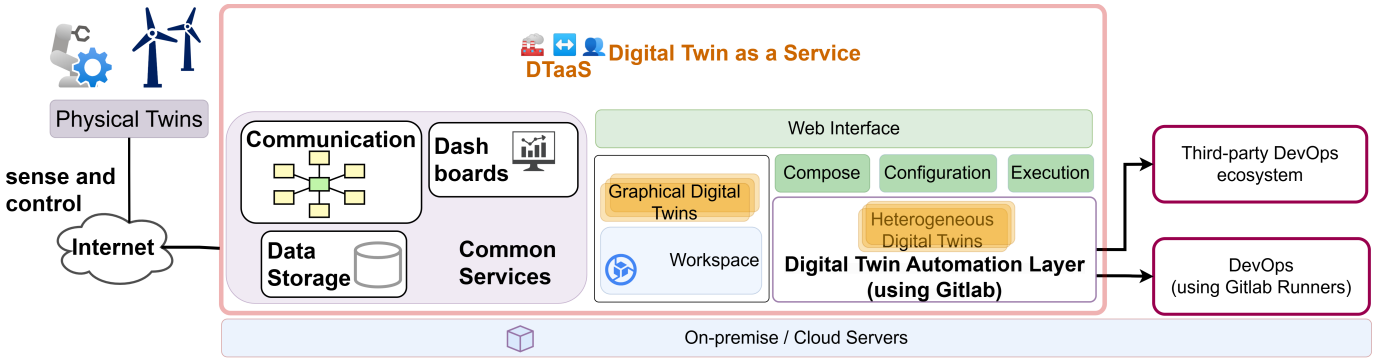
3.2.1 Advantages

The DTaaS software platform provides the following advantages:

- Support for heterogeneous Digital Twin implementations
- CFD, Simulink, co-simulation, FEM, ROM, ML, and other paradigms
- Integration with existing Digital Twin frameworks
- Provision of Digital Twin as a Service capabilities
- Facilitation of collaboration and asset reuse
- Private workspaces for verification of reusable assets and trial executions of DTs
- Cost effectiveness through shared infrastructure

3.2.2 Software Features

Each installation of the DTaaS platform includes the features illustrated in the following diagram.



All users are provided with dedicated workspaces. These workspaces are containerized implementations of Linux Desktops. The user desktops are isolated, ensuring that installations and customisations performed in one workspace do not affect other user workspaces. Graphical digital twins can be executed within these private workspaces.

Each user workspace is provisioned with pre-installed development tools. These tools are accessible directly through a web browser. The following tools are currently available:

Tool	Advantage
Jupyter Lab	Enables flexible creation and use of digital twins and their components through a web browser. All native JupyterLab use cases are supported.
Jupyter Notebook	Facilitates web-based management of files and library assets.
VS Code in the browser	A widely-adopted IDE for software development. Digital twin-related assets can be developed within this environment.
ungit	An interactive git client enabling repository management through a web browser.

In addition, an xfce-based remote desktop is accessible via a VNC client. The VNC client is available directly in the web browser. Desktop software supported by xfce can also be executed within the workspace.

The workspaces maintain Internet connectivity, enabling Digital Twins running in the workspace to interact with both internal and external services.

A DT automation layer is provided for managing DT automation tasks. This layer facilitates creation, modification, and execution of DTs on both on-premise infrastructure and commercial cloud (DevOps) service providers.

The DTaaS software platform includes several pre-installed services. The currently available services are:

Service	Advantage
InfluxDB	Internet of Things (IoT) device management and data visualization platform. This service stores data for digital twins and provides alerting capabilities.
RabbitMQ	Communication broker facilitating message exchange between physical and digital twins.
Grafana	Visualization dashboard service for digital twin data presentation.
MQTT	Lightweight data transfer broker for IoT devices and physical twins providing data to digital twins.
MongoDB	NoSQL document database for storing metadata from physical twins.
PostgreSQL	SQL database server for storing historical and time-series data.
ThingsBoard	an Internet of Things (IoT) device management and data visualization platform

Users can publish and reuse digital twin assets available on the platform. Additionally, digital twins can be executed and made available as services to external clients. These clients need not be registered users of the DTaaS installation.

3.2.3 References

Talasila, Prasad, et al. "Composable digital twins on Digital Twin as a Service platform." Simulation 101.3 (2025): 287-311.

3.3 Website

3.3.1 DTaaS Website Screenshots

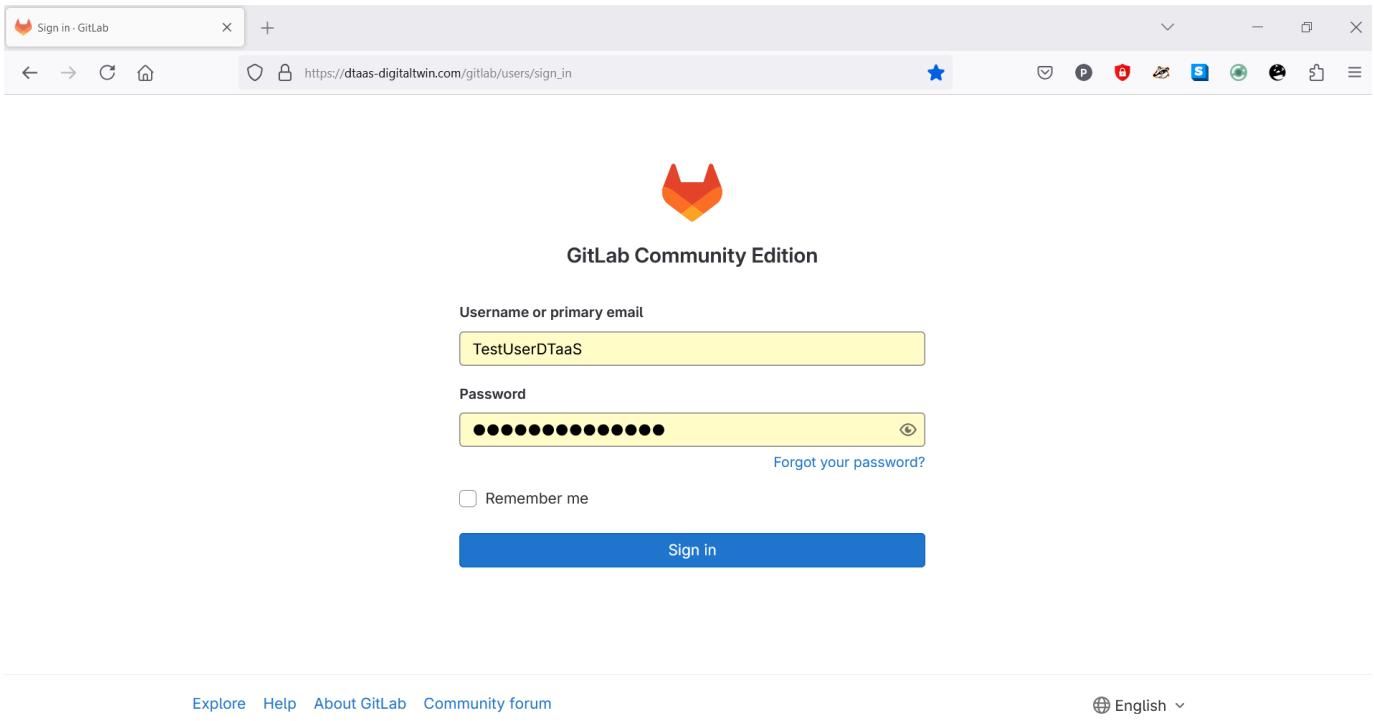
This page provides a screenshot-driven preview of the website serving the DTaaS software platform.

Visit the DTaaS Installation

Navigation begins by visiting the website of the DTaaS instance for which the user is registered.

Redirected to Authorization Provider

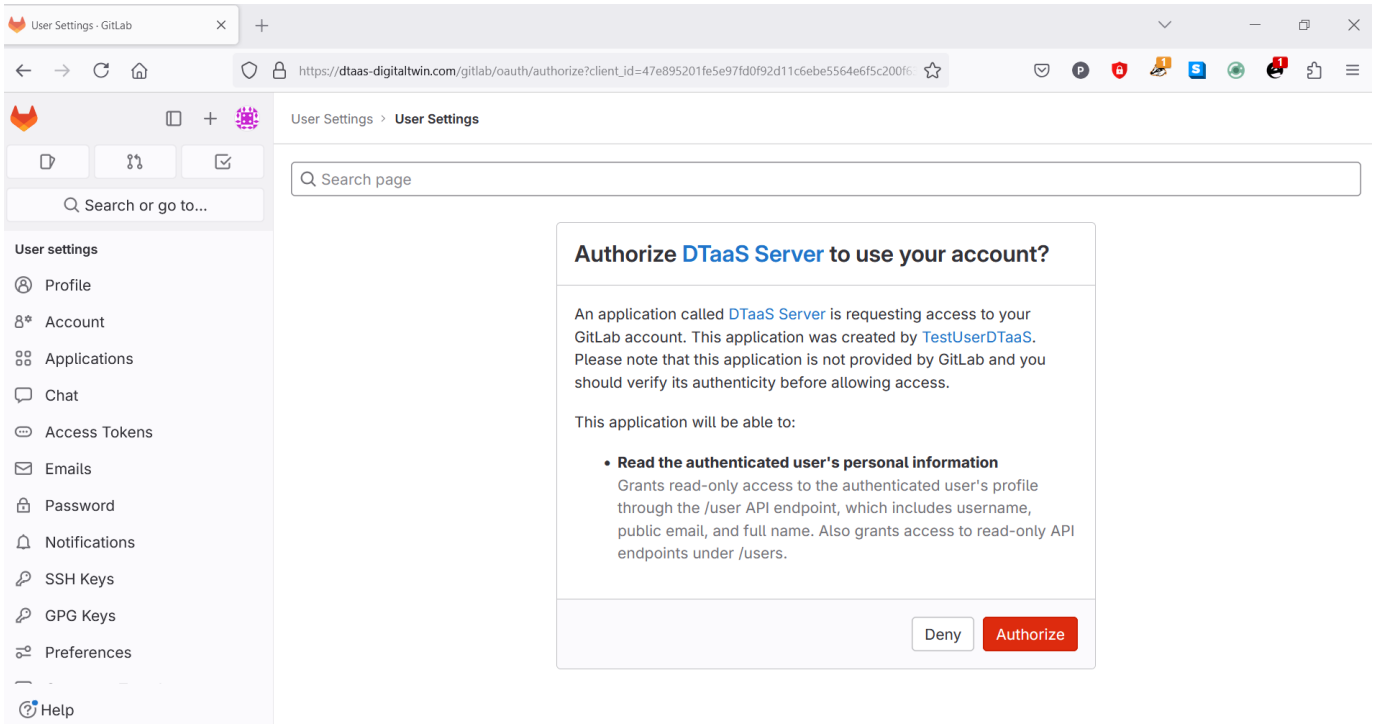
The browser redirects to the GitLab Authorization page for the DTaaS.



The email/username and password should be entered. If the email ID registered with the DTaaS matches a GitLab Login email ID.

The browser redirects to the OAuth 2.0 Application page.

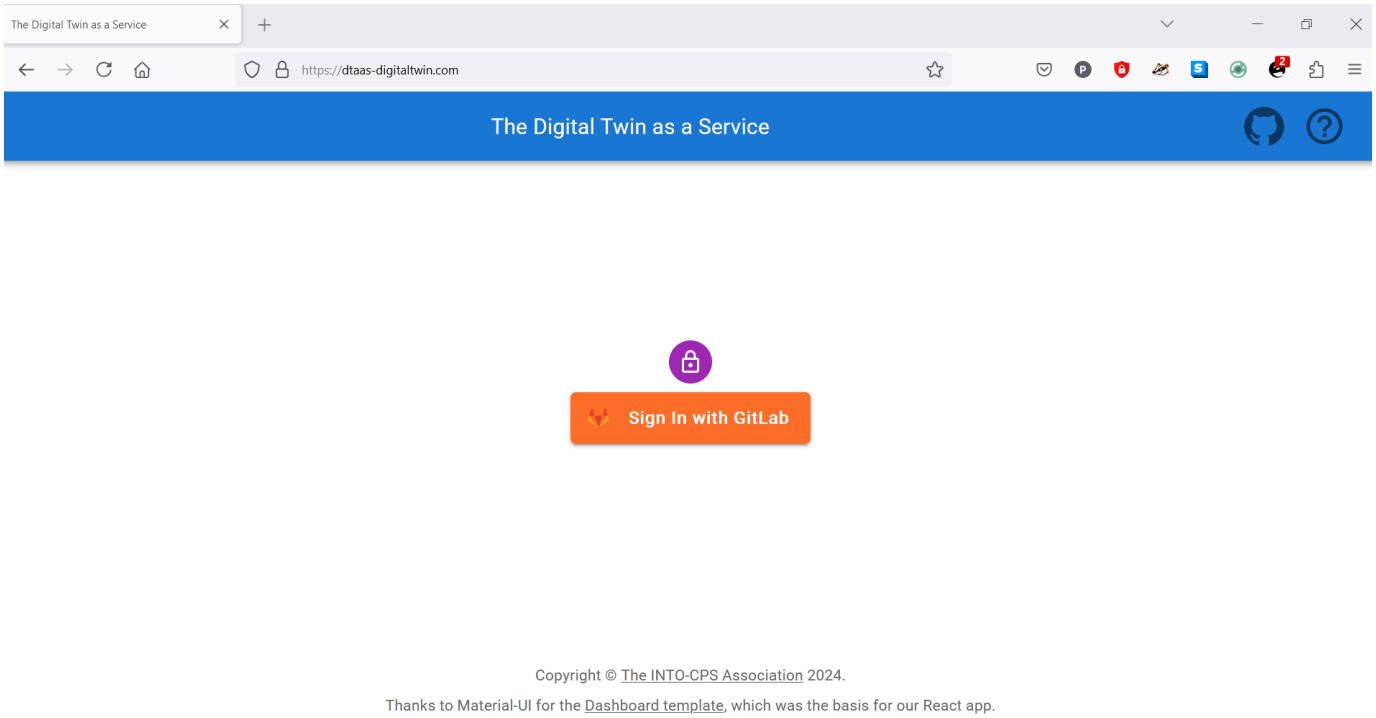
Permit DTaaS Server to Use GitLab



Clicking on Authorize permits the OAuth 2.0 application to access the information associated with the GitLab account. This is a required step.

After successful authentication, redirection to the login page of the DTaaS website occurs.

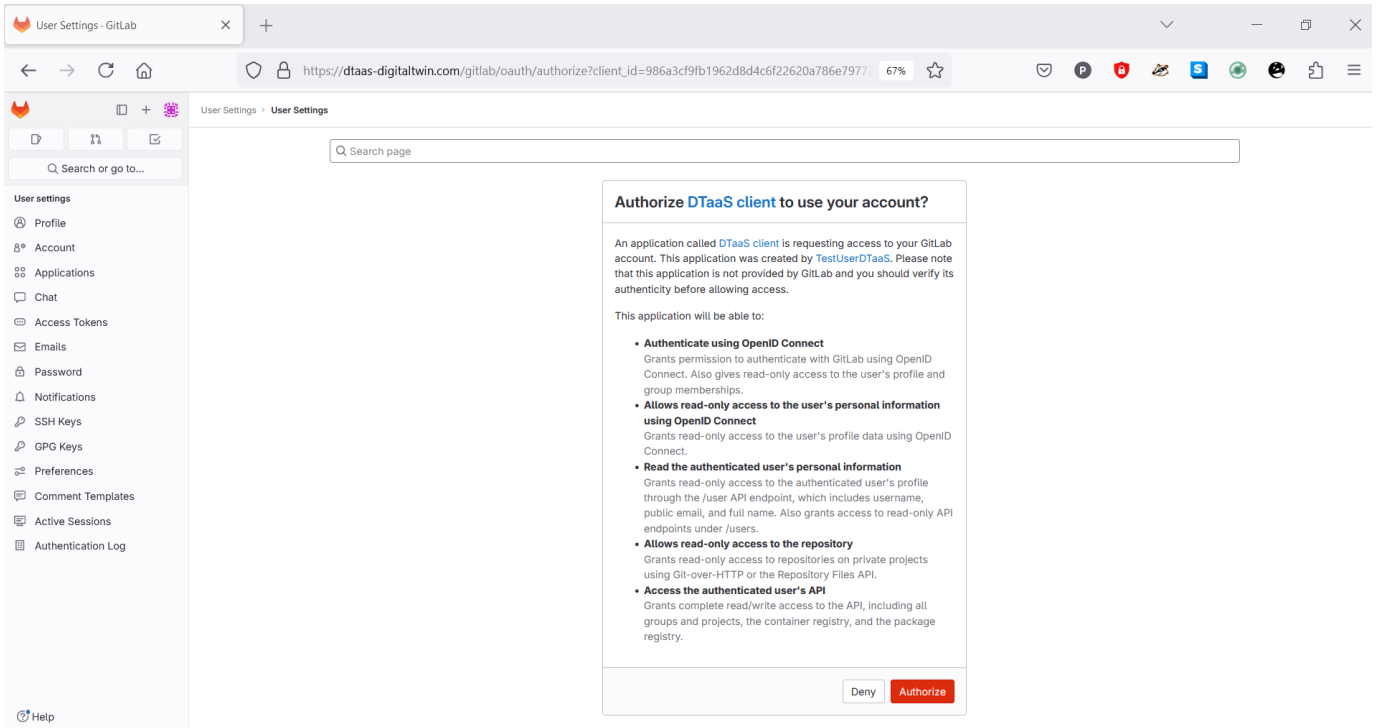
The DTaaS website employs an additional layer of security - the third-party authorization protocol known as OAuth 2.0. This protocol provides secure access to a DTaaS installation for users with active accounts at the selected OAuth 2.0 service provider. This implementation also uses GitLab as the OAuth 2.0 provider.



The GitLab signin button is displayed. Clicking this button redirects to the GitLab instance providing authorization for DTaaS. Re-authentication to GitLab is not required, unless explicit logout from the GitLab account has occurred.

Permit DTaaS Website to Use GitLab

The DTaaS website requires permission to use the GitLab account for authorization. The **Authorize** button must be clicked.

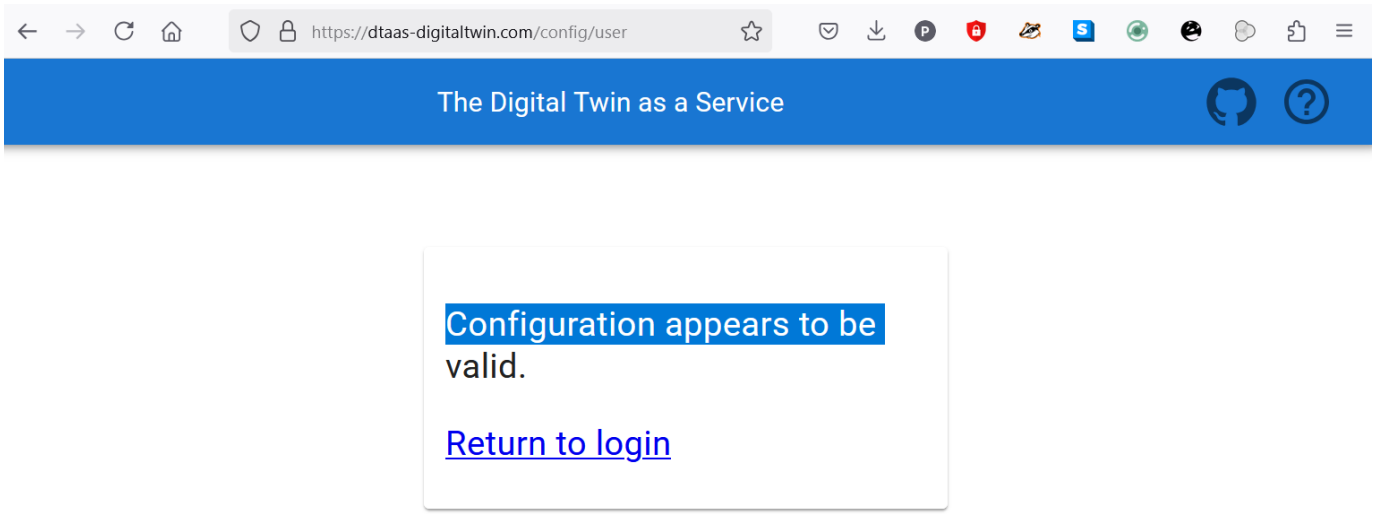


After successful authorization, redirection to the **Library** page of the DTaaS website occurs.

Two icons are located on the top-right of the webpage. The hyperlink on the **question mark icon** redirects to the help page, while the hyperlink on the **github icon** redirects to the GitHub code repository.

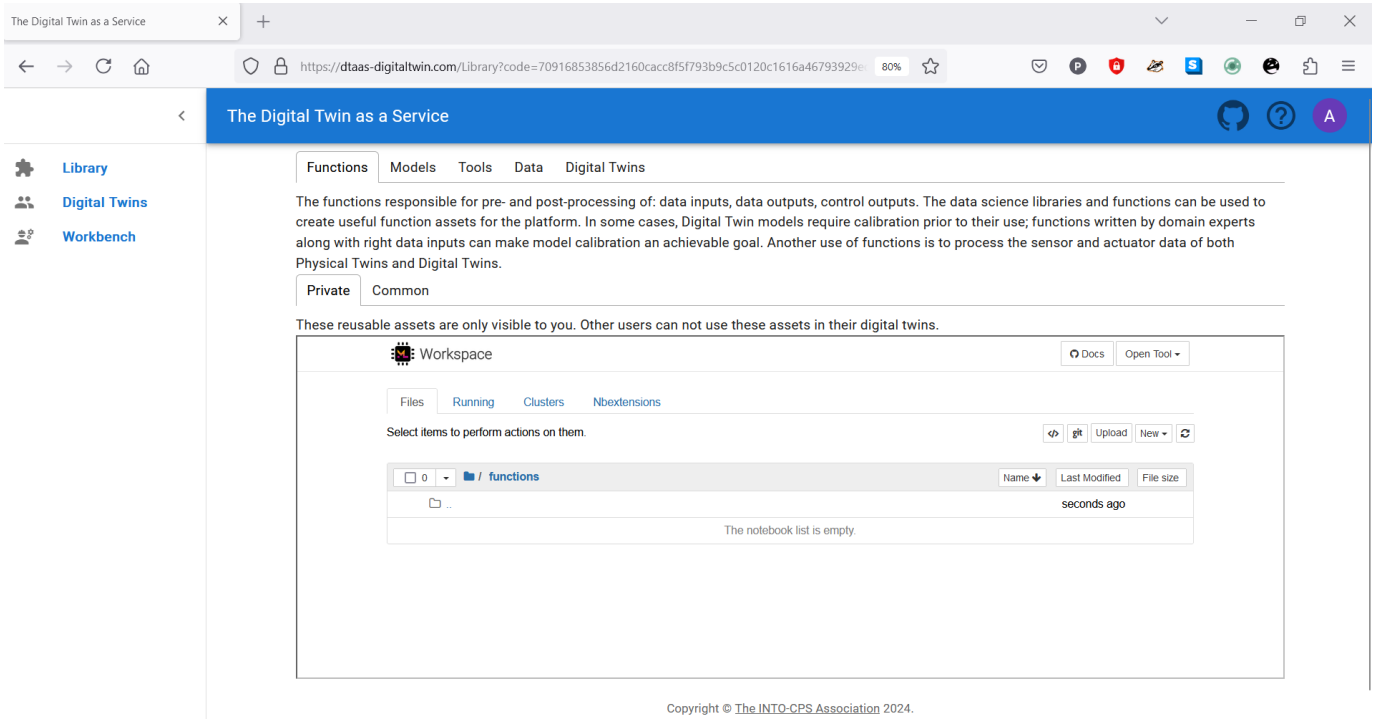
Check Website Access

For troubleshooting login issues, the website configuration can be verified by navigating to https://intocps.org/config/user. The following display indicates a correctly configured application.



Menu Items

The menu is hidden by default. Only the icons of menu items are visible. Clicking on the ☰ icon in the top-left corner of the page reveals the menu.



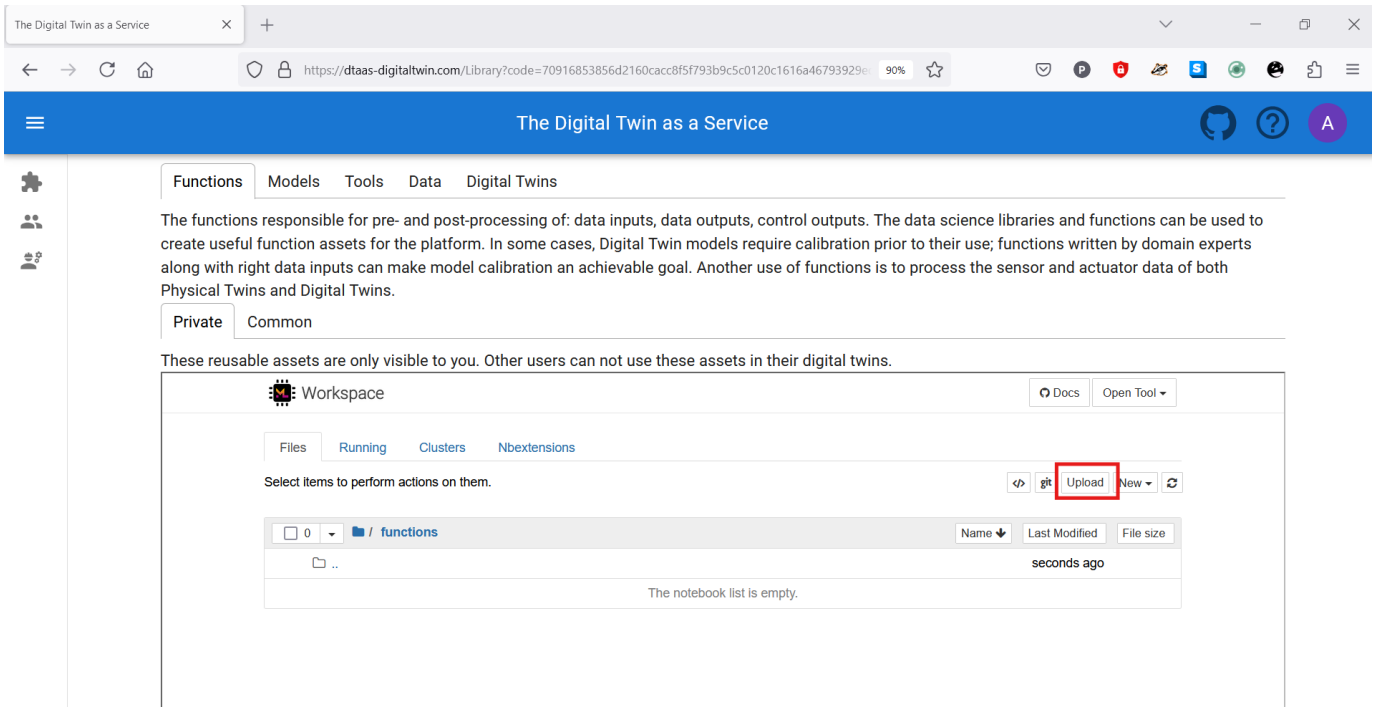
Three menu items are available:

Library: For management of reusable library assets. Files can be uploaded, downloaded, created, and modified on this page.

Digital Twins: For management of digital twins. A Jupyter Lab page is presented from which digital twins can be executed.

Workbench: Not all digital twins can be managed within Jupyter Lab. Additional tools are available on this page.

Library Page



Five tabs are displayed, each corresponding to one type of digital twin asset. Each tab provides help text to guide users on the asset type.

Functions

The functions responsible for pre- and post-processing of: data inputs, data outputs, control outputs. The data science libraries and functions can be used to create useful function assets for the platform. In some cases, Digital Twin models require calibration prior to their use; functions written by domain experts along with right data inputs can make model calibration an achievable goal. Another use of functions is to process the sensor and actuator data of both Physical Twins and Digital Twins.

Data

The data sources and sinks available to a digital twins. Typical examples of data sources are sensor measurements from Physical Twins, and test data provided by manufacturers for calibration of models. Typical examples of data sinks are visualization software, external users and data storage services. There exist special outputs such as events, and commands which are akin to control outputs from a Digital Twin. These control outputs usually go to Physical Twins, but they can also go to another Digital Twin.

Models

The model assets are used to describe different aspects of Physical Twins and their environment, at different levels of abstraction. Therefore, it is possible to have multiple models for the same Physical Twin. For example, a flexible robot used in a car production plant may have structural model(s) which will be useful in tracking the wear and tear of parts. The same robot can have a behavioural model(s) describing the safety guarantees provided by the robot manufacturer. The same robot can also have a functional model(s) describing the part manufacturing capabilities of the robot.

Tools

The software tool assets are software used to create, evaluate and analyse models. These tools are executed on top of a computing platforms, i.e., an operating system, or virtual machines like Java virtual machine, or inside docker containers. The tools tend to be platform specific, making them less reusable than models. A tool can be packaged to run on a local or distributed virtual machine environments thus allowing selection of most suitable execution environment for a Digital Twin. Most models require tools to evaluate them in the context of data inputs. There exist cases where executable packages are run as binaries in a computing environment. Each of these packages are a pre-packaged combination of models and tools put together to create a ready to use Digital Twins.

Digital Twins

These are ready to use digital twins created by one or more users. These digital twins can be reconfigured later for specific use cases.

Two sub-tabs exist: **private** and **common**. Library assets in the private category are visible only to the logged-in user, while library assets in the common category are available to all users.

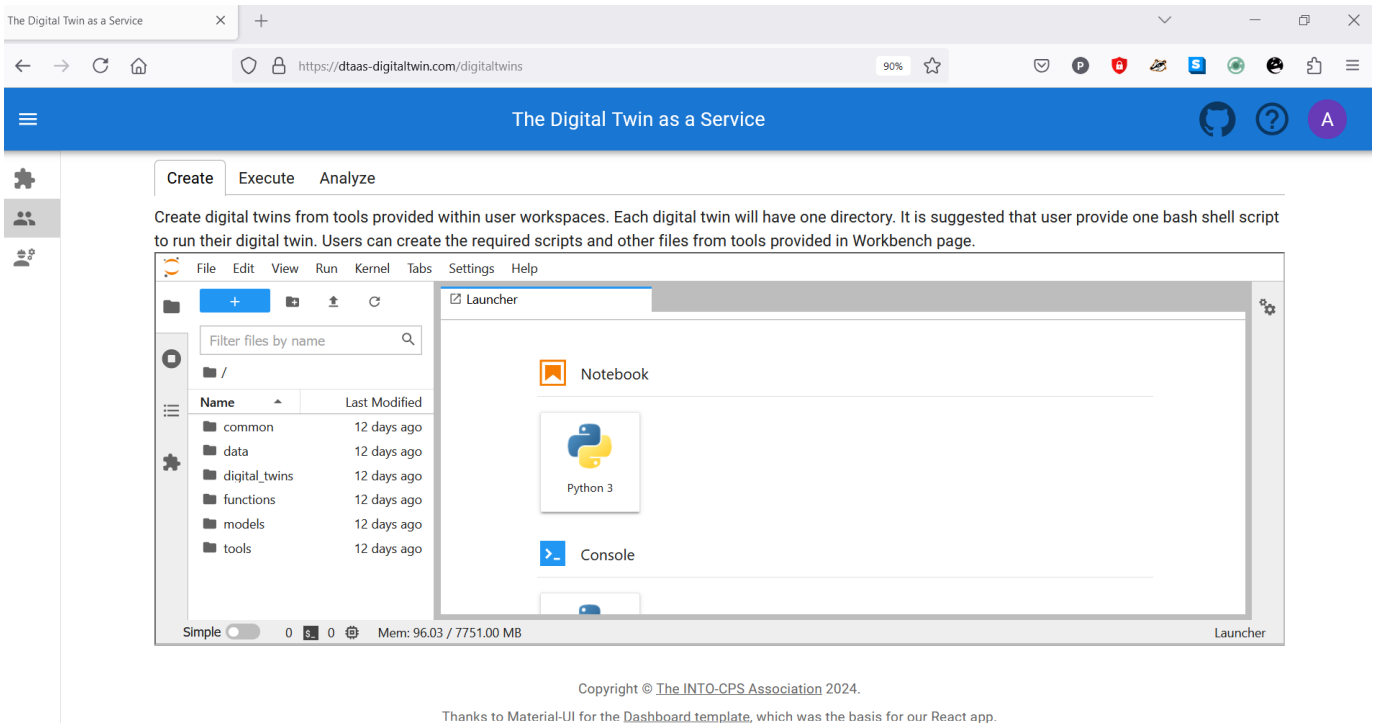
Further explanation on the placement of reusable assets within each type and the underlying directory structure on the server is available on the assets page.

Note

Assets (files) can be uploaded using the **upload** button.

The file manager is based on Jupyter Notebook, and all tasks available in Jupyter Notebook can be performed here.

Digital Twins Page



The digital twins page contains three tabs, and the central pane opens Jupyter Lab. The three tabs provide helpful instructions on suggested tasks for the **Create - Execute - Analyze** lifecycle phases of a digital twin. More explanation is available on the lifecycle phases of digital twin.

Create

Create digital twins from tools provided within user workspaces. Each digital twin will have one directory. It is suggested that user provide one bash shell script to run their digital twin. Users can create the required scripts and other files from tools provided in Workbench page.

Execute

Digital twins are executed from within user workspaces. The given bash script gets executed from digital twin directory. Terminal-based digital twins can be executed from VSCode and graphical digital twins can be executed from VNC GUI. The results of execution can be placed in the data directory.

Analyze

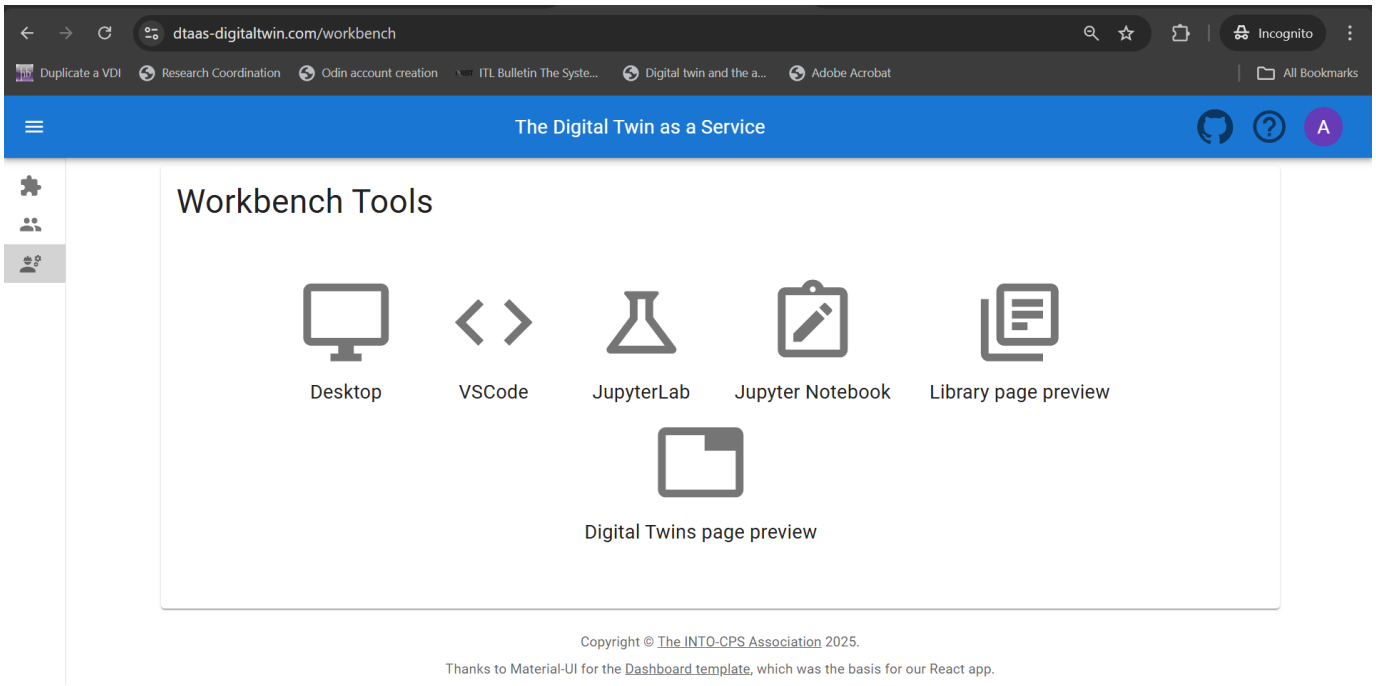
The analysis of digital twins requires running of digital twin script from user workspace. The execution results placed within data directory are processed by analysis scripts and results are placed back in the data directory. These scripts can either be executed from VSCode and graphical results or can be executed from VNC GUI. The analysis of digital twins requires running of digital twin script from user workspace. The execution results placed within data directory are processed by analysis scripts and results are placed back in the data directory. These scripts can either be executed from VSCode and graphical results or can be executed from VNC GUI.

The reusable assets (files) displayed in the file manager are also available in Jupyter Lab. Additionally, a git plugin is installed in Jupyter Lab that enables linking files with external git repositories.

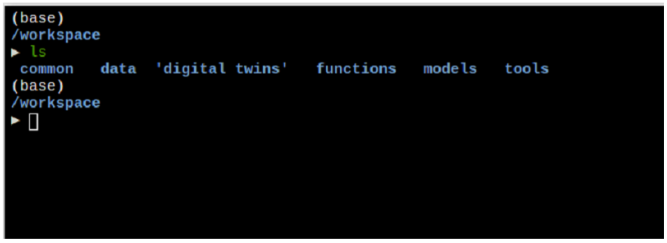
Workbench

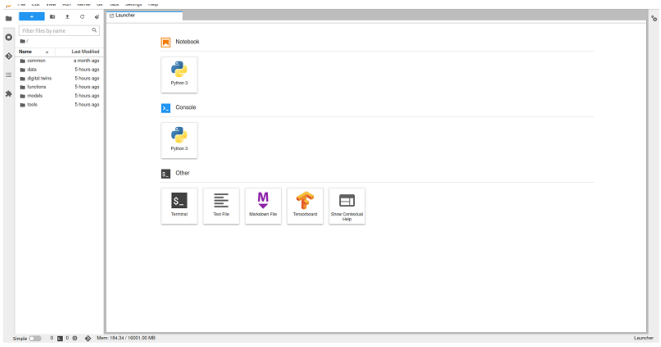
The **workbench** page provides links to four integrated tools:

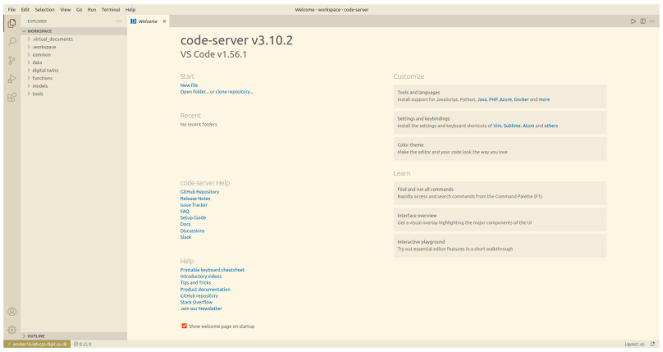
- Desktop
- VS Code
- Jupyter Lab
- Jupyter Notebook

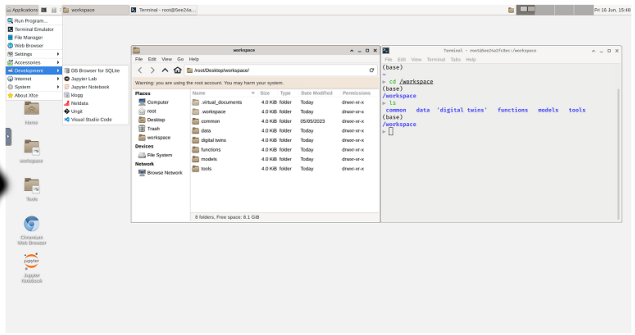


Screenshots of the pages opened in new browsers are shown:









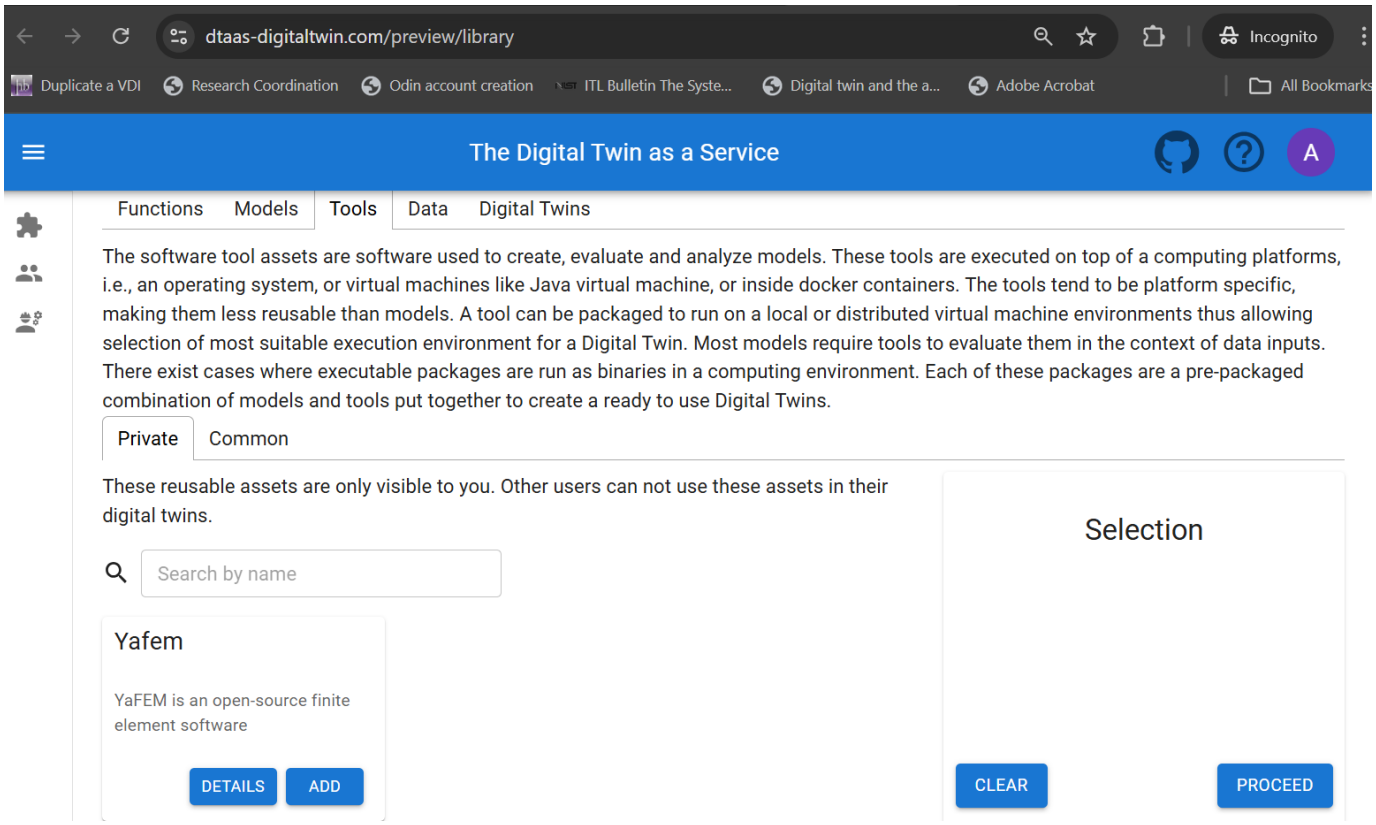
The hyperlinks open in new browser tabs.

The **workbench** also has two links to DevOps-based implementation of composable digital twins.

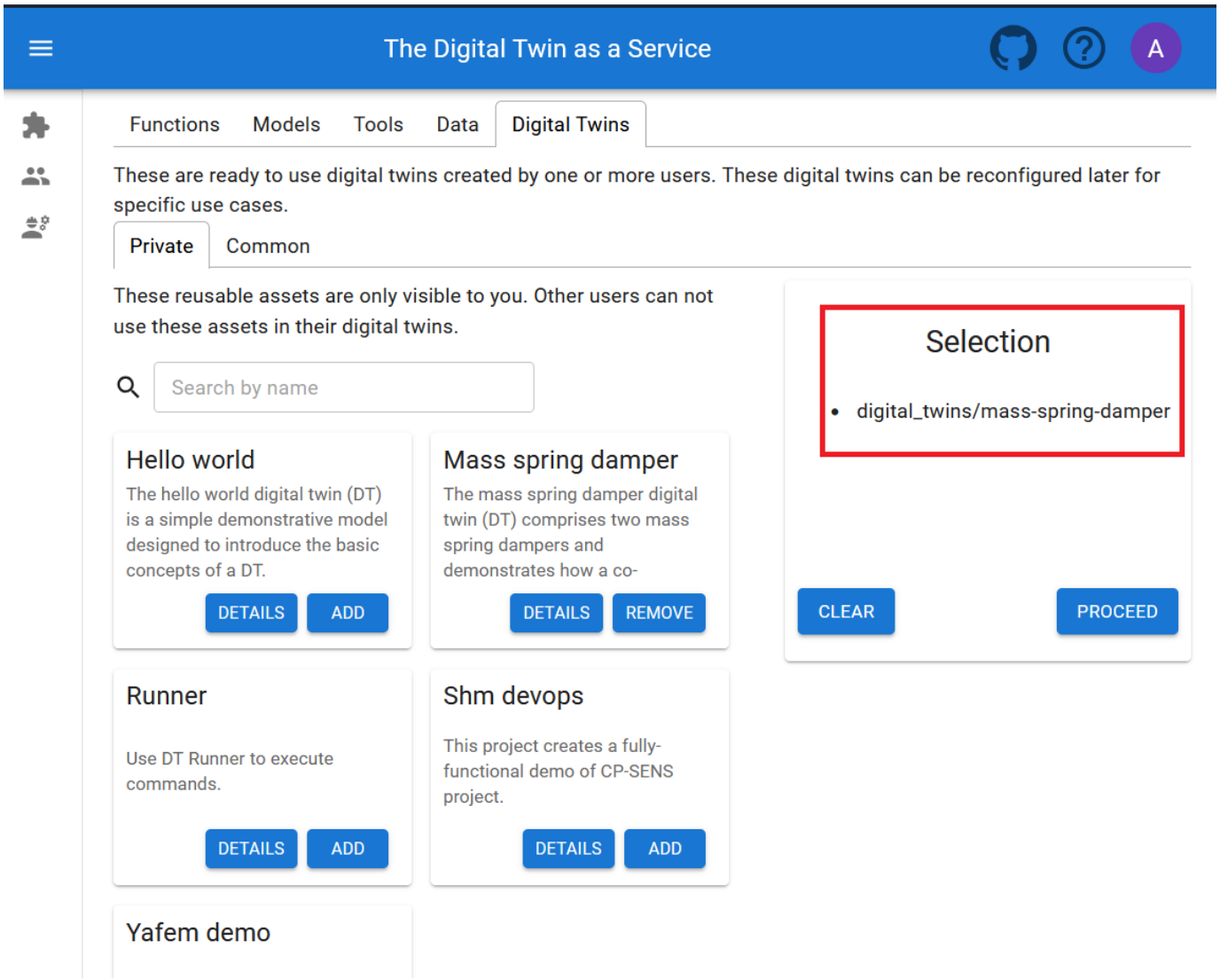
- Library Page Preview
- Digital Twins Page Preview

LIBRARY PREVIEW PAGE

This page has the same philosophy of Library page and provides similar user interface.



Unlike the Library page, this preview page uses digital twin assets stored in a GitLab repository. New digital twins can be composed by selecting the required library assets.



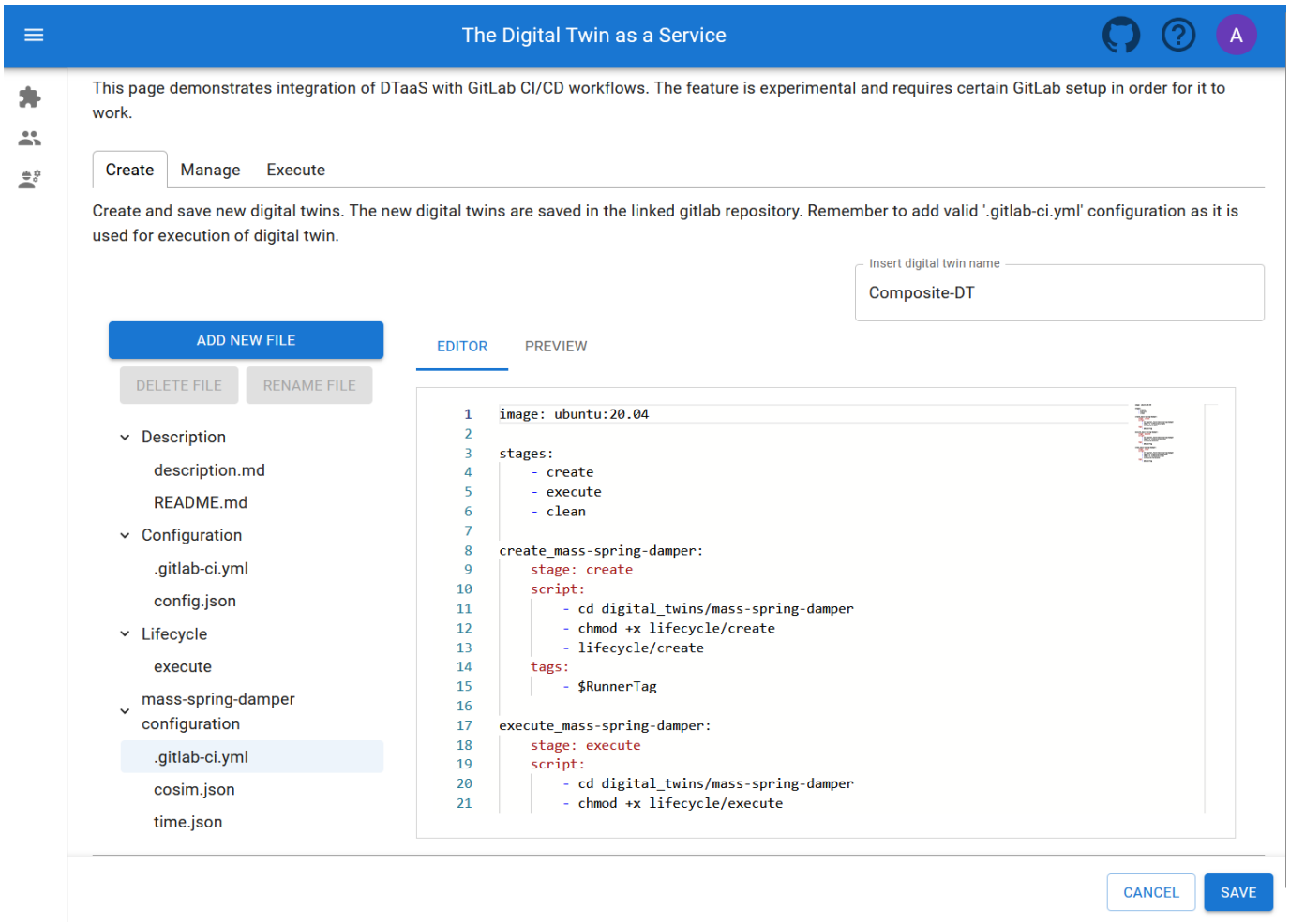
Upon clicking **Proceed** button, the digital twins create tab is opened.

Digital Twins Preview Page

The **Digital Twins Preview Page** provides means of managing digital twins using the DevOps methodology. This page has three tabs, namely **Create**, **Manage** and **Execute**.

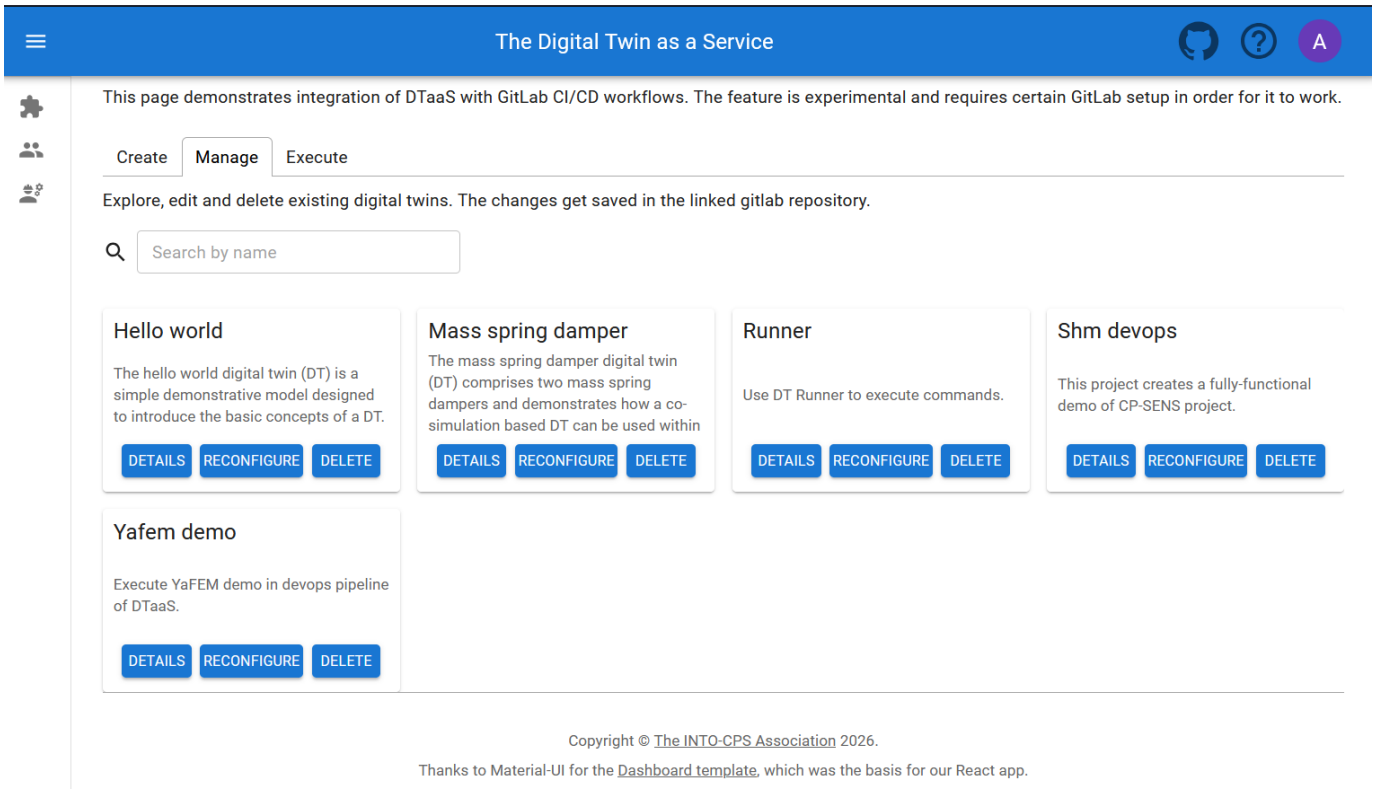
CREATE TAB

The library assets selected will be used on the **Create Tab** for creating new digital twins. The new digital twins are saved in the linked GitLab repository. Remember to add valid `.gitlab-ci.yml` configuration as it is used for execution of digital twin.

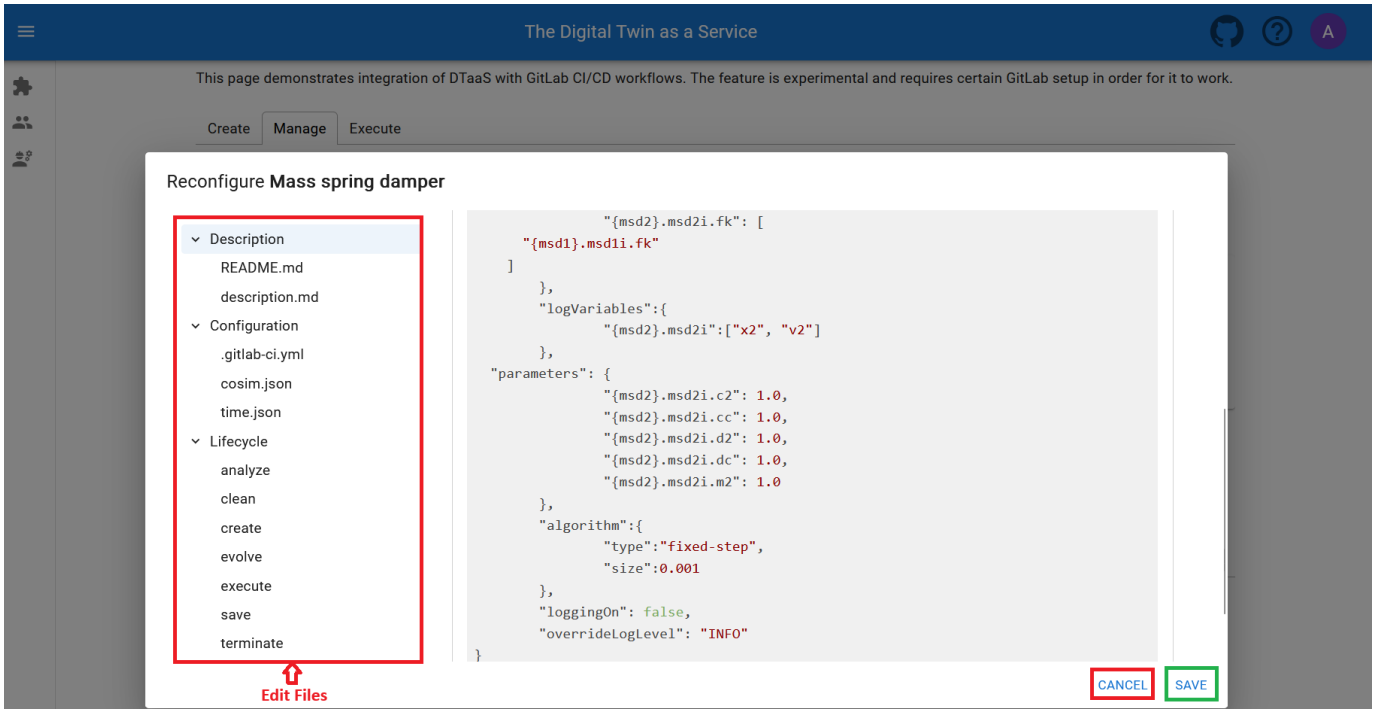


MANAGE TAB

Complete descriptions of digital twins can be read.

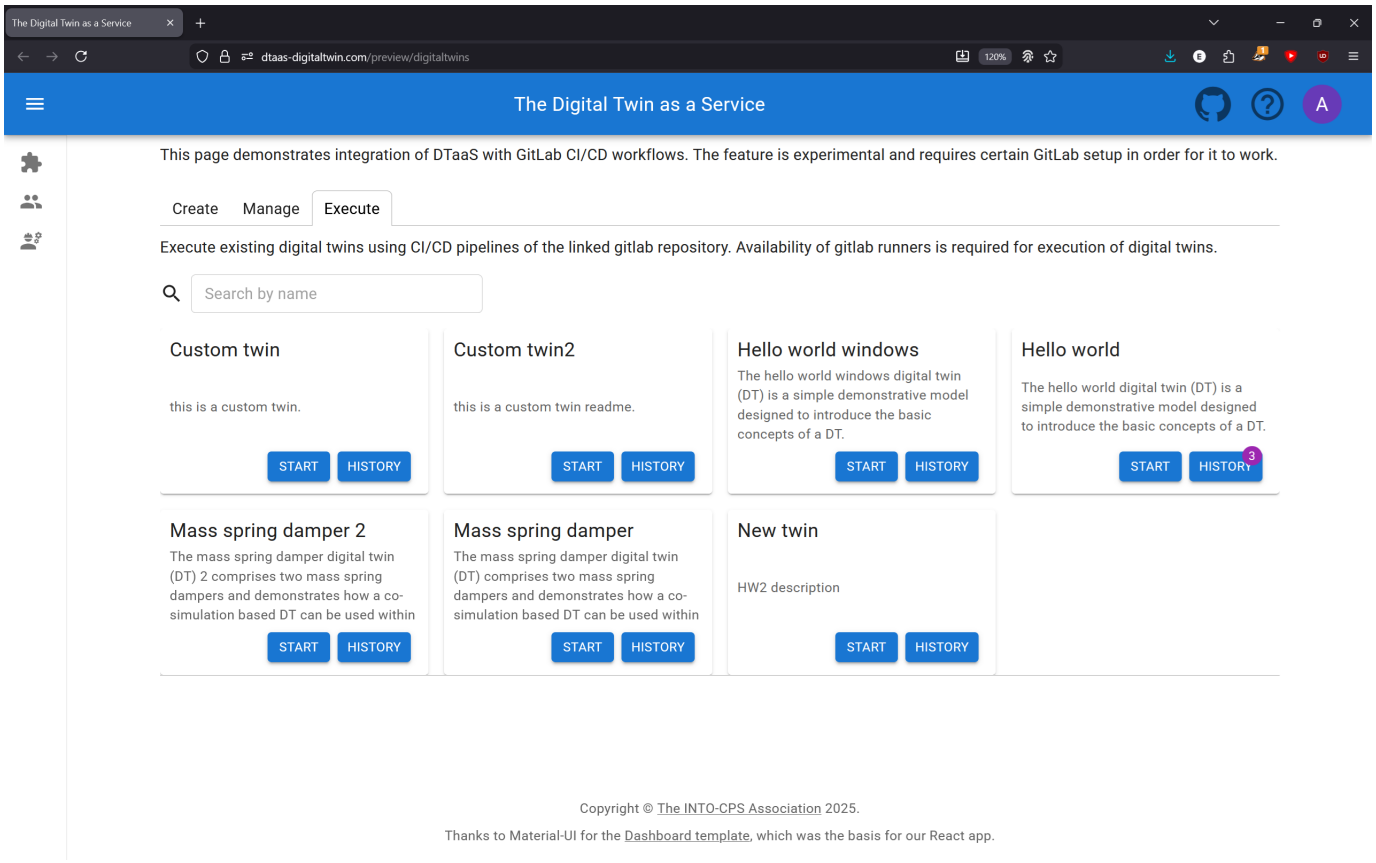


If necessary, a digital twin can be deleted, removing it from the workspace along with all associated data. Digital twins can also be reconfigured.

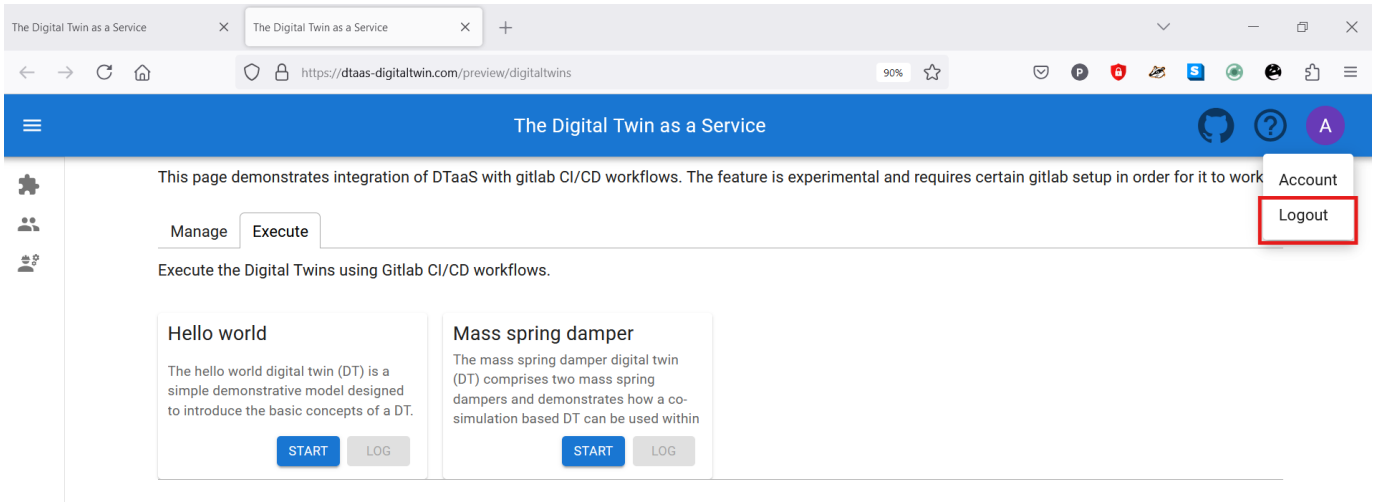


EXECUTE TAB

Digital Twins can be executed using GitLab CI/CD workflows. Multiple digital twins can be executed simultaneously.



Finally logout



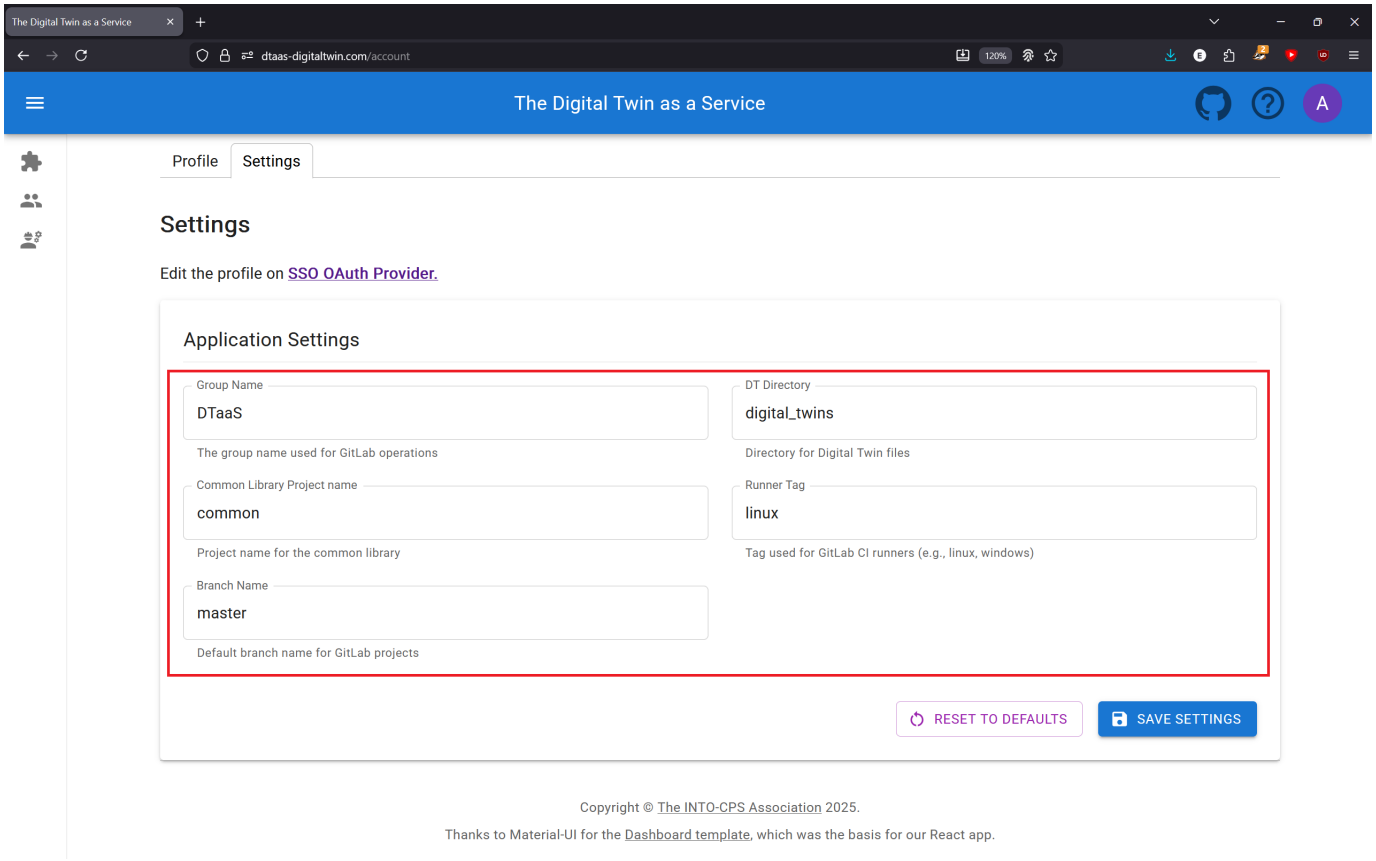
The browser must be closed to completely exit the DTaaS software platform.

3.3.2 Settings

Settings are important both during initial DTaaS setup and during use when working with different configurations. All the most important settings have been consolidated on one page for both of these scenarios.

⚙️ Changing Settings

The parameters used for sending and receiving information to the storage and execution services (e.g., GitLab) may need adjustment to match the infrastructure. Navigation to **Account** using the top-right purple 🅰️ icon followed by selection of the **Settings** tab displays the following adjustable parameters:



1. Group Name
2. DT Directory
3. Common Library Project name
4. Runner Tag
5. Branch Name

Following is a description of what each parameter does.

GROUP NAME

The Group Name denotes the highest level of organizational abstraction concerned with on the storage service, namely Groups. A GitLab group is required to use the DTaaS. Within the group, projects reside, which must match the usernames of system users. More information about the file organization is available. This parameter must be set to the case-insensitive name of the group.

Default: DTaaS

COMMON LIBRARY PROJECT NAME

One project within the group serves as the Digital Twins *Library*. Through the DTaaS, the files inside the Library are accessible to all users and can be copied to individual user projects as needed. This parameter specifies the project name of the Library, and must match that name.

Default: common

DT DIRECTORY

Within the common library and user projects, files related to Digital Twins are stored within a designated folder. This is the name chosen for that folder.

Default: Digital_Twins

BRANCH NAME

This parameter determines which branch to search for data (Twins, Functions, etc.) within user and library projects. This parameter also determines which branch's Digital Twins are executed.

Default: master

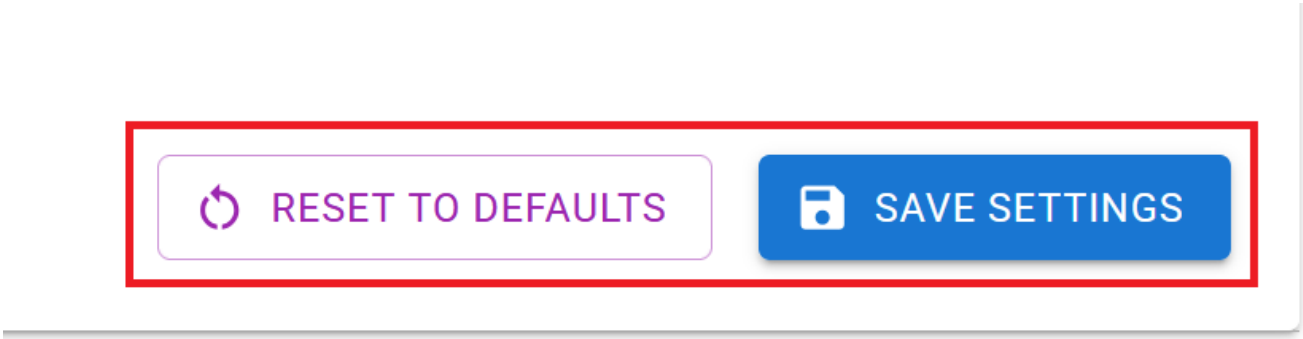
RUNNER TAG

The (GitLab) runners responsible for executing Digital Twin code must be associated with a tag. Only one tag can be specified, and it **cannot** be left blank, or the job of running the twin will not be processed. This is a limitation of the DTaaS.

Default: linux

💾 Saving and Resetting Changes

When satisfied with the changes, **SAVE SETTINGS** must be pressed for them to persist after leaving the Settings page. If a mistake was made, the settings can be reset to their default values by pressing the **RESET TO DEFAULTS** button. The reset values are saved automatically, so additional saving is not required. The Saving and Resetting buttons on the page:



The default values can be found and modified in the code if needed.

Return to the DevOps pages (i.e., Digital Twin Preview) is now possible. **Refreshing ensures fresh data from the remote repository is fetched**, and the Digital Twins should be visible, ready to be executed, edited, and shared.

💭 Summary

This document has described how to edit the settings for initializing the DTaaS to a project and for continuous use (i.e., modifying Runner Tag and Branch). The need to save changes and how to return to default values if a mistake is made have been discussed.

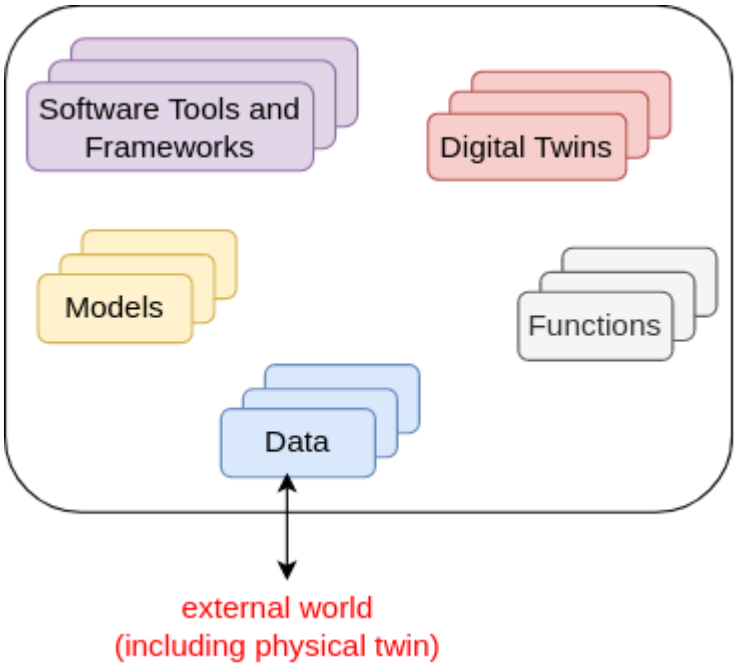
3.4 Reusable Assets

3.4.1 Reusable Assets

The reusability of digital twin assets facilitates efficient work with digital twins. Reusability of assets is a fundamental feature of the platform[1].

Kinds of Reusable Assets

The DTaaS platform categorizes all reusable library assets into six categories:



DATA

The data sources and sinks available to a digital twins. Typical examples of data sources are sensor measurements from Physical Twins, and test data provided by manufacturers for calibration of models. Typical examples of data sinks are visualization software, external users and data storage services. There exist special outputs such as events, and commands which are akin to control outputs from a Digital Twin. These control outputs usually go to Physical Twins, but they can also go to another Digital Twin.

MODELS

The model assets are used to describe different aspects of Physical Twins and their environment, at different levels of abstraction. Therefore, it is possible to have multiple models for the same Physical Twin. For example, a flexible robot used in a car production plant may have structural model(s) which will be useful in tracking the wear and tear of parts. The same robot can have a behavioural model(s) describing the safety guarantees provided by the robot manufacturer. The same robot can also have a functional model(s) describing the part manufacturing capabilities of the robot.

TOOLS

The software tool assets are software used to create, evaluate and analyse models. These tools are executed on top of a computing platforms, i.e., an operating system, or virtual machines like Java virtual machine, or inside docker containers. The tools tend to be platform specific, making them less reusable than models. A tool can be packaged to run on a local or distributed virtual machine environments thus allowing selection of most suitable execution environment for a Digital Twin. Most models require tools to evaluate them in the context of data inputs. There exist cases where executable packages are run as binaries in a computing environment. Each of these packages are a pre-packaged combination of models and tools put together to create a ready to use Digital Twins.

FUNCTIONS

The functions responsible for pre- and post-processing of: data inputs, data outputs, control outputs. The data science libraries and functions can be used to create useful function assets for the platform. In some cases, Digital Twin models require calibration prior to their use; functions written by domain experts along with right data inputs can make model calibration an achievable goal. Another use of functions is to process the sensor and actuator data of both Physical Twins and Digital Twins.

DIGITAL TWINS

These are ready to use digital twins created by one or more users. These digital twins can be reconfigured later for specific use cases.

File System Structure

Each user has their assets put into five different directories named above. In addition, there will also be common library assets that all users have access to. A simplified example of the structure is as follows:

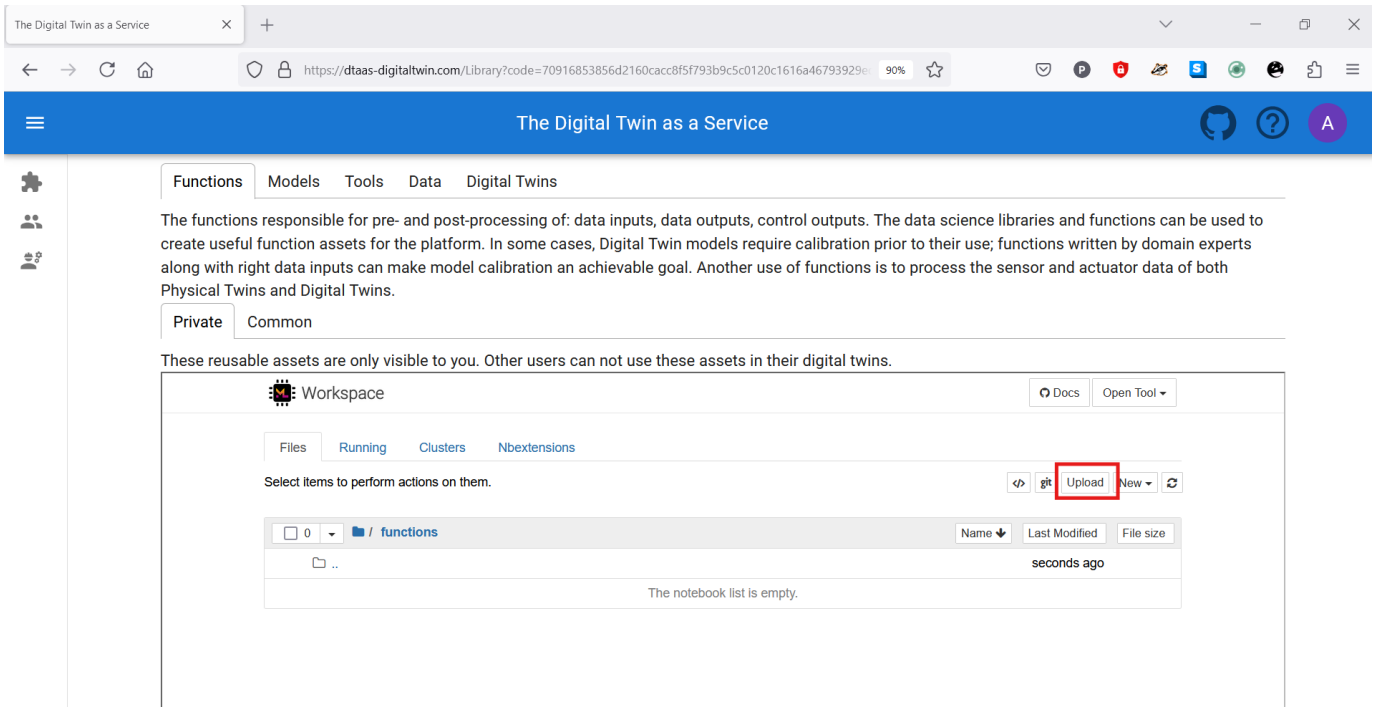
```
workspace/
  data/
    data1/ (ex: sensor)
      filename (ex: sensor.csv)
      README.md
    data2/ (ex: turbine)
      README.md (remote source; no local file)
    ...
  digital_twins/
    digital_twin-1/ (ex: incubator)
      config (yaml and json)
      README.md (usage instructions)
      description.md (short summary of digital twin)
      lifecycle/ (directory containing lifecycle scripts)
    digital_twin-2/ (ex: mass spring damper)
      config (yaml and json)
      README.md (usage instructions)
      description.md (short summary of digital twin)
      lifecycle/ (directory containing lifecycle scripts)
    digital_twin-3/ (ex: model swap)
      config (yaml and json)
      README.md (usage instructions)
      description.md (short summary of digital twin)
      lifecycle/ (directory containing lifecycle scripts)
    ...
  functions/
    function1/ (ex: graphs)
      filename (ex: graphs.py)
      README.md
    function2/ (ex: statistics)
      filename (ex: statistics.py)
      README.md
    ...
  models/
    model1/ (ex: spring)
      filename (ex: spring.fmu)
      README.md
    model2/ (ex: building)
      filename (ex: building.skp)
      README.md
    model3/ (ex: rabbitmq)
      filename (ex: rabbitmq.fmu)
      README.md
    ...
  tools/
    tool1/ (ex: maestro)
      filename (ex: maestro.jar)
      README.md
    ...
  common/
    data/
    functions/
    models/
    tools/
```

The DTaaS is agnostic to the format of assets. The only requirement is that they are files which can be uploaded on the Library page. Directories can be compressed as single files and uploaded. The files can be decompressed into directories from a Terminal or xfce Desktop available on the Workbench page.

A recommended file system structure for storing assets is also available in DTaaS examples.

Upload Assets

Users can upload assets into their workspace using the Library page of the website.



Navigation into a directory followed by clicking on the **upload** button allows uploading files or directories into the workspace. These assets then become available in all workbench tools. New assets can also be created on the page by clicking on the **new** dropdown menu. This simple web interface allows creation of text-based files. Other files must be uploaded using the **upload** button.

The user workbench provides the following services:

- Jupyter Notebook and Lab
- VS Code
- XFCE Desktop Environment available via VNC
- Terminal

Users can also bring DT assets into user workspaces from external sources using any of the above-mentioned services. Developers using *git* repositories can clone from and push to remote git servers. Users can also use widely-used file transfer protocols such as FTP and SCP to bring the required DT assets into their workspaces.

References

[1]: Talasila, Prasad, et al. "Composable digital twins on Digital Twin as a Service platform." Simulation 101.3 (2025): 287-311.

3.4.2 Library Microservice

The lib microservice is responsible for handling and serving the contents of library assets of the DTaaS platform. It provides API endpoints for clients to query, and fetch these assets.

This document provides instructions for using the library microservice.

The assets page describes suggested storage conventions for library assets.

Once assets are stored in the library, they become available in the user workspace.

Application Programming Interface (API)

The lib microservice application provides services at two end points:

GraphQL API Endpoint: `http://intocps.org/lib`

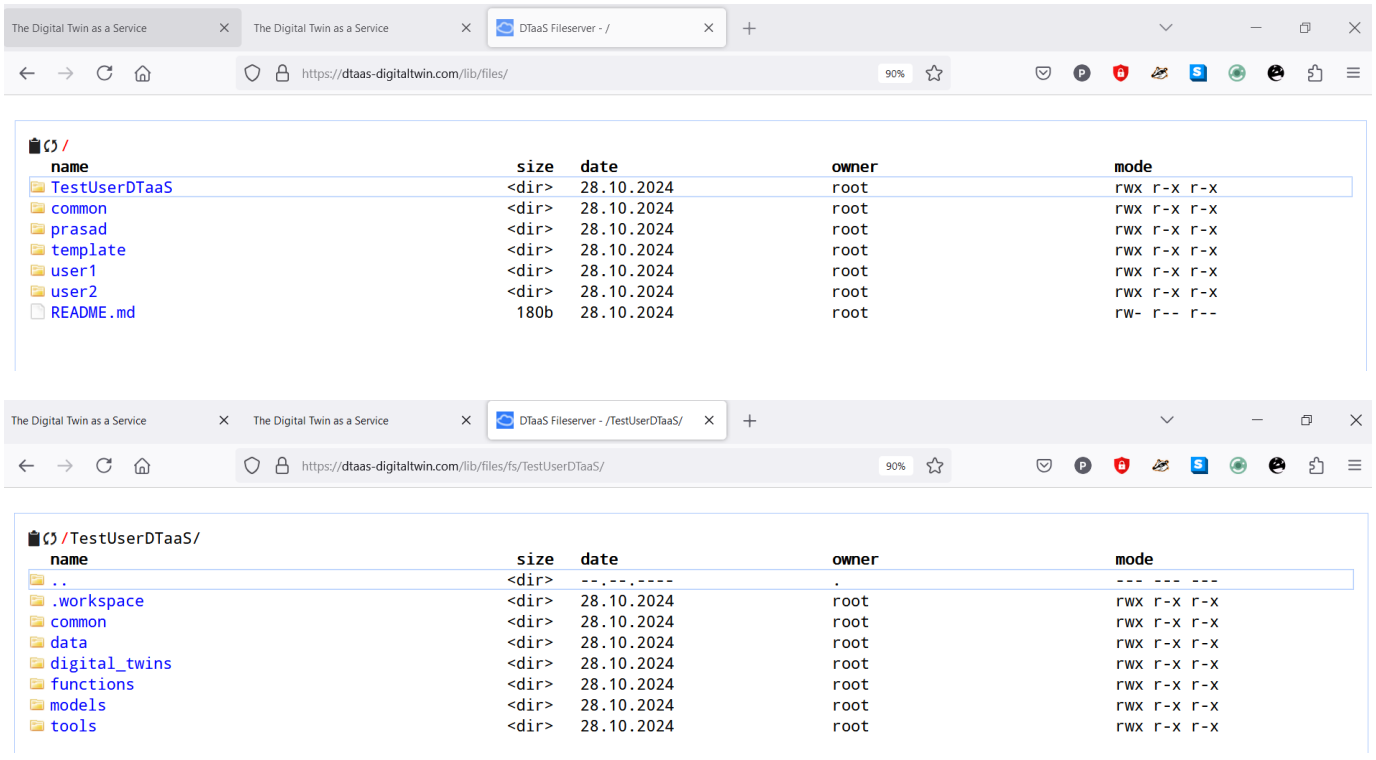
HTTP Endpoint: `http://intocps.org/lib/files`

HTTP PROTOCOL

Endpoint: `localhost:PORT/lib/files`

This option needs to be enabled with `-H http.json` flag. The regular file upload and download options become available.

Here are sample screenshots.



GRAPHQL PROTOCOL

Endpoint: `localhost:PORT/lib`

The `http://intocps.org/lib` URL opens a GraphQL playground.

The query schema can be examined and sample queries can be tested there. GraphQL queries must be sent as HTTP POST requests to receive responses.

The library microservice provides two API calls:

- Provide a list of contents for a directory
- Fetch a file from the available files

The API calls are accepted over GraphQL and HTTP API endpoints. The format of accepted queries is:

PROVIDE LIST OF CONTENTS FOR A DIRECTORY

To retrieve a list of files in a directory, use the following GraphQL query.

Replace `path` with the desired directory path.

send requests to: https://intocps.org/lib

GraphQL query for list of contents

```
query {
  listDirectory(path: "user1") {
    repository {
      tree {
        blobs {
          edges {
            node {
              name
              type
            }
          }
        }
        trees {
          edges {
            node {
              name
              type
            }
          }
        }
      }
    }
  }
}
```

GraphQL response for list of contents

```
{
  "data": {
    "listDirectory": {
      "repository": {
        "tree": {
          "blobs": {
            "edges": []
          },
          "trees": {
            "edges": [
              {
                "node": {
                  "name": "common",
                  "type": "tree"
                }
              },
              {
                "node": {
                  "name": "data",
                  "type": "tree"
                }
              },
              {
                "node": {
                  "name": "digital twins",
                  "type": "tree"
                }
              },
              {
                "node": {
                  "name": "functions",
                  "type": "tree"
                }
              },
              {
                "node": {
                  "name": "models",
                  "type": "tree"
                }
              },
              {
                "node": {
                  "name": "tools",
                  "type": "tree"
                }
              }
            ]
          }
        }
      }
    }
  }
}
```

HTTP request for list of contents

```
POST /lib HTTP/1.1
Host: intocps.org
Content-Type: application/json
Content-Length: 388

{
  "query":"query {\n  listDirectory(path: \"user1\") {\n    repository {\n      tree {\n        blobs {\n          edges {\n            node {\n              name\n              type\n            }\n          }\n        }\n        trees {\n          edges {\n            node {\n              name\n              type\n            }\n          }\n        }\n      }\n    }\n  }\n}"
}
```

HTTP response for list of contents

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
Connection: close
Content-Length: 306
Content-Type: application/json; charset=utf-8
Date: Tue, 26 Sep 2023 20:26:49 GMT
X-Powered-By: Express
{"data":{"listDirectory":{"repository":{"tree":{"blobs":{"edges":[]},"trees":{"edges":[{"node":{"name":"data","type":"tree"}},{"node":{"name":"digital twins","type":"tree"}},{"node":{"name":"functions","type":"tree"}},{"node":{"name":"models","type":"tree"}},{"node":{"name":"tools","type":"tree"}}]}}}}}}
```

FETCH A FILE FROM THE AVAILABLE FILES

This query receives directory path and send the file contents to user in response.

To check this query, create a file `files/user2/data/welcome.txt` with content of `hello world`.

GraphQL query for fetch a file

```
query {
  readFile(path: "user2/data/sample.txt") {
    repository {
      blobs {
        nodes {
          name
          rawBlob
          rawTextBlob
        }
      }
    }
  }
}
```

GraphQL response for fetch a file

```
{
  "data": {
    "readFile": {
      "repository": {
        "blobs": {
          "nodes": [
            {
              "name": "sample.txt",
              "rawBlob": "hello world",
              "rawTextBlob": "hello world"
            }
          ]
        }
      }
    }
  }
}
```

HTTP request for fetch a file

```
POST /lib HTTP/1.1
Host: intocps.org
Content-Type: application/json
Content-Length: 217
{
  "query":"query {\n  readFile(path: \"user2/data/welcome.txt\") {\n    repository {\n      blobs {\n        nodes {\n          name\n          rawBlob\n
rawTextBlob\n        }\n      }\n    }\n  }\n}"
}
```

HTTP response for fetch a file

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
Connection: close
Content-Length: 134
Content-Type: application/json; charset=utf-8
Date: Wed, 27 Sep 2023 09:17:18 GMT
X-Powered-By: Express
{"data":{"readFile":{"repository":{"blobs":{"nodes":[{"name":"welcome.txt","rawBlob":"hello world","rawTextBlob":"hello world"}]}}}}}
```

The *path* refers to the file path to look at: For example, *user1* looks at files of **user1**; *user1/functions* looks at contents of *functions/* directory.

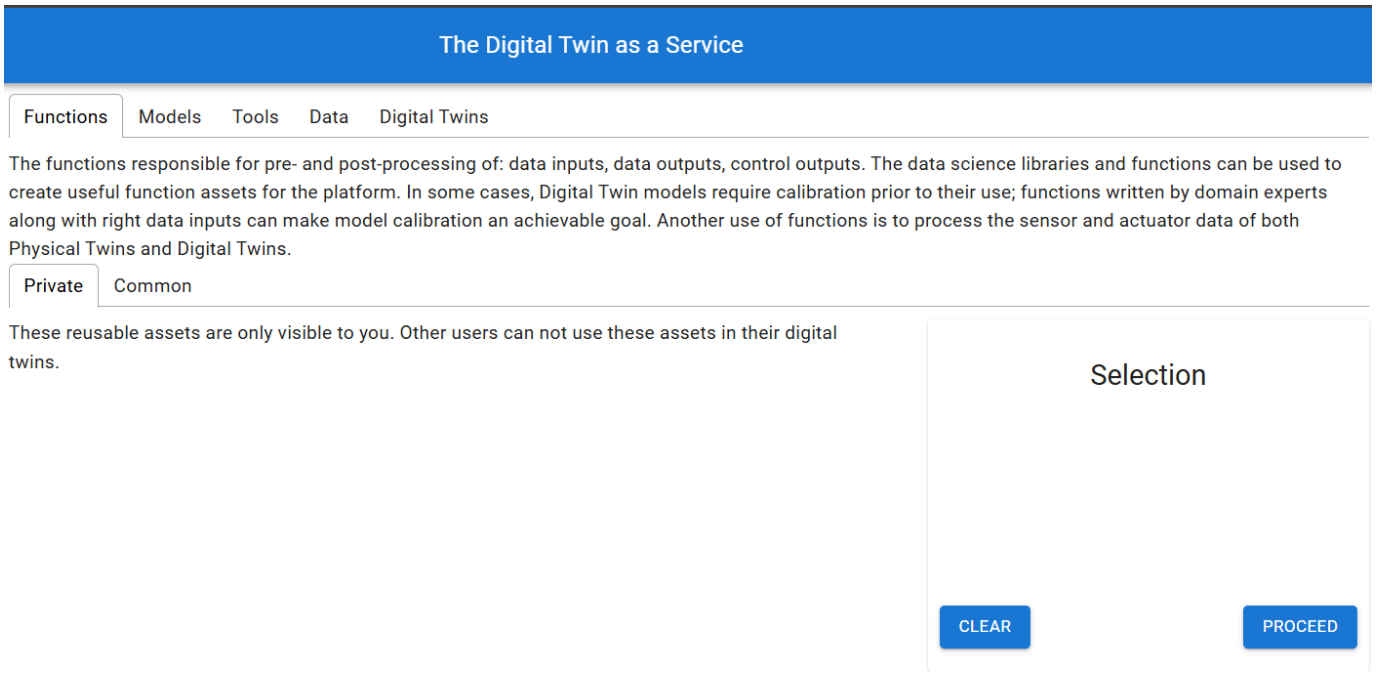
3.4.3 Reusable Assets and DevOps

DevOps has been a well established software development practice. An experimental feature integrating DevOps is being introduced in the DTaaS platform.

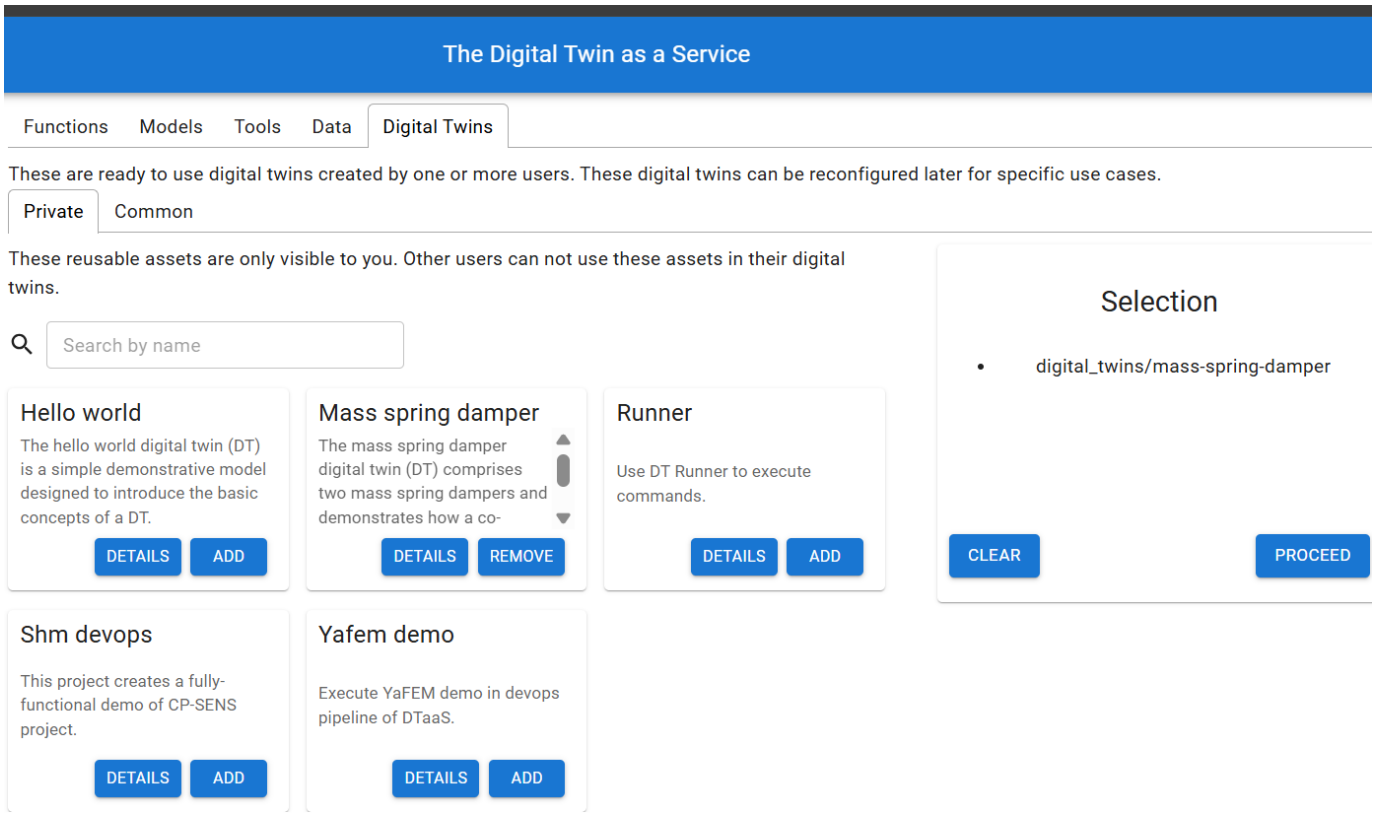
This feature requires specific installation setup.

1. Integrated GitLab installation
2. A valid GitLab repository for the logged in user. See an example repository. This repository can be cloned and customised as needed.
3. A linked GitLab Runner to the user GitLab repository.

Once these requirements are satisfied, the **Library** page shows all the reusable assets available stored in the linked GitLab repository. An empty list is shown if there are no assets of a specific category.



The page gets populated with any existing assets. All available DTs are shown in the following figure.



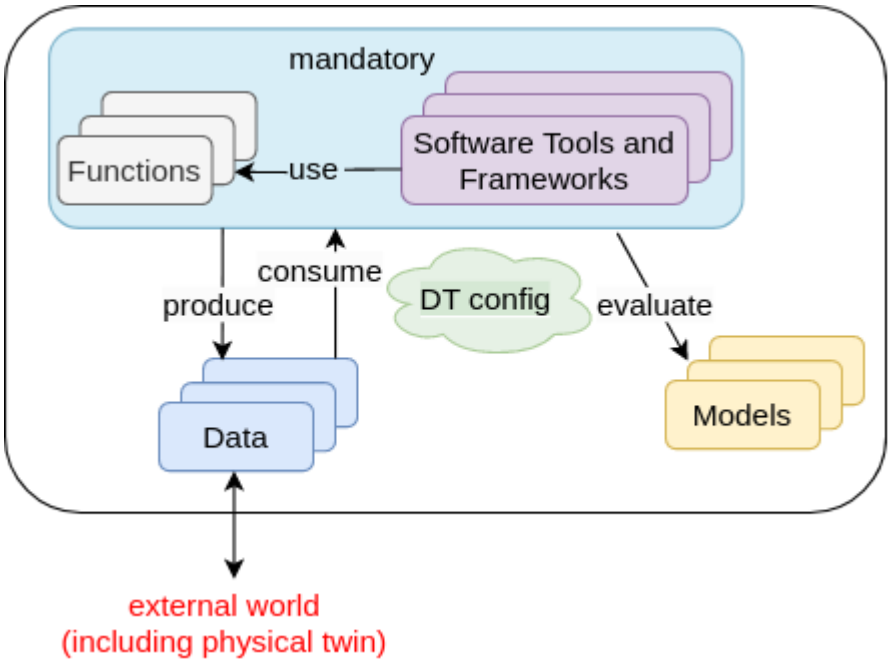
Any existing DT asset can be selected and it gets added to the **Selection** pane on the right. After selecting all the required assets, clicking on the *Proceed* button transitions to DT create stage

3.5 Digital Twins

3.5.1 Create a Digital Twin

The first step in digital twin creation involves utilizing the available assets within the workspace. For assets or files residing on a local computer that need to be accessible in the DTaaS workspace, the instructions provided in library assets should be followed.

Dependencies exist among the library assets. These dependencies are illustrated below.



A digital twin can only be created by linking assets in a meaningful way. This relationship can be expressed using the following mathematical equation:

where D denotes data, M denotes models, F denotes functions, T denotes tools, denotes DT configuration, and is a symbolic notation for a digital twin itself. The expression denotes composition of a DT from D, M, T, and F assets. The indicates zero or more instances of an asset, and indicates one or more instances of an asset.

The DT configuration specifies the relevant assets to use and the potential parameters to be set for these assets. When a DT requires RabbitMQ, InfluxDB, or similar services supported by the platform, the DT configuration must include access credentials for these services.

This generic DT definition is based on DT examples observed in practice. Deviation from this definition is permissible. The only requirement is the ability to execute the DT from either the command line or a graphical desktop environment.

> **Tip**
>
> For users new to Digital Twins who may not have distinct digital twin assets but rather a single directory containing all components, it is recommended to upload this monolithic digital twin into the **digital_twin/your_digital_twin_name** directory.

Example

The Examples repository contains a co-simulation setup for a mass-spring-damper system. This example demonstrates the application of co-simulation techniques for digital twins.

The file system contents for this example are:

```
workspace/
  data/
    mass-spring-damper
        input/
        output/

  digital_twins/
    mass-spring-damper/
      cosim.json
      time.json
      lifecycle/
        analyze
        clean
        evolve
        execute
        save
        terminate
      README.md

  functions/
  models/
    MassSpringDamper1.fmu
    MassSpringDamper2.fmu

  tools/
  common/
    data/
    functions/
    models/
    tools/
        maestro-2.3.0-jar-with-dependencies.jar
```

The `workspace/data/mass-spring-damper/` directory contains `input` and `output` data for the mass-spring-damper digital twin.

The two FMU models required for this digital twin are located in the `models/` directory.

The co-simulation digital twin requires the Maestro co-simulation orchestrator. As this is a reusable asset for all co-simulation-based DTs, the tool has been placed in the `common/tools/` directory.

The digital twin configuration is specified in the `digital twins/mass-spring-damper` directory. The co-simulation configuration is defined in two JSON files: `cosim.json` and `time.json`. Documentation for the digital twin can be placed in `digital twins/mass-spring-damper/document.md`.

The launch program for this digital twin is located in `digital twins/mass-spring-damper/lifecycle/execute`. This launch program executes the co-simulation digital twin, which runs until completion and then terminates. The programs in `digital twins/mass-spring-damper/lifecycle` are responsible for lifecycle management of this digital twin. The lifecycle page provides further explanation of these programs.

Execution of a Digital Twin

A frequent question arises on the run time characteristics of a digital twin. The natural intuition is to say that a digital twin must operate as long as its physical twin is in operation. **If a digital twin runs for a finite time and then ends, can it be called a digital twin? The answer is a resounding YES**. The Industry 4.0 usecases seen among SMEs have digital twins that run for a finite time. These digital twins are often run at the discretion of the user.

Execution of this digital twin involves the following steps:

1. Navigate to the Workbench tools page of the DTaaS website and open VNC Desktop. This opens a new tab in the browser.
2. A page with VNC Desktop and a connect button is displayed. Click on Connect to establish a connection to the Linux Desktop of the workspace.
3. Open a Terminal (the black rectangular icon in the top left region of the tab) and enter the following commands.
4. Download the example files by following the instructions provided in the examples overview.
5. Navigate to the digital twin directory and execute:

```
cd /workspace/examples/digital_twins/mass-spring-damper
lifecycle/execute
```

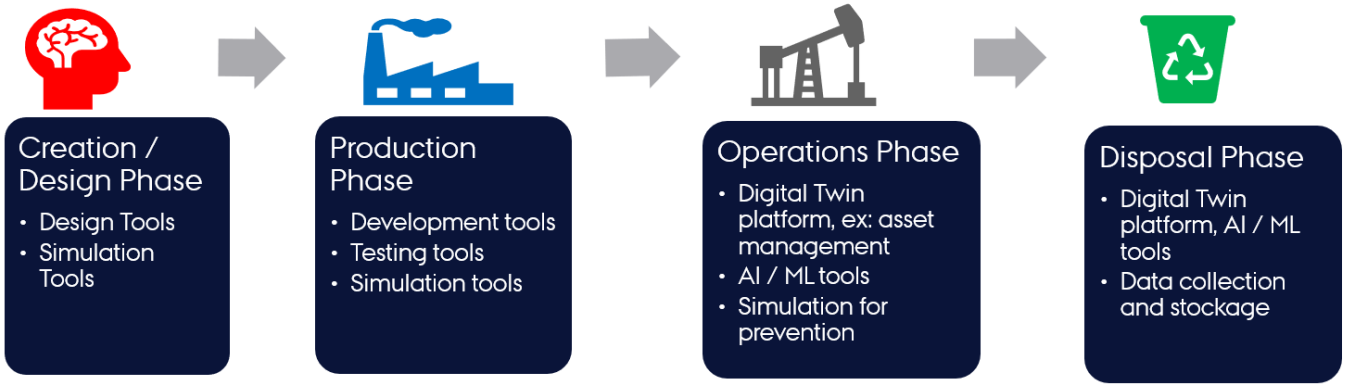
The final command executes the mass-spring-damper digital twin and stores the co-simulation output in `data/mass-spring-damper/output`.

3.5.2 ♻ Digital Twin Lifecycle

Physical products in the real world undergo a product lifecycle. A simplified four-stage product lifecycle is illustrated here.

A digital twin tracking physical products (twins) must evolve in conjunction with the corresponding physical twin.

The possible activities undertaken in each lifecycle phase are illustrated in the figure.



(Ref: Minerva, R, Lee, GM and Crespi, N (2020) Digital Twin in the IoT context: a survey on technical features, scenarios and architectural models. Proceedings of the IEEE, 108 (10). pp. 1785-1824. ISSN 0018-9219.)

Lifecycle Phases

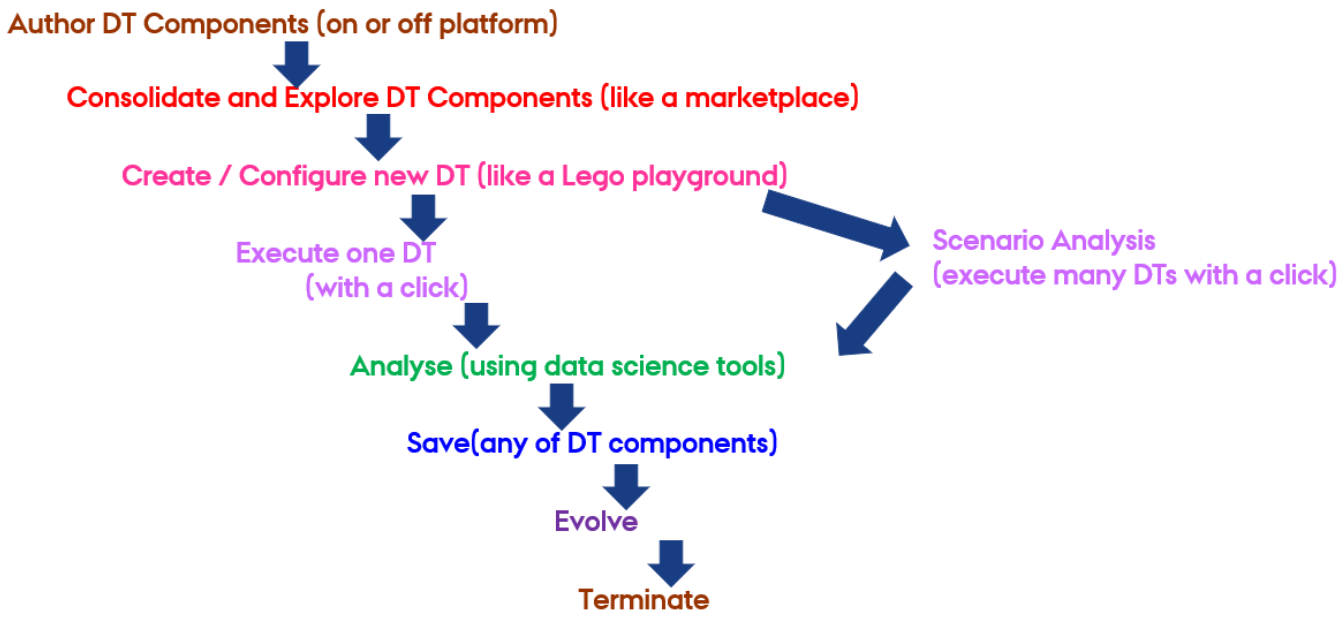
The four-phase lifecycle has been extended to a lifecycle with eight phases. The new phase names and the typical activities undertaken in each phase are outlined in this section[1].

A DT lifecycle consists of **explore, create, execute, save, analyse, evolve** and **terminate** phases.

Phase	Main Activities
explore	Selection of suitable assets based on user requirements and verification of their compatibility for DT creation.
create	Specification of DT configuration. For existing DTs, no creation phase is required at the time of reuse.
execute	Automated or manual execution of a DT based on its configuration. The DT configuration must be verified before starting execution.
analyse	Examination of DT outputs and decision-making. Outputs may include text files or visual dashboards.
evolve	Reconfiguration of DT primarily based on analysis results.
save	Preservation of DT state to enable future recovery.
terminate	Cessation of DT execution.

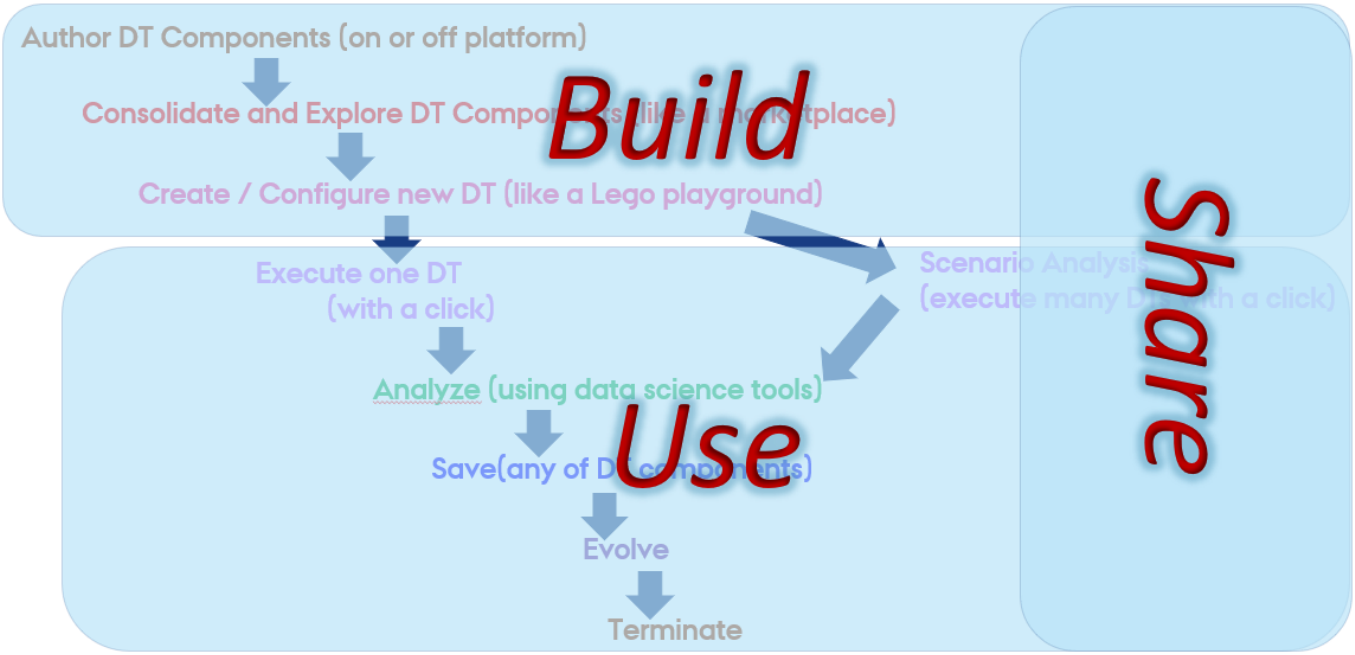
A digital twin faithfully tracking the physical twin lifecycle must support all the phases. Digital twin engineers may also add additional phases to their implementations. Consequently, the DTaaS platform is designed to accommodate the needs of diverse DTs.

A potential linear representation of the tasks undertaken in a digital twin lifecycle is shown here.



This representation shows only one possible pathway. The sequence of steps may be altered as needed.

It is possible to map the lifecycle phases to the **Build-Use-Share** approach of the DTaaS platform.



Although not mandatory, maintaining a matching code structure facilitates DT creation and management within the DTaaS platform. The following structure is recommended:

```
workspace/
  digital_twins/
    digital-twin-1/
      lifecycle/
        analyze
        clean
        evolve
        execute
        save
        terminate
```

A dedicated program exists for each phase of the DT lifecycle. Each program can be as simple as a script that launches other programs or sends messages to a live digital twin.

The recommended approach for implementing lifecycle phases within DTaaS is to create scripts. These scripts can be implemented as shell scripts.

Example Lifecycle Scripts

The following example programs/scripts demonstrate management of three phases in the lifecycle of the **mass-spring-damper DT**.

lifecycle/execute

```
#!/bin/bash
mkdir -p /workspace/data/mass-spring-damper/output
#cd ..
java -jar /workspace/common/tools/maestro-2.3.0-jar-with-dependencies.jar \
    import -output /workspace/data/mass-spring-damper/output \
    --dump-intermediate sg1 cosim.json time.json -i -vi FMI2 \
    output-dir>debug.log 2>&1
```

The execute phase utilizes the DT configuration, FMU models, and Maestro tool to execute the digital twin. The script also stores the output of co-simulation in `/workspace/data/mass-spring-damper/output`.

A DT may not support a specific lifecycle phase. This intention can be expressed with an empty script and a helpful message if deemed necessary.

lifecycle/analyze

```
#!/bin/bash
printf "operation is not supported on this digital twin"
```

The lifecycle programs can invoke other programs in the codebase. In the case of the `lifecycle/terminate` program, it calls another script to perform the necessary operations.

lifecycle/terminate

```
#!/bin/bash
lifecycle/clean
```

References

[1]: Talasila, Prasad, et al. "Composable digital twins on Digital Twin as a Service platform." Simulation 101.3 (2025): 287-311.

3.5.3 DevOps

Digital Twin File Structure in GitLab

GitLab is used as a file store for performing DevOps on Digital Twins. The user interface page is a front-end for this gitlab-backed file storage.

Each DTaaS installation comes with an integrated GitLab. There must be a GitLab group named **dtaas** and a GitLab repository for each user where repository name matches the username. For example, if there are two users, namely *user1* and *user2* on a DTaaS installation, then the following repositories must exist on the linked GitLab installation.

```
https://intocps.org/gitlab/dtaas/common.git
https://intocps.org/gitlab/dtaas/user1.git
https://intocps.org/gitlab/dtaas/user2.git
```

> **Warning**
>
> The assets being displayed on the Library preview page come from the `master` branch of the backing GitLab project. Please create a branch named `master` and make it the default branch. This must be done for all the user repositories including the common repository.

Each user repository must also have a specific structure. The required structure is as follows.

```
<username>/
 ├─── common/
 ├─── data/
 ├─── digital_twins/
 ├─── functions/
 ├─── models/
 ├─── tools/
 ├─── .gitlab-ci.yml
 └─── README.md
```

This file structure follows the same pattern user sees on the existing **Library** page.

DIGITAL TWIN STRUCTURE

The `digital_twins` folder contains DTs that have been pre-built by one or more users. The intention is that they should be sufficiently flexible to be reconfigured as required for specific use cases.

Let us look at an example of such a configuration. The dtaas/user1 repository on gitlab.com contains the `digital_twins` directory with a `hello_world` example. Its file structure looks like this:

```
hello_world/
 ├─── lifecycle/ (at least one lifecycle script)
 |    ├─── clean
 |    ├─── create
 |    ├─── execute
 |    └─── terminate
 ├─── .gitlab-ci.yml (GitLab DevOps config for executing lifecycle scripts)
 └─── description.md (optional but is recommended)
 └─── README.md (optional but is recommended)
```

The `lifecycle` directory here contains four files - `clean`, `create`, `execute` and `terminate`, which are simple BASH scripts. These correspond to stages in a digital twin's lifecycle. Further explanation of digital twin is available on lifecycle stages.

Digital Twins and DevOps

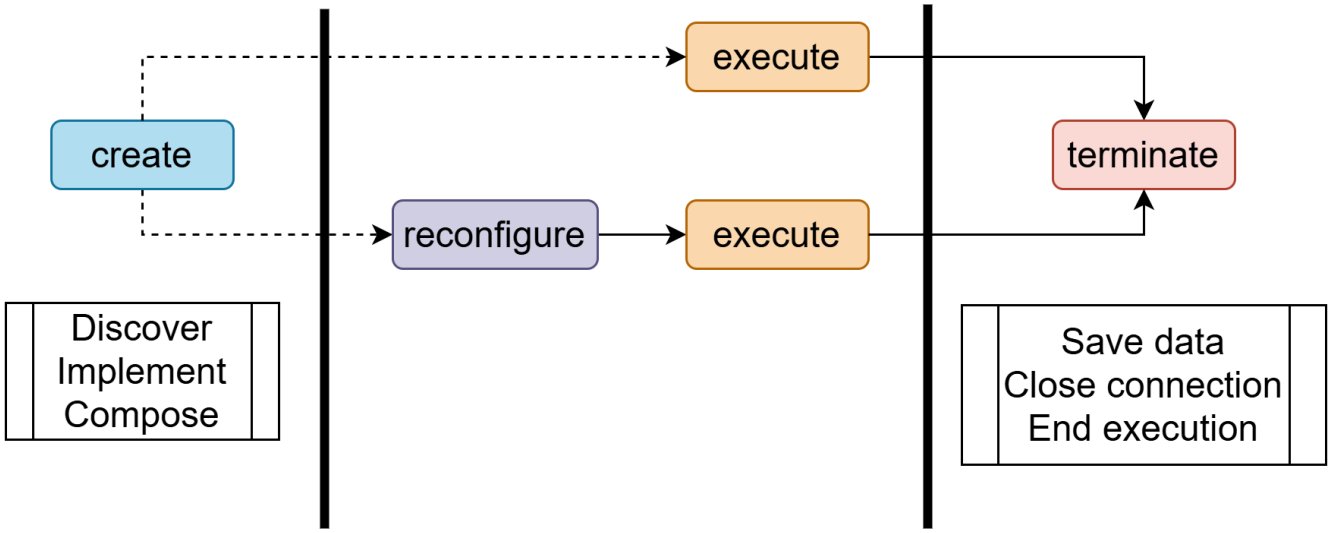
DevOps has been a well established software development practice. An experimental feature integrating DevOps is being introduced in the DTaaS.

This feature requires specific installation setup.

1. Integrated GitLab installation
2. A valid GitLab repository for the logged in user. See an example repository. This repository can be cloned and customised as needed.
3. A linked GitLab Runner to the user gitlab repository.

DT LIFECYCLE

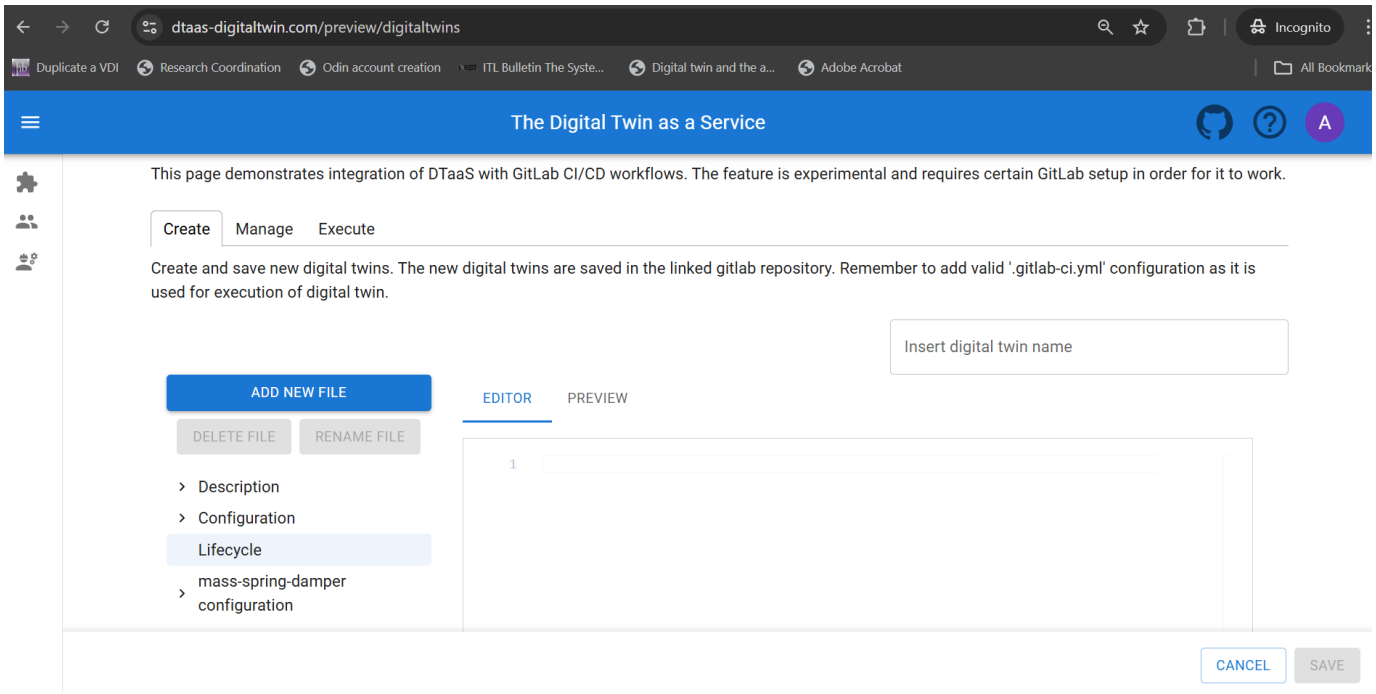
The DT preview implements the **Create**, **Manage** and **Execute** stages of a DT lifecycle. The suggested sequence of use for different lifecycle stages are:



There are dedicated tabs for **Create**, **Manage** and **Execute** stages. The selection of DT assets for Create stage happens via Library preview page. The Manage tab fulfills reconfigure feature. The Execute and Terminate are managed on the Execute tab.

CREATE TAB

The users select reusable DT assets and arrive on the Create tab. The following figure shows DT creation page after selecting *mass-spring-damper* as a reusable asset for the new DT.



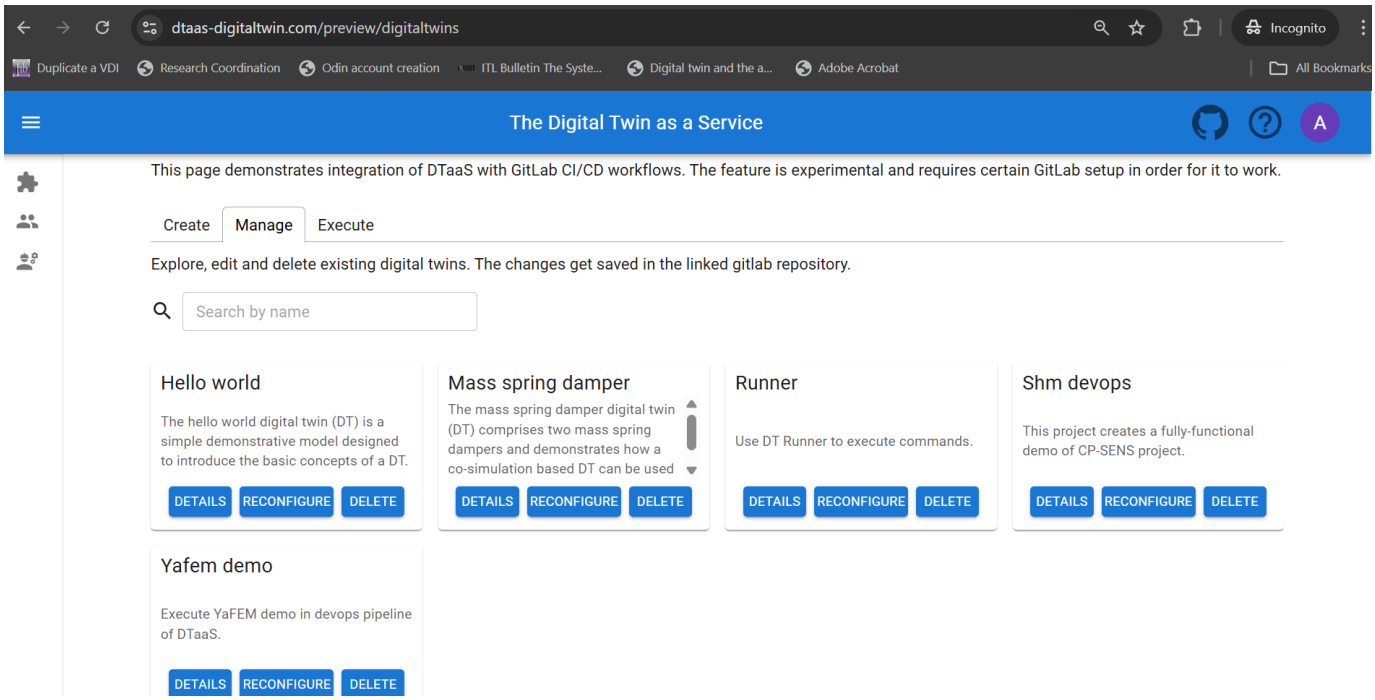
The left-side menu shows the possibility of creating necessary structure and elements for a new DT. Each reusable asset selected for this new DT appears on the left menu. Its configuration can be updated as well.

These files on the left menu correspond to three categories.

- **Description**: contains *README.md* providing comprehensive description of DT and *description.md* providing a brief description. The brief description is shown in the DT tabs and clicking on the *Details* button shows the complete README.md The *Details* button is available only on the Manage page.
- **Configuration**: Contains a *.gitlab-ci.yaml* for running the required lifecycle scripts and operations of the DT. Additional json and yaml files can be added to create configuration for a new DT.
- **Lifecycle**: These are the DT lifecycle scripts.

The *Add New File* button can be used to add new files in all the three categories. Finally, click on *SAVE* button to save the new DT. Newly created DTs become immediately available on the **Manage** and **Execute** tabs.

MANAGE TAB

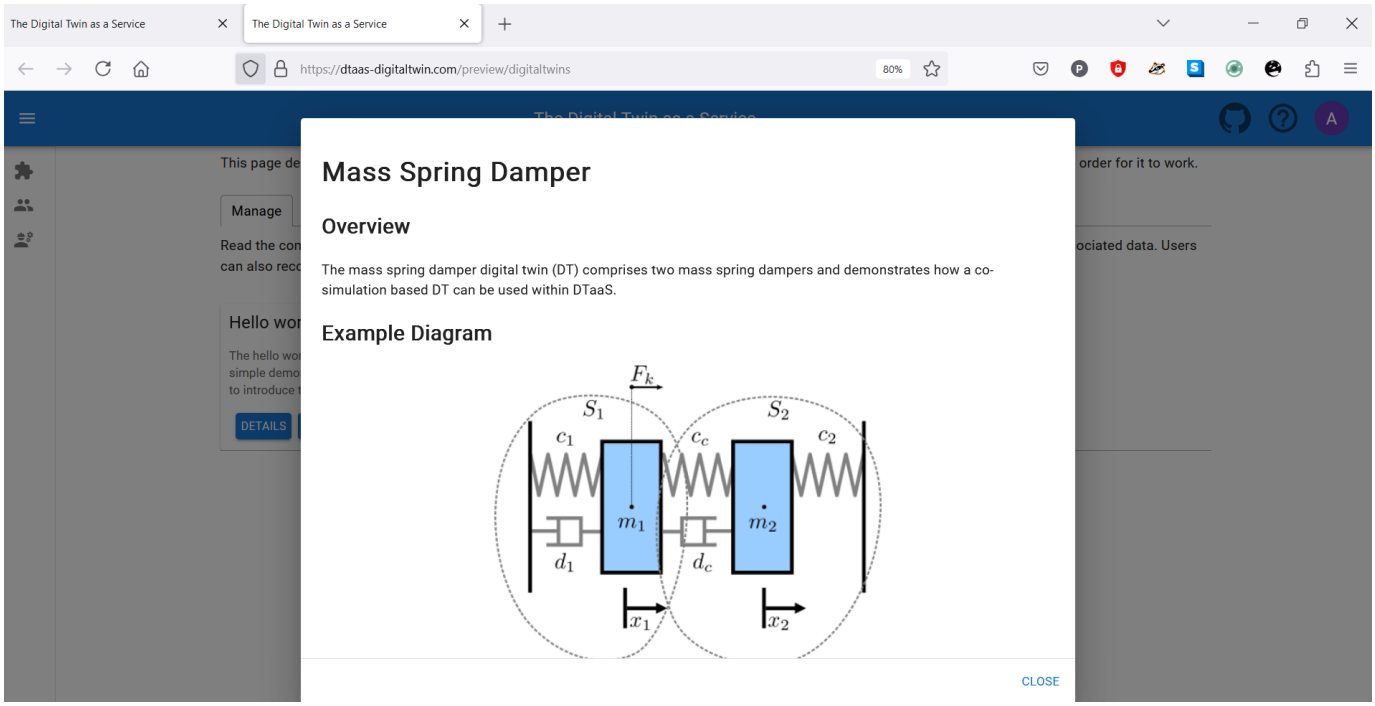


The manage tab allows for different operations on a digital twin:

- Checking the details (**Details** button)
- Delete (**Delete** button)
- Modify / Reconfigure (**Reconfigure** button)

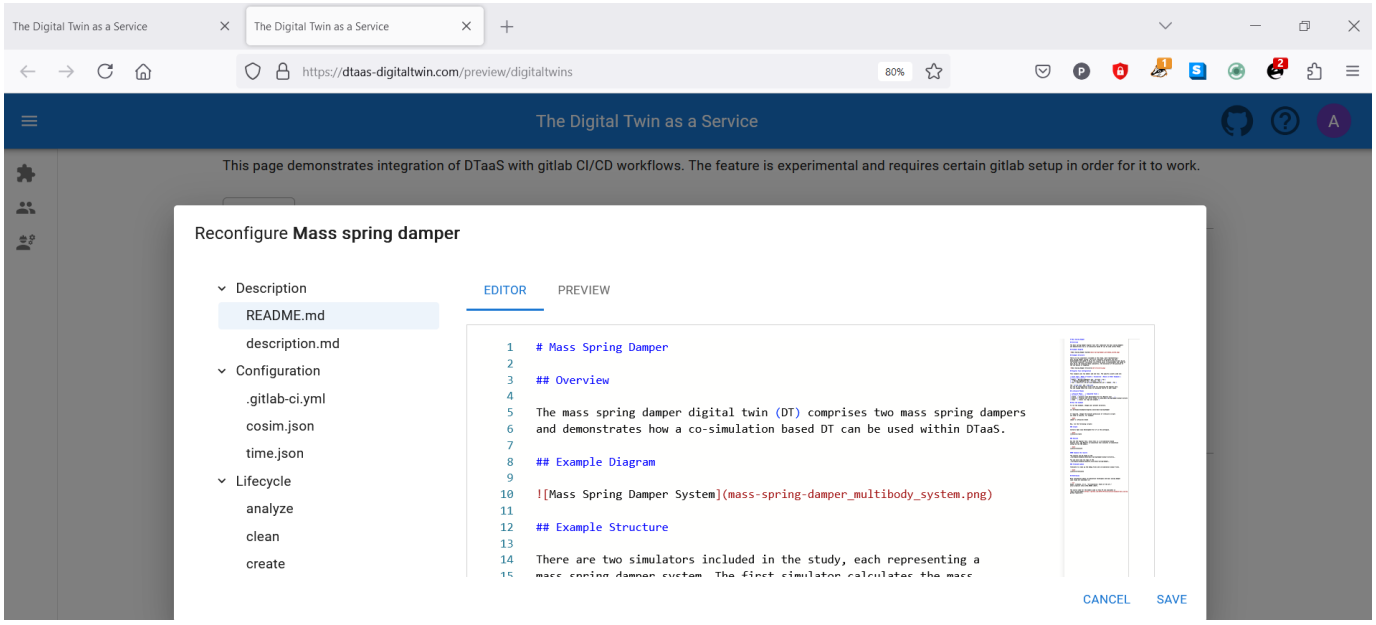
A digital twin placed in the DTaaS has a certain recommended structure. See the assets page for an explanation and this example.

The information page shown using the Details button, shows the README.md information stored inside the digital twin directory.

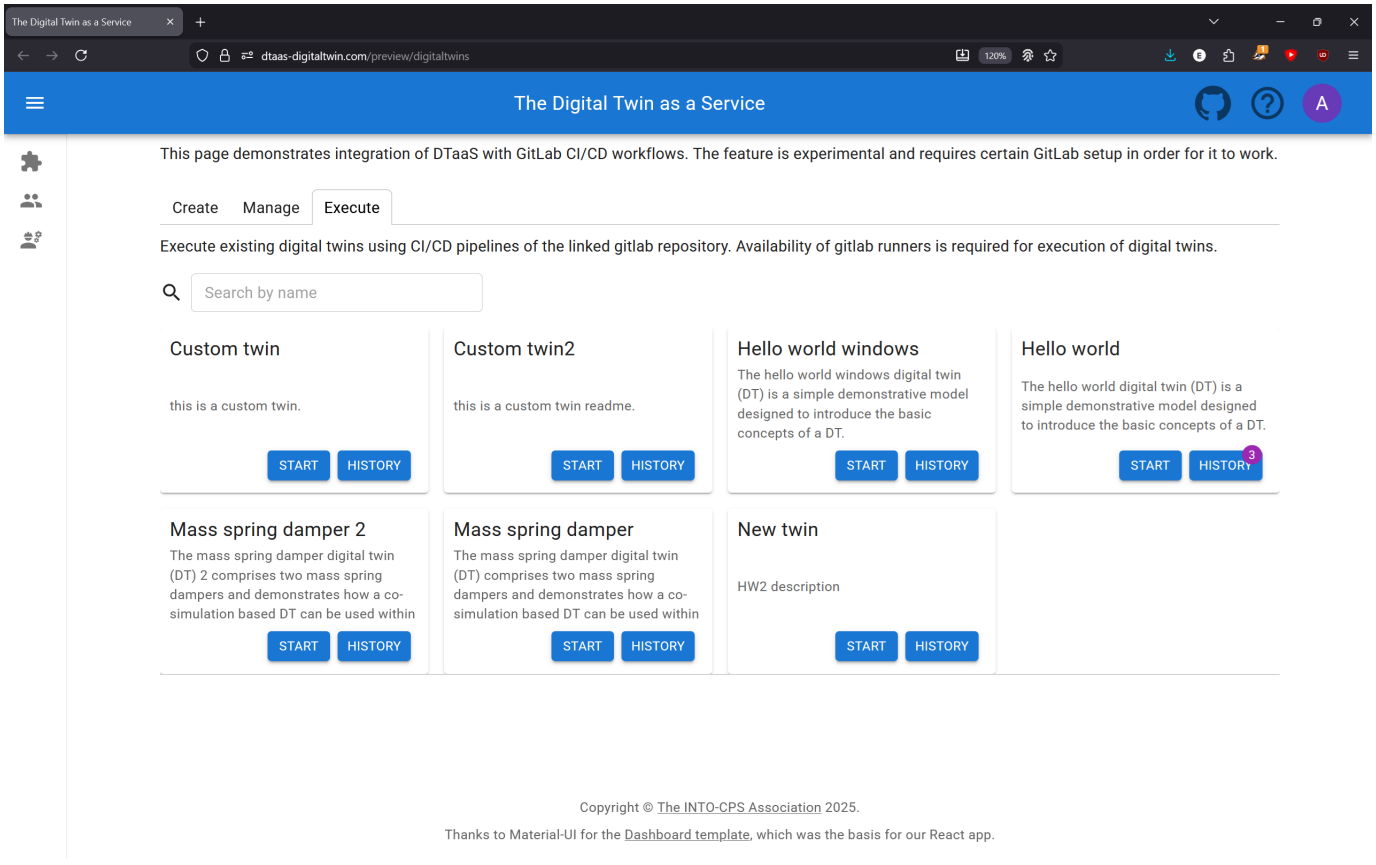


A reconfigure button opens an editor and shows all the files corresponding to a digital twin. All of these files can be updated. These files correspond to three categories.

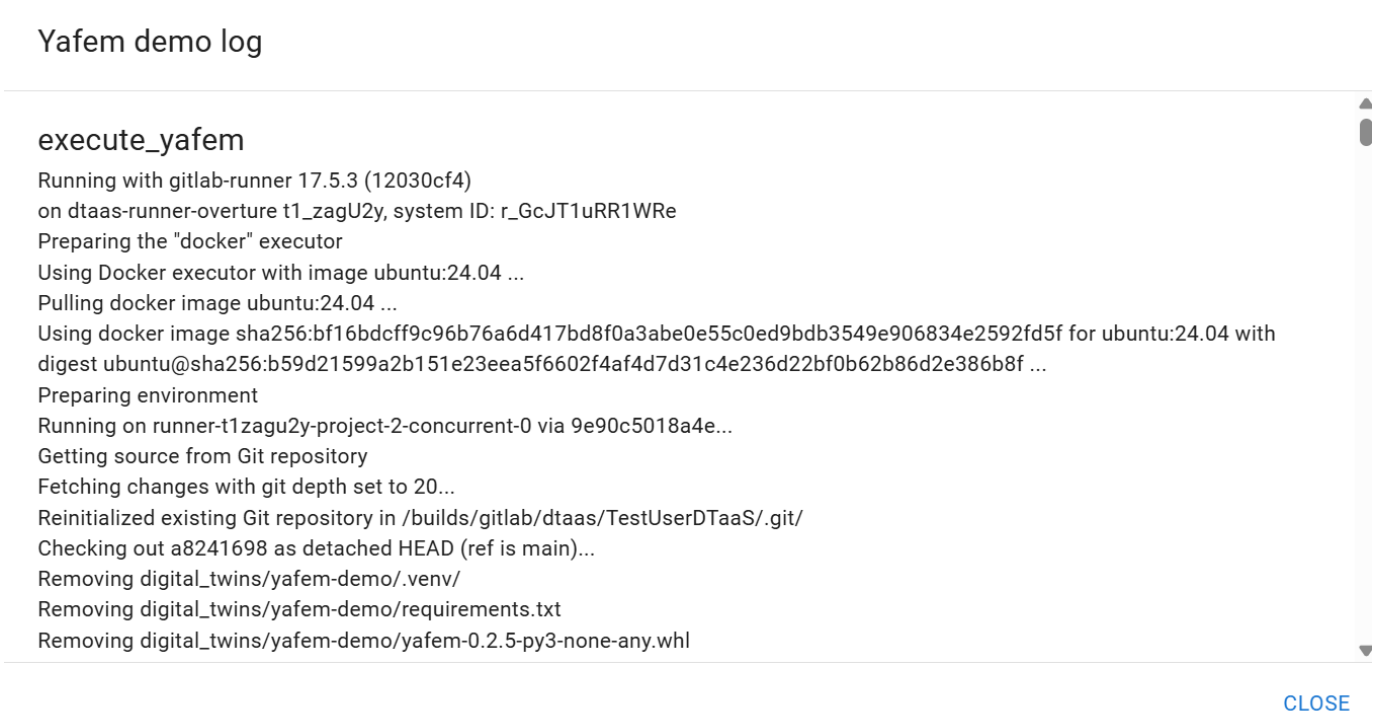
- **Description**
- **Configuration**
- **Lifecycle**



EXECUTE TAB



The execute tabs shows the possibility of executing multiple digital twins. Once an execution of digital twin is complete, the execution log is also available.



Setting Allowed Values

Some Setting values will cause problems if used. This document provides an in-depth examination of the values that are allowed and values that are not. Expected errors and troubleshooting steps for this page are also covered.

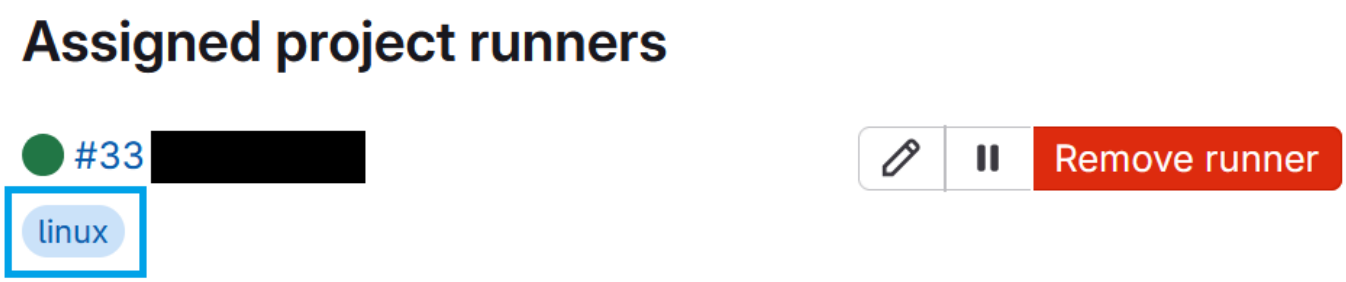
To understand what each parameter does, refer to the settings document.

Note: The user must have the appropriate access rights to all the resources connected to the application.

RUNNER TAG

- Allowed values

This parameter has to be a text string like "linux", "ubuntu", "2" and match some Runner's tags on the execution service:



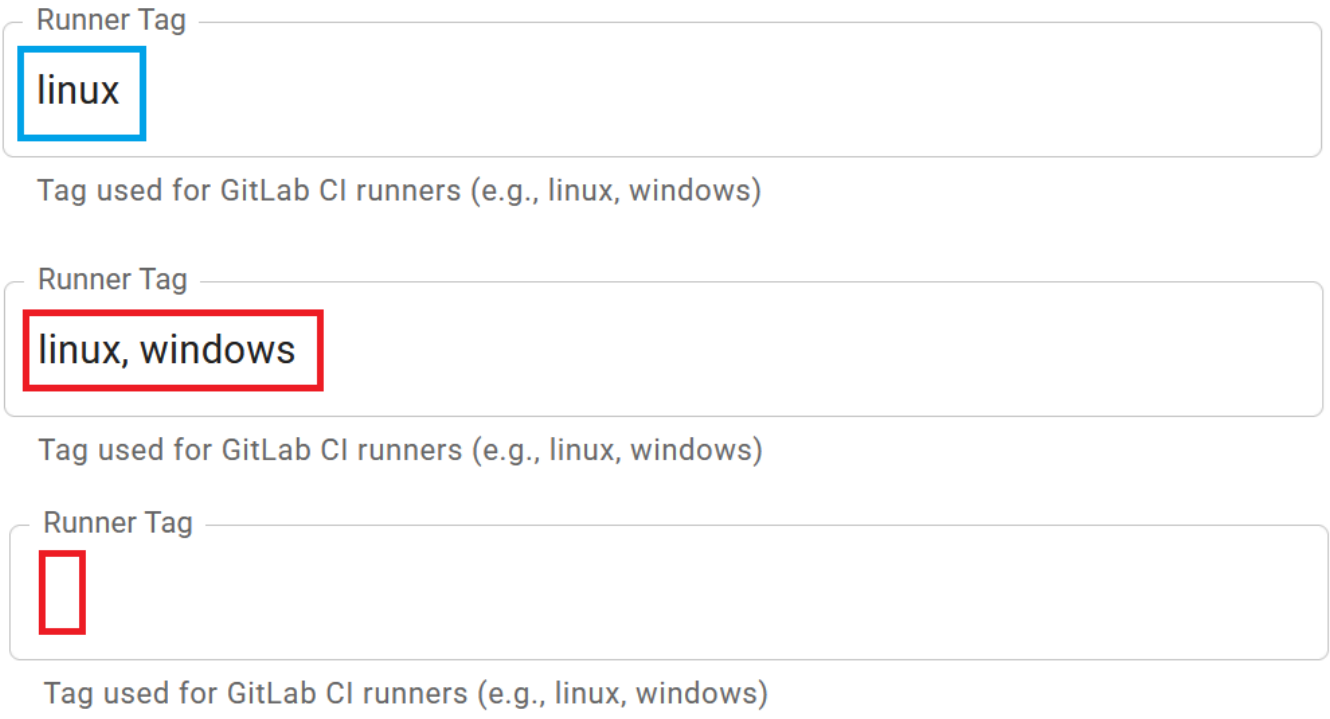
Multiple values, like "linux, windows" is not supported and will be treated as one tag.

- Not allowed values

If the **Runner Tag** field *does not* match a Runner, no jobs will be picked up and the job will hence time out. If the twins unexpectedly time out, verify that the tag has been spelt correctly.

No tag (i.e. a blank field) is permitted by some services like GitLab, meaning that *any* Runner that is configured to pick up tag-less jobs can pick these jobs up. A limitation of DTaaS is that *some* tag is required. No tags, like other unallowed tags, will be ignored.

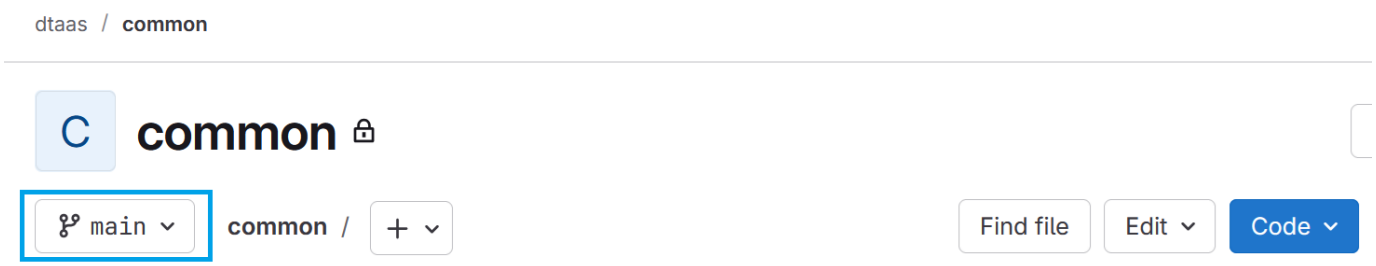
- Visual examples



BRANCH

- Allowed values

The **Branch** field must be a text string matching a branch in the user *and* common library repositories. For example: "main" or "master":



- Not allowed values

This field should not be left empty or set to a value not matching a branch in both repositories. While technically allowed, this may cause errors or unexpected behaviours.

Expected error:

An error occurred while fetching assets: GitbeakerRequestError: 404 Tree Not Found

- Visual examples

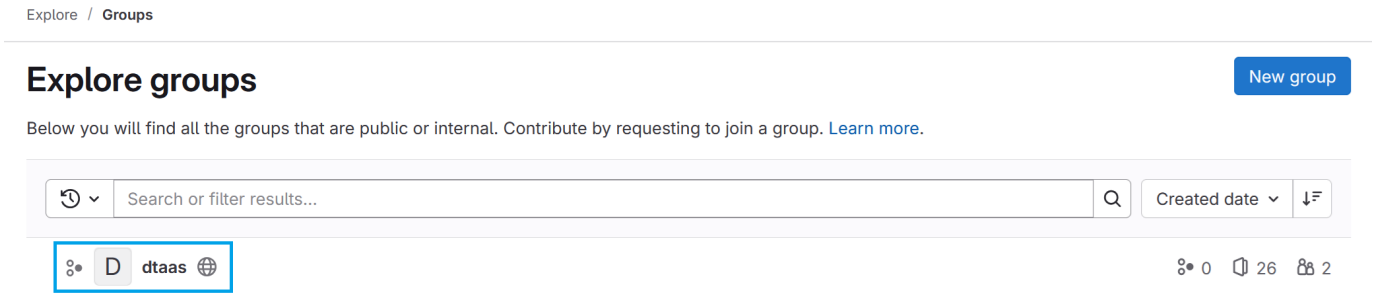
GROUP NAME

- Allowed values

The **Group Name** field again requires a text string, and it must further match some group in the storage service (e.g. GitLab).

Example:

DTaaS



Group names are case insensitive on GitLab, but matching the case exactly is recommended for consistency and to remove it as a source of error.

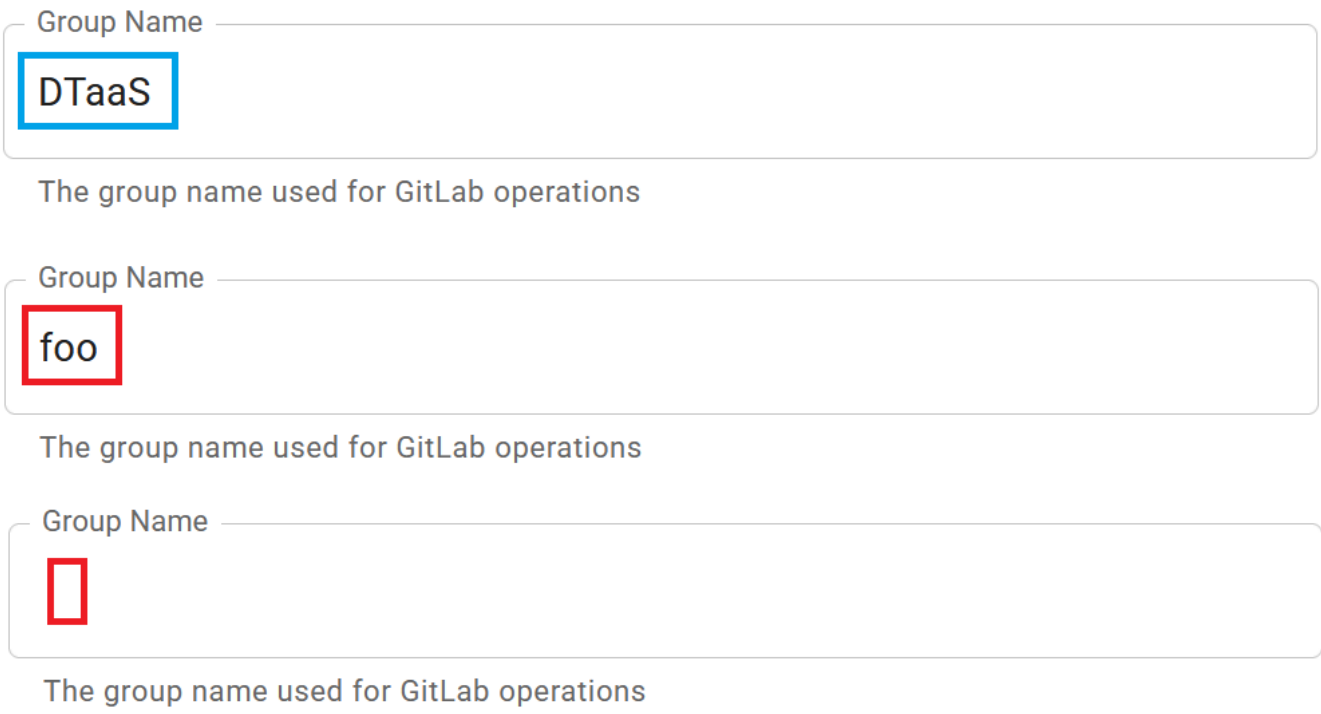
- Not allowed values

If the group does not exist or the field is left blank, errors will occur upon accessing Digital Twins, etc. Ensure that a value is entered.

Expected error:

An error occurred while fetching assets: GitbeakerRequestError: 404 Group Not Found

- Visual examples



COMMON LIBRARY PROJECT NAME

- Allowed values The **Common Library Project name** parameter is a text string. It must correlate to the repository responsible for keeping shared resources. Examples: "common" and "library":

Failure to match a repository will break the DevOps page.

Expected error:

An error occurred while fetching assets: Error: Common project not found

- Not allowed values

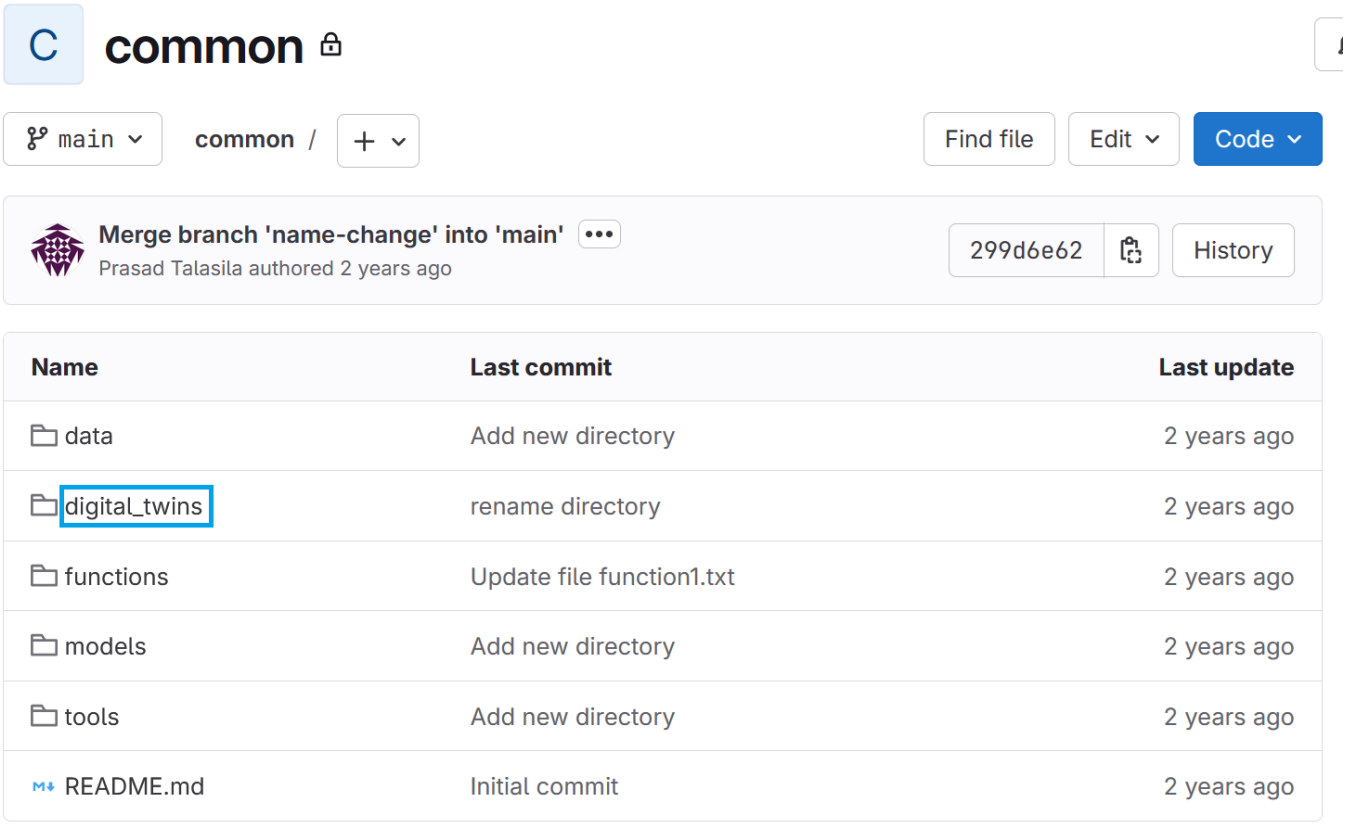
This field should not be left blank or set to a value not matching a repository. It is inadvisable for it to match a user repository.

- Visual examples

DT DIRECTORY

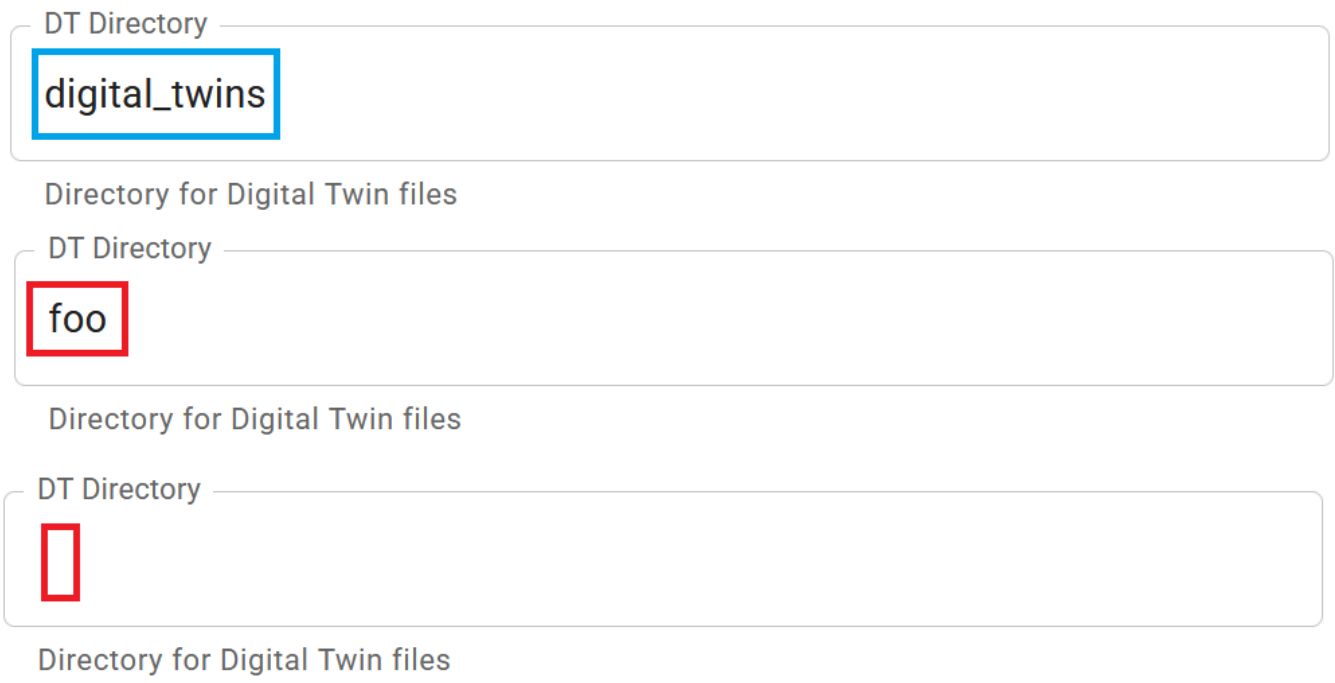
- Allowed values

Like the other fields, this must be a text string. It should match the name of the folder within the common library and most importantly the user repository where the Digital Twins are stored.



• Not allowed values

Do not leave this blank or erroneously mapped. This will lead to silent failure with the Digital Twins not loading or Digital Twins from a previous set up to show up.



TROUBLESHOOTING

The Digital Twins are not showing up:

Verify that Group, Branch, common and DT directory all are configured exactly. like it is on the backend.

The Digital Twins are timing out:

Verify that the Runner Tag parameter is not a list of tags, that the runner is active and matches its tag.

SUMMARY

The possible values for correctly connecting DTaaS to the relevant storage and service instances have been specified, along with potential pitfalls. The key point is not to leave any of the fields blank; they should be of the text string format and properly correspond to the relevant services and runners. Some troubleshooting steps have further been provided to overcome common hurdles.

Capabilities

In addition to creating and editing Digital Twins, it is also possible to run them and adjust their execution parameters. The DTaaS permits running multiple Digital Twins simultaneously and even changing settings while they are running, without the need to manually fetch the data: it will load upon returning to whichever branch and group was used for execution. The settings include both repository and execution options, while all twins can readily be run and checked from the Execution tab under the Preview page.

Read more about Settings, Setting Values and Concurrent Execution on their respective pages.

The following sections describe the specific capabilities of each of these features.

⚙ SETTINGS

The table below describes the different test setups (valid, invalid, etc.), the resultant behaviour and what was deemed to be expected behaviour across the features. Following every table, there is a summary and list of potential problems.

Setting	Expected behaviour	Observed behaviour	Test method
Runner tag **valid**	Job is picked up by runner with relevant tag and completes without error.	✅ Same as expected.	Goto Preview page. Go to Account. Change Runner Tag. Go back one page. Execute Hello world twin. Verify runner name based on tag (repeat with 2nd runner)
Runner tag **invalid**	Job is never picked up. Runner times out after 10 minutes.	✅ Same as expected.	Change Runner Tag to "foo" (inexistent). Run Hello World twin.
Runner Tag **no value**	Configuration saves. Running a twin succeeds if appropriate runner exists.	❌ Not picked up, times out.	Set `run_untagged = false` in local gitlab runner config. Mark "Run untagged jobs" as true in gitlab instance (https://dtaas-digitaltwin.com/gitlab). Change Runner Tag to the empty string on application. Run Hello world twin.
Branch **valid**	Job runs with the correct branch and completes without error.	✅ Same as expected.	Make new branch in GitLab instance project. Change Branch name to "master-2". Execute Hello world twin. Verify ref matches branch in execution log.
Branch **invalid**	Execution tab gracefully displays no twins as branch does not exist.	❌ IF twins are not cached: Throws an error displayed to user: An error occurred while fetching assets: GitbeakerRequestError. IF twins are cached: Job fails. Snackbar says "Execution error for [twin name]". No log available in execution history.	Change Branch name to "master-1" (inexistent). IF twins are cached: Execute Hello world twin.
Group name **valid**	The twin is runnable.	✅ Same as expected.	Click "reset to defaults". Run twin.
Group name **invalid**	Execution tab gracefully displays no twins as group does not exist.	❌ Same as Branch invalid case.	Change group name to "Foo" (inexistent). IF twins are cached: Execute Hello world twin.
Common name **valid**	Library twins are visible.	✅ Same as expected. *Private twins also show up in Common twins.	Click "reset to defaults". Goto library. Goto common twins. Inspect twin visibility.
Common name **invalid**	Library twins gracefully not visible.	❌ Displays error: An error occurred while fetching assets: Error: Common project foo not found	Change Common Library Project name to "Foo" (inexistent). Goto common twins. Inspect visibility.
DT directory **valid**	Twins are visible.	✅ Same as expected.	Click "reset to defaults". Goto library. Goto Execution tab. Inspect twin visibility.
DT directory name **invalid**	Twins gracefully not visible.	❌ Depends on the cache. Displays error if there is none: An error occurred while fetching assets: GitbeakerRequestError	Change Common DT directory name to "Foo" (inexistent). Goto Execution tab. Inspect twin visibility.
Group Name, DT Directory, Common Library Project	Invalid value caught early. Cannot save, appropriate	❌ Same as Branch invalid case.	Change Group Name, DT Directory, Common Library Project Name, Branch name to the empty string.

Setting	Expected behaviour	Observed behaviour	Test method
Name, Branch name **no value**	feedback of invalid form fill out.		

Summary: Changing to valid settings works. Invalid settings in the best case display an error in the HTML (no cache) and otherwise shows stale twins.

Problems:

- Stale state maintained after settings are updated unless page is refreshed.
- Displaying technical errors to the user when the settings are invalid.
- Permitting invalid values such as empty strings when they are required
- No tags runners may not work, but possibly local problem.
- Private twins show up as common twins

⏳ CONCURRENT EXECUTION

Expected behaviour	Observed behaviour	Test method
Both twins run successfully simultaneously.	✅ Same as expected.	Run the same twin twice at the same time.
All twins run successfully simultaneously.	✅ Same as expected.	Run different twins at the same time.

Concurrent Execution Across Different Runners

Expected behaviour	Observed behaviour	Test method
Both twins run successfully simultaneously. They report the correct runner name in the Execution logs.	✅ Same as expected.	Set up 2 ubuntu GitLab runners with different tags. Change Runner Tag to first runner's tag. Execute twin. Change Runner Tag to second runner tag. Execute another twin.
History stays after editing, it is still executing.	✅ Same as expected.	Execute → Edit settings → Execute. Check – is history still there? Does it look correct?

Summary:

All functionality works according to the tests. The logs are still there after changing settings and running any combination of twins and runners work (Only Ubuntu runners tested).

🧩 IMPLEMENTING BACKENDS

The DTaaS is by default set up to work with GitLab as execution and storage backend, but other combinations may be more suitable. As such, the code base is designed with this flexibility in mind, so reimplementation from scratch is not required when a new backend is needed. In future versions of the DTaaS, more backends may be available by default, such as GitHub and Azure.

💭 SUMMARY

Digital Twins can be queued as needed and live settings changes will not interfere with data retention, while enabling the testing of multiple setups simultaneously. For greater flexibility, those with the requisite technical expertise can readily expand upon the available backends.

Running Multiple Digital Twins at the Same Time

The DTaaS platform allows for executing multiple Digital Twins in tandem with one another. This can save hours of time when dealing with intensive Digital Twins or when deploying them across different resources.

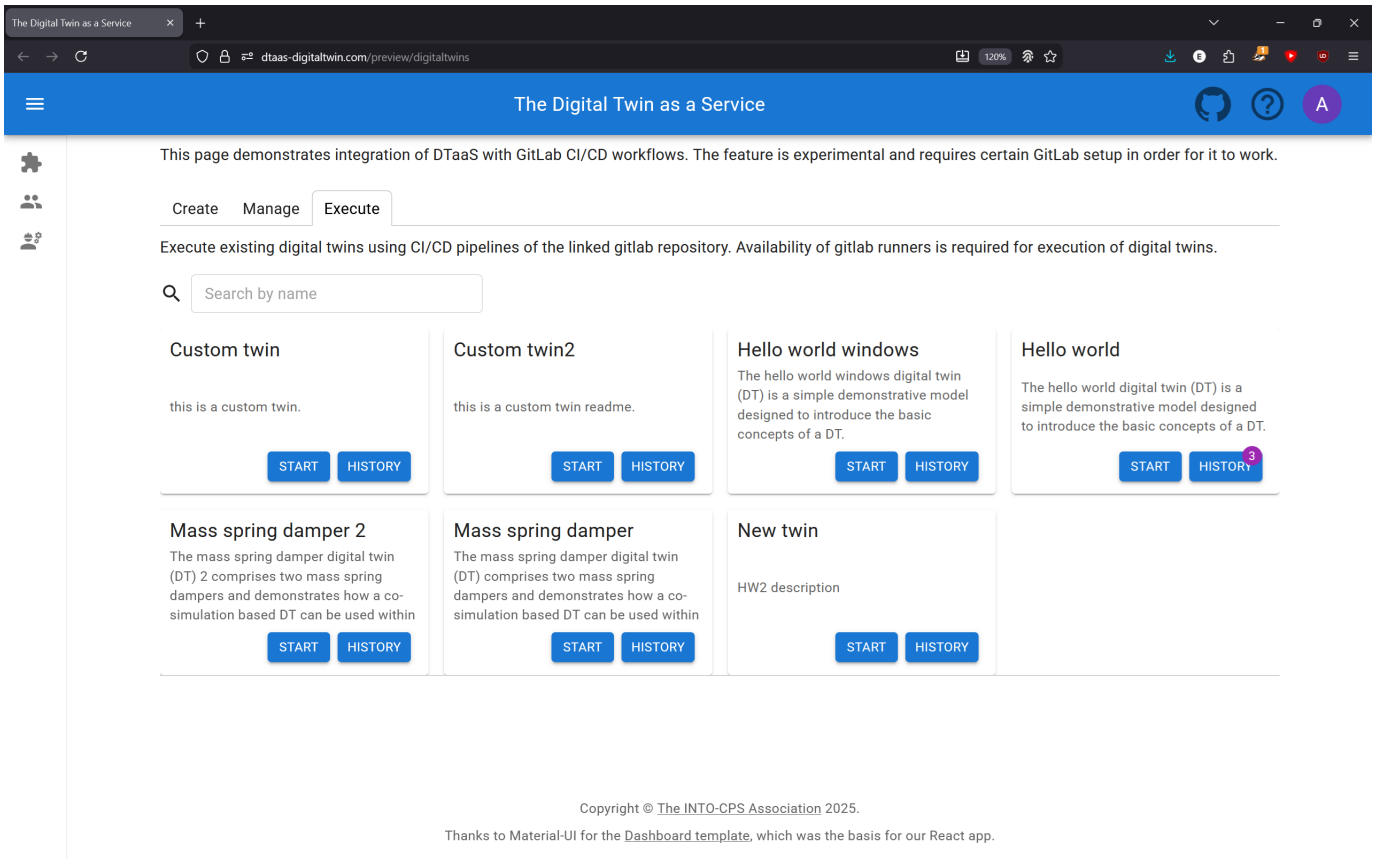
⏯️ RUNNING DIGITAL TWINS

To deploy a Digital Twin, navigate to the *Execution tab* on the *Digital Twin Preview page*. This can be found by clicking on the Workbench icon  in the sidebar on the left hand side and locating *Digital Twins Preview* among the other links. Click to open the *Digital Twin Preview page* in a new tab.

In the newly opened tab you will see a series of cards with a name, short description of the Digital Twins and **START** and **HISTORY** buttons. The **START** button sends a *job* to the backend, telling it to run the simulation associated with that Digital Twin, which is defined by its Life Cycle files. These can be inspected and changed in the *Edit* tab.

Running Multiple Twins

Pressing the **START** button multiple times queues multiple simulations. It is also possible to queue different kinds of Digital Twins.



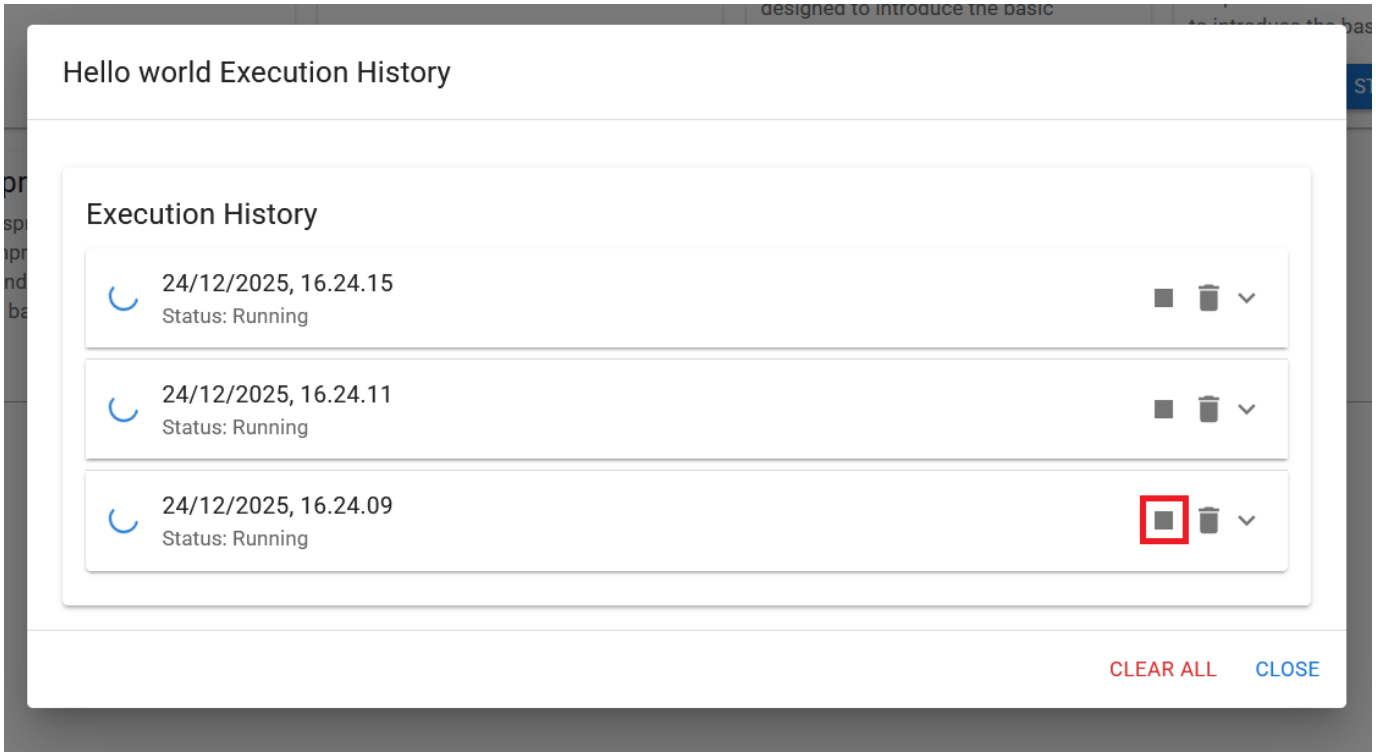
The deployment service (e.g. GitLab) automatically load balances across available runners, so it can be configured once and left to run autonomously.

🗃️ THE EXECUTION LOG

Pushing the **HISTORY** button provides an overview of all current and past simulation trials, distinguished by time of execution and status (Running, Failed, Succeeded or Timed Out). To review the results of a simulation, click on an entry (or the arrow on the right of the entry) to expand it. Each step in the Life Cycle will then be presented.

To delete an entry, click the trash icon. A verification dialogue will confirm the entry to be deleted. All entries of a Digital Twin can also be deleted with the **CLEAR ALL** button.

Click the stop symbol in the log to stop an execution:



🔧 CHANGING RUNNERS

The runners that pick up jobs can be changed in the settings. Further information on these and other settings is available.

💭 SUMMARY

This document has described how to navigate the DevOps Execution page: executing multiple Digital Twins simultaneously and inspecting and deleting their logs.

3.6 Working with GitLab

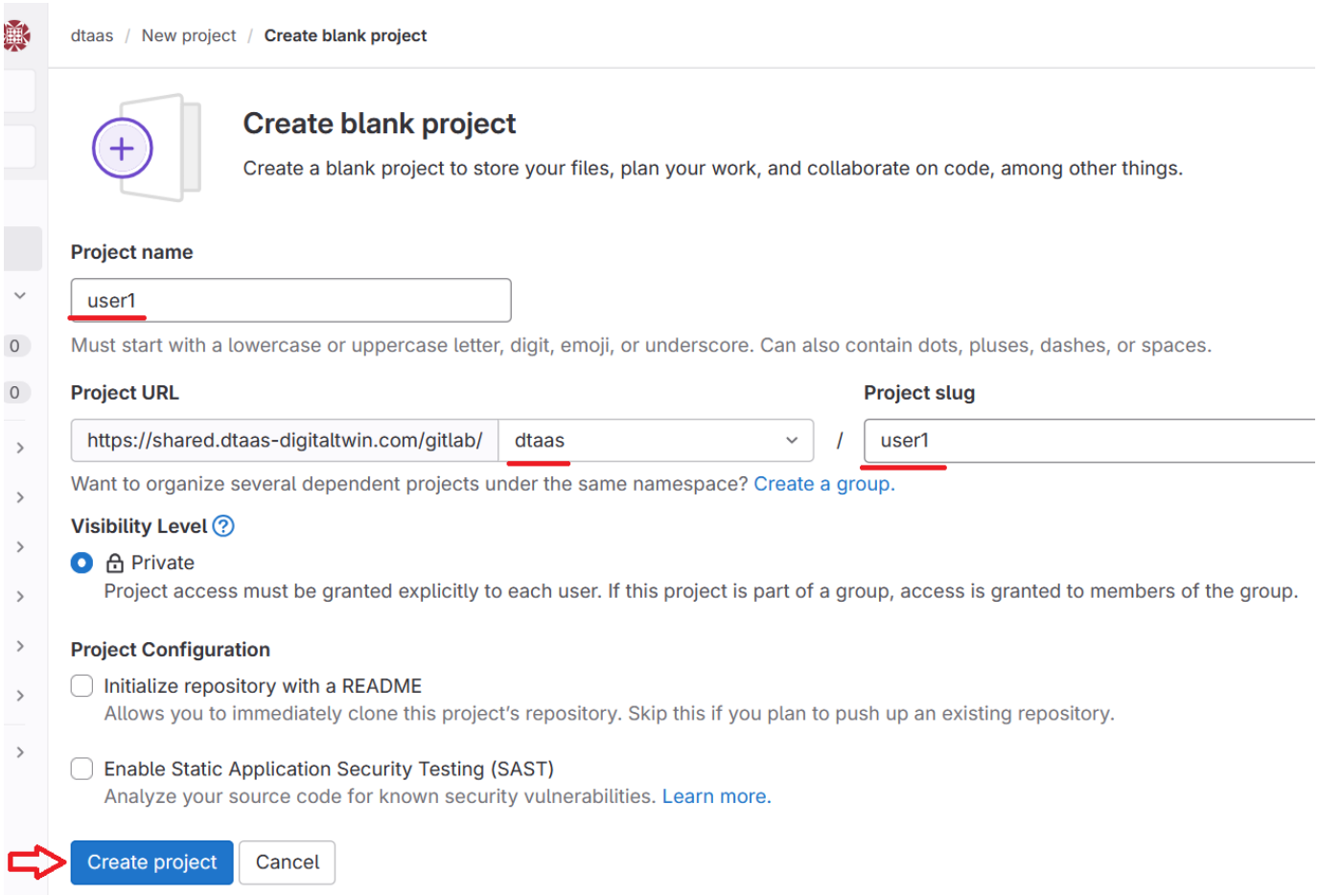
The DTaaS platform relies on GitLab for two purposes:

1. OAuth 2.0 authorization service
2. DevOps service

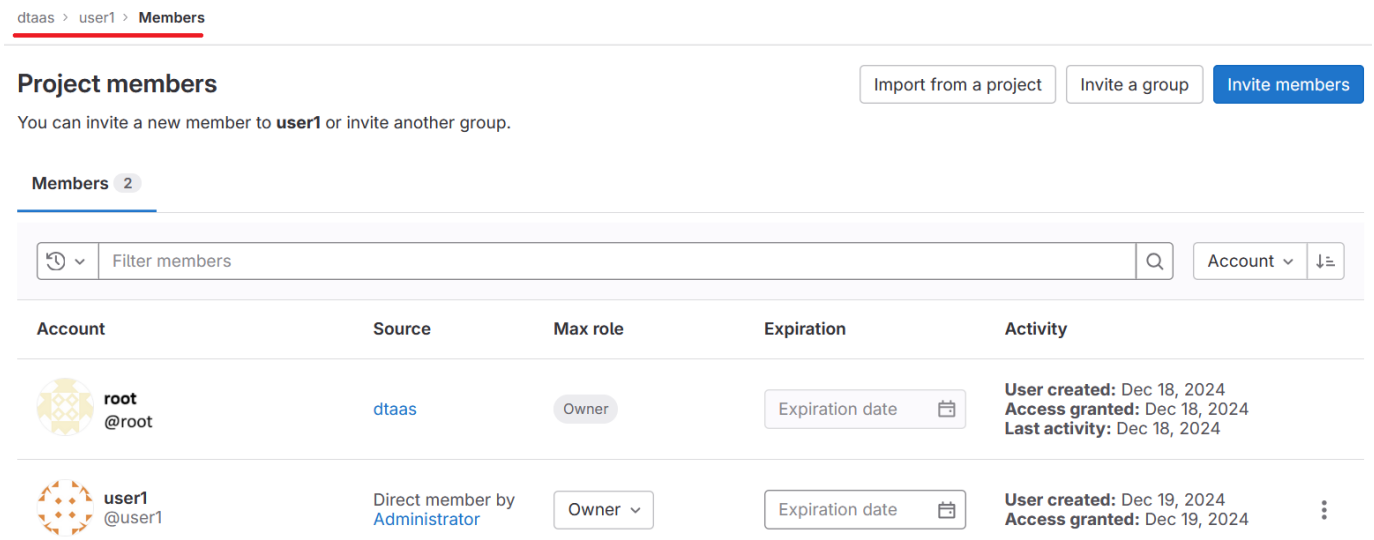
The admin documentation covers the OAuth 2.0 authorization configuration. This guide addresses the use of git commands and project structure for the GitLab DevOps service within the DTaaS.

3.6.1 Preparation

The first step is to create a GitLab project with the *username* in the GitLab user group named *dtaas*.



The user must have ownership permissions over the project.



Warning

The DTaaS website expects a default branch named `main` to exist. The website client performs all git operations on this branch. The preferred default branch name can be changed in the settings page.

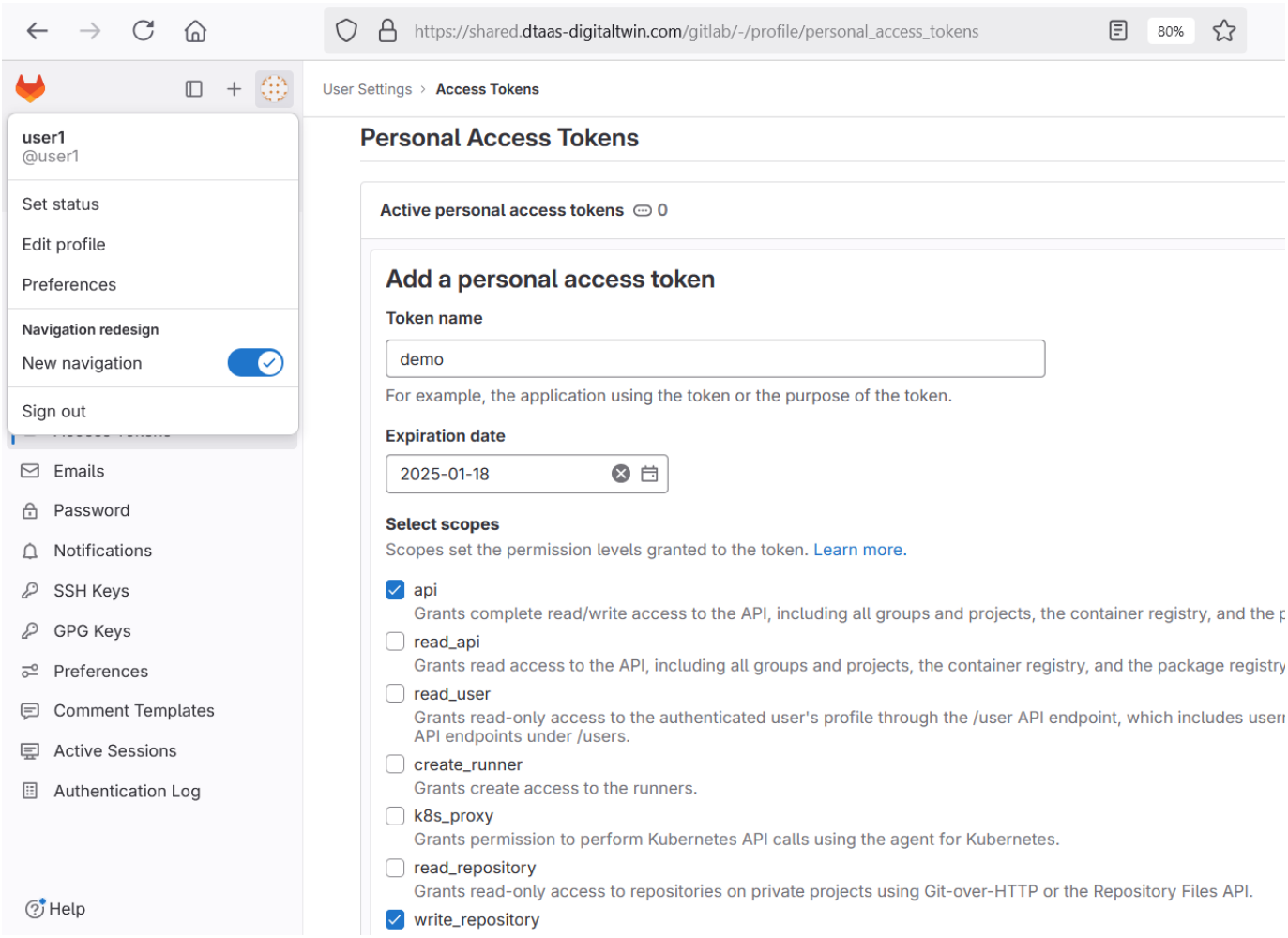
3.6.2 Git commands

Standard git commands and workflows should be utilised. There are two methods for using a GitLab project as a remote git server:

1. Over SSH using a personal SSH key
2. Over HTTPS using personal access tokens (PAT)

This tutorial demonstrates the use of PAT for working with the GitLab server.

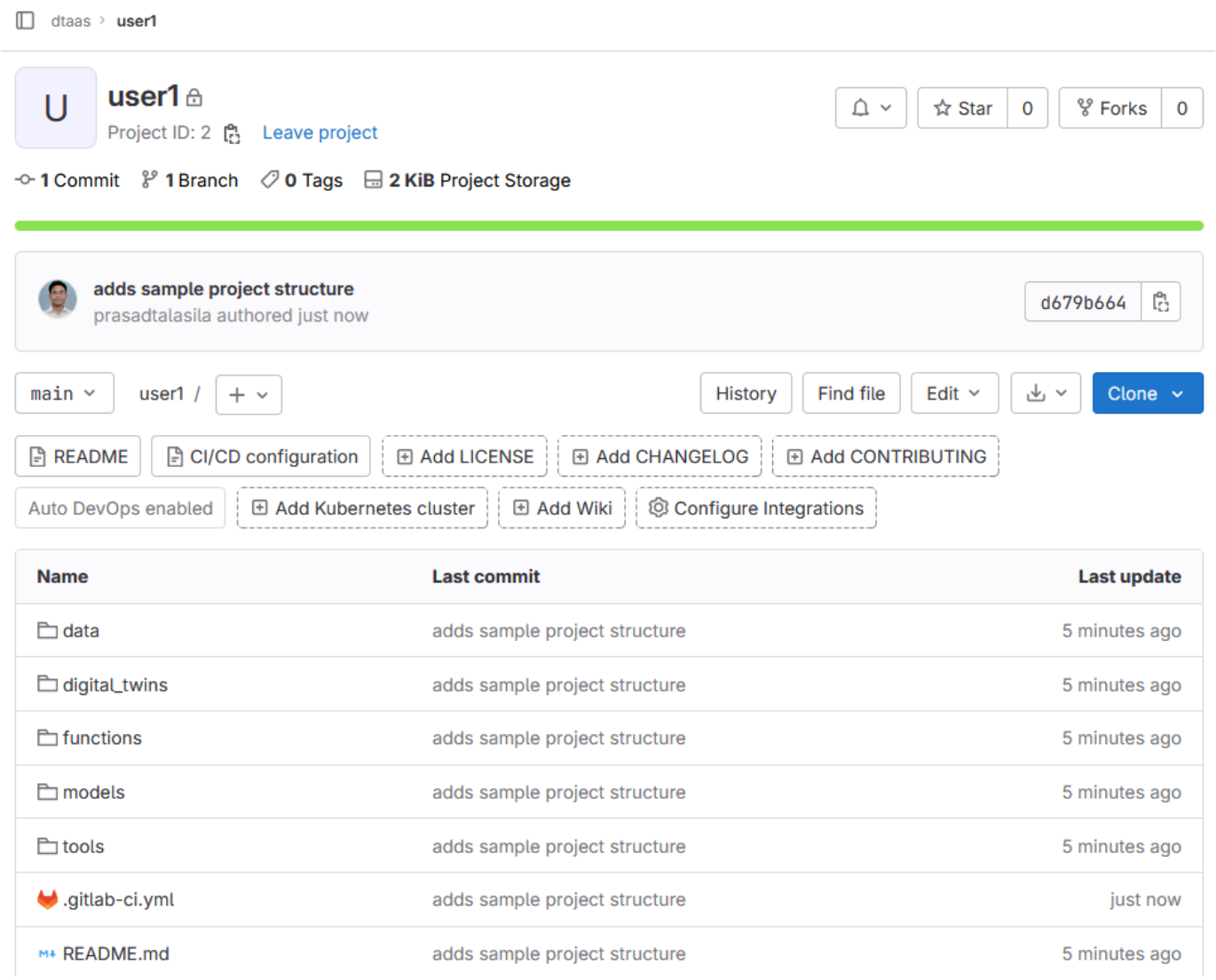
The first step is to create a PAT.



This token should be copied and used to clone the git repository.

3.6.3 Library Assets

GitLab is used to store the reusable **Library** assets of all users. A mandatory structure exists for storing and using Library assets including digital twins. A properly initialized GitLab project should have the following structure.



Pay special attention to `.gitlab-ci.yml`. It must be a valid GitLab DevOps configuration. The example repo provides a sample structure.

For example, with `PAT1` as the PAT for the `dtaas/user1` repository, the command to clone the repository is:

```
$git clone \
  https://user1:PAT1@dtaas-digitaltwin.com/gitlab/dtaas/user1.git
$cd user1
```

After adding the required Library assets:

```
$git push origin
```

3.6.4 Next Steps

A GitLab runner should be integrated with the project repository. Runners may already be installed with the DTaaS platform. These can be verified on the runners page. Additionally, custom runners can be installed and integrated with the repository.

The Digital Twins Preview can then be used to access the DevOps features of the DTaaS platform.

3.7 Runner

A utility service to manage safe execution of remote scripts / commands. The user launches this from the command line and lets the utility manage the commands to be executed.

The runner utility runs as a service and provides REST API interface to safely execute remote commands. Multiple runners can be active simultaneously on one computer. The commands are sent via the REST API and are executed on the computer with active runner.

> **Warning**
>
> This npm package works only on Linux platforms

3.7.1 Install

NPM Registry

The package is available on npmjs.

Install the package with the following command:

```
sudo npm install -g @into-cps-association/runner
```

GitHub Registry

The package is available in GitHub packages registry.

Set the registry and install the package with the following commands

```
sudo npm config set @into-cps-association:registry \
  https://npm.pkg.github.com
sudo npm install -g @into-cps-association/runner
```

The *npm install* command asks for username and password. The username is the GitHub username and the password is the GitHub personal access token. In order for the npm to download the package, the personal access token needs to have *read:packages* scope.

3.7.2 Configure

The utility requires config specified in YAML format. The template configuration file is:

```
port: 5000
location: 'script' #directory location of scripts
commands: #list of permitted scripts
  - create
  - execute
  - terminate
```

It is suggested that the configuration file be named as *runner.yaml* and placed in the directory in which the *runner* microservice is run.

The `location` refers to the relative location of the scripts directory with respect to the location of *runner.yaml* file.

However, there is no limitation on either the configuration filename or the `location`. The path to *runner.yaml* can either be relative or absolute path. However, the `location` path is always relative path with respect to the path of *runner.yaml* file.

The commands must be executable. Ensure that the commands have execute permission on Linux platforms.

3.7.3 Create Commands

The runner requires commands / scripts to be run. These need to be placed in the `location` specified in *runner.yaml* file. The location must be relative to the directory in which the **runner** microservice is being run.

3.7.4 🚀 Use

Display help.

```
$runner -h
Usage: runner [options]

Remote code execution for humans

Options:
  -v --version          package version
  -c --config <string>  runner config file specified in yaml format (default: "runner.yaml")
  -h --help             display help
```

The config option is not mandatory. If it is not used, **runner** looks for *runner.yaml* in the directory from which it is being run. Once launched, the utility runs at the port specified in *runner.yaml* file.

```
runner  #use runner.yaml of the present working directory
runner -c FILE-PATH       #absolute or relative path to config file
runner --config FILE-PATH #absolute or relative path to config file
```

If launched on one computer, the service is accessible at `http://localhost:<port>`.

Access to the service on the network is available at `http://<ip or hostname>:<port>/`.

Application Programming Interface (API)

Three REST API methods are active. The route paths and the responses given for these two sources are:

REST API Route	HTTP Method	Return Value	Comment
localhost:port	POST	Returns the execution status of command	Executes the command provided. Each invocation appends to *array* of commands executed so far.
localhost:port	GET	Returns the execution status of the last command sent via POST request.	
localhost:port/history	GET	Returns the array of POST requests received so far.	

POST REQUEST TO /

Executes a command. The command name given here must exist in *location* directory.

Valid HTTP Request **HTTP Response - Valid Command** **HTTP Response - Inalid Command**

```
POST / HTTP/1.1
Host: intocps.org
Content-Type: application/json
Content-Length: 388

{
  "name": "<command-name>"
}
```

```
Connection: close
Content-Length: 134
Content-Type: application/json; charset=utf-8
Date: Tue, 09 Apr 2024 08:51:11 GMT
Etag: W/"86-ja15r8P5HJu72JcROfBTV4sAn2I"
X-Powered-By: Express

{
  "status": "success"
}
```

```
Connection: close
Content-Length: 28
Content-Type: application/json; charset=utf-8
Date: Tue, 09 Apr 2024 08:51:11 GMT
Etag: W/"86-ja15r8P5HJu72JcROfBTV4sAn2I"
X-Powered-By: Express

{
  "status": "invalid command"
}
```

GET REQUEST TO /

Shows the status of the command last executed.

Valid HTTP Request

```
GET / HTTP/1.1
Host: intocps.org
Content-Type: application/json
Content-Length: 388

{
  "name": "<command-name>"
}
```

HTTP Response - Valid Command

```
Connection: close
Content-Length: 134
Content-Type: application/json; charset=utf-8
Date: Tue, 09 Apr 2024 08:51:11 GMT
Etag: W/"86-ja15r8P5HJu72JcROfBTV4sAn2I"
X-Powered-By: Express

{
  "name": "<command-name>",
  "status": "valid",
  "logs": {
    "stdout": "<output log of command>",
    "stderr": "<error log of command>"
  }
}
```

HTTP Response - Inalid Command

```
Connection: close
Content-Length: 70
Content-Type: application/json; charset=utf-8
Date: Tue, 09 Apr 2024 08:51:11 GMT
Etag: W/"86-ja15r8P5HJu72JcROfBTV4sAn2I"
X-Powered-By: Express

{
  "name": "<command-name>",
  "status": "invalid",
  "logs": {
    "stdout": "",
    "stderr": ""
  }
}
```

GET REQUEST TO /HISTORY

Returns the array of POST requests received so far. Both valid and invalid commands are recorded in the history.

```
[
  {
    "name": "valid command"
  },
  {
    "name": "valid command"
  },
  {
    "name": "invalid command"
  }
]
```

3.8 Examples

3.8.1 DTaaS Examples

Several example digital twins have been created for the DTaaS platform. These examples can be explored, and the steps provided in this **Examples** section can be followed to experience features of the DTaaS platform and understand best practices for managing digital twins within the platform. The following slides and video provide an overview of these examples.

Please see the following demos illustrating the use the DTaaS in two projects:

Project	Slides and Videos
CP-SENS project	Project Introduction: slides and video
	Wind turbine testing with the demo inside user workspace
	Python package and demo video
	Videos demonstrating digital twin for structural health monitoring applications: Data Acquisition System: Part-1, Data Acquisition System: Part-2, Operational Model Analysis, and Model Updating
Incubator	video

Copy Examples

The first step is to copy all example code into the user workspace within a user workspace. The provided shell script copies all examples into the `/workspace/examples` directory.

```
wget https://raw.githubusercontent.com/INTO-CPS-Association/DTaaS-examples/main/getExamples.sh
bash getExamples.sh
```

Example List

The digital twins provided in examples vary in complexity. It is recommended to use the examples in the following order.

1. Mass Spring Damper
2. Water Tank Fault Injection
3. Water Tank Model Swap
4. Desktop Robotti and RabbitMQ
5. Water Treatment Plant and OPC-UA
6. Three Water Tanks with DT Manager Framework
7. Flex Cell with Two Industrial Robots
8. Incubator
9. Firefighters in Emergency Environments
10. Mass Spring Damper with NuRV Runtime Monitor FMU
11. Water Tank Fault Injection with NuRV Runtime Monitor FMU
12. Incubator Co-Simulation with NuRV Runtime Monitor FMU
13. Incubator with NuRV Runtime Monitor as Service
14. Incubator with NuRV Runtime Monitor FMU as Service

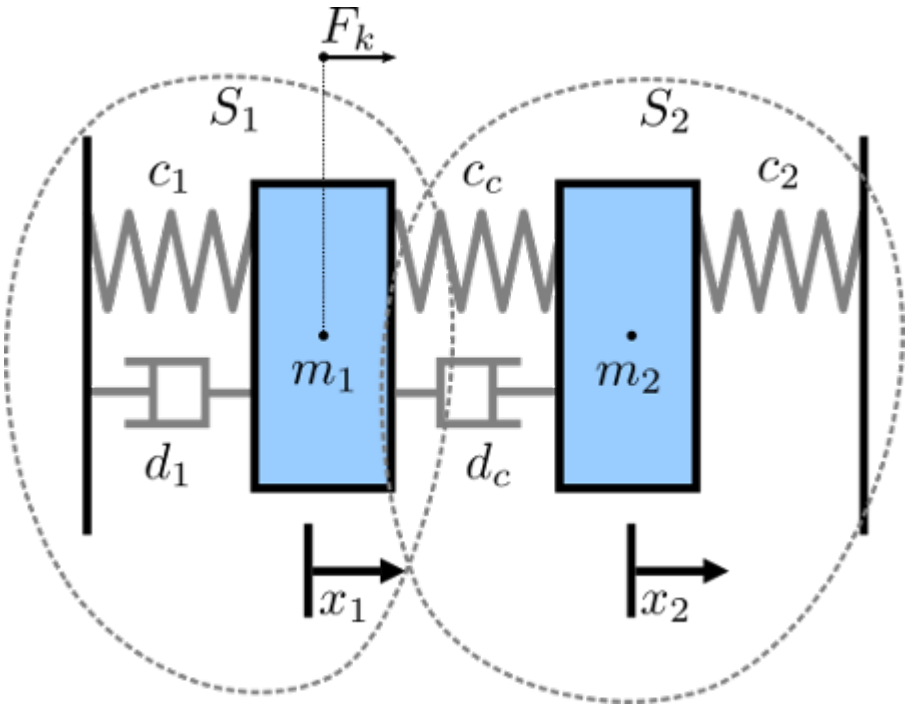
DTaaS examples

3.8.2 Mass Spring Damper

Overview

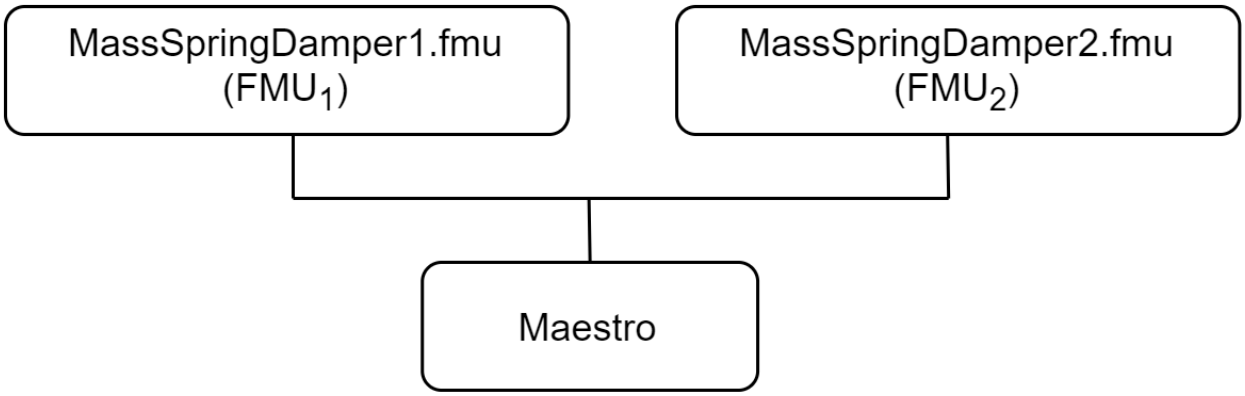
The mass spring damper digital twin (DT) comprises two mass spring dampers and demonstrates how a co-simulation based DT can be used within the DTaaS.

Example Diagram



Example Structure

There are two simulators included in the study, each representing a mass spring damper system. The first simulator calculates the mass displacement and speed of for a given force acting on mass . The second simulator calculates force given a displacement and speed of mass . By coupling these simulators, the evolution of the position of the two masses is computed.



Digital Twin Configuration

This example uses two models and one tool. The specific assets used are:

Asset Type	Names of Assets	Visibility	Reuse in Other Examples
Models	MassSpringDamper1.fmu	Private	Yes
	MassSpringDamper2.fmu	Private	Yes
Tool	maestro-2.3.0-jar-with-dependencies.jar	Common	Yes

The `co-sim.json` and `time.json` are two DT configuration files used for executing the digital twin. These two files can be modified to customise the DT for specific requirements.

Lifecycle Phases

Lifecycle Phase	Completed Tasks
Create	Installs Java Development Kit for Maestro tool
Execute	Produces and stores output in data/mass-spring-damper/output directory
Clean	Clears run logs and outputs

Run the example

To run the example, change the present directory.

```
cd /workspace/examples/digital_twins/mass-spring-damper
```

If required, change the execute permission of lifecycle scripts that need to be executed, for example:

```
chmod +x lifecycle/create
```

Now, run the following scripts:

CREATE

Installs Open Java Development Kit 17 in the workspace.

```
lifecycle/create
```

EXECUTE

Run the the Digital Twin. Since this is a co-simulation based digital twin, the Maestro co-simulation tool executes co-simulation using the two FMU models.

```
lifecycle/execute
```

Examine the Results

The results can be found in the */workspace/examples/data/mass-spring-damper/output directory*.

Run logs can also be viewed in the */workspace/examples/digital_twins/mass-spring-damper*.

TERMINATE PHASE

Terminate to clean up the debug files and co-simulation output files.

```
lifecycle/terminate
```

References

More information about co-simulation techniques and mass spring damper case study are available in:

```
Gomes, Cláudio, et al. "Co-simulation: State of the art."
arXiv preprint arXiv:1702.00686 (2017).
```

The source code for the models used in this DT are available in mass spring damper github repository.

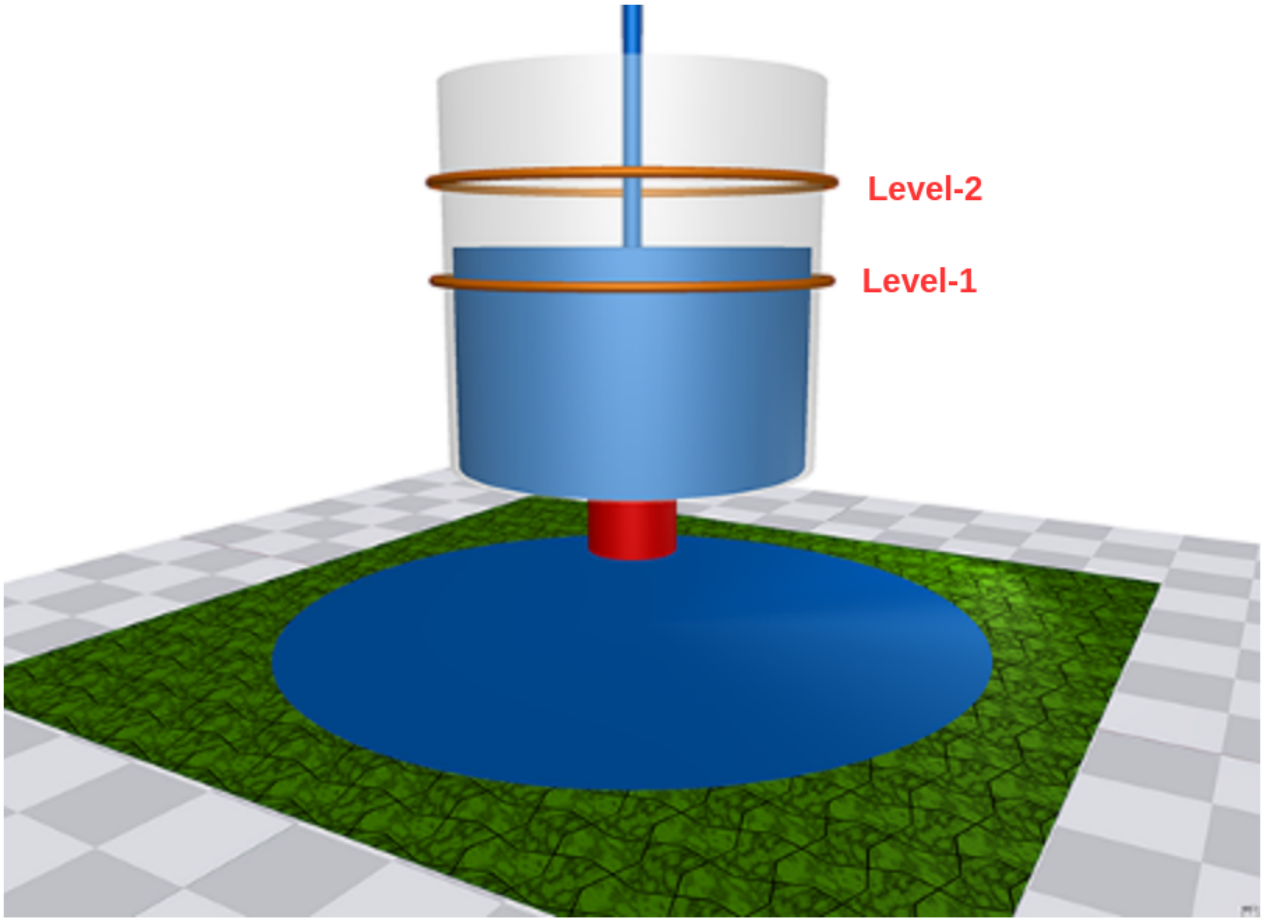
3.8.3 Water Tank Fault Injection

Overview

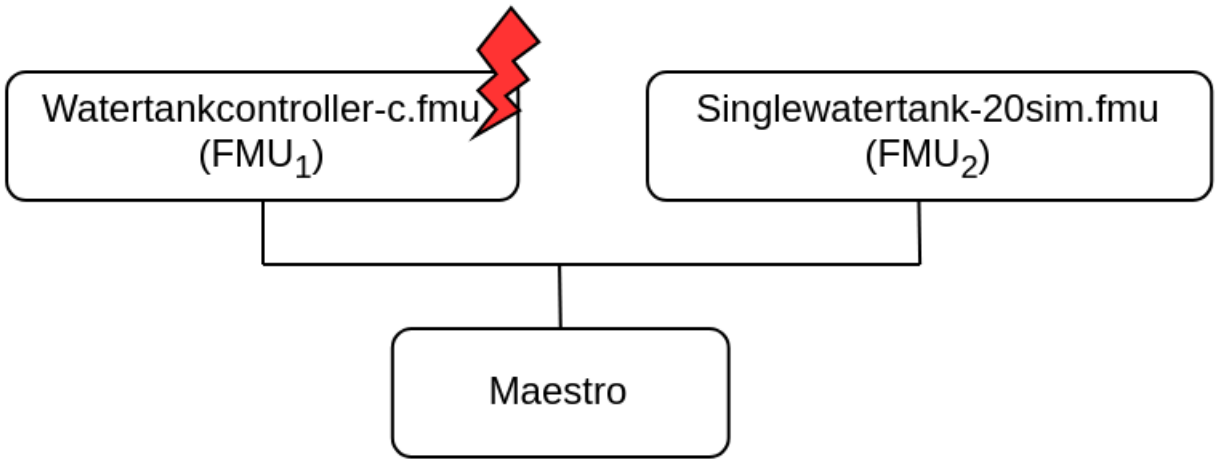
This example demonstrates a fault injection (FI) enabled digital twin (DT). A live DT is subjected to simulated faults received from the environment. The simulated faults are specified as part of DT configuration and can be changed for new instances of DTs.

In this co-simulation based DT, a watertank case-study is used; co-simulation consists of a tank and controller. The goal of which is to keep the level of water in the tank between `Level-1` and `Level-2`. The faults are injected into output of the water tank controller (**Watertankcontroller-c.fmu**) from 12 to 20 time units, such that the tank output is closed for a period of time, leading to the water level increasing in the tank beyond the desired level (`Level-2`).

Example Diagram



Example Structure



Digital Twin Configuration

This example uses two models and one tool. The specific assets used are:

Asset Type	Names of Assets	Visibility	Reuse in Other Examples
Models	watertankcontroller-c.fmu	Private	Yes
	singlewatertank-20sim.fmu	Private	Yes
Tool	maestro-2.3.0-jar-with-dependencies.jar	Common	Yes

The `multimodelFI.json` and `simulation-config.json` are two DT configuration files used for executing the digital twin. These two files can be modified to customise the DT for specific requirements.

The faults are defined in **wt_fault.xml**.

Lifecycle Phases

Lifecycle Phase	Completed Tasks
Create	Installs Java Development Kit for Maestro tool
Execute	Produces and stores output in data/water_tank_FI/output directory
Clean	Clears run logs and outputs

Run the example

To run the example, change the present directory.

```
cd /workspace/examples/digital_twins/water_tank_FI
```

If required, change the execute permission of lifecycle scripts that need to be executed, for example:

```
chmod +x lifecycle/create
```

Now, run the following scripts:

CREATE

Installs Open Java Development Kit 17 and pip dependencies. The pandas and matplotlib are the pip dependencies installated.

```
lifecycle/create
```

EXECUTE

Run the co-simulation. Generates the co-simulation output.csv file at `/workspace/examples/data/water_tank_FI/output`.

```
lifecycle/execute
```

ANALYZE PHASE

Process the output of co-simulation to produce a plot at: `/workspace/examples/data/water_tank_FI/output/plots/`.

```
lifecycle/analyze
```

Examine the results

The results can be found in the */workspace/examples/data/water_tank_FI/output directory*.

Run logs can also be viewed in the */workspace/examples/digital_twins/water_tank_FI*.

TERMINATE PHASE

Clean up the temporary files and delete output plot

```
lifecycle/terminate
```

References

More details on this case-study can be found in the paper:

```
M. Frasheri, C. Thule, H. D. Macedo, K. Lausdahl, P. G. Larsen and
L. Esterle, "Fault Injecting Co-simulations for Safety,"
2021 5th International Conference on System Reliability and Safety (ICSRS),
Palermo, Italy, 2021.
```

The fault-injection plugin is an extension to the Maestro co-orchestration engine that enables injecting inputs and outputs of FMUs in an FMI-based co-simulation with tampered values. More details on the plugin can be found in fault injection git repository. The source code for this example is also in the same github repository in a example directory.

3.8.4 Water Tank Model Swap

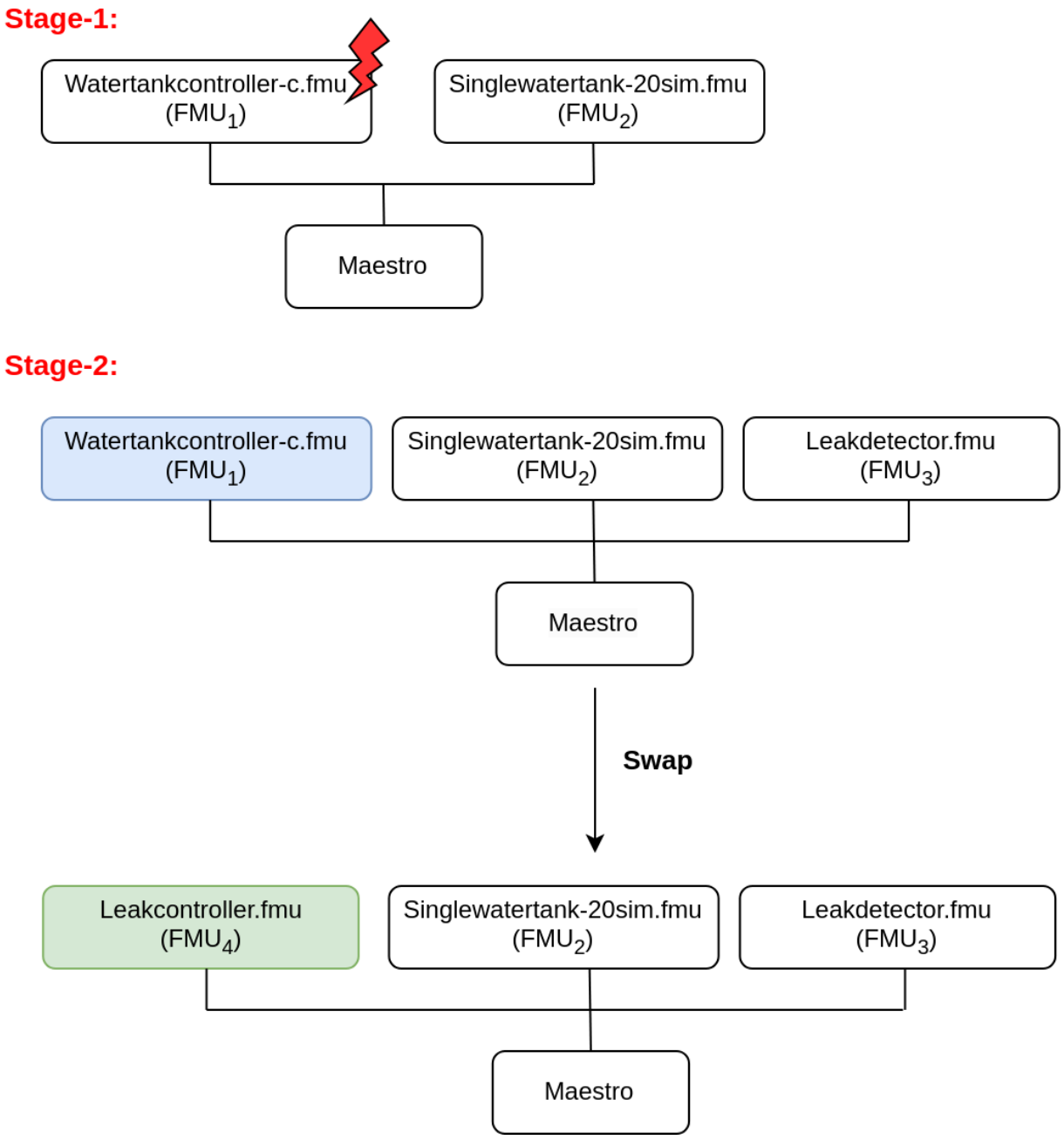
Overview

This example demonstrates multi-stage execution and dynamic reconfiguration of a digital twin (DT). Two features of DTs are demonstrated:

- Fault injection into live DT
- Dynamic auto-reconfiguration of live DT

The co-simulation methodology is used to construct this DT.

Example Structure



Configuration of assets

This example uses four models and one tool. The specific assets used are:

Asset Type	Names of Assets	Visibility	Reuse in Other Examples
Models	Watertankcontroller-c.fmu	Private	Yes
	Singlewatertank-20sim.fmu	Private	Yes
	Leak_detector.fmu	Private	No
	Leak_controller.fmu	Private	No
Tool	maestro-2.3.0-jar-with-dependencies.jar	Common	Yes

This DT has many configuration files. The DT is executed in two stages. There exist separate DT configuration files for each stage. The following table shows the configuration files and their purpose.

Configuration file name	Execution Stage	Purpose
mm1. json	stage-1	DT configuration
wt_fault.xml, FaultInject.mabl	stage-1	faults injected into DT during stage-1
mm2.json	stage-2	DT configuration
simulation-config.json	Both stages	Configuration for specifying DT execution time and output logs

Lifecycle Phases

Lifecycle Phase	Completed Tasks
Create	Installs Java Development Kit for Maestro tool
Execute	Produces and stores output in data/water_tank_swap/output directory
Analyze	Process the co-simulation output and produce plots
Clean	Clears run logs, outputs and plots

Run the example

To run the example, change the present directory.

```
cd /workspace/examples/digital_twins/water_tank_swap
```

If required, change the permission of files that need to be executed, for example:

```
chmod +x lifecycle/create
```

Now, run the following scripts:

CREATE

Installs Open Java Development Kit 17 and pip dependencies. The matplotlib pip package is also installated.

```
lifecycle/create
```

EXECUTE

This DT has two-stage execution. In the first-stage, a co-simulation is executed. The Watertankcontroller-c.fmu and Singlewatertank-20sim.fmu models are used to execute the DT. During this stage, faults are injected into one of the models (Watertankcontroller-c.fmu) and the system performance is checked.

In the second-stage, another co-simulation is run in which three FMUs are used. The FMUs used are: watertankcontroller, singlewatertank-20sim, and leak_detector. There is an in-built monitor in the Maestro tool. This monitor is enabled during the stage and a swap condition is set at the beginning of the second-stage. When the swap condition is satisfied, the Maestro swaps out Watertankcontroller-c.fmu model and swaps in Leakcontroller.fmu model. This swapping of FMU models demonstrates the dynamic reconfiguration of a DT.

The end of execution phase generates the co-simulation output.csv file at `/workspace/examples/data/water_tank_swap/output`.

```
lifecycle/execute
```

ANALYZE PHASE

Process the output of co-simulation to produce a plot at: `/workspace/examples/data/water_tank_FI/output/plots/`.

```
lifecycle/analyze
```

Examine the results

The results can be found in the *workspace/examples/data/water_tank_swap/output directory*.

Run logs can also be viewed in the *workspace/examples/digital_twins/water_tank_swap*.

TERMINATE PHASE

Clean up the temporary files and delete output plot

```
lifecycle/terminate
```

References

The complete source of this example is available on model swap github repository.

The runtime model (FMU) swap mechanism demonstrated by the experiment is detailed in the paper:

```
Ejersbo, Henrik, et al. "fmiSwap: Run-time Swapping of Models for
Co-simulation and Digital Twins." arXiv preprint arXiv:2304.07328 (2023).
```

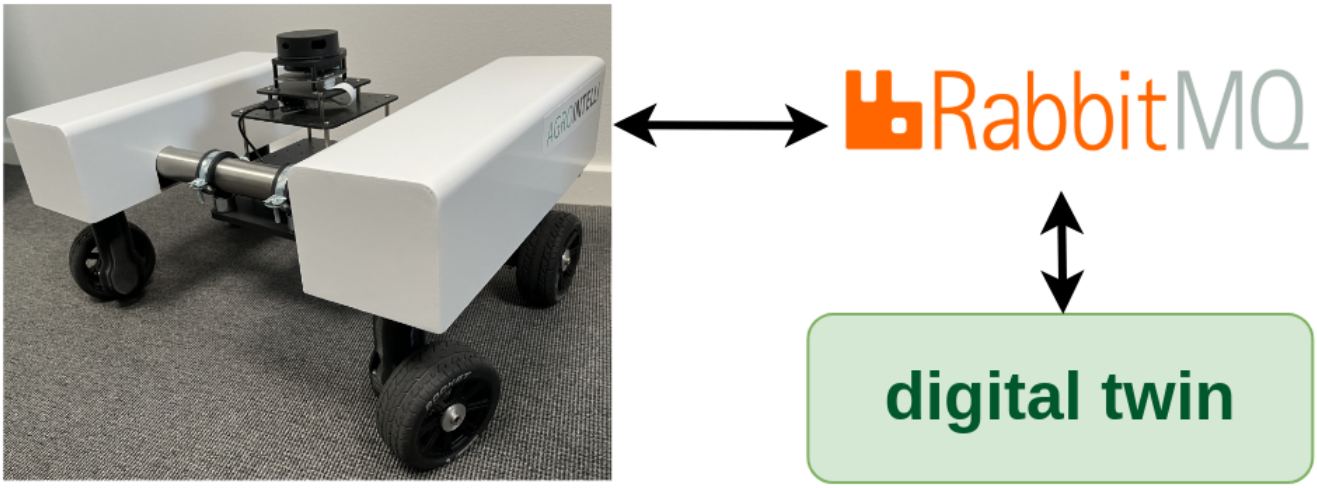
The runtime reconfiguration of co-simulation by modifying the Functional Mockup Units (FMUs) used is further detailed in the paper:

```
Ejersbo, Henrik, et al. "Dynamic Runtime Integration of
New Models in Digital Twins." 2023 IEEE/ACM 18th Symposium on
Software Engineering for Adaptive and Self-Managing Systems
(SEAMS). IEEE, 2023.
```

3.8.5 Desktop Robotti with RabbitMQ

Overview

This example demonstrates bidirectional communication between a mock physical twin and a digital twin of a mobile robot (Desktop Robotti). The communication is enabled by RabbitMQ Broker.

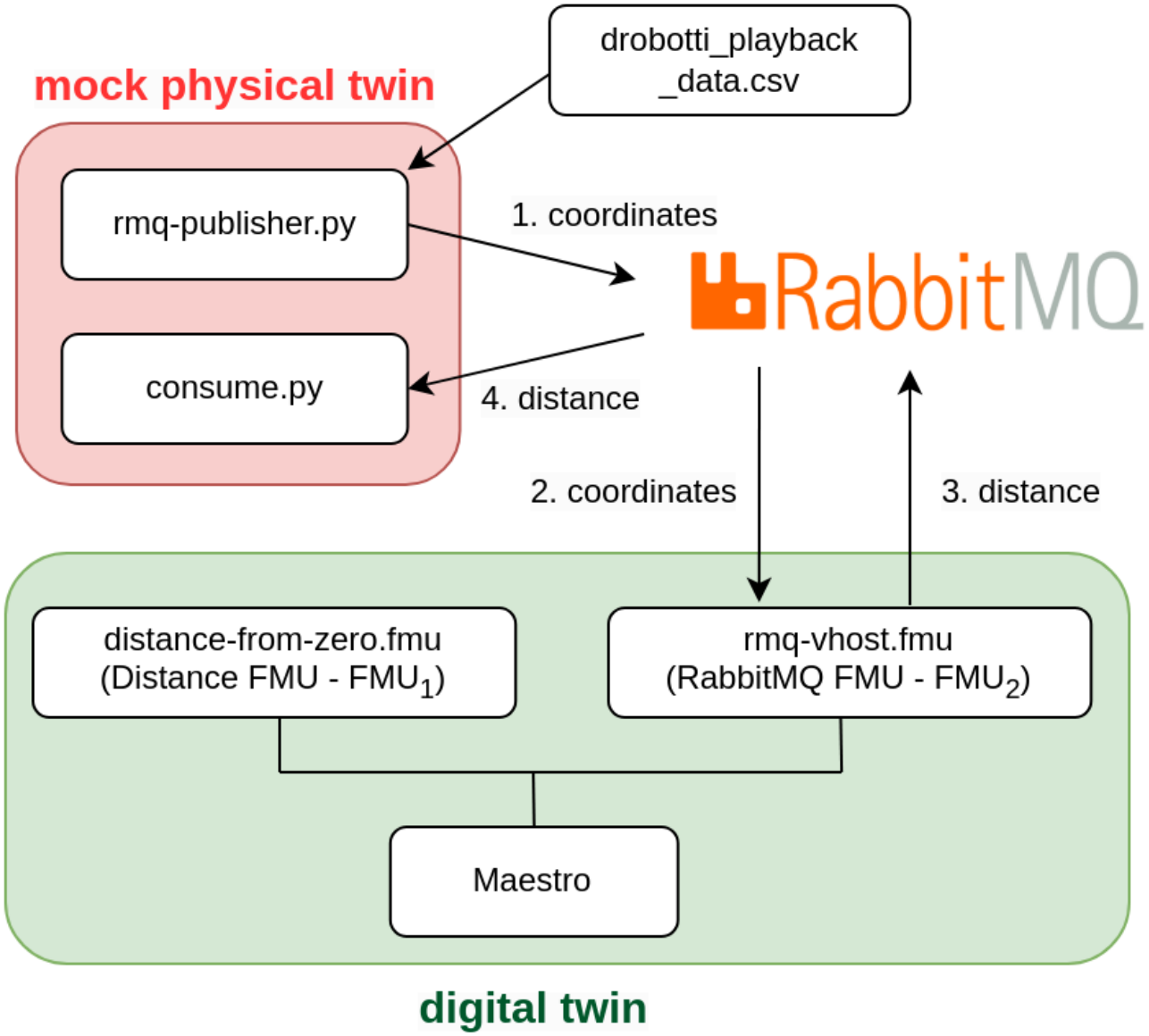


Example Structure

The mock physical twin of mobile robot is created using two python scripts

1. data/drobotti_rmqfmu/rmq-publisher.py
2. data/drobotti_rmqfmu/consume.py

The mock physical twin sends its physical location in *(x,y)* coordinates and expects a cartesian distance calculated from digital twin.



The *rmq-publisher.py* reads the recorded *(x,y)* physical coordinates of mobile robot. The recorded values are stored in a data file. These *(x,y)* values are published to RabbitMQ Broker. The published *(x,y)* values are consumed by the digital twin.

The *consume.py* subscribes to RabbitMQ Broker and waits for the calculated distance value from the digital twin.

The digital twin consists of a FMI-based co-simulation, where Maestro is used as co-orchestration engine. In this case, the co-simulation is created by using two FMUs - RMQ FMU (rabbitmq-vhost.fmu) and distance FMU (distance-from-zero.fmu). The RMQ FMU receives the *(x,y)* coordinates from *rmq-publisher.py* and sends calculated distance value to *consume.py*. The RMQ FMU uses RabbitMQ broker for communication with the mock mobile robot, i.e., *rmq-publisher.py* and *consume.py*. The distance FMU is responsible for calculating the distance between *(0,0)* and *(x,y)*. The RMQ FMU and distance FMU exchange values during co-simulation.

Digital Twin Configuration

This example uses two models, one tool, one data, and two scripts to create mock physical twin. The specific assets used are:

Asset Type	Names of Assets	Visibility	Reuse in Other Examples
Models	distance-from-zero.fmu	Private	No
	rmq-vhost.fmu	Private	Yes
Tool	maestro-2.3.0-jar-with-dependencies.jar	Common	Yes
Data	drobotti_playback_data.csv	private	No
Mock PT	rmq-publisher.py	Private	No
	consume.py	Private	No

This DT has many configuration files. The `coe.json` and `multimodel.json` are two DT configuration files used for executing the digital twin. These two files can be modified to customise the DT for specific requirements.

The RabbitMQ access credentials need to be provided in `multimodel.json`. The `rabbitMQ-credentials.json` provides RabbitMQ access credentials for mock PT python scripts. The appropriate credentials should be added in both these files.

Lifecycle Phases

Lifecycle Phase	Completed Tasks
Create	Installs Java Development Kit for Maestro tool and pip packages for python scripts
Execute	Runs both DT and mock PT
Clean	Clears run logs and outputs

Run the example

To run the example, change the present directory.

```
cd /workspace/examples/digital_twins/drobotti_rmqfmu
```

If required, change the execute permission of lifecycle scripts that need to be executed, for example:

```
chmod +x lifecycle/create
```

Now, run the following scripts:

CREATE

Installs Open Java Development Kit 17 in the workspace. Also install the required python pip packages for *rmq-publisher.py* and *consume.py* scripts.

```
lifecycle/create
```

EXECUTE

Run the python scripts to start mock physical twin. Also run the the Digital Twin. Since this is a co-simulation based digital twin, the Maestro co-simulation tool executes co-simulation using the two FMU models.

```
lifecycle/execute
```

Examine the results

The results can be found in the */workspace/examples/digital_twins/drobotti_rmqfmu directory*.

Executing the DT will generate and launch a co-simulation (RMQFMU and distance FMU), and two python scripts. One to publish data that is read from a file. And one to consume what is sent by the distance FMU.

In this examples the DT will run for 10 seconds, with a stepsize of 100ms. Thereafter it is possible to examine the logs produce in `/workspace/examples/digital_twins/drobotti_rmqfmu/target`. The outputs for each FMU, xpos and ypos for the RMQFMU, and the distance for the distance FMU are recorded in the `outputs.csv` file. Other logs can be examined for each FMU and the publisher scripts. Note that, the RMQFMU only sends data, if the current input is different to the previous one.

TERMINATE PHASE

Terminate to clean up the debug files and co-simulation output files.

```
lifecycle/terminate
```

References

The RabbitMQ FMU github repository contains complete documentation and source code of the rmq-vhost.fmu.

More information about the case study is available in:

```
Frasheri, Mirgita, et al. "Addressing time discrepancy between digital
and physical twins." Robotics and Autonomous Systems 161 (2023): 104347.
```

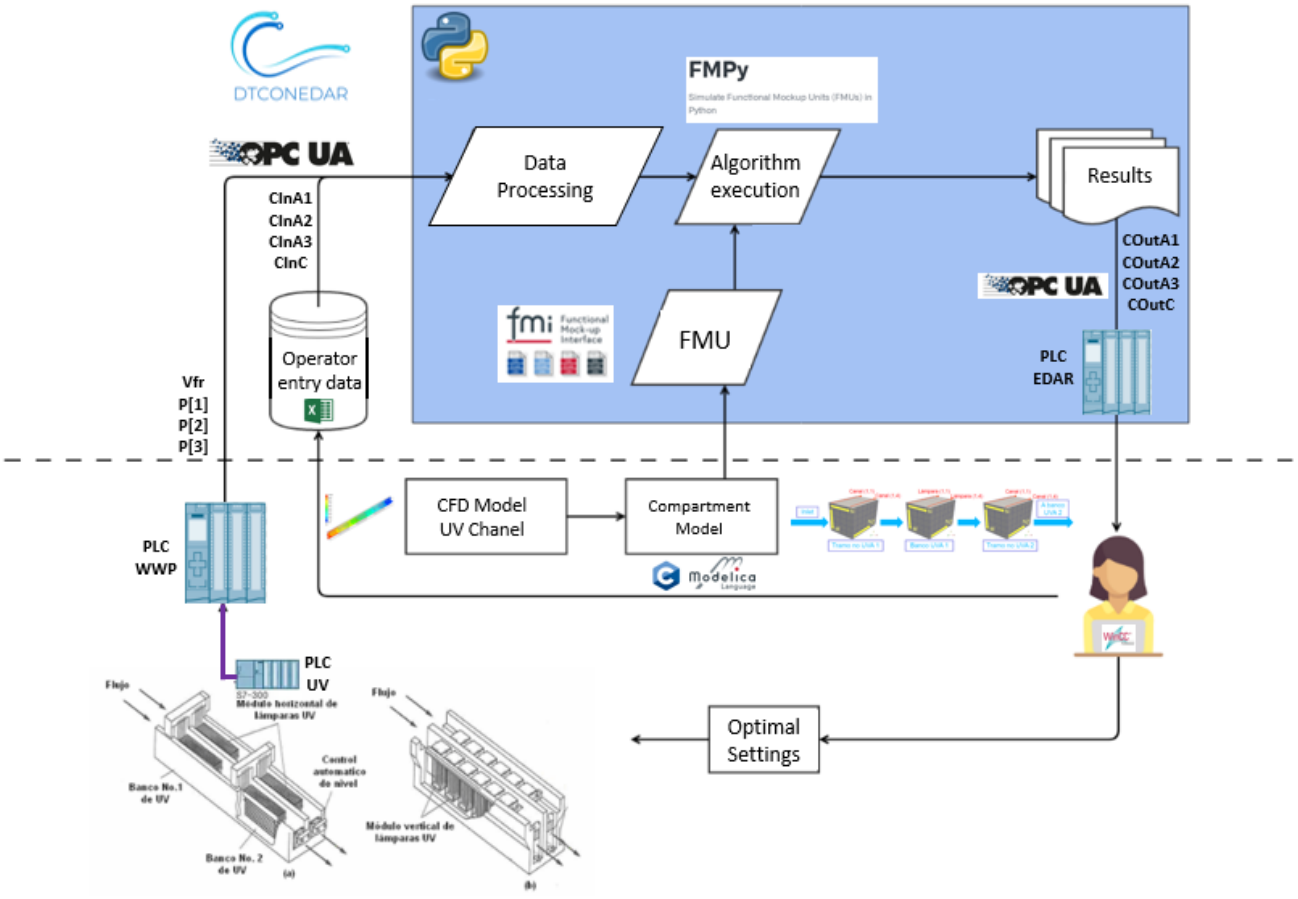
3.8.6 Waste Water Plant with OPC-UA

Introduction

Waste water treatment (WWT) plants must comply with substance and species concentration limits established by regulation in order to ensure the good quality of the water. This is usually done taking periodic samples that are analysed in the laboratory. This means that plant operators do not have continuous information for making decisions and, therefore, operation setpoints are set to higher values than needed to guarantee water quality. Some of the processes involved in WWT plants consume a lot of power, thus adjusting setpoints could significantly reduce energy consumption.

Physical Twin Overview

This example demonstrates the communication between a physical ultraviolet (UV) disinfection process (the tertiary treatment of a WWT plant) and its digital twin, which is based on Computational Fluid Dynamics (CFD) and compartment models. The aim of this digital twin is to develop "virtual sensors" that provide continuous information that facilitates the decision making process for the plant operator.



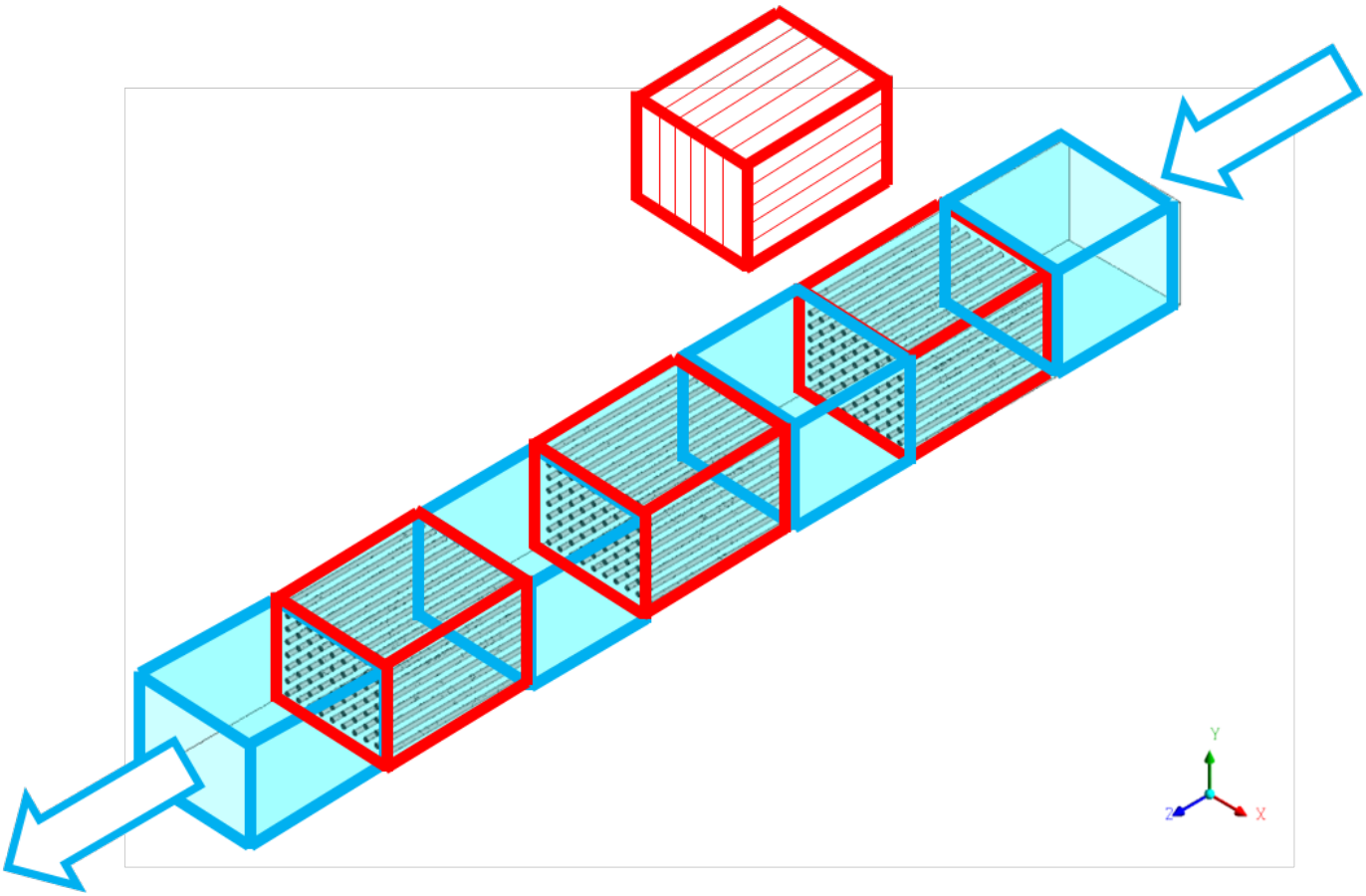
The physical twin of the waste water plant is composed of an ultraviolet channel controlled by a PLC that controls the power of the UV lamps needed to kill all the pathogens of the flow. The channel has 3 groups of UV lamps, therefore the real channel (and is mathematical model) is subdivided into 7 zones: 4 correspond to zones without UV lamps (2 for the entrance and exit of the channel + 2 zones between UV lamps) and the 3 reamaining for the UV lamps.

The dose to be applied (related with the power) changes according to the residence time (computed from the measure of the volume flow) and the UV intensity (measured by the intensity sensor).

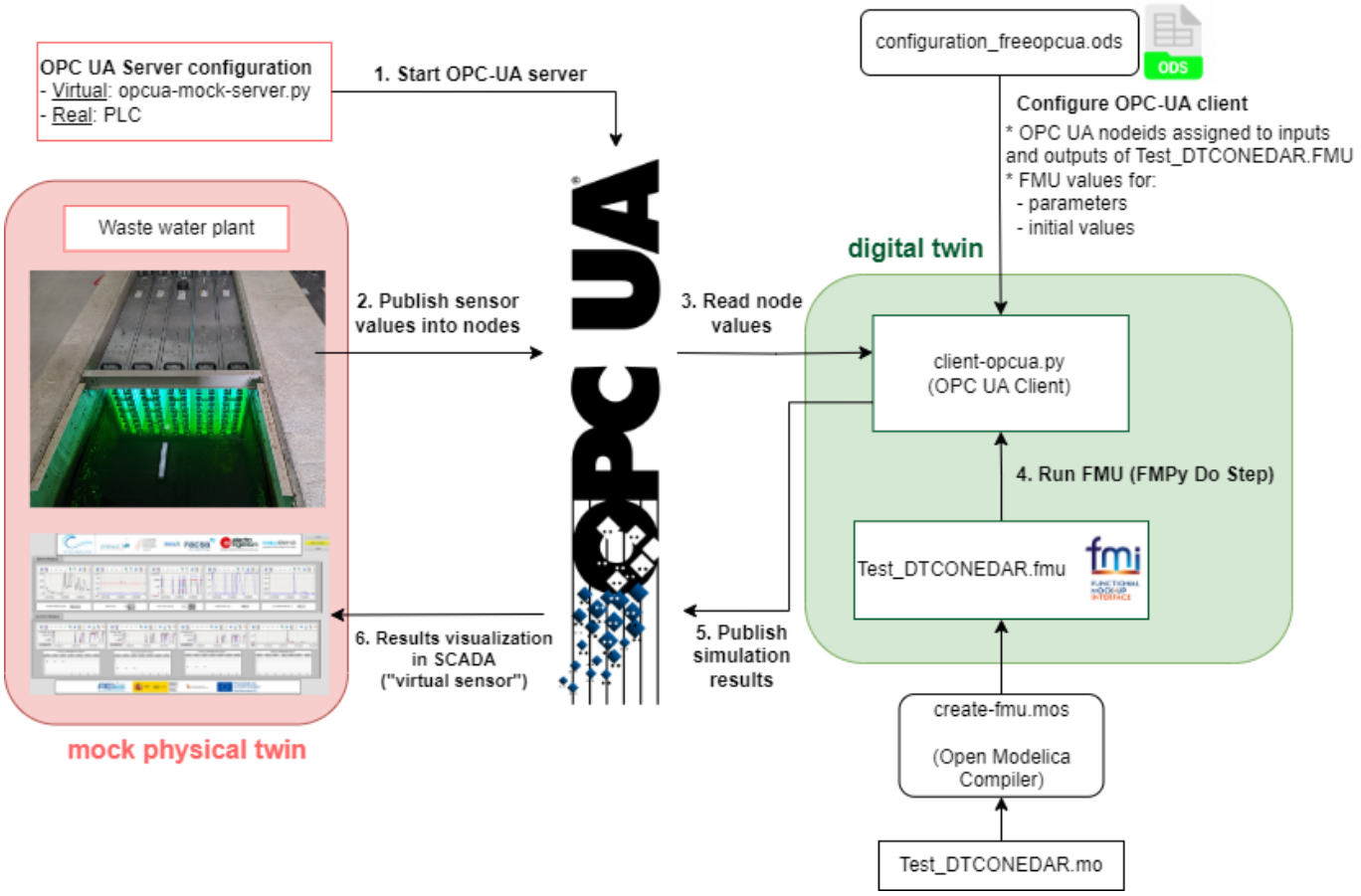
The information of the volumetric flow and power (in the three parts of the channel) is transmitted to the PLC of the plant. Furthermore, the PLC is working as OPC UA Server to send and receive data to and from an OPC UA Client. Additionally, some sizing parameters and initial values are read from a spreadsheet filled in by the plant operator. In this case, the spreadsheet is an Open Office file (.ods) due to the software installed in the SCADA PC. Some of the variables like initial concentration of disinfectant and pathogens are included, among

others. Some values defined by the plant operator correspond to input signals that are not currently being measured, but are expected to be measured in the future.

Digital Twin Overview



The digital twin is a reduced model (developed in C) that solves physical conservation laws (mass, energy and momentum), but simplifies details (geometry, mainly) to ensure real-time calculations and accurate results. The results are compared to the ones obtained by the CFD. C solver developed is used by the OpenModelica model. OpenModelica converts it into the FMI standard, to be integrated in the OPC UA Client (*client-opcua.py*).



Digital Twin Configuration

Asset Type	Name of Asset	Visibility	Reuse in Other Examples
Model	Test_DTCONEDAR.mo	private	No
Data	configuration_freopcua.ods	private	No
	model_description.csv (generated by client-asyncua.py)	private	No
	Mock OPC UA Server: opcua-mock-server.py	private	No
Tool	**OPC UA Client**: client-opcua.py	private	No
	FMU builder: create-fmu.mos	private	No

In this example, a dummy model representation of the plant is used, instead of the real model. The simplified model (with not the real equations) is developed in **Open Modelica (Test_DTCONEDAR.mo)**. The FMU is generated from the Open Modelica interface to obtain the needed binaries to run the FMU. It is possible to run an FMU previously generated, however, to ensure that the right binaries are being used, it is recommended to install Open Modelica Compiler and run `script.mos` to build the FMU from the Modelica file `Test_DTCONEDAR.mo`.

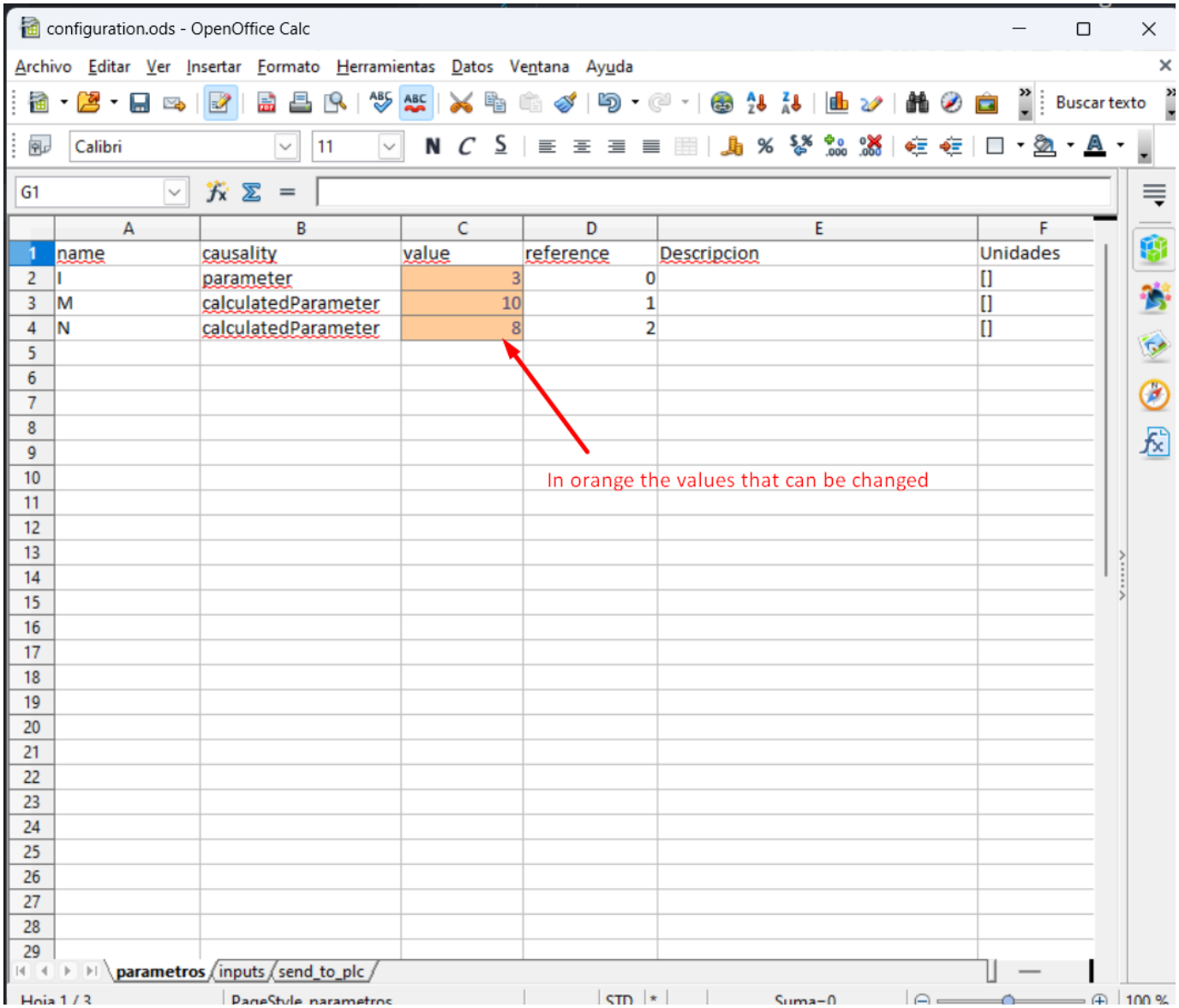
The FMU model description file (`modelDescription.xml` file inside `Test_DTCONEDAR.fmu`) has the information of the value references configuration_freeopcua.ods has the information of the OPC-UA node IDs And both have in common the variable name

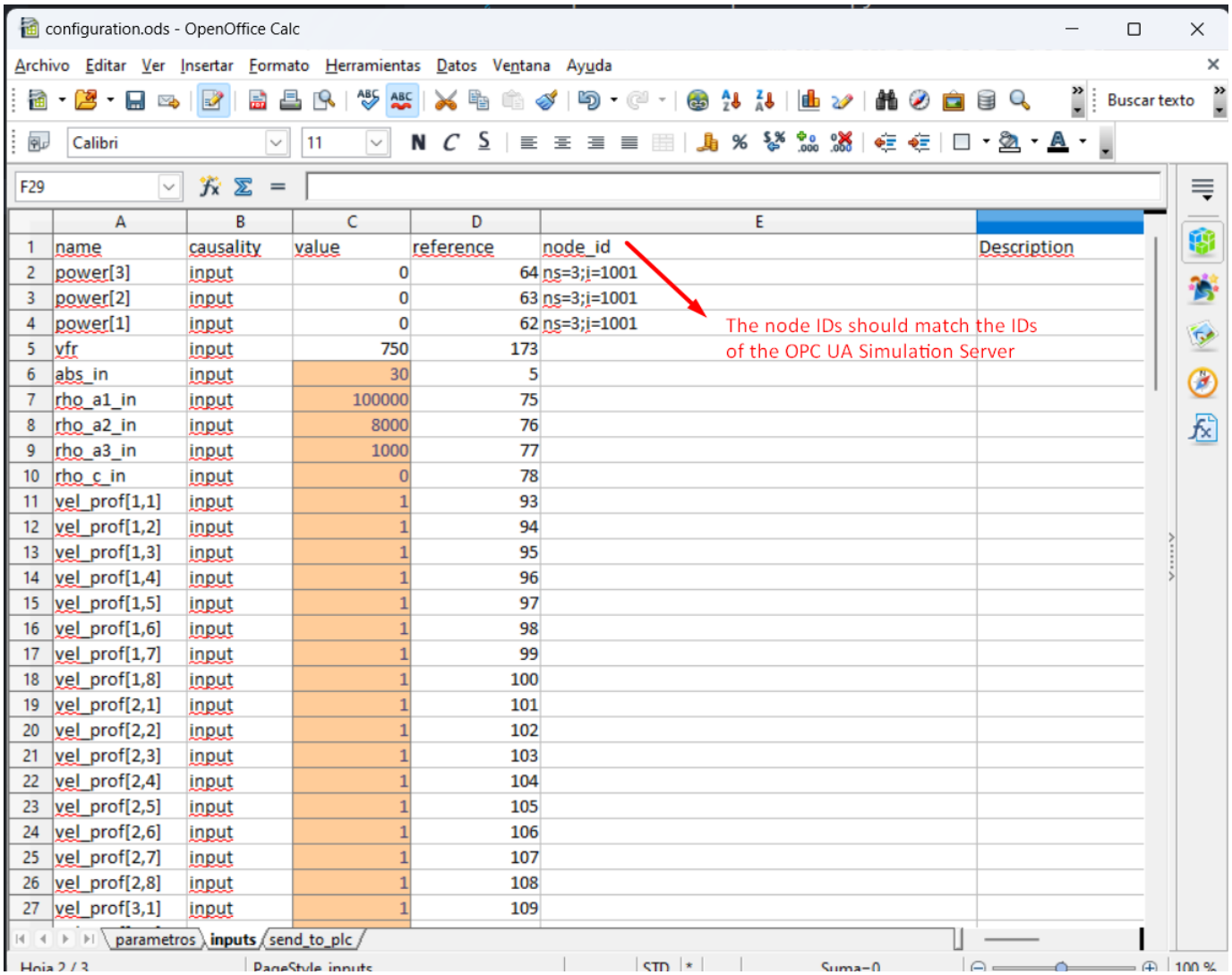
The client python script (`client-opcua.py`) does the following actions:

- Reads the variable names and the variable value references from the model description file of the Test_DTCONEDAR.FMU.
- Reads *configuration_freeopcua.ods* to obtain opcua node IDs and assigns those node IDs to the variables read from the FMU
- Read *configuration_freeopcua.ods* to fix initial values, parameters and some inputs (those inputs that are not being measured, a reasonable value is assumed).
- Read values from PLC using a client OPC.
- Execute the algorithm with the FMPy library using the .fmu created from the compartment model (based on CFD)
- Obtain results.
- Send by OPC UA protocol the result values to the PLC, to visualise them in the SCADA and with the aim to improve the decision-making process of the plant operator.

INPUT DATA VARIABLES

The *configuration_freeopcua.ods* date file is used for customising the initial input data values used by the server.





	A	B	C	D	E	
1	name	causality	value	reference	node_id	Description
2	power[3]	input	0	64	ns=3;i=1001	
3	power[2]	input	0	63	ns=3;i=1001	
4	power[1]	input	0	62	ns=3;i=1001	
5	vfr	input	750	173		
6	abs_in	input	30	5		
7	rho_a1_in	input	100000	75		
8	rho_a2_in	input	8000	76		
9	rho_a3_in	input	1000	77		
10	rho_c_in	input	0	78		
11	vel_prof[1,1]	input	1	93		
12	vel_prof[1,2]	input	1	94		
13	vel_prof[1,3]	input	1	95		
14	vel_prof[1,4]	input	1	96		
15	vel_prof[1,5]	input	1	97		
16	vel_prof[1,6]	input	1	98		
17	vel_prof[1,7]	input	1	99		
18	vel_prof[1,8]	input	1	100		
19	vel_prof[2,1]	input	1	101		
20	vel_prof[2,2]	input	1	102		
21	vel_prof[2,3]	input	1	103		
22	vel_prof[2,4]	input	1	104		
23	vel_prof[2,5]	input	1	105		
24	vel_prof[2,6]	input	1	106		
25	vel_prof[2,7]	input	1	107		
26	vel_prof[2,8]	input	1	108		
27	vel_prof[3,1]	input	1	109		

DT CONFIG

The *config.json* specifies the configuration parameters for the OPC UA client.

config.json M

```
{
    "url"             : "opc.tcp://0.0.0.0:4840",
    "config_ods"      : "configuration_freeopcua.ods",
    "fmu_filename"    : "Test_DTCONEDAR_linux.fmu",
    "stop_time"       : 10.0,
    "step_size"       : 0.5,
    "record"          : false,
    "record_interval"    : 5.0,
    "record_variables"   : [],
    "enable_send"        : true
}
```

Optional parameters can be modified:

- stop_time
- step_size
- record = True, to save the results of the simulation
- record_interval. Sometimes the simulation step_size is small and a the size of the results file can be too big. For instance, if the simulation step_size is 0.01 seconds, the record_interval can be increased so as to reduce the result file size.
- record_variables: the list of variables to record can be specified.
- enable_send = True, to send results to the OPC UA Server.

Lifecycle Phases

The lifecycles that are covered include:

Lifecycle Phase	Completed Tasks
Install	Installs Open Modelica, Python 3.10 and the required pip dependencies
Create	Create FMU from Open Modelica file
Execute	Run OPC UA mock server and normal OPC-UA client
Clean	Delete the temporary files

Run the example

To run the example, change the present directory.

```
cd /workspace/examples/digital twins/opc-ua-waterplant
```

If required, change the execute permission of lifecycle scripts.

```
chmod +x lifecycle/*
```

Now, run the following scripts:

INSTALL

Installs Open Modelica, Python 3.10 and the required pip dependencies

```
lifecycle/install
```

CREATE

Create `Test_DTCONEDAR.fmu` co-simulation model from `Test_DTCONEDAR.mo open modelica file.

```
lifecycle/create
```

EXECUTE

Start the mock OPC UA server in the background. Run the OPC UA client.

```
lifecycle/execute
```

CLEAN

Remove the temporary files created by Open Modelica and output files generated by OPC UA client.

```
lifecycle/clean
```

References

More explanation about this example is available at:


```
Royo, L., Labarías, A., Arnau, R., Gómez, A., Ezquerra, A., Cilla, I., &
Díez-Antoñazas, L. (2023). Improving energy efficiency in the tertiary
treatment of Alguazas WWTP (Spain) by means of digital twin development
based on Physics modelling . Cambridge Open Engage.
[doi:10.33774/coe-2023-1vjcw](https://doi.org/10.33774/coe-2023-1vjcw)
```

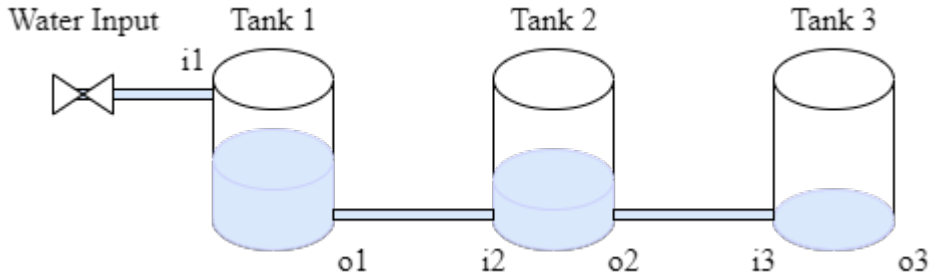

Acknowledgements

The work on this example was done in a project subsidised thanks to the support grants for Agrupación Empresarial Innovadora (AEI) of the Ministry of Industry, Trade and Tourism (MINCOTUR) with the aim of improving the competitiveness of small and medium-sized enterprises within the Recovery, Transformation and Resilience Plan (PRTR) financed by the Next Generation EU (NGEU) funds, with Grant Number: AEI-010500-2022b-196.

3.8.7 Three-Tank System Digital Twin

Overview

The three-tank system is a simple case study that represents a system composed of three individual components coupled in a cascade as follows: The first tank is connected to the input of the second tank, and the output of the second tank is connected to the input of the third tank.



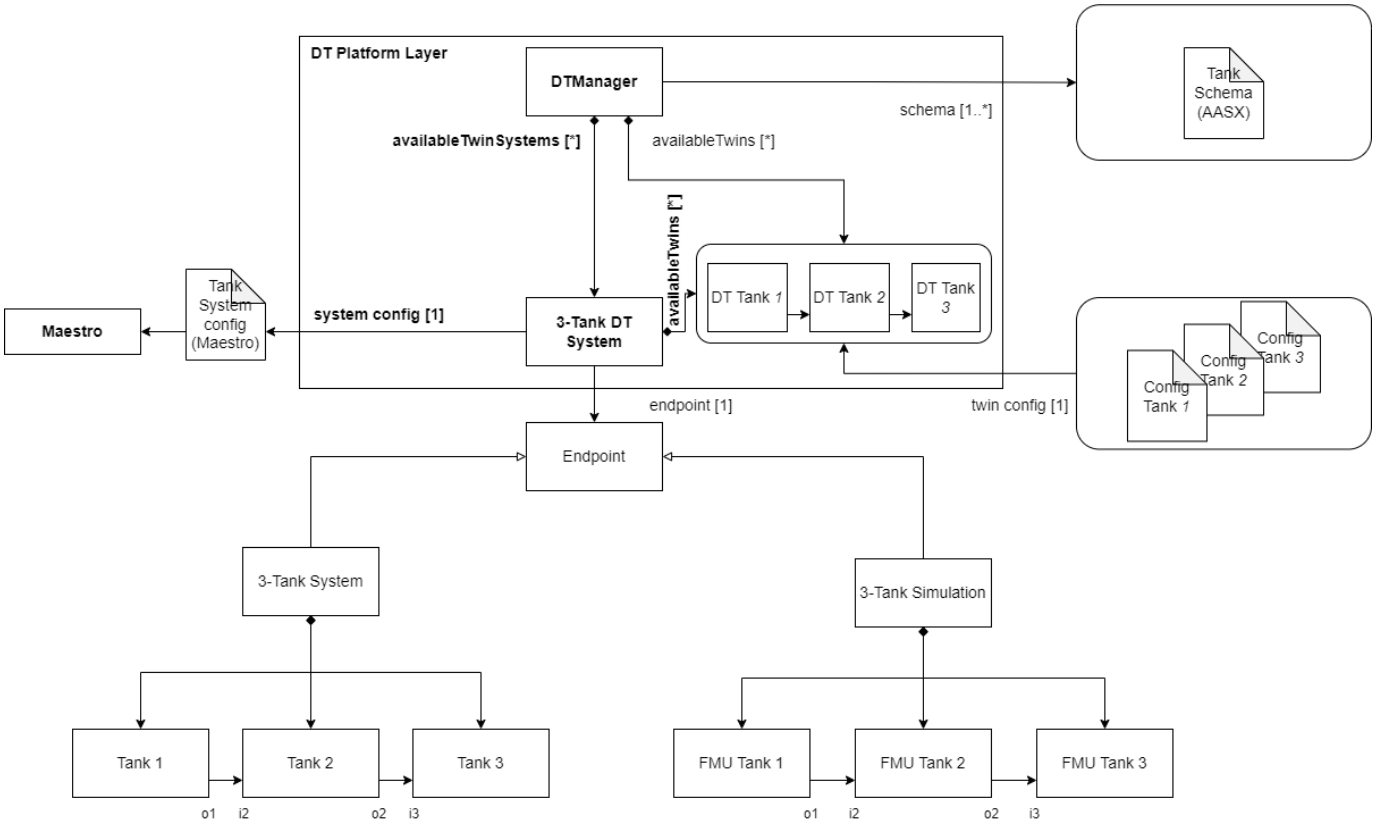
This example contains only the simulated components for demonstration purposes; therefore, there is no configuration for the connection with the physical system.

The three-tank system case study is managed using the `DTManager`, which is packed as a jar library in the tools, and run from a java main file. The `DTManager` uses Maestro as a slave for co-simulation, so it generates the output of the co-simulation.

The main file can be changed according to the application scope, i.e., the `/workspace/examples/tools/three-tank/TankMain.java` can be manipulated to get a different result.

The `/workspace/examples/models/three-tank/` folder contains the `Linear.fmu` file, which is a non-realistic model for a tank with input and output and the `TankSystem.aasx` file for the schema representation with Asset Administration Shell. The three instances use the same `.fmu` file and the same schema due to being of the same object class. The `DTManager` is in charge of reading the values from the co-simulation output.

Example Structure



Digital Twin Configuration

This example uses two models, two tools, one data, and one script. The specific assets used are:

Asset Type	Names of Assets	Visibility	Reuse in Other Examples
Model	Linear.fmu	Private	No
	TankSystem.aasx	Private	No
Tool	DTManager-0.0.1-Maestro.jar (wraps Maestro)	Common	Yes
	maestro-2.3.0-jar-with-dependencies.jar (used by DTManager)	Common	Yes
	TankMain.java (main script)	Private	No
Data	outputs.csv	Private	No

This DT has multiple configuration files. The *coe.json* and *multimodel.json* are used by Maestro tool. The *tank1.conf*, *tank2.conf* and *tank3.conf* are the config files for three different instances of one model (Linear.fmu).

Lifecycle Phases

The lifecycles that are covered include:

Lifecycle Phase	Completed Tasks
Create	Installs Java Development Kit for Maestro tool
Execute	The DT Manager executes the three-tank digital twin and produces output in `data/three-tank/output` directory
Terminate	Terminating the background processes and cleaning up the output

Run the example

To run the example, change the present directory.

```
cd /workspace/examples/digital twins/three-tank
```

If required, change the execute permission of lifecycle scripts that need to be executed, for example:

```
chmod +x lifecycle/create
```

Now, run the following scripts:

CREATE

Installs Open Java Development Kit 11 and pip dependencies. Also creates `DTManager` tool (DTManager-0.0.1-Maestro.jar) from source code.

```
lifecycle/create
```

EXECUTE

Execute the three-tank digital twin using DTManager. DTManager in-turn runs the co-simulation using Maestro. Generates the co-simulation output.csv file at `/workspace/examples/data/three-tank/output`.

```
lifecycle/execute
```

TERMINATE

Stops the Maestro running in the background. Also stops any other jvm process started during **execute** phase.

```
lifecycle/terminate
```

CLEAN

Removes the output generated during execute phase.

```
lifecycle/terminate
```

Examining the results

Executing this Digital Twin will generate a co-simulation output, but the results can also be monitored from updating the `/workspace/examples/tools/three-tank/TankMain.java` with a specific set of `getAttributeValue` commands, such as shown in the code.

That main file enables the online execution of the Digital Twin and its internal components.

The output of the co-simulation is generated to the `/workspace/examples/data/three-tank/output` folder.

In the default example, the co-simulation is run for 10 seconds in steps of 0.5 seconds. This can be modified for a longer period and different step size. The output stored in `outputs.csv` contains the level, in/out flow, and leak values.

No data from the physical twin are generated/used.

References

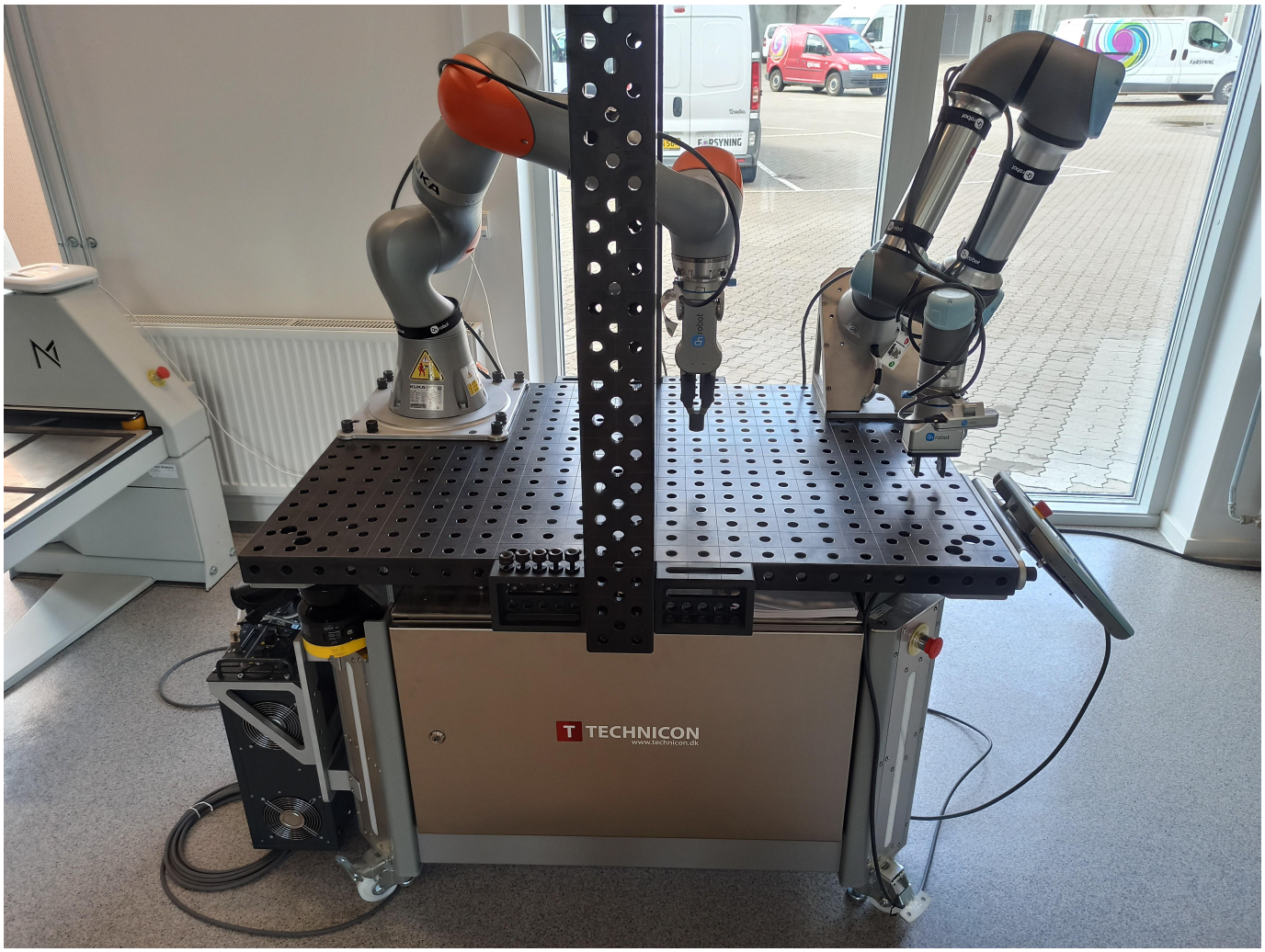
More information about the DT Manager is available at:

```
D. Lehner, S. Gil, P. H. Mikkelsen, P. G. Larsen and M. Wimmer,
"An Architectural Extension for Digital Twin Platforms to Leverage
Behavioral Models," 2023 IEEE 19th International Conference on
Automation Science and Engineering (CASE), Auckland, New Zealand,
2023, pp. 1-8, doi: 10.1109/CASE56687.2023.10260417.
```

3.8.8 Flex Cell Digital Twin with Two Industrial Robots

Overview

The flex-cell Digital Twin is a case study with two industrial robotic arms, a UR5e and a Kuka LBR iiwa 7, working in a cooperative setting on a manufacturing cell.



The case study focuses on the robot positioning in the discrete cartesian space of the flex-cell working space. Therefore, it is possible to send (X,Y,Z) commands to both robots, which refer to the target hole and height to which they should move.

The flex-cell case study is managed using the `TwinManager` (formerly `DT Manager`), which is packed as a jar library in the tools, and run from a java main file.

The `TwinManager` uses Maestro as a slave for co-simulation, so it generates the output of the co-simulation and can interact with the real robots at the same time (with the proper configuration and setup). The mainfile can be changed according to the application scope, i.e., the `/workspace/examples/tools/flex-cell/FlexCellDTaaS.java` can be manipulated to get a different result.

The `/workspace/examples/models/flex-cell/` folder contains the `.fmu` files for the kinematic models of the robotic arms, the `.urdf` files for visualization (including the grippers), and the `.aasx` files for the schema representation with Asset Administration Shell.

The case study also uses RabbitMQFMU to inject values into the co-simulation, therefore, there is the rabbitmqfmu in the models folder as well. Right now, RabbitMQFMU is only used for injecting values into the co-simulation, but not the other way around. The `TwinManager` is in charge of reading the values from the co-simulation output and the current state of the physical twins.

Example Structure

The example structure represents the components of the flex-cell DT implementation using the `TwinManager` architecture.

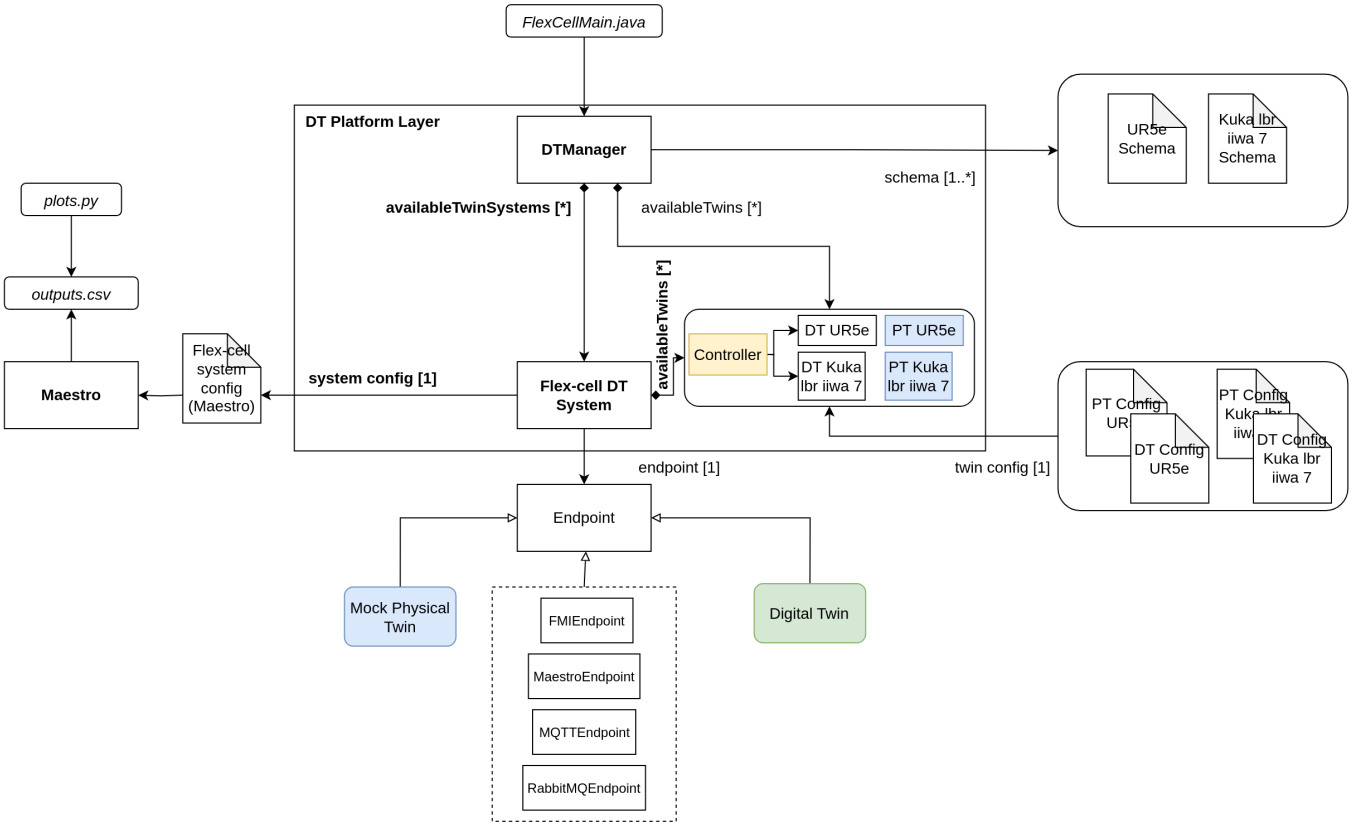
The TwinManager orchestrates the flex-cell DT via the *Flex-cell DT System*, which is composed of 2 smaller Digital Twins, namely, the *DT UR5e* and the *DT Kuka lbr iiwa 7*. The TwinManager also provides the interface for the Physical Twins, namely, *PT UR5e* and *PT Kuka lbr iiwa 7*. Each Physical Twin and Digital Twin System has a particular endpoint (with a different specialization), which is initialized from configuration files and data model (twin schema).

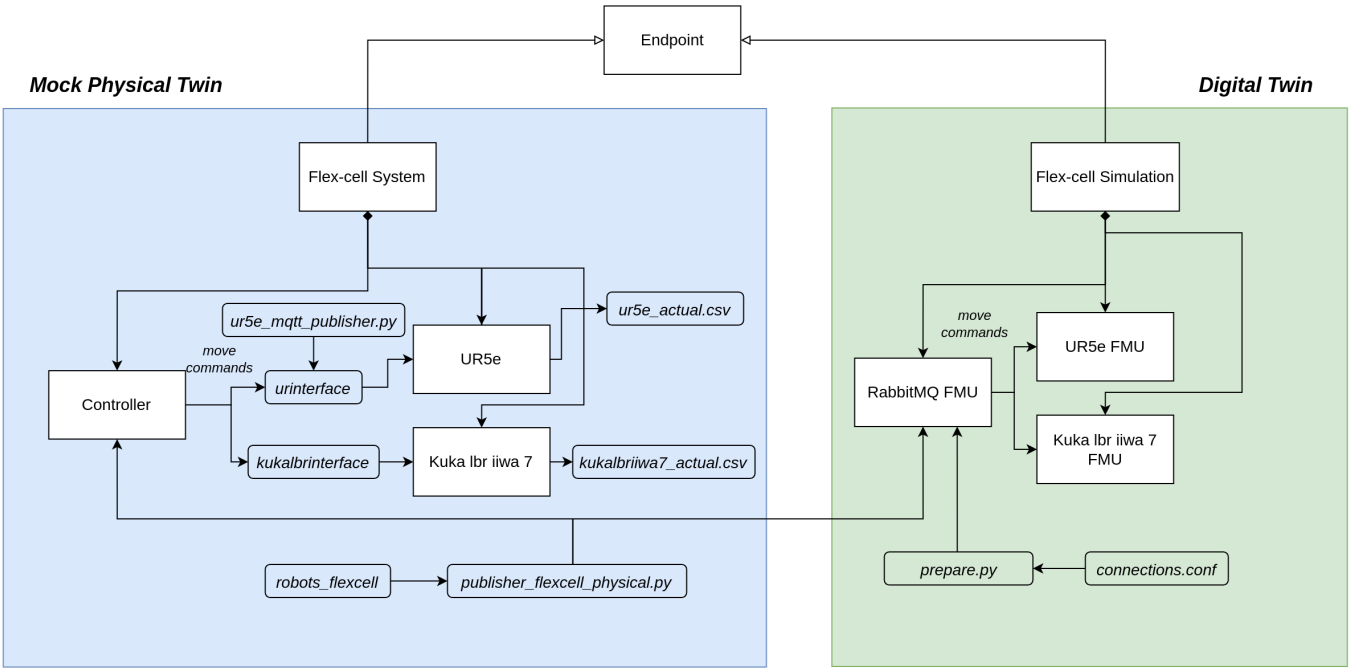
The current endpoints used in this implementation are:

Digital or Physical Twin	Endpoint
Flex-cell DT System	MaestroEndpoint
DT UR5e	FMIEndpoint
DT Kuka lbr iiwa 7	FMIEndpoint
PT UR5e	MQTTEndpoint and RabbitMQEndpoint
PT Kuka lbr iiwa 7	MQTTEndpoint and RabbitMQEndpoint

The Flex-cell DT System uses another configuration to be integrated with the Maestro co-simulation engine.

In the lower part, the Flex-cell System represents the composed physical twin, including the two robotic arms and controller and the Flex-cell Simulation is the mock-up representation for the real system, which is implemented by FMU blocks and their connections.





Digital Twin Configuration

This example uses seven models, five tools, six data files, two functions, and one script. The specific assets used are:

Asset Type	Names of Assets	Visibility	Reuse in Other Examples
Model	kukalbriiwa_model.fmu	Private	No
	kuka_irw_gripper_rg6.urdf	Private	No
	kuka.aasx	Private	No
	ur5e_model.fmu	Private	No
	ur5e_gripper_2fg7.urdf	Private	No
	ur5e.aasx	Private	No
	rmq-vhost.fmu	Private	Yes
Tool	maestro-2.3.0-jar-with-dependencies.jar	Common	Yes
	TwinManagerFramework-0.0.2.jar	Private	Yes
	urinterface (installed with pip)	Private	No
	kukalbrinterface	Private	No
	robots_flexcell	Private	No
	FlexCellDTaaS.java (main script)	Private	No
Data	publisher-flexcell-physical.py	Private	No
	ur5e_mqtt_publisher.py	Private	No
	connections.conf	Private	No
	outputs.csv	Private	No
	kukalbriiwa7_actual.csv	Private	No
	ur5e_actual.csv	Private	No
Function	plots.py	Private	No
	prepare.py	Private	No

Lifecycle Phases

The lifecycles that are covered include:

1. Installation of dependencies in the create phase.
2. Preparing the credentials for connections in the prepare phase.
3. Execution of the experiment in the execution phase.
4. Saving experiments in the save phase.
5. Plotting the results of the co-simulation and the real data coming from the robots in the analyze phase.
6. Terminating the background processes and cleaning up the outputs in the termination phase.

Lifecycle Phase	Completed Tasks
Create	Installs Java Development Kit for Maestro tool, Compiles source code of TwinManager to create a usable jar package (used as tool)
Prepare	Takes the RabbitMQ and MQTT credentials in connections.conf file and configures different assets of DT.
Execute	The TwinManager executes the flex-cell DT and produces output in `data/flex-cell/output` directory
Save	Save the experimental results
Analyze	Uses plotting functions to generate plots of co-simulation results
Terminate	Terminating the background processes
Clean	Cleans up the output data

Run the example

To run the example, change the present directory.

```
cd /workspace/examples/digital_twins/flex-cell
```

If required, change the execute permission of lifecycle scripts that need to be executed, for example:

```
chmod +x lifecycle/create
```

This example requires Java 11. The **create** script installs Java 11; however if other Java versions have already been installed, the default *java* may be pointing to another version. The default version can be checked and modified using the following commands.

```
java -version
update-alternatives --config java
```

Now, run the following scripts:

CREATE

Installs Open Java Development Kit 11 and a python virtual environment with pip dependencies. Also builds the `TwinManager` tool (TwinManagerFramework-0.0.2.jar) from source code.

```
lifecycle/create
```

PREPARE

This step configures different assets of the DT with connection credentials. The `functions/flex-cell/prepare.py` script is used for this purpose. The only step needed to set up the connection is to update the file `/workspace/examples/data/flex-cell/input/connections.conf` with the connection parameters for MQTT and RabbitMQ and then execute the `prepare` script.

```
lifecycle/prepare
```

The following files are updated with the configuration information:

1. `/workspace/examples/digital_twins/flex-cell/kuka_actual.conf`
2. `/workspace/examples/digital_twins/flex-cell/ur5e_actual.conf`
3. `/workspace/examples/data/flex-cell/input/publisher-flexcell-physical.py`
4. `modelDescription.xml` for the RabbitMQFMU require special credentials to connect to the RabbitMQ and the MQTT brokers.

EXECUTE

Execute the flex-cell digital twin using TwinManager. TwinManager in-turn runs the co-simulation using Maestro. Generates the co-simulation output.csv file at `/workspace/examples/data/flex-cell/output`. The execution needs to be stopped with `control + c` since the `TwinManager` runs the application in a non-stopping loop.

```
lifecycle/execute
```

SAVE

Each execution of the DT is treated as a single run. The results of one execution are saved as time-stamped co-simulation output file in The TwinManager executes the flex-cell digital twin and produces output in `data/flex-cell/output/saved_experiments` directory.

```
lifecycle/save
```

The `execute` and `save` scripts can be executed in that order any number of times. A new file `data/flex-cell/output/saved_experiments` directory with each iteration.

ANALYZE

There are dedicated plotting functions in `functions/flex-cell/plots.py`. This script plots the co-simulation results against the recorded values from the two robots.

```
lifecycle/analyze
```

TERMINATE

Stops the Maestro running in the background. Also stops any other jvm process started during **execute** phase.

```
lifecycle/terminate
```

CLEAN

Removes the output generated during execute phase.

```
lifecycle/clean
```

Examining the Results

Executing this Digital Twin generates a co-simulation output. The results can also be monitored by updating the `/workspace/examples/tools/flex-cell/FlexCellDTaaS.java` with a specific set of `getAttributeValue` commands, as shown in the code. That main file enables the online execution and comparison of Digital Twin and Physical Twin at the same time and at the same abstraction level.

The output is generated to the `/workspace/examples/data/flex-cell/output` folder. In case a specific experiments is to be saved, the **save** lifecycle script stores the co-simulation results into the `/workspace/examples/data/flex-cell/output/saved_experiments` folder.

In the default example, the co-simulation is run for 11 seconds in steps of 0.2 seconds. This can be modified for a longer period and different step size. The output stored in `outputs.csv` contains the joint position of both robotic arms and the current discrete (X,Y,Z) position of the TCP of the robot. Additional variables can be added, such as the discrete (X,Y,Z) position of the other joints.

When connected to the real robots, the tools `urinterface` and `kukalbrinterface` log their data at a higher sampling rate.

References

The RabbitMQ FMU github repository contains complete documentation and source code of the **rmq-vhost.fmu**.

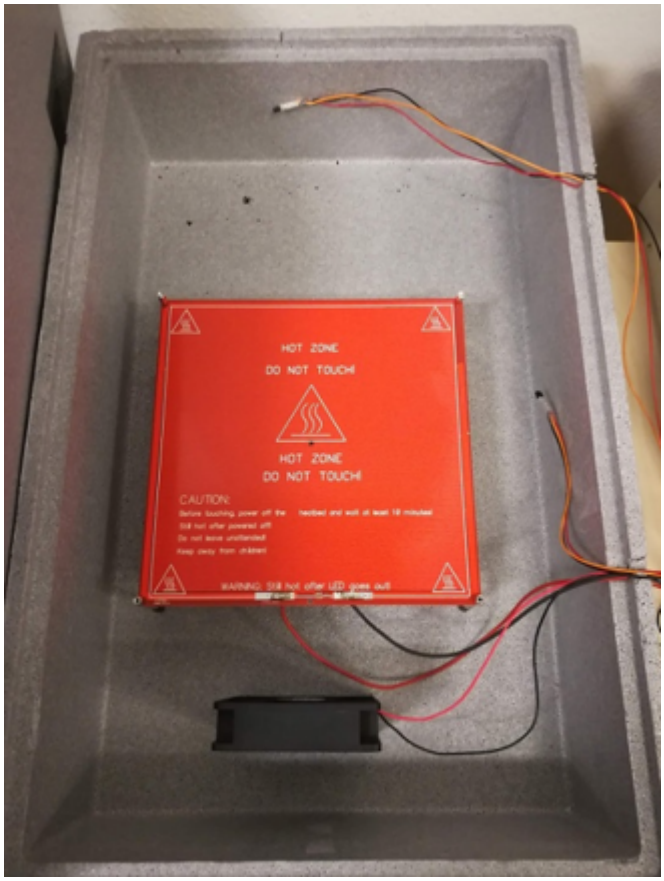
More information about the TwinManager (formerly DT Manager) and the case study is available in:

1. D. Lehner, S. Gil, P. H. Mikkelsen, P. G. Larsen and M. Wimmer, "An Architectural Extension for Digital Twin Platforms to Leverage Behavioral Models," 2023 IEEE 19th International Conference on Automation Science and Engineering (CASE), Auckland, New Zealand, 2023, pp. 1-8, doi: 10.1109/CASE56687.2023.10260417.
2. S. Gil, P. H. Mikkelsen, D. Tola, C. Schou and P. G. Larsen, "A Modeling Approach for Composed Digital Twins in Cooperative Systems," 2023 IEEE 28th International Conference on Emerging Technologies and Factory Automation (ETFA), Sinaia, Romania, 2023, pp. 1-8, doi: 10.1109/ETFA54631.2023.10275601.
3. S. Gil, C. Schou, P. H. Mikkelsen, and P. G. Larsen, “Integrating Skills into Digital Twins in Cooperative Systems,” in 2024 IEEE/SICE International Symposium on System Integration (SII), 2024, pp. 1124–1131, doi: 10.1109/SII58957.2024.10417610.

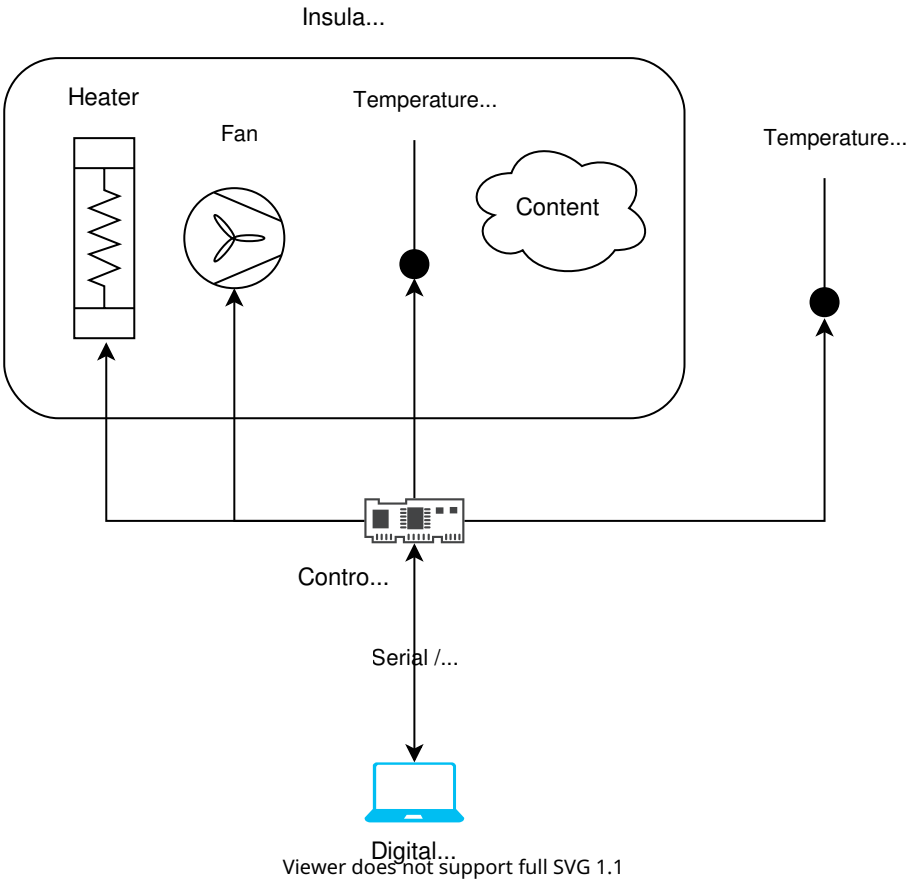
3.8.9 Incubator Digital Twin

Overview

This is a case study of an Incubator intended to demonstrate the steps and processes involved in developing a digital twin system. This incubator is an insulated container with the ability to maintain a temperature and provide heating, but not cooling. A picture of the incubator is provided below.



The overall purpose of the system is to reach a certain temperature within a box and keep the temperature regardless of content. An overview of the system can be seen below:



The system consists of:

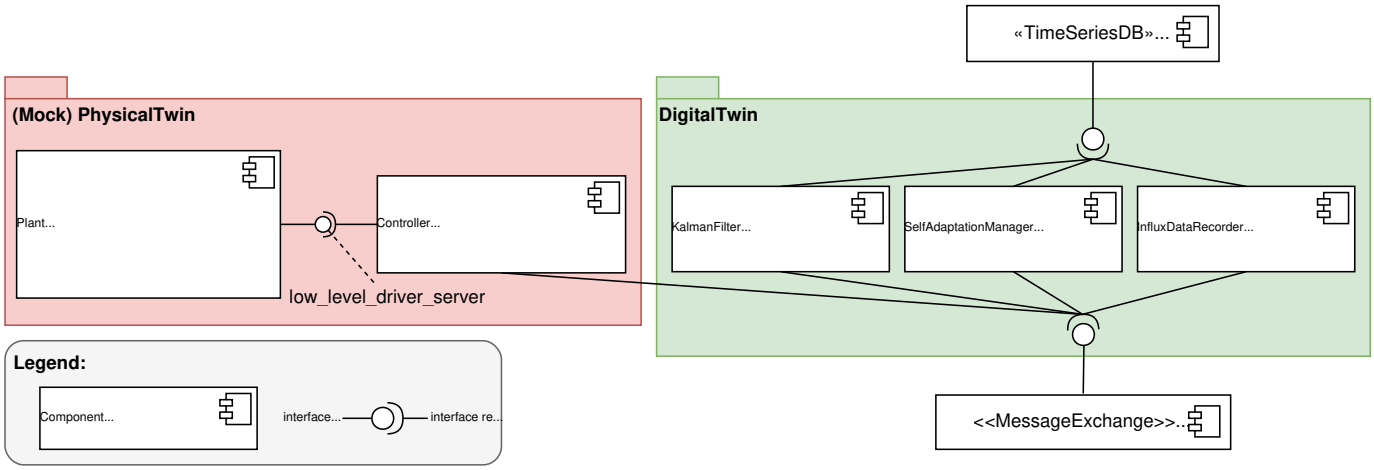
- 1x styrofoam box in order to have an insulated container
- 1x heat source to heat up the content within the Styrofoam box.
- 1x fan to distribute the heating within the box
- 2x temperature sensor to monitor the temperature within the box
- 1x temperature Sensor to monitor the temperature outside the box
- 1x controller to actuate the heat source and the fan and read sensory information from the temperature sensors, and communicate with the digital twin.

The original repository for the example can be found: Original repository. This trimmed version of the codebase does not have the following:

- docker support
- tests
- datasets

The original repository contains the complete documentation of the example, including the full system architecture, instructions for running with a physical twin, and instructions for running a 3D visualization of the incubator.

Digital Twin Structure



This diagrams shows the main components and the interfaces they use to communicate. All components communicate via the RabbitMQ message exchange, and the data is stored in a time series database. The RabbitMQ and InfluxDB are platform services hosted by the DTaaS.

The Incubator digital twin is a pre-packaged digital twin. It can be used as is or integrated with other digital twins.

The mock physical twin is executed from `incubator/mock_plant/real_time_model_solver.py` script.

Digital Twin Configuration

This example uses a plethora of Python scripts to run the digital twin. By default, it is configured to run with a mock physical twin. Furthermore, it depends on RabbitMQ and InfluxDB instances.

There is one configuration file: `simulation.conf`. The RabbitMQ and InfluxDB configuration parameters must be updated.

Lifecycle Phases

The lifecycles that are covered include:

Lifecycle Phase	Completed Tasks
Create	Potentially updates the system and installs Python dependencies
Execute	Executes the Incubator digital twin and produces output in the terminal and in *incubator/log.log*.
Clean	Removes the log file.

Run the example

To run the example, change the present directory.

```
cd /workspace/examples/digital_twins/incubator
```

If required, change the execute permission of lifecycle scripts that need to be executed, for example:

```
chmod +x lifecycle/create
```

Now, run the following scripts:

CREATE

Potentially updates the system and installs Python dependencies.

```
lifecycle/create
```

EXECUTE

Executes the Incubator digital twin with a mock physical twin. Pushes the results in the terminal, *incubator/log.log*, and in InfluxDB.

```
lifecycle/execute
```

CLEAN

Removes the output log file.

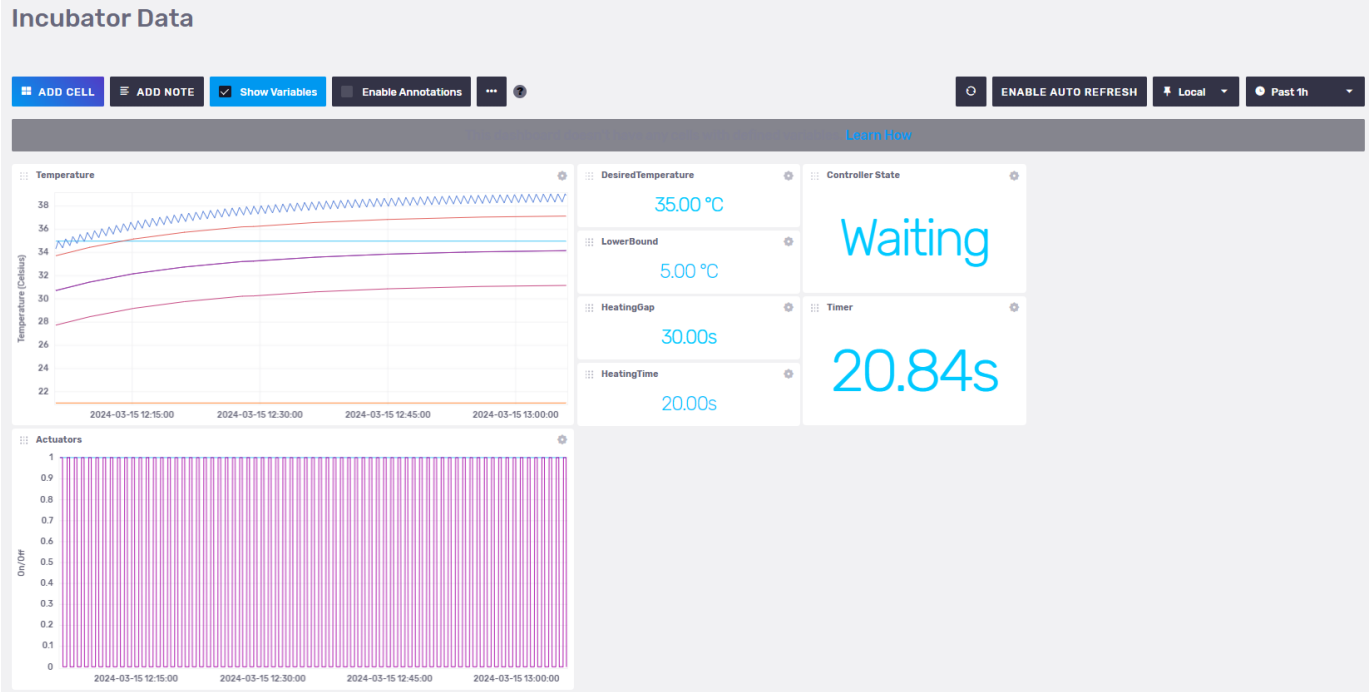
```
lifecycle/clean
```

Examining the results

After starting all services successfully, the controller service will start producing output that looks like the following:

```
time           execution_interval  elapsed  heater_on  fan_on   room   box_air_temperature  state
19/11 16:17:59  3.00                0.01     True       False    10.70  19.68                Heating
19/11 16:18:02  3.00                0.03     True       True     10.70  19.57                Heating
19/11 16:18:05  3.00                0.01     True       True     10.70  19.57                Heating
19/11 16:18:08  3.00                0.01     True       True     10.69  19.47                Heating
19/11 16:18:11  3.00                0.01     True       True     10.69  19.41                Heating
```

An InfluxDB dashboard can be setup based on `incubator/digital_twin/data_access/influxdbserver/dashboards/incubator_data.json`. If the dashboard on the InfluxDB is setup properly, the following visualization can be seen:



References

📋 Forked from: Incubator repository with commit ID: 989ccf5909a684ad26a9c3ec16be2390667643aa

To understand what a digital twin is, one or more of the following resources are recommended:

1. Feng, Hao, Cláudio Gomes, Casper Thule, Kenneth Lausdahl, Alexandros Iosifidis, and Peter Gorm Larsen. “Introduction to Digital Twin Engineering.” In 2021 Annual Modeling and Simulation Conference (ANNSIM), 1–12. Fairfax, VA, USA: IEEE, 2021. https://doi.org/10.23919/ANNSIM52504.2021.9552135.
2. Video recording of presentation by Claudio Gomes

3.8.10 Firefighter Mission in a Burning Building

In a firefighter mission, it is important to monitor the oxygen levels of each firefighter's Self Contained Breathing Apparatus (SCBA) in the context of their mission.

Physical Twin Overview

Image: Schematic overview of a firefighter mission. Note the mission commander on the lower left documenting the air supply pressure levels provided by radio communication from the firefighters inside and around the burning building. This image was created with the assistance of DALL·E.

The following scenario is assumed:

- a set of firefighters work to extinguish a burning building
- they each use an SCBA with pressurised oxygen to breath
- a mission commander on the outside coordinates the efforts and surveills the oxygen levels

Digital Twin Overview

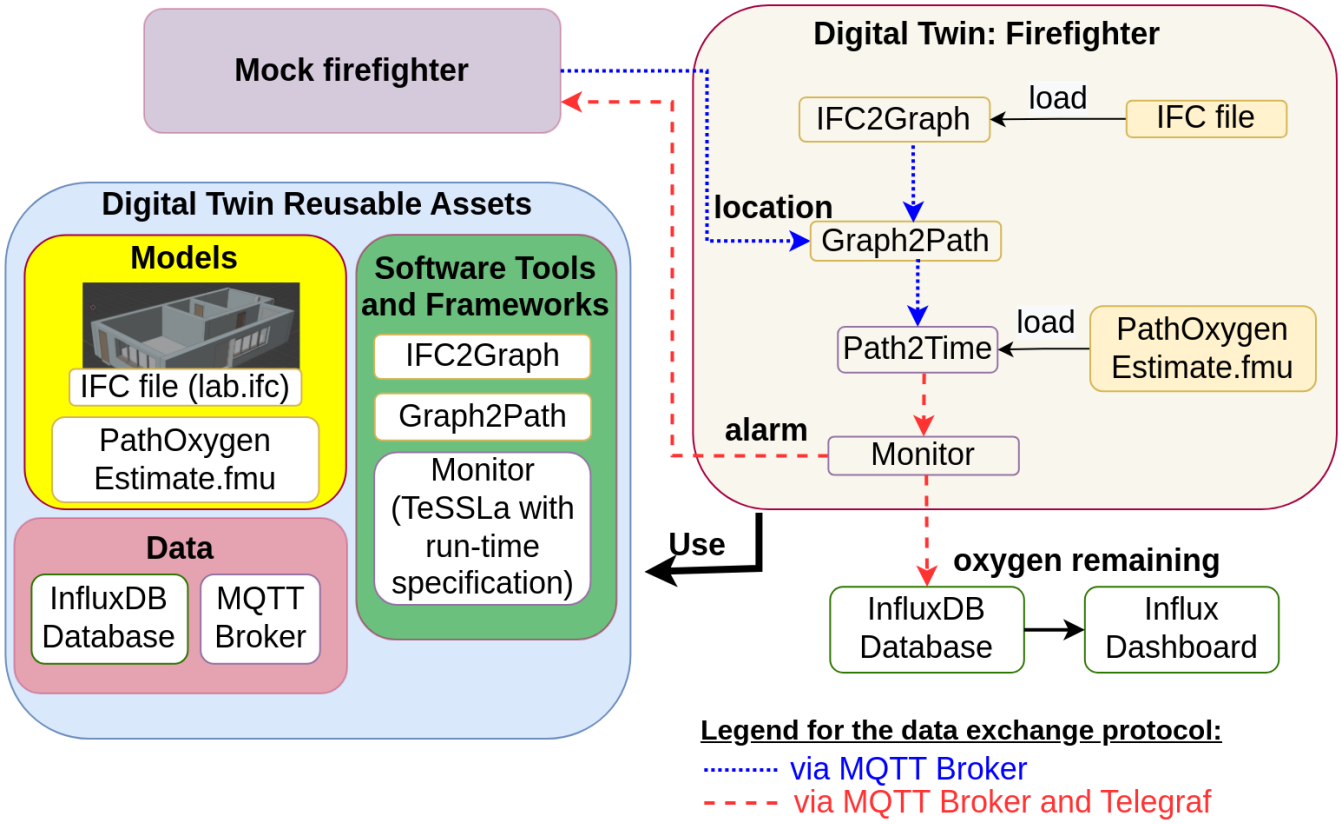
In this example a monitor is implemented, that calculates how much time the firefighers have left, until they need to leave the building. To that end, the inputs used are:

- 3D-model of the building in which the mission takes place,
- pressure data of a firefighters SCBA and
- firefighters location inside of the building

are used to estimate:

- the shortest way out,
- how much time this will need and
- how much time is left until all oxygen from the SCBA is used up.

The remaining mission time is monitored and the firefighter receive a warning if it drops under a certain threshold.



This example is an implementation of the the paper *Digital Twin for Rescue Missions--a Case Study* by Leucker et al.

QUICK CHECK

Before running this example, the following files must be verified to be at the correct locations:

```
/workspace/examples/
   data/o5g/input/
      runTessla.sh
      sensorSimulation.py
      telegraf.conf

   models/
    lab.ifc
    makefmu.mos
      PathOxygenEstimate.mo

  tools/
    graphToPath.py
    ifc_to_graph
    pathToTime.py
    tessla-telegraf-connector/
    tessla-telegraf-connector/
      tessla.jar
      specification.tessla (run-time specification)

   digital_twins/o5g/
      main.py
      config
      lifecycle/ (scripts)
```

DIGITAL TWIN CONFIGURATION

All configuration for this example is contained in `digital_twins/o5g/config`.

To use the MQTT-Server, account information needs to be provided. The topics are set to their default values, which allow the DT to access the mock physical twins sensor metrics and to send back alerts.

```
export O5G_MQTT_SERVER=
export O5G_MQTT_PORT=
export O5G_MQTT_USER=
export O5G_MQTT_PASS=

export O5G_MQTT_TOPIC_SENSOR='vgiot/ue/metric'
export O5G_MQTT_TOPIC_AIR_PREDICTION='vgiot/dt/prediction'
export O5g_MQTT_TOPIC_ALERT='vgiot/dt/alerts'
```

This example uses InfuxDB as a data storage, which will need to be configured to use the relevant access data. The following configuration steps are needed:

- Log into the InfluxDB Web UI
- Obtain **org** name (below the *username* in the sidebar)
- Create a data bucket if one does not already exist in `Load Data -> Buckets`
- Create an API access token in `Load Data -> API Tokens`, Copy and save this token immediately; it cannot be accessed subsequently.

```
export O5G_INFLUX_SERVER=
export O5G_INFLUX_PORT=
export O5G_INFLUX_TOKEN=
export O5G_INFLUX_ORG=
export O5G_INFLUX_BUCKET=
```

Lifecycle Phases

The lifecycles that are covered include:

Lifecycle Phase	Completed Tasks
Install	Installs Open Modelica, Rust, Telegraf and the required pip dependencies
Create	Create FMU from Open Modelica file
Execute	Execute the example in the background tmux terminal session
Terminate	Terminate the tmux terminal session running in the background
Clean	Delete the temporary files

Run the example

INSTALL

Run the install script by executing

```
lifecycle/install
```

This will install all the required dependencies from apt and pip, as well as Open Modelica, Rust, Telegraf and the required pip dependencies from their respective repos.

Create

Run the create script by executing

```
lifecycle/create
```

This will compile the modelica model to an Functional Mockup Unit (FMU) for the correct platform.

Exceute

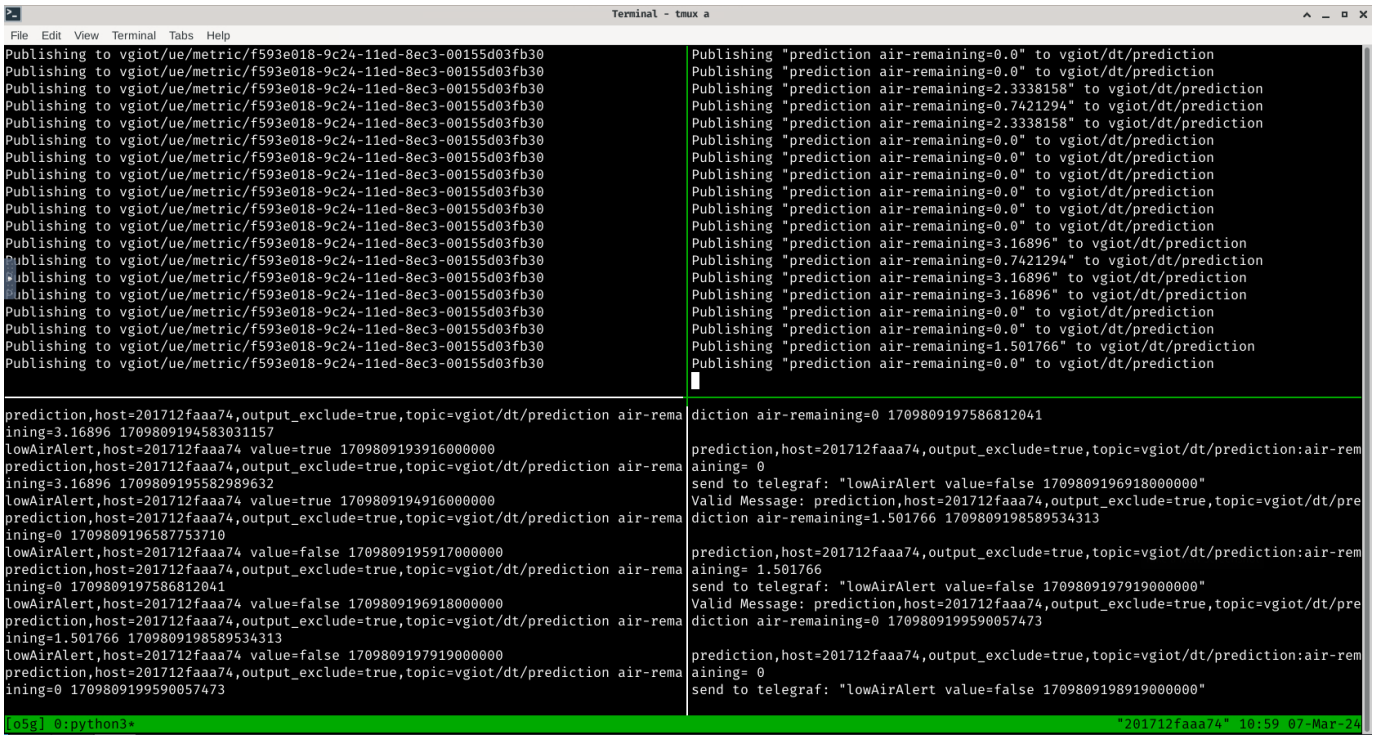
To run the Digital Twin execute

```
lifecycle/execute
```

This will start all the required components in a single tmux session called `o5g` in the background. To view the running Digital Twin attatch to this tmux session by executing

```
tmux a -t o5g
```

To detatch press `Ctrl-b` followed by `d`.



The *tmux* session contains 4 components of the digital twin:

Panel location	Purpose
Top Left	Sensor simulator generating random location and O2-level data
Top Right	Main Digital Twin receives the sensor data and calculates an estimate of how many minutes of air remain
Bottom Left	Telegraf to convert between different message formats, also displays all messages between components
Bottom Right	TeSSLa monitor raises an alarm, if the remaining time is to low.

Examine the Results

For additional mission awareness, it is recommended to utilise the Influx data visualisation. A dashboard configuration is provided in the file *influx-dashoard.json*. Log in to the Influx Server to import (usually port 8086). A screenshot of the dashboard is given here.

The data gets stored in `o5g->prediction->air-remaining->37ae3e4fb3ea->true->vgiot/dt/prediction` variable of the InfluxDB. In addition to importing dashboard configuration given above, it is possible to create custom dashboards using the stored data.

Terminate

To stop the all components and close the *tmux* session execute

```
lifecycle/terminate
```

Clean

To remove temoporary files created during execution

```
lifecycle/clean
```

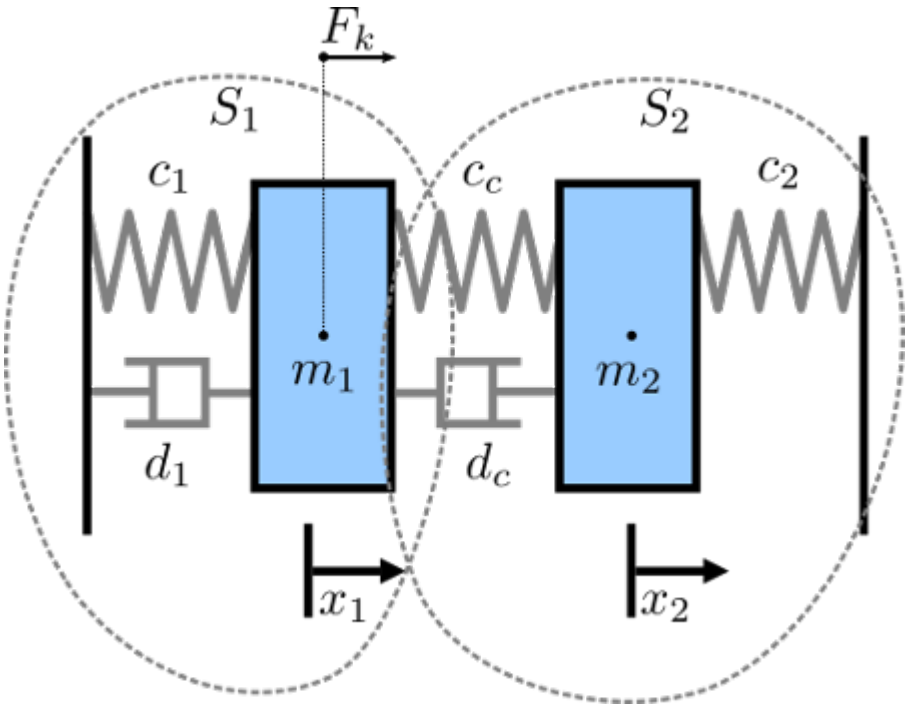
3.8.11 Mass Spring Damper with NuRV Runtime Monitor

Overview

This digital twin is derived from the Mass Spring Damper digital twin.

The mass spring damper digital twin (DT) comprises two mass spring dampers and demonstrates how a co-simulation based DT can be used within the DTaaS. This version of the example is expanded with a monitor generated by NuRV. More information about NuRV is available here.

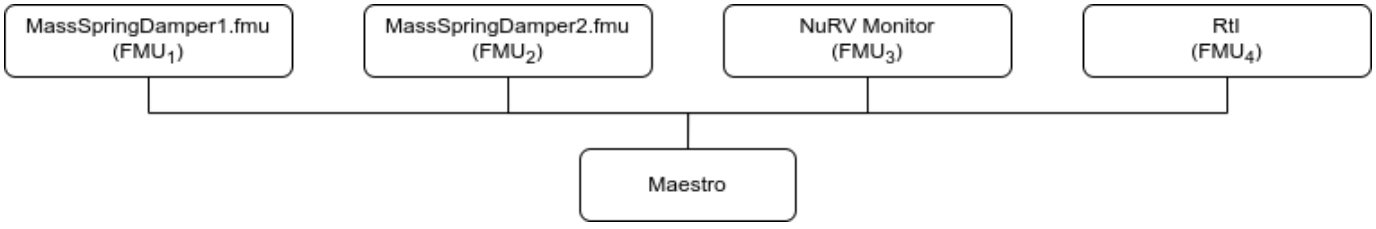
Example Diagram



Example Structure

There are two simulators included in the study, each representing a mass spring damper system. The first simulator calculates the mass displacement and speed of for a given force acting on mass . The second simulator calculates force given a displacement and speed of mass . By coupling these simulators, the evolution of the position of the two masses is computed.

Additionally, a monitor is inserted in the simulation to check at runtime whether the displacement of the two masses stays below a maximum threshold.



Digital Twin Configuration

This example uses two models and one tool. The specific assets used are:

Asset Type	Names of Assets	Visibility	Reuse in Other Examples
Models	MassSpringDamper1.fmu	Private	Yes
	MassSpringDamper2.fmu	Private	Yes
	m2.fmu	Private	No
	RtI.fmu	Private	Yes
Specification	m2.smv	Private	No
Tool	maestro-2.3.0-jar-with-dependencies.jar	Common	Yes

The `co-sim.json` and `time.json` are two DT configuration files used for executing the digital twin. These two files can be modified to customise the DT for specific requirements.

Lifecycle Phases

Lifecycle Phase	Completed Tasks
Create	Installs Java Development Kit for Maestro tool Generates and compiles the monitor FMU
Execute	Produces and stores output in data/mass-spring-damper-monitor/output directory
Clean	Clears run logs and outputs

Run the example

To run the example, change the present directory.

```
cd /workspace/examples/digital_twins/mass-spring-damper-monitor
```

If required, change the execute permission of lifecycle scripts that need to be executed, for example:

```
chmod +x lifecycle/create
```

Now, run the following scripts:

CREATE

- Installs Open Java Development Kit 17 in the workspace.
- Generates and compiles the monitor FMU from the NuRV specification

```
lifecycle/create
```

EXECUTE

Run the the Digital Twin. Since this is a co-simulation based digital twin, the Maestro co-simulation tool executes co-simulation using the two FMU models.

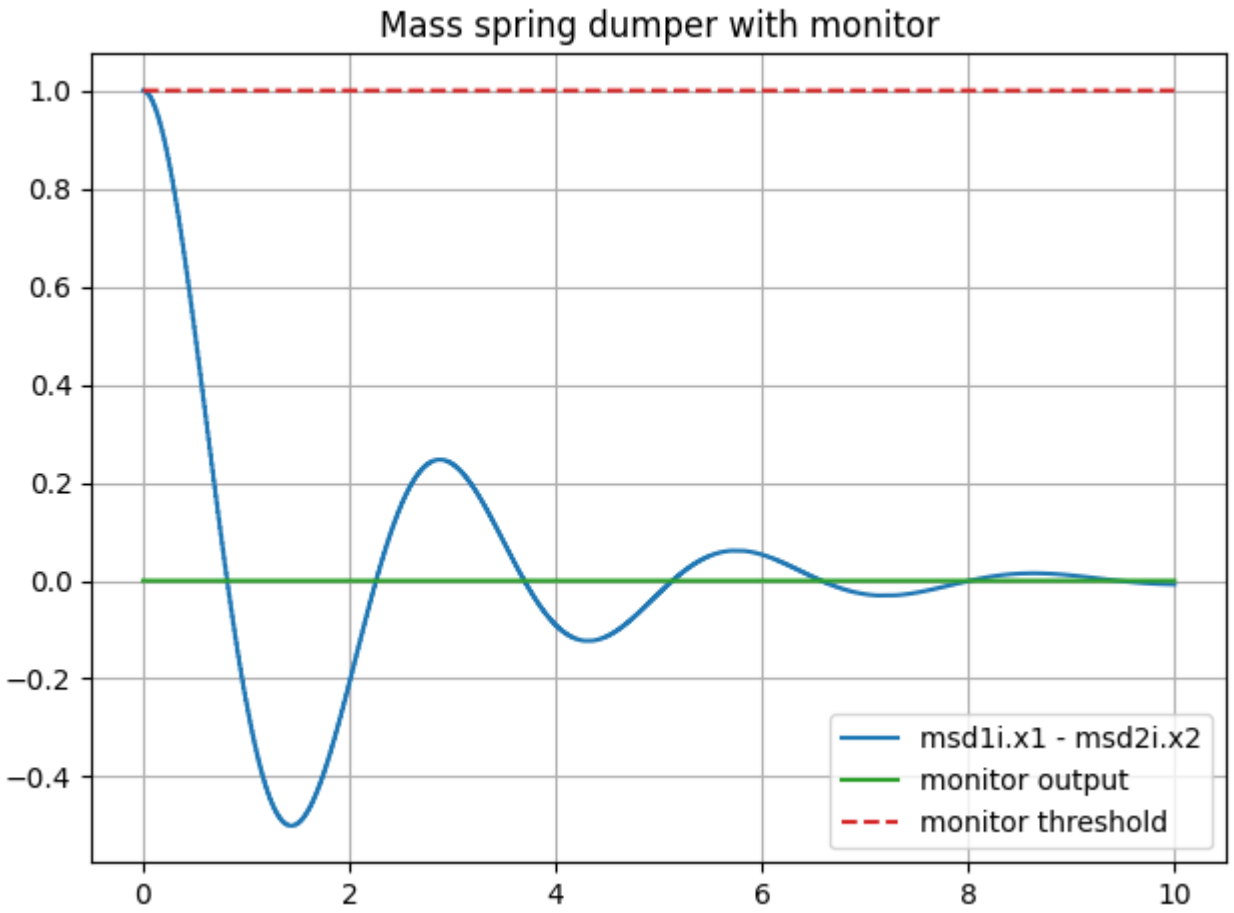
```
lifecycle/execute
```

ANALYZE PHASE

Process the output of co-simulation to produce a plot at: `/workspace/examples/data/mass-spring-damper-monitor/output/plots`.

```
lifecycle/analyze
```

A sample plot is given here.



In the plot, three colour-coded indicators are used to represent different values. The blue line shows the distance between the two masses, while the green indicates the monitor's verdict. A red dashed line serves as a reference point, marking the distance checked by the monitor. Since the distance of the masses is always below the threshold, the output of the monitor is fixed to `unknown` (`0`).

Examine the results

The results can be found in the */workspace/examples/data/mass-spring-damper-monitor/output directory*.

Run logs can also be viewed in the */workspace/examples/digital_twins/mass-spring-damper-monitor*.

TERMINATE PHASE

Terminate to clean up the debug files and co-simulation output files.

```
lifecycle/terminate
```

References

More information about co-simulation techniques and mass spring damper case study are available in:

```
Gomes, Cláudio, et al. "Co-simulation: State of the art."
arXiv preprint arXiv:1702.00686 (2017).
```

The source code for the models used in this DT are available in mass spring damper github repository.

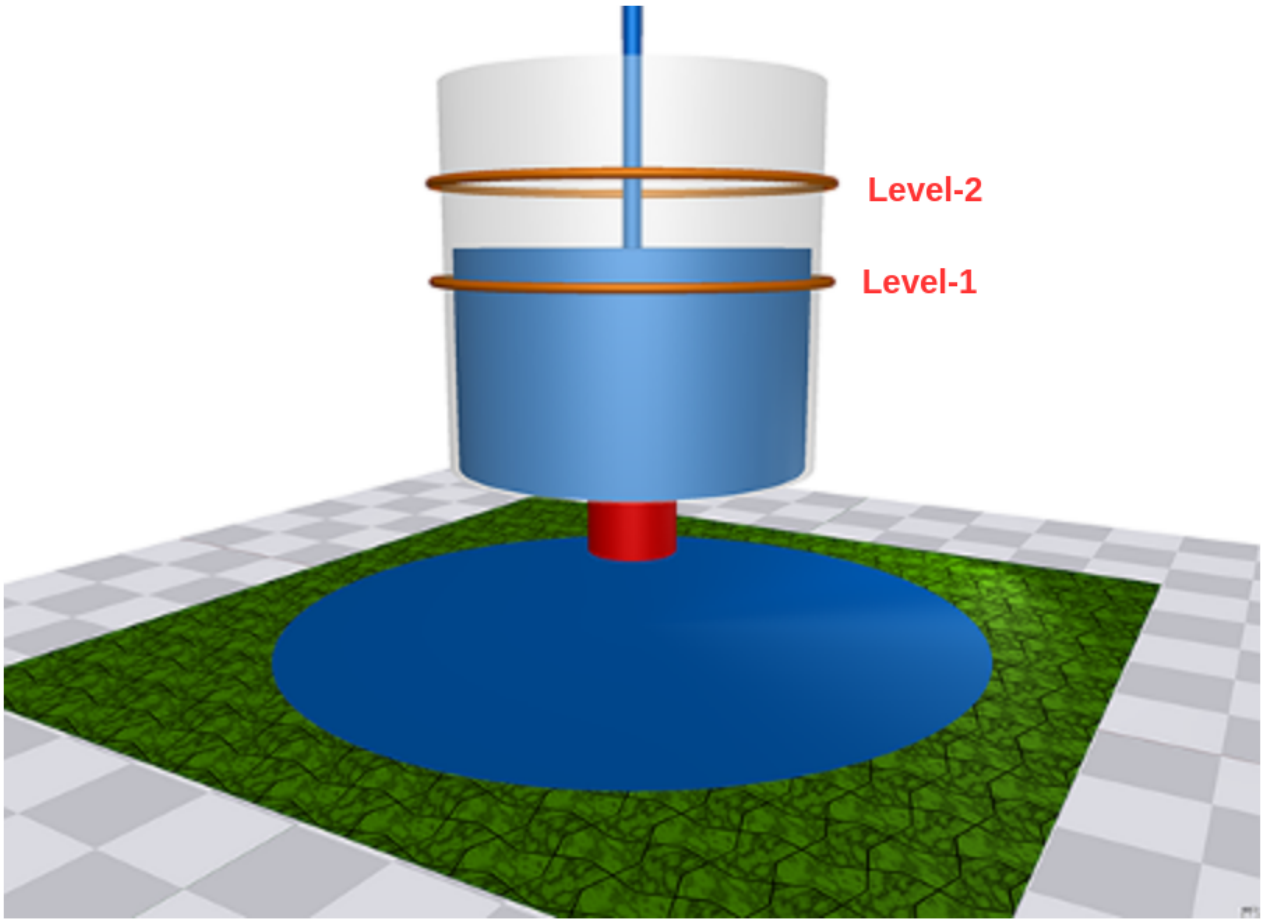
3.8.12 Water Tank Fault Injection with NuRV Runtime Monitor

Overview

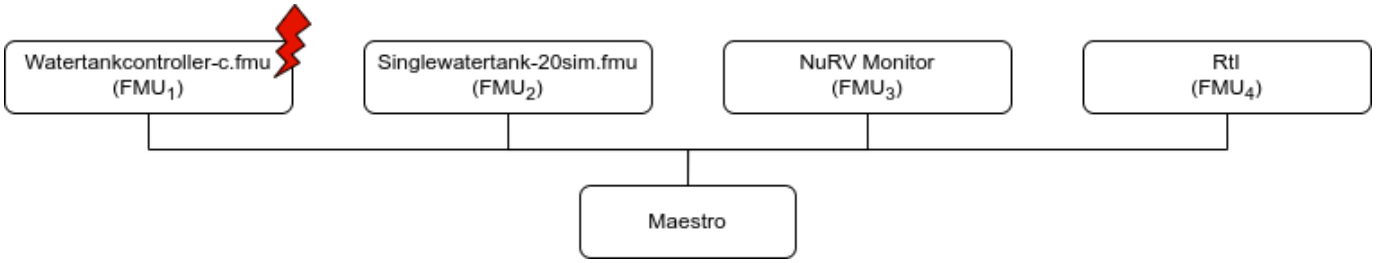
This example demonstrates a fault injection (FI) enabled digital twin. A live DT is subjected to simulated faults received from the environment. The simulated faults are specified as part of DT configuration and can be changed for new instances of DTs. This version of the example is expanded with a monitor generated by NuRV. More information about NuRV is available here.

In this co-simulation based DT, a watertank case-study is used; co-simulation consists of a tank and controller. The goal of which is to keep the level of water in the tank between `Level-1` and `Level-2`. The faults are injected into output of the water tank controller (**Watertankcontroller-c.fmu**) from 12 to 20 time units, such that the tank output is closed for a period of time, leading to the water level increasing in the tank beyond the desired level (`Level-2`). Additionally, a monitor is inserted in the simulation to check at runtime whether the level of the water stays below a maximum threshold.

Example Diagram



Example Structure



Digital Twin Configuration

This example uses two models and one tool. The specific assets used are:

Asset Type	Names of Assets	Visibility	Reuse in Other Examples
Models	watertankcontroller-c.fmu	Private	Yes
	singlewatertank-20sim.fmu	Private	Yes
	m1.fmu	Private	No
	RtI.fmu	Private	Yes
Specification	m1.smv	Private	No
Tool	maestro-2.3.0-jar-with-dependencies.jar	Common	Yes

The `multimodelFI.json` and `simulation-config.json` are two DT configuration files used for executing the digital twin. These two files can be modified to customise the DT for specific requirements.

The faults are defined in **wt_fault.xml**.

Lifecycle Phases

Lifecycle Phase	Completed Tasks
Create	Installs Java Development Kit for Maestro tool Generates and compiles the monitor FMU
Execute	Produces and stores output in data/water_tank_FI_monitor/output directory
Clean	Clears run logs and outputs

Run the example

To run the example, change the present directory.

```
cd /workspace/examples/digital_twins/water_tank_FI_monitor
```

If required, change the execute permission of lifecycle scripts that need to be executed, for example:

```
chmod +x lifecycle/create
```

Now, run the following scripts:

CREATE

Installs Open Java Development Kit 17 and pip dependencies. The pandas and matplotlib are the pip dependencies installated. The monitor FMU from the NuRV specification is generated and compiled.

```
lifecycle/create
```

EXECUTE

Run the co-simulation. Generates the co-simulation output.csv file at `/workspace/examples/data/water_tank_FI_monitor/output`.

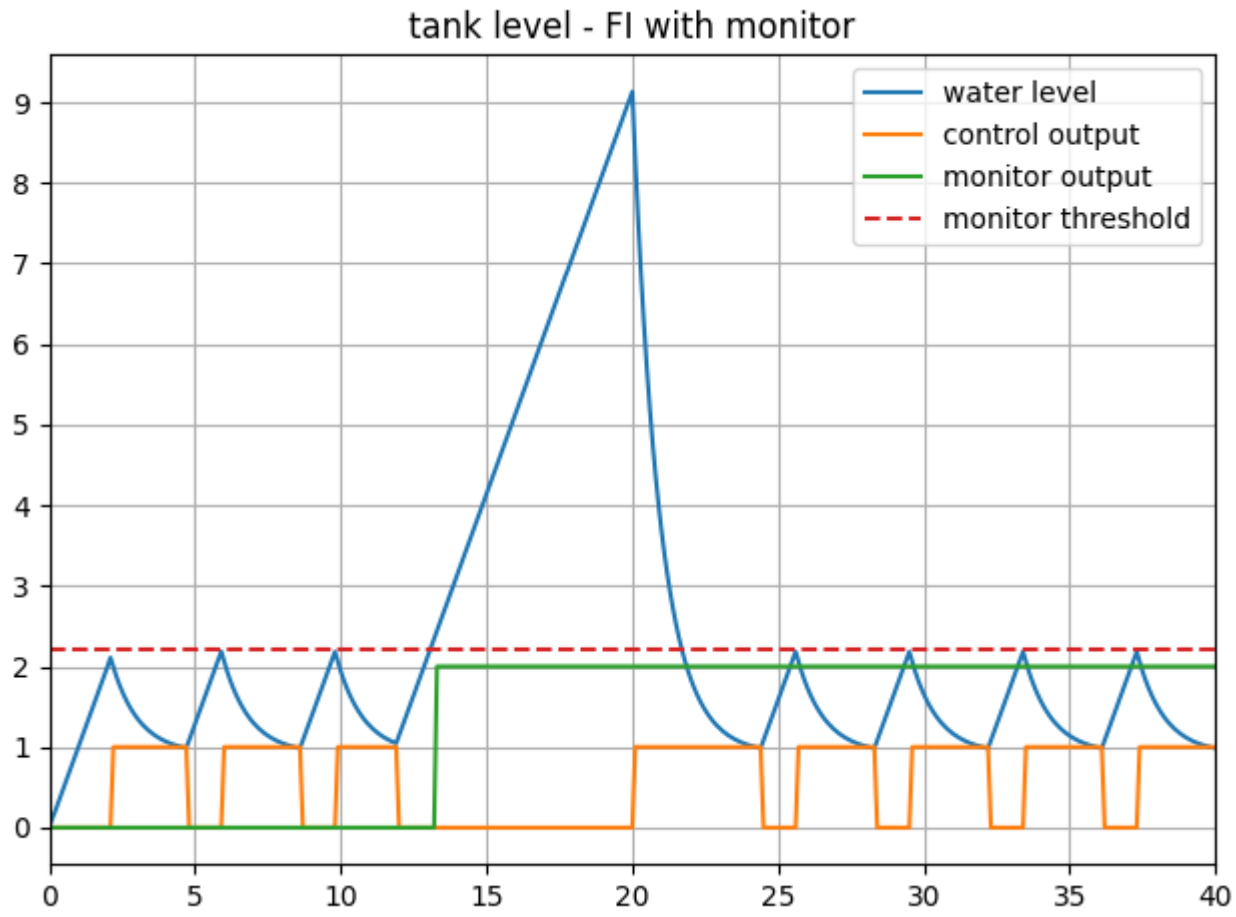
```
lifecycle/execute
```

ANALYZE PHASE

Process the output of co-simulation to produce a plot at: `/workspace/examples/data/water_tank_FI_monitor/output/plots/`.

```
lifecycle/analyze
```

A sample plot is given here.



In the plot, four colour-coded indicators are used to represent different values. The blue line shows the water tank level, while orange represents the control output and green indicates the monitor's verdict. A red dashed line serves as a reference point, marking the level checked by the monitor. As the water level exceeds this threshold, the monitor's verdict changes from `unknown` (`0`) to `false` (`2`).

Examine the results

The results can be found in the */workspace/examples/data/water_tank_FI_monitor/output directory*.

Run logs can also be viewed in the */workspace/examples/digital_twins/water_tank_FI_monitor*.

TERMINATE PHASE

Clean up the temporary files and delete output plot

```
lifecycle/terminate
```

References

More details on this case-study can be found in the paper:

```
M. Frasheri, C. Thule, H. D. Macedo, K. Lausdahl, P. G. Larsen and
L. Esterle, "Fault Injecting Co-simulations for Safety,"
2021 5th International Conference on System Reliability and Safety (ICSRS),
Palermo, Italy, 2021.
```

The fault-injection plugin is an extension to the Maestro co-orchestration engine that enables injecting inputs and outputs of FMUs in an FMI-based co-simulation with tampered values. More details on the plugin can be found in fault injection git repository. The source code for this example is also in the same github repository in a example directory.

3.8.13 Incubator Co-Simulation Digital Twin validation with NuRV Monitor

Overview

This example demonstrates how to validate some digital twin components using FMU monitors (in this example, the monitors are generated with NuRV[1]).

Simulated scenario

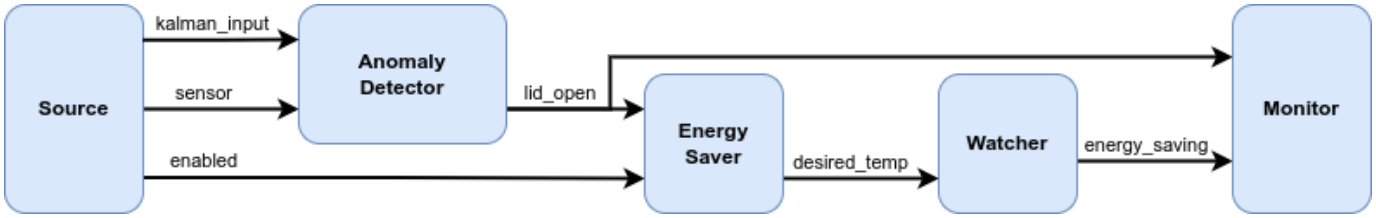
This example validates some components of the Incubator digital twin, by performing a simulation in which the commponents are wrapped inside FMUs, and are then inspected at runtime by some a FMU monitor generated by NuRV. Please note that the link to Incubator digital twin is only provided to know the details of the incubator physical twin. The digital twin (DT) presented here is a co-simulation DT of the Incubator.

The input data for the simulation is generated by a purpose-built FMU component named *source*, which supplies testing data to the *anomaly detector*, simulating an anomaly occurring at time t=60s. An additional component, *watcher*, is employed to verify whether the *energy saver* activates in response to an anomaly reported by the *anomaly detector*.

The output of the watcher is the passed to the monitor, which ensures that when an anomaly is detected, the *energy saver* activates within a maximum of three simulation cycles.

Example structure

A diagram depicting the logical software structure of the example can be seen below.



Digital Twin Configuration

The example uses the following assets:

Asset Type	Names of Assets	Visibility	Reuse in other Examples
Models	anomaly_detection.fmu	Private	No
	energy_saver.fmu	Private	No
	Source.fmu	Private	No
	Watcher.fmu	Private	No
Specification	safe-operation.smv	Private	No
Tool	maestro-2.3.0-jar-with-dependencies.jar	Common	Yes

The *safe-operation.smv* file contains the default monitored specification as described in the Simulated scenario section.

Lifecycle phases

The lifecycle phases for this example include:

Lifecycle Phase	Completed Tasks
Create	Installs Java Development Kit for Maestro tool Generates and compiles the monitor FMU
Execute	Produces and stores output in data/incubator-NuRV-monitor-validation/output directory
Clean	Clears run logs and outputs

If required, change the execute permissions of lifecycle scripts that need to be executed. This can be done using the following command

```
chmod +x lifecycle/{script}
```

where {script} is the name of the script, e.g. *create*, *execute* etc.

Run the example

To run the example, change the present directory.

```
cd /workspace/examples/digital_twins/incubator-NuRV-monitor-validation
```

If required, change the execute permission of lifecycle scripts that need to be executed, for example:

```
chmod +x lifecycle/create
```

Now, run the following scripts:

CREATE

- Installs Open Java Development Kit 17 in the workspace.
- Generates and compiles the monitor FMU from the NuRV specification

```
lifecycle/create
```

EXECUTE

Run the the Digital Twin. Since this is a co-simulation based digital twin, the Maestro co-simulation tool executes co-simulation using the FMU models.

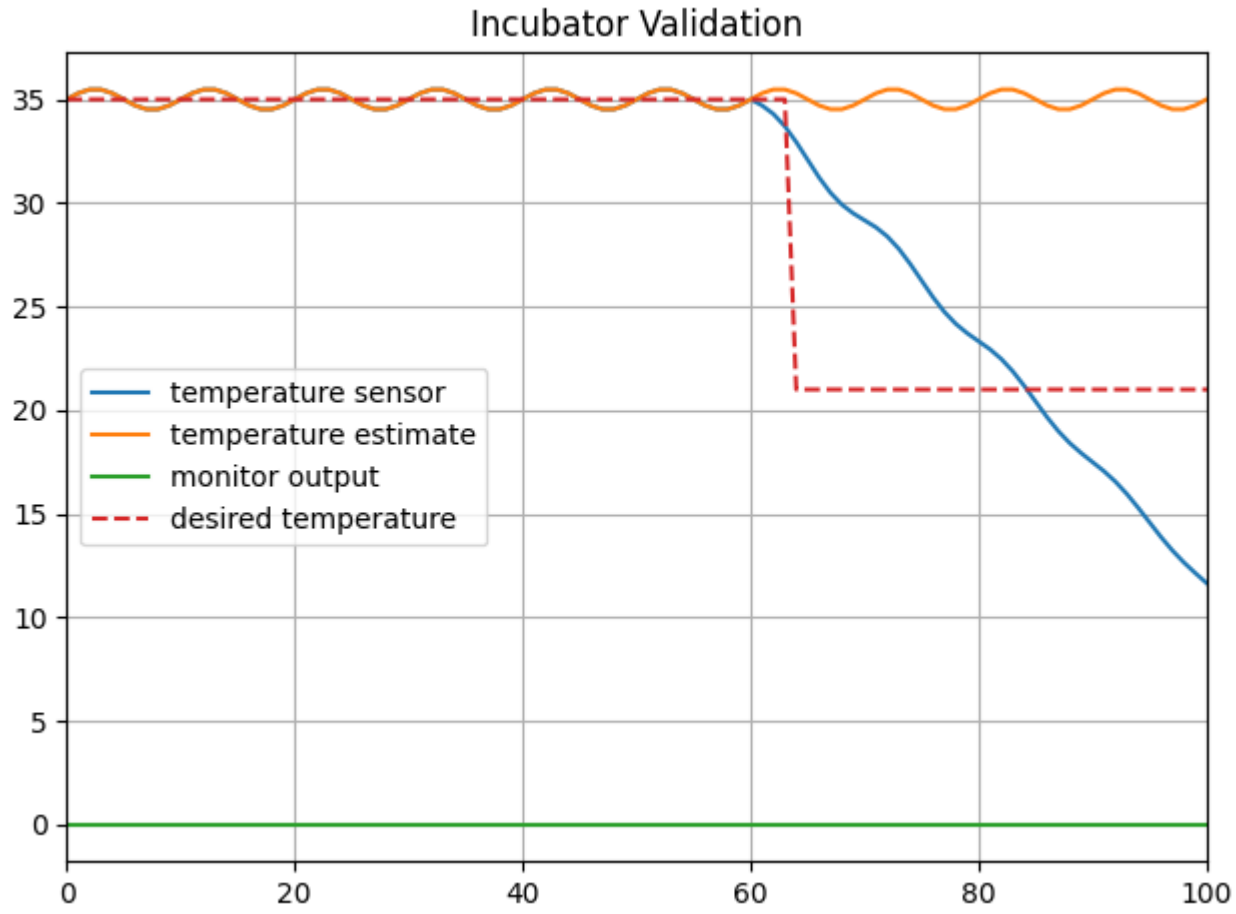
```
lifecycle/execute
```

ANALYZE PHASE

Process the output of co-simulation to produce a plot at: `/workspace/examples/data/incubator-NuRV-monitor-validation/output/plots`.

```
lifecycle/analyze
```

A sample plot is given here.



In the plot, four colour-coded indicators provide a visual representation of distinct values. The blue line depicts the simulated temperature, while orange one represents the temperature estimate. A red dashed line indicates the target temperature set by the energy saver component. The green line shows the monitor's output verdict. As observed, when there is a disparity between the estimated and actual temperatures, the energy saver adjusts the target temperature downward, indicating that the component is working properly. Thus, the output of the monitor is fixed at `unknown` (`0`), signifying that the monitoring property is not violated.

Examine the results

The results can be found in the */workspace/examples/data/incubator-NuRV-monitor-validation/output* directory where the logs are also included.

Figures of the output results can be found in the */workspace/examples/data/incubator-NuRV-monitor-validation/output* directory.

TERMINATE PHASE

Terminate to clean up the debug files and co-simulation output files.

```
1  lifecycle/terminate
```

References

1. More information about NuRV is available here.

3.8.14 Incubator Digital Twin with NuRV monitoring service

Overview

This example demonstrates how a runtime monitoring service (in this example NuRV[1]) can be connected with the Incubator digital twin to verify runtime behaviour of the Incubator.

Simulated scenario

This example simulates a scenario where the lid of the Incubator is removed and later put back on. The Incubator is equipped with anomaly detection capabilities, which can detect anomalous behaviour (i.e. the removal of the lid). When an anomaly is detected, the Incubator triggers an energy saving mode where the heater is turned off.

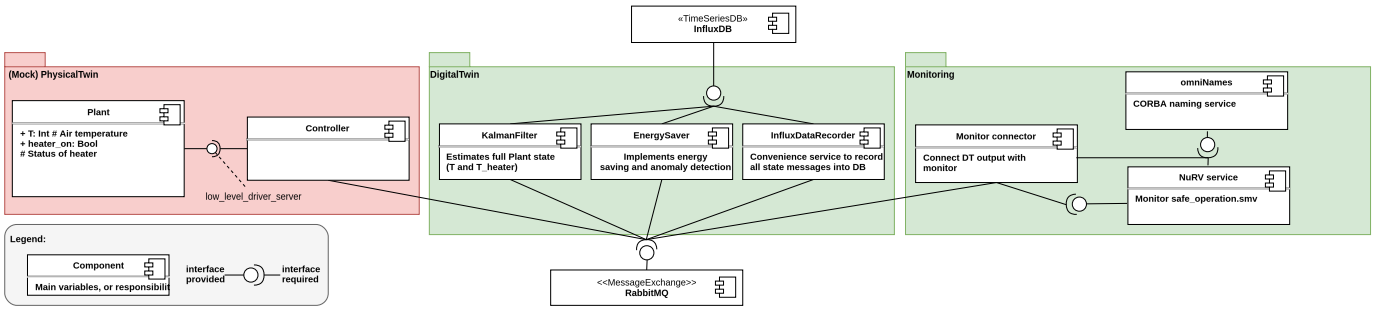
From a monitoring perspective, the aim is to verify that within 3 simulation steps of an anomaly detection, the energy saving mode is turned on. To verify this behaviour, the following property is constructed: . Whenever a True or False verdict is produced by the monitor, it is reset, allowing for the detection of repeated satisfaction/violation detections of the property.

The simulated scenario progresses as follows:

- *Initialization*: The services are initialized and the Kalman filter in the Incubator is given 2 minutes to stabilize. Sometimes, the anomaly detection algorithm will detect an anomaly at startup even though the lid is still on. It will disappear after approx 15 seconds.
- *After 2 minutes*: The lid is lifted and an anomaly is detected. The energy saver is turned on shortly after
- *After another 30 seconds*: The energy saver is manually disabled producing a False verdict.
- *After another 30 seconds*: The lid is put back on and the anomaly detection is given time to detect that the lid is back on. The simulation then ends.

Example structure

A diagram depicting the logical software structure of the example can be seen below.



The *execute.py* script is responsible for orchestrating and starting all the relevant services in this example. This includes the Incubator DT, CORBA naming service (omniNames) and the NuRV monitor server as well as implementing the *Monitor connector* component that connects the DT output to the NuRV monitor server.

The NuRV monitor server utilizes a CORBA naming service where it registers under a specific name. A user can then query the naming service for the specific name, to obtain a reference to the monitor server. For more information on how the NuRV monitor server works, please refer to [1].

After establishing connection with the NuRV monitor server, the Incubator DT is started and a RabbitMQ client is created that subscribes to changes in the *anomaly* and *energy_saving* states of the DT. Each time an update is received of either state, the full state (the new updated state and the previous other state) is pushed to the NuRV monitor server whereafter the verdict is printed to the console.

Digital Twin Configuration

Before running the example, the *simulation.conf* file should be configured with the appropriate RabbitMQ credentials.

The example uses the following assets:

Asset Type	Names of Assets	Visibility	Reuse in other Examples
Service	common/services/NuRV_orbit	Common	Yes
DT	common/digital_twins/incubator	Common	Yes
Specification	safe-operation.smv	Private	No
Script	execute.py	Private	No

The *safe-operation.smv* file contains the default monitored specification as described in the Simulated scenario section. These can be configured as desired.

Lifecycle phases

The lifecycle phases for this example include:

Lifecycle phase	Completed tasks
create	Downloads the necessary tools and creates a virtual python environment with the necessary dependencies
execute	Runs a python script that starts up the necessary services as well as the Incubator simulation. Various status messages are printed to the console, including the monitored system states and monitor verdict.
clean	Removes created *data* directory and incubator log files.

If required, change the execute permissions of lifecycle scripts that need to be executed. This can be done using the following command

```
chmod +x lifecycle/{script}
```

where {script} is the name of the script, e.g. *create*, *execute* etc.

Running the example

To run the example, first run the following command in a terminal:

```
cd /workspace/examples/digital_twins/incubator-monitor-server/
```

Then, first execute the *create* script (this can take a few minutes depending on the network connection) followed by the *execute* script using the following command:

```
lifecycle/{script}
```

The *execute* script will then start outputting system states and the monitor verdict approx every 3 seconds. The output is printed as follows "**State: {anomaly state} & {energy_saving state}, verdict: {Verdict}**" where "*anomaly*" indicates that an anomaly is detected and "!anomaly" indicates that an anomaly is not currently detected. The same format is used for the energy_saving state.

The monitor verdict can be True, False or Unknown, where the latter indicates that the monitor does not yet have sufficient information to determine the satisfaction of the property.

An example output trace is provided below:

```
....
Running scenario with initial state: lid closed and energy saver on
Setting energy saver mode: enable
Setting G_box to: 0.5763498
State: !anomaly & !energy_saving, verdict: True
State: !anomaly & !energy_saving, verdict: True
....
State: anomaly & !energy_saving, verdict: Unknown
State: anomaly & energy_saving, verdict: True
State: anomaly & energy_saving, verdict: True
```

There is currently some startup issues with connecting to the NuRV server, and it will likely take a few tries before the connection is established. This is however handled automatically.

References

1. Information on the NuRV monitor can be found on FBK website.

3.8.15 Incubator Digital Twin with NuRV FMU Monitoring Service

Overview

This example demonstrates how an FMU can be used as a runtime monitoring service (in this example NuRV[1]) and connected with the Incubator digital twin to verify runtime behaviour of the Incubator.

Simulated scenario

This example simulates a scenario where the lid of the Incubator is removed and later put back on. The Incubator is equipped with anomaly detection capabilities, which can detect anomalous behaviour (i.e. the removal of the lid). When an anomaly is detected, the Incubator triggers an energy saving mode where the heater is turned off.

From a monitoring perspective, the aim is to verify that within 3 messages of an anomaly detection, the energy saving mode is turned on. To verify this behaviour, the following property is constructed:

.

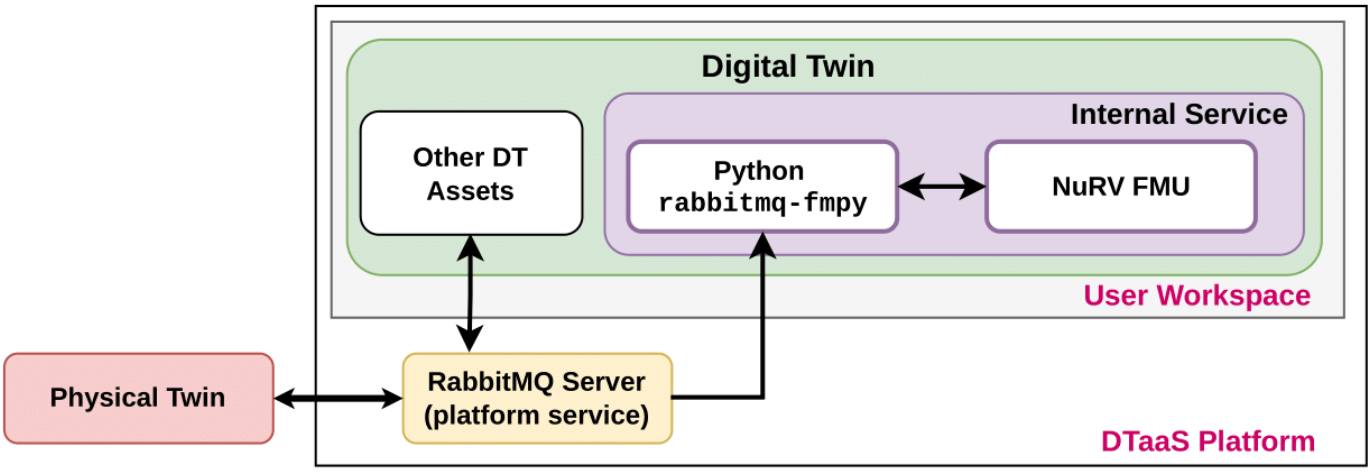
The monitor will output the *unknown* state as long as the property is satisfied and will transition to the *false* state once a violation is detected.

The simulated scenario progresses as follows:

- *Initialization*: The services are initialized and the Kalman filter in the Incubator is given 2 minutes to stabilize. Sometimes, the anomaly detection algorithm will detect an anomaly at startup even though the lid is still on. It will disappear after approx 15 seconds.
- *After 2 minutes*: The lid is lifted and an anomaly is detected. The energy saver is turned on shortly after.
- *After another 30 seconds*: The energy saver is manually disabled producing a *false* verdict.
- *After another 30 seconds*: The lid is put back on and the anomaly detection is given time to detect that the lid is back on. The monitor is then reset producing an Unknown verdict again. The simulation then ends.

Example structure

A diagram depicting the logical software structure of the example can be seen below.



The *execute* script is responsible for starting the NuRV service and running the Python script that controls the scenario (*execute.py*).

The *execute.py* script starts the Incubator services and runs the example scenario. Once the Incubator DT is started, a RabbitMQ client is created that subscribes to changes in the *anomaly* and *energy_saving* states of the DT, as well as the verdicts produced by the NuRV service. Each time an update is received, the full state and verdict is printed to the console.

Digital Twin Configuration

Before running the example, the *simulation.conf* file should be configured with the appropriate RabbitMQ credentials.

The example uses the following assets:

Asset Type	Names of Assets	Visibility	Reuse in other Examples
Tools	common/tool/NuRV/NuRV	Common	Yes
Other	common/fmi2_headers	Common	Yes
DT	common/digital_twins/incubator	Common	Yes
Specification	safe-operation.smv	Private	No
Script	execute.py	Private	No

The *safe-operation.smv* file contains the default monitored specification as described in the Simulated scenario section. These can be configured as desired.

Lifecycle phases

The lifecycle phases for this example include:

Lifecycle phase	Completed tasks
create	Downloads the necessary tools and creates a virtual python environment with the necessary dependencies
execute	Runs a python script that starts up the necessary services as well as the Incubator simulation. Various status messages are printed to the console, including the monitored system states and monitor verdict.

If required, change the execute permissions of lifecycle scripts that need to be executed. This can be done using the following command.

```
chmod +x lifecycle/{script}
```

where {script} is the name of the script, e.g. *create*, *execute* etc.

Running the example

To run the example, first run the following command in a terminal:

```
cd /workspace/examples/digital_twins/incubator-NuRV-fmu-monitor-service/
```

Then, first execute the *create* script followed by the *execute* script using the following command:

```
lifecycle/{script}
```

The *execute* script will then start outputting system states and the monitor verdict approx every 3 seconds. The output is printed as follows.

"**State: {anomaly state} & {energy_saving state}**"

where "*anomaly*" indicates that an anomaly is detected and "!anomaly" indicates that an anomaly is not currently detected. The same format is used for the energy_saving state. NuRV verdicts are printed as follows

"**Verdict from NuRV: {verdict}**".

The monitor verdict can be false or unknown, where the latter indicates that the monitor does not yet have sufficient information to determine the satisfaction of the property. The monitor will never produce a true verdict as the entire trace must be verified to ensure satisfaction due to the G operator. Thus the unknown state can be viewed as a tentative true verdict.

An example output trace is provided below:

```
....
Using LIFECYCLE_PATH: /workspace/examples/digital_twins/incubator-NuRV-fmu-monitor-service/lifecycle
Using INCUBATOR_PATH: /workspace/examples/digital_twins/incubator-NuRV-fmu-monitor-service/lifecycle/../../../common/digital_twins/incubator
Starting NuRV FMU Monitor Service, see output at /tmp/nurv-fmu-service.log
NuRVService.py PID: 13496
Starting incubator
Connected to rabbitmq server.
Running scenario with initial state: lid closed and energy saver on
Setting energy saver mode: enable
Setting G_box to: 0.5763498
State: !anomaly & !energy_saving
State: !anomaly & !energy_saving
Verdict from NuRV: unknown
State: !anomaly & !energy_saving
State: !anomaly & !energy_saving
Verdict from NuRV: unknown
State: !anomaly & !energy_saving
State: !anomaly & !energy_saving
Verdict from NuRV: unknown
State: !anomaly & !energy_saving
State: !anomaly & !energy_saving
Verdict from NuRV: unknown
State: !anomaly & !energy_saving
State: !anomaly & !energy_saving
Verdict from NuRV: unknown
```

References

1. Information on the NuRV monitor can be found on FBK website.

4. Admin

4.1 Installation Scenarios

4.1.1 Overview

Installation Scenarios

The DTaaS repository provides installation assets in the following locations:

- `deploy/dtaas/docker/` for DTaaS application deployments
- `deploy/workspace/` for workspace-centred deployments
- `deploy/services/` for optional platform services
- `deploy/vagrant/` for virtual-machine-based deployment

Use the scenario pages listed below to select the most appropriate setup.

DTAAS

DTaaS scenarios focus on complete platform deployments that bundle the web client, gateway, user workspaces, and core services. Choose a scenario based on target environment, security needs, and GitLab topology. Each scenario now provides dedicated Install and Config pages, plus scenario-specific operational documents.

Scenario	Purpose	Source Directory
localhost	Single-user DTaaS package over HTTP	`deploy/dtaas/docker/localhost`
localhost on portainer	GUI-based localhost deployment with Portainer for single-users	`deploy/dtaas/docker/localhost` and `deploy/workspace/dex/localhost`
secure server	Compatibility package for secure server installs	`deploy/dtaas/docker/secure-server`
secure server and GitLab	Multi-user DTaaS package with integrated GitLab	`deploy/dtaas/docker/secure-server_with_integrated-gitlab`

WORKSPACE

Workspace scenarios focus on user workbench access, identity-provider setup, and route-level protection. Choose localhost for rapid onboarding with Dex, or secure server for production-style Keycloak/OIDC deployment. Scenario pages are organized by Install, Configuration, and identity-provider setup to make operations and troubleshooting predictable.

Scenario	Purpose	Source Directory
localhost	Single-user workspace deployment with Dex	`deploy/workspace/dex/localhost`
secure server	Multi-user workspace deployment with Keycloak	`deploy/workspace/keycloak/production`

OTHER

- Platform services
- Vagrant
- Independent packages

The installation steps page remains the recommended sequence guide.

Administration

- DTaaS CLI

- GitLab server guidance
- GitLab runner guidance

4.1.2 Installation Steps

Complete the DTaaS Platform

Current DTaaS deployments are package-driven. Start by selecting one of these package roots:

- `deploy/dtaas/docker/localhost` (DTaaS localhost package over HTTP)
- `deploy/dtaas/docker/secure-server` (DTaaS secure server compatibility package)
- `deploy/dtaas/docker/secure-server_with_integrated-gitlab` (DTaaS secure server package with integrated GitLab)
- `deploy/workspace/dex/localhost` (workspace localhost deployment using Dex)
- `deploy/workspace/keycloak/production` (workspace secure server deployment using Keycloak)

Recommended Sequence

1. SELECT INSTALLATION SCENARIO

Use installation scenarios to select the right package and guide.

2. CONFIGURE OAUTH AND CLIENT SETTINGS

- DTaaS packages:
- Frontend OAuth setup: client auth
- Backend OAuth setup (forward-auth): servers auth
- Client runtime settings: client config
- Workspace Dex localhost:
- Configure `.env` and `config/dex-config.yaml`
- Workspace Keycloak secure server:
- Follow package guides in `deploy/workspace/keycloak/production/CONFIGURATION.md` and `deploy/workspace/keycloak/production/KEYCLOAK_SETUP.md`

3. CONFIGURE FILESYSTEM AND CERTIFICATES

For DTaaS or workspace server deployments:

- Create user workspaces under `files/`
- Provide TLS certs under `certs/`

For workspace Dex localhost deployment:

- Create `.env` and Dex config from examples

4. START SERVICES

Run compose commands from the selected package directory.

- DTaaS localhost package (HTTP): `deploy/dtaas/docker/localhost`
- DTaaS secure server package: `deploy/dtaas/docker/secure-server`
- DTaaS secure server with integrated GitLab package: `deploy/dtaas/docker/secure-server_with_integrated-gitlab`
- Workspace secure server: `deploy/workspace/keycloak/production`
- Workspace localhost (HTTP): `deploy/workspace/dex/localhost`

5. VALIDATE AND ITERATE

- Confirm login flow in browser
- Validate workspace routes (`/user1` , `/user2`)
- Check service logs and health

Platform Services (Optional)

To install additional data/infrastructure services, use the Platform Services CLI rooted in `deploy/services/cli`.

Independent Packages

Reusable package details are listed on packages.

4.2 DTaaS

4.2.1 Localhost

Install

🏭 ↔️ 👥 Digital Twin as a Service DTaaS

🎉 Thank you for downloading **Digital Twin as a Service**.

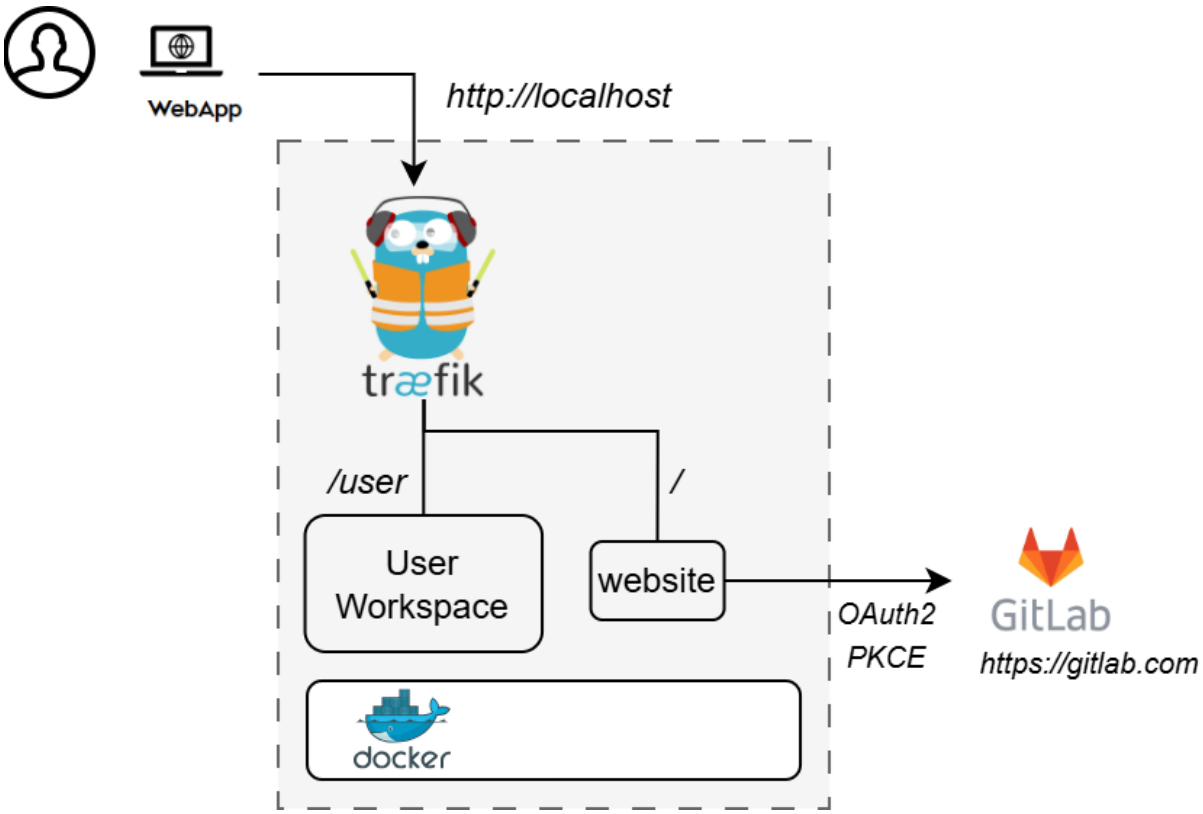
This README provides a quick-start installation guide for DTaaS on localhost over **HTTP** with a single user workspace.

For HTTP localhost deployment, use `deploy/dtaas/docker/localhost`.

For detailed configuration reference, see config.md.

DESIGN

An illustration of the docker containers used in this package is shown here.



REQUIREMENTS

The installation requirements to run this package are:

- Docker Engine with Compose plugin
- GitLab account for OAuth sign-in

QUICK START

1. Create Configuration Files

```
cp config/.env.example config/.env
cp config/client.js.example config/client.js
```

2. Create User Workspace Directory

Edit `config/.env` and set `DEFAULT_USER`, then create a matching workspace:

```
cp -R files/template files/<DEFAULT_USER>
sudo chown -R 1000:100 files/*
```

3. Start Services

```
docker compose --env-file config/.env up -d
```

4. Open DTaaS

- http://localhost

Sign in using the configured OAuth provider in `config/client.js` (default authority is `https://gitlab.com/`).

RUN

The commands to start and stop the application are:

```
docker compose --env-file config/.env up -d
docker compose --env-file config/.env down
```

NOTES

- This package does not include `libms` or backend forward-auth.
- For secure production deployments, see `deploy/dtaas/docker/secure-server_with_integrated-gitlab`.

DOCUMENTATION

See https://into-cps-association.github.io/DTaaS/development/index.html for complete documentation.

REFERENCES

Image sources: Traefik logo, gitlab

Configuration Reference

This document provides a detailed reference for each configuration file in this package.

For the quick-start installation guide, see install.md.

CONFIG/.ENV

Source: `config/.env.example`

```
cp config/.env.example config/.env
```

Variable	Example	Description
`DEFAULT_USER`	`user1`	Workspace path prefix and workspace folder name
`COMPOSE_PROJECT_NAME`	`dtaas`	Docker Compose project name

The `DEFAULT_USER` value is used in:

- Router path prefix (`/${DEFAULT_USER}`)
- Workspace volume mount (`./files/${DEFAULT_USER}`)
- Workspace container `MAIN_USER` environment variable

CONFIG/CLIENT.JS

Source: `config/client.js.example`

```
cp config/client.js.example config/client.js
```

Variable	Example	Description
`REACT_APP_URL`	`http://localhost`	Base URL of DTaaS
`REACT_APP_CLIENT_ID`	*(OAuth client id)*	Client OAuth application ID
`REACT_APP_AUTH_AUTHORITY`	`https://gitlab.com/`	OAuth authority URL
`REACT_APP_REDIRECT_URI`	`http://localhost/Library`	Redirect URI after sign-in
`REACT_APP_LOGOUT_REDIRECT_URI`	`http://localhost/`	Redirect URI after sign-out

FILES/

`files/common` is mounted read-write to `/workspace/common`.

Create a per-user workspace from `files/template`:

```
cp -R files/template files/<DEFAULT_USER>
sudo chown -R 1000:100 files/*
```

RELOAD CLIENT CONFIG

After editing `config/client.js`, reload only the client service:

```
docker compose --env-file config/.env up -d --force-recreate client
```

4.2.2 Secure Server

Install

🏭 ↔️ 👥 Digital Twin as a Service DTaaS

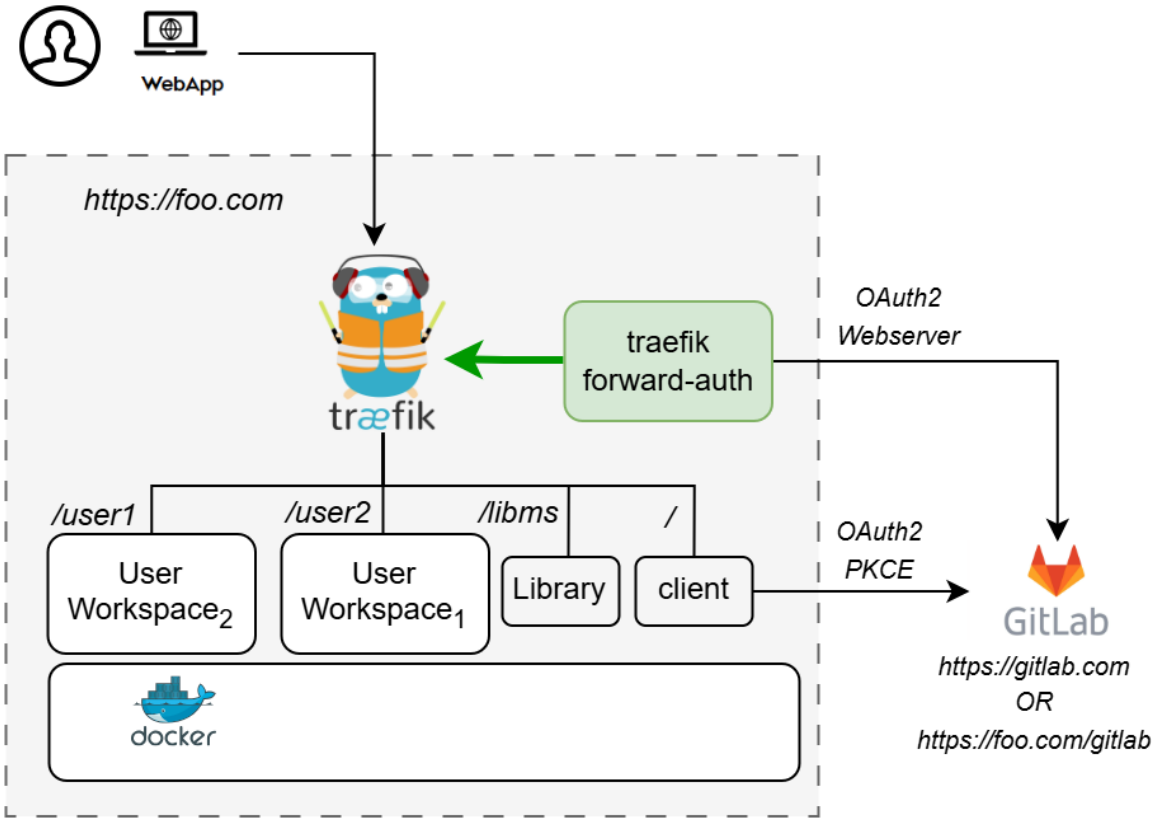
🎉 Thank you for downloading **Digital Twin as a Service**.

This README provides a quick-start installation guide for a secure, multi-user DTaaS deployment that uses an **external GitLab instance** for OAuth 2.0 authorisation.

For a full configuration reference, see config.md.

OVERVIEW

This package deploys DTaaS with Traefik, OAuth forward-auth, and multiple user workspaces.



The `docker-compose.yml` starts the following services:

Service	Purpose
traefik	Reverse proxy with TLS termination
client	DTaaS React frontend
user1 / **user2**	JupyterLab user workspaces
libms	Library management microservice
traefik-forward-auth	OAuth 2.0 authorisation middleware

PREREQUISITES

Requirement	Details
Docker Engine	v28 or later with Compose plugin
Domain name	Public DNS name or server IP
TLS certificates	`fullchain.pem` and `privkey.pem` for your domain
OAuth provider	External GitLab (`gitlab.com` or self-hosted GitLab)

QUICK START

1. Create Configuration Files

```
cp config/.env.example config/.env
cp config/conf.server.example config/conf.server
cp config/client.js.example config/client.js
```

Edit `config/.env` :

- `SERVER_DNS`
- `USERNAME1`
- `USERNAME2`
- `OAUTH_URL`
- `OAUTH_CLIENT_ID`
- `OAUTH_CLIENT_SECRET`
- `OAUTH_SECRET`

Edit `config/client.js` :

- `REACT_APP_CLIENT_ID`
- `REACT_APP_AUTH_AUTHORITY`
- `REACT_APP_REDIRECT_URI`
- `REACT_APP_LOGOUT_REDIRECT_URI`

See config.md for full details.

2. Create User Workspace Directories

```
cp -R files/template files/<USERNAME1>
cp -R files/template files/<USERNAME2>
sudo chown -R 1000:100 files/*
```

3. Add TLS Certificates

```
cp /path/to/fullchain.pem certs/fullchain.pem
cp /path/to/privkey.pem certs/privkey.pem
```

If certificates are missing or invalid, Traefik runs with self-signed certificates.

4. Configure OAuth 2.0 Applications in GitLab 🦊

1. Create user accounts (see GitLab docs). The usernames **must** match `USERNAME1` / `USERNAME2` in `config/.env`.
2. Register two OAuth 2.0 applications in GitLab (Admin Area → Applications):
3. **DTaaS Client Authorization** — for the React SPA frontend. See client auth docs.
4. **DTaaS Server Authorization** — for Traefik forward-auth backend. See server auth docs.
5. Update configuration files (`config/.env` and `config/client.js`) with the generated OAuth 2.0 tokens:
6. Set `REACT_APP_CLIENT_ID` and `REACT_APP_AUTH_AUTHORITY` in `config/client.js`.
7. Set `OAUTH_URL`, `OAUTH_CLIENT_ID`, and `OAUTH_CLIENT_SECRET` in `config/.env`.

Update `config/client.js` and `config/.env` with generated credentials, then reload affected services:

5. Start Services

```
docker compose --env-file config/.env up -d
```

6. Verify

URL	Expected result
`https://<SERVER_DNS>`	DTaaS web interface
`https://<SERVER_DNS>/user1`	User 1 workspace after sign-in
`https://<SERVER_DNS>/user2`	User 2 workspace after sign-in
`https://<SERVER_DNS>/lib`	Library service

STOP

```
docker compose --env-file config/.env down
```

DIRECTORY LAYOUT

```
.
|- certs/                # TLS certificates (fullchain.pem, privkey.pem)
|- config/
|  |- .env               # Docker Compose environment variables
|  |- conf.server        # Traefik forward-auth authorization rules
|  |- client.js          # DTaaS React client configuration
|  |- .env.example
|  |- conf.server.example
|  |- client.js.example
|  \- tls.yml           # Traefik TLS provider configuration
|- files/
|  |- common/            # Shared files across all workspaces
|  \- template/          # sample user workspace files
|- docker-compose.yml    # Service definitions
|- CONFIG.md             # Detailed configuration reference
\- README.md             # This file
```

Configuration Reference

This document provides a detailed reference for each configuration file in this package.

For the quick-start installation guide, see install.md.

TABLE OF CONTENTS

CONFIG/.ENV - DOCKER COMPOSE ENVIRONMENT

Source: `config/.env.example`

This file provides environment variables consumed by `docker-compose.yml`.

```
cp config/.env.example config/.env
```

Server Settings

Variable	Example	Description
`SERVER_DNS`	`intocps.org`	Domain name or IP address of the server. Do not include `https://`.
`USERNAME1`	`user1`	Path prefix and workspace name for the first user
`USERNAME2`	`user2`	Path prefix and workspace name for the second user
`COMPOSE_PROJECT_NAME`	`dtaas`	Docker Compose project name

OAuth 2.0 Settings

These are set from the external GitLab OAuth 2.0 applications.

Variable	Example	Description
`OAUTH_URL`	`https://gitlab.com`	GitLab base URL used by traefik-forward-auth
`OAUTH_CLIENT_ID`	*(from GitLab)*	Application ID from the **DTaaS Server Authorization** OAuth app
`OAUTH_CLIENT_SECRET`	*(from GitLab)*	Secret from the **DTaaS Server Authorization** OAuth app
`OAUTH_SECRET`	*(random string)*	Encryption key for OAuth session cookies. Example generation: `openssl rand -base64 32`

How Variables Map to Services

Variable	Used by
`SERVER_DNS`	traefik, client, user1, user2, libms, traefik-forward-auth
`USERNAME1` / `USERNAME2`	user1, user2 (routing and workspace volumes)
`OAUTH_URL`	traefik-forward-auth (authorize, token, userinfo endpoints)
`OAUTH_CLIENT_ID` / `OAUTH_CLIENT_SECRET`	traefik-forward-auth
`OAUTH_SECRET`	traefik-forward-auth

CONFIG/CLIENT.JS - DTAAS WEB CLIENT

Source: `config/client.js.example`

This JavaScript file is mounted into the React client container and configures DTaaS web behaviour at runtime.

```
cp config/client.js.example config/client.js
```

Variable Reference

Variable	Example	Description
`REACT_APP_ENVIRONMENT`	`prod`	Environment name
`REACT_APP_URL`	`https://intocps.org`	Base URL of the DTaaS web application
`REACT_APP_URL_BASENAME`	`''`	Optional URL base path
`REACT_APP_URL_DTLINK`	`/lab`	URL path for the Digital Twin workbench
`REACT_APP_URL_LIBLINK`	`''`	URL path for the Library
`REACT_APP_WORKBENCHLINK_LIBRARY_PREVIEW`	`/preview/library`	Library preview page
`REACT_APP_WORKBENCHLINK_DT_PREVIEW`	`/preview/digitaltwins`	Digital Twins preview page
`REACT_APP_CLIENT_ID`	*(from GitLab)*	Application ID from **DTaaS Client Authorization** OAuth app
`REACT_APP_AUTH_AUTHORITY`	`https://gitlab.com`	OAuth issuer URL
`REACT_APP_REDIRECT_URI`	`https://intocps.org/Library`	Redirect URI after sign-in
`REACT_APP_LOGOUT_REDIRECT_URI`	`https://intocps.org/`	Redirect URI after sign-out
`REACT_APP_GITLAB_SCOPES`	`openid profile read_user read_repository api`	Requested OAuth scopes

CONFIG/CONF.SERVER - TRAEFIK FORWARD-AUTH RULES

Source: `config/conf.server.example`

This file defines per-path authorisation rules for traefik-forward-auth. Each rule restricts a URL path to specific email addresses.

```
cp config/conf.server.example config/conf.server
```

Format

```
rule.<NAME>.action=auth
rule.<NAME>.rule=PathPrefix(`/<path>`)
rule.<NAME>.whitelist=<email>
```

Default Rules

```
rule.libms.action=auth
rule.libms.rule=PathPrefix(`/lib`)

rule.onlyu1.action=auth
rule.onlyu1.rule=PathPrefix(`/user1`)
rule.onlyu1.whitelist=user1@emailservice.com

rule.onlyu2.action=auth
rule.onlyu2.rule=PathPrefix(`/user2`)
rule.onlyu2.whitelist=user2@emailservice.com
```

Replace usernames and email addresses to match the actual users.

Important Rules

- Usernames in `config/.env` (`USERNAME1`, `USERNAME2`) must match `PathPrefix` values in `config/conf.server`.
- If a route exists in `docker-compose.yml` but has no rule in `config/conf.server`, the default behaviour allows any signed-in user.
- If a rule exists in `config/conf.server` but no router serves that path, the URL returns 404.

CERTS - TLS CERTIFICATES

Place these TLS certificate files in `certs/`:

```
certs/
|- fullchain.pem   # Public certificate or certificate chain
\- privkey.pem     # Private key
```

Certificates must be valid for `SERVER_DNS`.

FILES - USER WORKSPACE DIRECTORIES

Each user workspace container mounts a directory from `files/` as its `/workspace` volume. `files/common/` is shared across all workspaces.

```
files/
|- common/    # Shared files (mounted to /workspace/common)
|- user1/     # User 1 workspace files
\- user2/     # User 2 workspace files
```

Create user directories:

```
cp -R files/template files/<USERNAME>
sudo chown -R 1000:100 files/*
```

OAUTH 2.0 APPLICATION SETUP

Two OAuth 2.0 applications are needed in the external GitLab instance.

DTaaS Client Authorization (React Frontend)

1. In GitLab, open **Applications**.
2. Create an application:
3. **Name**: DTaaS Client Authorization

4. **Redirect URI**: `https://<SERVER_DNS>/Library`
5. **Confidential**: unticked (public SPA client)
6. **Scopes**: `openid`, `profile`, `read_user`, `read_repository`, `api`
7. Save the **Application ID** and set `REACT_APP_CLIENT_ID` in `config/client.js`.
8. Set `REACT_APP_AUTH_AUTHORITY` in `config/client.js` to the GitLab URL.

DTaaS Server Authorization (Traefik Forward-Auth)

1. In GitLab, create another application:
2. **Name**: DTaaS Server Authorization
3. **Redirect URI**: `https://<SERVER_DNS>/_oauth`
4. **Confidential**: ticked
5. **Scopes**: `read_user`
6. Save **Application ID** and **Secret**.
7. Set `OAUTH_CLIENT_ID`, `OAUTH_CLIENT_SECRET`, `OAUTH_URL` in `config/.env`.
8. Generate `OAUTH_SECRET` and set it in `config/.env`.

Reload After Configuration

```
docker compose --env-file config/.env up -d --force-recreate client traefik-forward-auth
```

ADDING MORE USERS

To add a third user:

1. Add service `user3` in `docker-compose.yml` based on `user1` / `user2`.
2. Add `USERNAME3=<name>` in `config/.env`.
3. Create `files/<name>` directory.
4. Add matching authorisation rule in `config/conf.server`.
5. Restart:

```
docker compose --env-file config/.env up -d
```

TROUBLESHOOTING

Authentication redirect loop

- Verify `OAUTH_URL` in `config/.env`.
- Verify `REACT_APP_AUTH_AUTHORITY` in `config/client.js`.
- Clear browser cookies for the domain.
- Check logs:

```
docker compose --env-file config/.env logs traefik-forward-auth
```

404 on user workspace

- Verify usernames are consistent across:
- `config/.env`
- `config/conf.server`
- `docker-compose.yml`

TLS warning in browser

- Replace `certs/fullchain.pem` and `certs/privkey.pem` with valid certificates.

4.2.3 Secure Server and GitLab

Install

Digital Twin as a Service
DTaaS

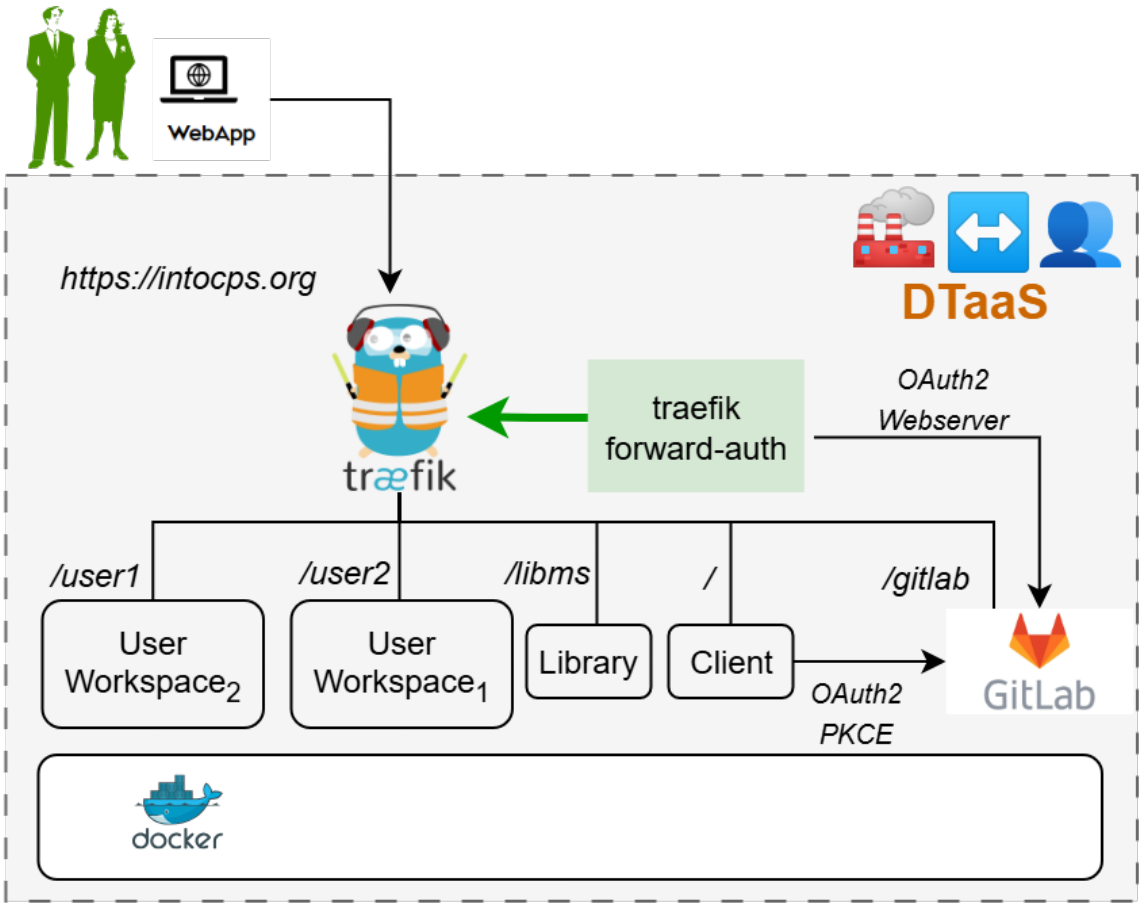
🎉 Thank you for downloading **Digital Twin as a Service**.

This README provides a quick-start installation guide. For detailed configuration reference, see config.md.

[!IMPORTANT] The hostname `intocps.org` is used for illustration throughout this guide. Replace it with the actual server hostname of the installation.

OVERVIEW

This package installs the DTaaS as a multi-user web application with an integrated GitLab CE instance. GitLab serves as both the OAuth 2.0 authorisation provider and the DevOps backend for digital twin projects.



Note: The text starting with / at the beginning indicates the URL route at which a certain service is available. For example, user workspace is available at https://intocps.org/user1

The `docker-compose.yml` starts the following services:

Service	Purpose
traefik	Reverse proxy with TLS termination
client	DTaaS React frontend
user1 / **user2**	JupyterLab user workspaces
libms	Library management microservice
traefik-forward-auth	OAuth 2.0 authorisation middleware
gitlab	Integrated GitLab CE (OAuth 2.0 provider)

📋 PREREQUISITES

Requirement	Details
Docker Engine	v28 or later with Compose plugin
Domain name	A domain name (e.g. `intocps.org`) or IP address
TLS certificate	`fullchain.pem` and `privkey.pem` for the domain. Obtain via certbot or a certificate provider

🚀 QUICK START

1. Create Configuration Files

```
cp config/.env.example       config/.env
cp config/conf.server.example config/conf.server
cp config/client.js.example     config/client.js
```

Edit `config/.env` — set `SERVER_DNS`, `USERNAME1`, `USERNAME2`. Leave the `OAUTH_*` variables as placeholders for now; they will be filled after the GitLab instance is running (see step 5).

See config.md for a complete reference of every variable.

2. Create User Workspace Directories

```
cp -R files/template files/<USERNAME1>
cp -R files/template files/<USERNAME2>
sudo chown -R 1000:100 files/*
```

3. Add TLS Certificates

```
cp /path/to/fullchain.pem certs/fullchain.pem
cp /path/to/privkey.pem   certs/privkey.pem
```

Traefik falls back to self-signed certificates if these files are absent or invalid.

4. Start Services

```
docker compose --env-file config/.env up -d
```

Wait a few minutes for GitLab to become healthy:

```
watch docker ps
```

5. Configure GitLab 🦊

Once the GitLab container shows as `healthy`:

1. Log in to `https://intocps.org/gitlab` as `root`. The initial password is inside `config/gitlab/initial_root_password`.

   [!WARNING] This file is **deleted 24 hours** after the first start. Save the password immediately.

1. Create user accounts (see GitLab docs). The usernames **must** match `USERNAME1` / `USERNAME2` in `config/.env`.
2. Register two OAuth 2.0 applications in GitLab (Admin Area → Applications):
3. **DTaaS Client Authorization** — for the React SPA frontend. See client auth docs.
4. **DTaaS Server Authorization** — for Traefik forward-auth backend. See server auth docs.
5. Update configuration files (`config/.env` and `config/client.js`) with the generated OAuth 2.0 tokens:
6. Set `REACT_APP_CLIENT_ID` and `REACT_APP_AUTH_AUTHORITY` in `config/client.js`.
7. Set `OAUTH_URL`, `OAUTH_CLIENT_ID`, and `OAUTH_CLIENT_SECRET` in `config/.env`.
8. Update user permissions in `config/conf.server`.
9. Reload the services:

```
docker compose --env-file config/.env up -d --force-recreate client traefik-forward-auth
```

6. Verify ✅

URL	Expected result
`https://intocps.org`	DTaaS web interface (redirects to GitLab sign-in)
`https://intocps.org/gitlab`	Integrated GitLab instance
`https://intocps.org/user1`	User 1 workspace (after sign-in)
`https://intocps.org/user2`	User 2 workspace (after sign-in)

🛑 STOP

```
docker compose --env-file config/.env down
```

📁 DIRECTORY LAYOUT

```
.
├── certs/                # TLS certificates (fullchain.pem, privkey.pem)
├── config/
│   ├── .env              # Docker compose environment variables
│   ├── conf.server       # Traefik forward-auth authorization rules
│   ├── client.js            # DTaaS React client configuration
│   ├── gitlab/           # GitLab config (mounted as /etc/gitlab)
│   └── tls.yml           # Traefik TLS provider configuration
├── data/                 # GitLab persistent data (/var/opt/gitlab)
├── files/
│   ├── common/           # Shared files across all workspaces
|  \- template/          # sample user workspace files
├── logs/                 # GitLab logs (/var/log/gitlab)
├── docker-compose.yml    # Service definitions
├── CONFIG.md             # Detailed configuration reference
└── README.md             # This file — quick-start guide
```

:FRAMED_PICTURE: REFERENCES

Image sources: Traefik logo, gitlab

⚙️ Configuration Reference

This document provides a detailed reference for every configuration file in this package. For the quick-start installation guide, see install.md.

[!IMPORTANT] The hostname `intocps.org` is used for illustration throughout this guide. Replace it with the actual server hostname of the installation.

TABLE OF CONTENTS

🔧 CONFIG/.ENV — DOCKER COMPOSE ENVIRONMENT

Source: `config/.env.example`

This file provides environment variables consumed by `docker-compose.yml`.

```
cp config/.env.example config/.env
```

Server Settings

Variable	Example	Description
`SERVER_DNS`	`intocps.org`	Domain name or IP address of the server. Do **not** include `https://`.
`USERNAME1`	`user1`	Path prefix and workspace name for the first user
`USERNAME2`	`user2`	Path prefix and workspace name for the second user
`COMPOSE_PROJECT_NAME`	`dtaas`	Docker Compose project name (rarely needs changing)

OAuth 2.0 Settings

These are populated after the GitLab instance is running and OAuth 2.0 applications have been created (see OAuth 2.0 Application Setup).

Variable	Example	Description
`OAUTH_URL`	`https://intocps.org/gitlab`	GitLab instance URL used for browser-side authorisation redirects. No trailing slash.
`OAUTH_CLIENT_ID`	*(from GitLab)*	Application ID from the **DTaaS Server Authorization** OAuth 2.0 application
`OAUTH_CLIENT_SECRET`	*(from GitLab)*	Secret from the **DTaaS Server Authorization** OAuth 2.0 application
`OAUTH_SECRET`	*(random string)*	Encryption key for OAuth session cookies. Generate with: `openssl rand -base64 32`

How Variables Map to Services

Variable	Used by
`SERVER_DNS`	traefik, client, user1, user2, libms, traefik-forward-auth, gitlab
`USERNAME1` / `USERNAME2`	user1, user2 (routing and workspace volumes)
`OAUTH_URL`	traefik-forward-auth (browser redirect URL)
`OAUTH_CLIENT_ID` / `OAUTH_CLIENT_SECRET`	traefik-forward-auth
`OAUTH_SECRET`	traefik-forward-auth

CONFIG/CLIENT.JS — DTAAS WEB CLIENT

Source: `config/client.js.example`

This JavaScript file is mounted into the React client container and configures the DTaaS web application at runtime.

```
cp config/client.js.example config/client.js
```

Variable Reference

Variable	Example	Description
`REACT_APP_ENVIRONMENT`	`prod`	Environment name. Use `prod` for production.
`REACT_APP_URL`	`https://intocps.org`	Base URL of the DTaaS web application
`REACT_APP_URL_BASENAME`	`''`	Optional URL base path (leave empty for root hosting)
`REACT_APP_URL_DTLINK`	`/lab`	URL path for the Digital Twin workbench
`REACT_APP_URL_LIBLINK`	`''`	URL path for the Library
`REACT_APP_WORKBENCHLINK_LIBRARY_PREVIEW`	`/preview/library`	Library preview page
`REACT_APP_WORKBENCHLINK_DT_PREVIEW`	`/preview/digitaltwins`	Digital Twins preview page
`REACT_APP_CLIENT_ID`	*(from GitLab)*	Application ID from the **DTaaS Client Authorization** OAuth 2.0 application
`REACT_APP_AUTH_AUTHORITY`	`https://intocps.org/gitlab`	URL of the GitLab instance (OAuth 2.0 issuer)
`REACT_APP_REDIRECT_URI`	`https://intocps.org/Library`	Where GitLab sends users after sign-in
`REACT_APP_LOGOUT_REDIRECT_URI`	`https://intocps.org/`	Where users land after sign-out
`REACT_APP_GITLAB_SCOPES`	`openid profile read_user read_repository api`	OAuth 2.0 scopes requested during sign-in

Example

```
if (typeof window !== 'undefined') {
  window.env = {
    REACT_APP_ENVIRONMENT: 'prod',
    REACT_APP_URL: 'https://intocps.org',
    REACT_APP_URL_BASENAME: '',
    REACT_APP_URL_DTLINK: '/lab',
    REACT_APP_URL_LIBLINK: '',
    REACT_APP_WORKBENCHLINK_LIBRARY_PREVIEW: '/preview/library',
    REACT_APP_WORKBENCHLINK_DT_PREVIEW: '/preview/digitaltwins',
    REACT_APP_CLIENT_ID: '<APPLICATION_ID>',
    REACT_APP_AUTH_AUTHORITY: 'https://intocps.org/gitlab',
    REACT_APP_REDIRECT_URI: 'https://intocps.org/Library',
    REACT_APP_LOGOUT_REDIRECT_URI: 'https://intocps.org/',
    REACT_APP_GITLAB_SCOPES: 'openid profile read_user read_repository api',
  };
};
```

CONFIG/CONF.SERVER — TRAEFIK FORWARD-AUTH RULES

Source: `config/conf.server.example`

This file defines per-path authorisation rules for traefik-forward-auth. Each rule restricts a URL path to specific GitLab email addresses.

```
cp config/conf.server.example config/conf.server
```

Format

```
rule.<NAME>.action=auth
rule.<NAME>.rule=PathPrefix(`/<path>`)
rule.<NAME>.whitelist=<email>
```

Default Rules

```
rule.libms.action=auth
rule.libms.rule=PathPrefix(`/lib`)

rule.onlyu1.action=auth
rule.onlyu1.rule=PathPrefix(`/user1`)
rule.onlyu1.whitelist=user1@emailservice.com

rule.onlyu2.action=auth
rule.onlyu2.rule=PathPrefix(`/user2`)
rule.onlyu2.whitelist=user2@emailservice.com
```

Replace `user1`, `user2`, and the email addresses to match the actual GitLab accounts.

Important Rules

[!WARNING] **Usernames must be consistent.** The usernames in `config/.env` (`USERNAME1`, `USERNAME2`) must match the `PathPrefix` values in `config/conf.server`. Mismatches cause routing or authorisation failures.

Scenario	Behaviour
Route in `config/.env` but **missing** from `config/conf.server`	Any signed-in user can access the route (default forward-auth behaviour)
Route in `config/conf.server` but **missing** from `config/.env`	Traefik returns **404** (route not served)
The `/lib` rule has **no whitelist**	Any signed-in user can access the library service

CERTS/ — TLS CERTIFICATES

Place the TLS certificate files here:

```
certs/
├── fullchain.pem   # Public certificate (or certificate chain)
└── privkey.pem     # Private key
```

The certificates must be valid for `SERVER_DNS` (e.g. `intocps.org` or `*.intocps.org`).

Obtain certificates via:

```
# Using certbot (Let's Encrypt)
sudo certbot certonly --standalone -d intocps.org
sudo cp /etc/letsencrypt/live/intocps.org/fullchain.pem certs/
sudo cp /etc/letsencrypt/live/intocps.org/privkey.pem   certs/
```

If the certificate files are absent or invalid, Traefik runs with self-signed certificates. Browsers will show a security warning.

FILES/ — USER WORKSPACE DIRECTORIES

Each user workspace container mounts a directory from `files/` as its `/workspace` volume. The `files/common/` directory is shared across all workspaces and mounted to `/workspace/common` in each container.

```
files/
├── common/    # Shared files (mounted to /workspace/common in each container)
└── template/  # template workspace files
```

Create directories for each user:

```
cp -R files/template files/<USERNAME>
sudo chown -R 1000:100 files/*
```

The UID `1000` and GID `100` match the default user inside the workspace container.

OAUTH 2.0 APPLICATION SETUP

After the GitLab instance is running, two OAuth 2.0 applications must be registered to connect DTaaS and Traefik forward-auth to the integrated GitLab.

DTaaS Client Authorization (React Frontend)

1. In GitLab, go to **Admin Area → Applications** (or the user's **Edit Profile → Applications**).
2. Create a new application:
3. **Name**: DTaaS Client Authorization
4. **Redirect URI**: `https://intocps.org/Library`
5. **Confidential**: unticked (public SPA client)
6. **Scopes**: `openid`, `profile`, `read_user`, `read_repository`, `api`
7. Save the **Application ID**.
8. Set `REACT_APP_CLIENT_ID` in `config/client.js` to this Application ID.
9. Set `REACT_APP_AUTH_AUTHORITY` in `config/client.js` to `https://intocps.org/gitlab`.

For full details, see the client auth documentation.

DTaaS Server Authorization (Traefik Forward-Auth)

1. In GitLab, go to **Admin Area → Applications**.
2. Create a new application:
3. **Name**: DTaaS Server Authorization
4. **Redirect URI**: `https://intocps.org/_oauth`
5. **Confidential**: ticked
6. **Scopes**: `read_user`
7. Save the **Application ID** and **Secret**.
8. Set `OAUTH_CLIENT_ID` and `OAUTH_CLIENT_SECRET` in `config/.env`.
9. Set `OAUTH_URL` in `config/.env` to `https://intocps.org/gitlab`.
10. Generate `OAUTH_SECRET`: `openssl rand -base64 32` and set it in `config/.env`.

For full details, see the server auth documentation.

Reload After Configuration

After updating the OAuth 2.0 tokens in the configuration files, reload the affected services:

```
docker compose --env-file config/.env up -d --force-recreate client traefik-forward-auth
```

👥 ADDING MORE USERS

To add a third user:

1. **Add service to `docker-compose.yml`:**

```
  user3:
    image: intocps/workspace:main-967bc10
    restart: unless-stopped
    environment:
      - MAIN_USER=${USERNAME3:-user3}
    volumes:
      - "./files/common:/workspace/common"
      - "./files/${USERNAME3:-user3}:/workspace"
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.u3.rule=Host(`${SERVER_DNS:-localhost}`) && PathPrefix(`/${USERNAME3:-user3}`)"
      - "traefik.http.routers.u3.tls=true"
      - "traefik.http.routers.u3.middlewares=traefik-forward-auth"
    networks:
      - users
```

1. **Add to `config/.env`:**

```
USERNAME3=alice
```

1. **Create workspace directory:**

```
cp -R files/user1 files/alice
sudo chown -R 1000:100 files/alice
```

1. **Add authorisation rule to `config/conf.server`:**

```
rule.onlyu3.action=auth
rule.onlyu3.rule=PathPrefix(`/alice`)
rule.onlyu3.whitelist=alice@emailservice.com
```

1. **Create a GitLab account** for `alice` in the integrated GitLab instance.
2. **Restart:**

```
docker compose --env-file config/.env up -d
```

🔍 TROUBLESHOOTING

⏳ GitLab Takes Too Long to Start

GitLab CE requires significant resources. The first startup may take 5–10 minutes. Monitor with `docker compose --env-file config/.env logs -f gitlab`. Ensure the host has at least 4 GB RAM available for GitLab.

🔁 Authentication Redirect Loop

1. Verify `OAUTH_URL` in `config/.env` matches the URL accessible from the user's browser (e.g. `https://intocps.org/gitlab`).
2. Verify `REACT_APP_AUTH_AUTHORITY` in `config/client.js` matches the same URL.
3. Clear browser cookies for the domain.
4. Check traefik-forward-auth logs: `docker compose --env-file config/.env logs traefik-forward-auth`

🚫 404 on User Workspace

- Ensure `USERNAME1` / `USERNAME2` in `config/.env` matches the `PathPrefix` in `config/conf.server`.
- Ensure a corresponding service exists in `docker-compose.yml`.

🚧 GitLab "502 Bad Gateway"

GitLab is still initializing. Wait until `docker ps` shows the container as `healthy`.

🔐 Self-Signed Certificate Warning in Browser

TLS certificate files are missing or invalid in `certs/`. Replace them with valid certificates for the domain.

4.2.4 DTaaS Command Line Interface

The DTaaS Command Line Interface (CLI) is a command line tool for managing a DTaaS installation.

Prerequisite

The DTaaS platform with base users and essential containers must be operational before the CLI can be utilised.

Installation

The CLI is available as a Python package that can be installed via pip.

It is recommended to install the CLI in a virtual environment.

The installation steps are as follows:

- Change the working folder:

```
cd <DTaaS-directory>/cli
```

- It is recommended to use a virtual environment. A virtual environment should be created and activated.
- To install the CLI:

```
pip install dtaas
```

Usage

Note

The base DTaaS platform should be up and running before adding/deleting users with the CLI.

CONFIGURE

The CLI uses *dtaas.toml* as configuration file. A sample configuration file is given here.

```
# This is the config for DTaaS CLI

name = "Digital Twin as a Service (DTaaS)"
version = "0.2.1"
owner = "The INTO-CPS-Association"
git-repo = "https://github.com/into-cps-association/DTaaS.git"

[common]
# Server hostname either localhost or a valid hostname, ex: intocps.org
server-dns = "localhost"
# absolute path to the DTaaS application directory
# Specify the directory of DTaaS installation
# Linux example
path = "/Users/username/DTaaS"
# Windows example
#path = "C:\\Users\\XXX\\DTaaS"
# Note: You have to either use / or \\ when specifying path, else you would get
# "Error while getting toml file: dtaas.toml, Invalid unicode value"

[common.resources]
# Default resource limits applied when creating user workspace containers.
# Keys:
# - cpus: integer count of virtual CPUs to allocate to the container
# - mem_limit: memory limit string accepted by Docker (e.g. "4G", "512M")
# - pids_limit: maximum number of processes the container may create
# - shm_size: size for /dev/shm (shared memory), e.g. "512m"
#
# Adjust these values to match your host capacity and tenancy policy.
cpus = 4
mem_limit = "4G"
pids_limit = 4960
shm_size = "512m"

# Example: Increase memory and lower CPU for heavier-memory workloads
# cpus = 2
# mem_limit = "8G"


[users]
# matching user info must present in this config file
add = ["username1","username2", "username3"]
delete = ["username2", "username3"]
...
```

Notes

- Edits to `dtaas.toml` affect new user containers created after the change.
- To apply updated limits to existing containers, recreate or restart the user container(s) (for example by removing and re-adding the user workspace via the CLI or by restarting the container in Docker Compose).
- Use units (`M`, `G`) for memory and shared memory values.

SELECT TEMPLATE

The *cli* uses YAML templates provided in this directory to create new user workspaces. The available templates are:

1. *user.local.yml*: localhost installation
2. *User.server.yml*: multi-user web application over HTTP
3. *user.server.secure.yml*: multi-user web application over HTTPS

It should be noted that the *cli* is not capable of detecting the difference between HTTP and HTTPS modes of the web application. When serving the web application over HTTPS, an additional step is required.

```
cp user.server.secure.yml user.server.yml
```

This will change the user template from insecure to secure.

ADD USERS

To add new users using the CLI, the *users.add* list in *dtaas.toml* should be populated with the GitLab instance usernames of the users to be added.

```
[users]
# matching user info must present in this config file
add = ["username1","username2", "username3"]
```

The working directory must be the *cli* directory.

Then execute:

```
dtaas admin user add
```

The command checks for the existence of `files/<username>` directory. If it does not exist, a new directory with correct file structure is created. The directory, if it exists, must be owned by the user executing **dtaas** command on the host operating system. If the files do not have the expected ownership rights, the command fails.

Caveats

This process brings up the containers, without the AuthMS authentication.

- Currently the *email* fields for each user in *dtaas.toml* are not in use, and are not necessary to complete. These emails must be configured manually for each user in the `deploy/dtaas/docker/server/config/conf.server` file and the *traefik-forward-auth* container must be restarted. This is accomplished as follows:
- Navigate to the secure server package directory

```
cd <DTaaS>/deploy/dtaas/docker/server
```

- Add three lines to `config/conf.server`

```
rule.onlyu3.action=auth
rule.onlyu3.rule=PathPrefix(`/user3`)
rule.onlyu3.whitelist = user3@emailservice.com
```

- Run the command for these changes to take effect:

```
docker compose --env-file config/.env up \
  -d --force-recreate traefik-forward-auth
```

The new users are now added to the DTaaS instance, with authorisation enabled.

DELETE USERS

- To delete existing users, the *users.delete* list in *dtaas.toml* should be populated with the GitLab instance usernames of the users to be deleted.

```
[users]
# matching user info must present in this config file
delete = ["username1","username2", "username3"]
```

- The working directory must be the *cli* directory.

Then execute:

```
dtaas admin user delete
```

- Remember to remove the rules for deleted users in *conf.server*.

ADDITIONAL POINTS TO REMEMBER

- The *user add* CLI will add and start a container for a new user. It can also start a container for an existing user if that container was somehow stopped. It shows a *Running* status for existing user containers that are already up and running, it does not restart them.
- *user add* and *user delete* CLIs return an error if the *add* and *delete* lists in *dtaas.toml* are empty, respectively.
- '.' is a special character. Currently, usernames which have '.'s in them cannot be added properly through the CLI. This is an active issue that will be resolved in future releases.

4.3 Workspace

4.3.1 Localhost

Install

🏭 ↔️ 👥 Digital Twin as a Service DTaaS

🎉 Thank you for downloading **Digital Twin as a Service**.

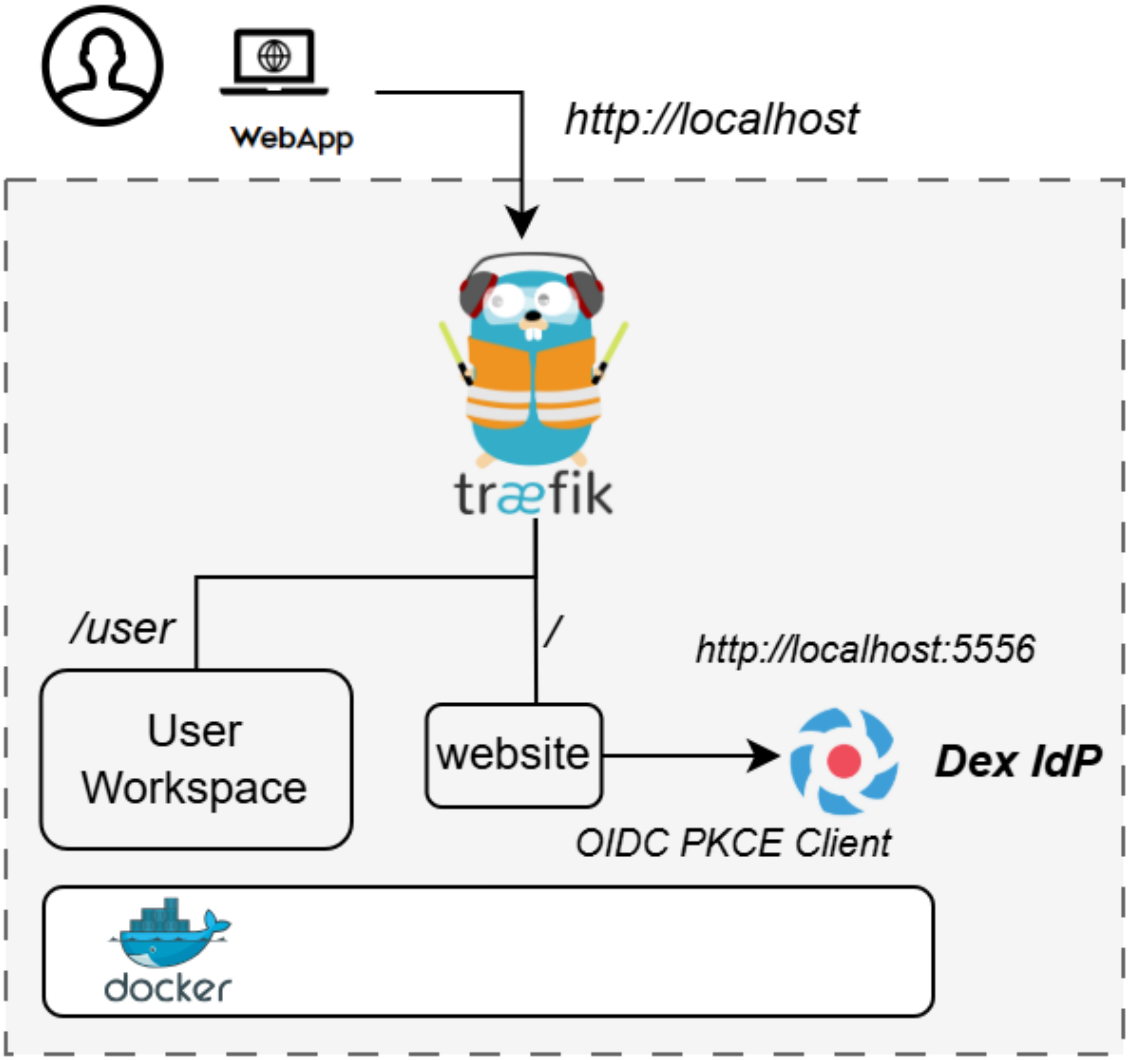
🛠️ INSTALL

The installation instructions provided in this README are ideal for running the **DTaaS on localhost served over HTTP connection**.

✅ This installation is ideal for single users intending to use DTaaS on their own computers.

🏗️ DESIGN

An illustration of the installation setup is shown here.



PREREQUISITES

- Docker Engine v27 or later
- Ports 80 and 5556 available on the host
- At least 2GB RAM available

⚡ QUICK DEMO

The description below refers to filenames. All the commands mentioned below are to be run this directory.

Copy example configuration files first:

```
cp .env.example .env
cp config/dex-config.yaml.example config/dex-config.yaml
```

▶️ Start the demo:

```
docker compose up -d
```

The application will be accessible at http://localhost from web browser. Login using the default credentials.

👤 **User email:** `user@intocps.org` 🔑 **Password:** `user`

🔲 Stop the demo:

```
docker compose down
```

🌵 LIMITATIONS

1. All the functionality of DTaaS except DevOps features should be available through the single page client now.
2. The installation has default user credentials. See instructions for help with changing the user credentials.

📚 DOCUMENTATION

See https://into-cps-association.github.io/DTaaS/development/index.html for complete documentation.

🔗 REFERENCES

Image sources:

Traefik logo: https://www.laub-home.de/wiki/

Dex IdP: https://dexidp.io/

🧩 Dex configuration guide

ℹ️ The `dex-config.yaml.example` file contains Dex configuration template. Duplicate `config/dex-config.yaml.example` to `config/dex-config.yaml`.

📁 FILES

- `dex-config.yaml.example`: template for local/passwordDB mode

⚙️ COMMON OPTIONS USED

🌐 **issuer**

Set to `http://localhost:5556/dex`.

This must match the URL used by the web client (`REACT_APP_AUTH_AUTHORITY`) and the Dex companion/proxy endpoint.

`storage.type: memory`

In-memory state for local/dev usage. Tokens, keys, and sessions reset when the Dex container is recreated.

🌍 **`web.http: 0.0.0.0:5556`**

Dex listens on container port `5556`.

🔓 **`web.allowedOrigins: ['*']`**

Permissive CORS setting for localhost development.

🪪 **`staticClients`**

A single client is configured:

- `id: mock`
- `redirectURIs: ['http://localhost/Library']`
- `public: true` (no client secret required; suitable for browser SPA)

🔐 LOCAL/PASSWORDDB FILE (**`DEX-CONFIG.YAML`**)

⏳ **`expiry.idTokens: "2h"`**

Matches the expected local-dev behaviour with 2-hour ID tokens.

✅ **`oauth2.skipApprovalScreen: true`**

Removes consent page during login for simpler local workflows.

👤 **`enablePasswordDB: true`** + **`staticPasswords`**

Enables Dex local user authentication with static users.

Configured user fields:

- `email`: login identifier used on Dex login screen
- `hash`: bcrypt hash of the password
- `username` / `name` / `preferredUsername`: identity claims returned by Dex
- `groups`: included when `groups` scope is requested
- `userID`: stable Dex subject identifier

ℹ️ In `dex-config.yaml.example`, the sample user password is `user` (bcrypt-hashed).

🔑 Generate a bcrypt password hash

Use the generated value for the `hash` field in `staticPasswords`.

Install the Python dependency if needed:

```
python3 -m pip install bcrypt
```

Generate a bcrypt hash interactively:

```
python3 - <<'PY'
import bcrypt
import getpass

password = getpass.getpass('Password: ').encode('utf-8')
print(bcrypt.hashpw(password, bcrypt.gensalt()).decode('utf-8'))
PY
```

✅ Copy the printed hash into `config/dex-config.yaml` as the value of `hash`.

👤 USERNAME ALIGNMENT WITH `.ENV`

DTaaS routes and workspace paths use `.env` value `DEFAULT_USER`.

For local/passwordDB mode, keep these aligned:

- `.env`: `DEFAULT_USER=<your-user>`
- `config/dex-config.yaml`: set static user `username` and `preferredUsername` to the same `<your-user>`

✅ This prevents path mismatches in user-scoped URLs.

Custom User

🏭 ↔️ 👥 Digital Twin as a Service DTaaS

ℹ️ The instructions in this document help update the username and password for the localhost version of the **DTaaS** installation.

⚙️ UPDATE CONFIGURATION

Duplicate `config/dex-config.yaml.example` to `config/dex-config.yaml`. Update these fields in `config/dex-config.yaml`:

- `email`
- `hash` (bcrypt hash of the chosen password)
- `username`, `name`, `preferredUsername`

Dex configuration details are documented in `dex.md`.

Edit `.env`.

URL Path	Example Value	Explanation
DEFAULT_USER	'user'	Dex username set in `dex-config.yaml`

⚠️ Important alignment for local/passwordDB mode:

- In `.env`, choose `DEFAULT_USER=<your-user>`.
- In `config/dex-config.yaml`, set static user `username` and `preferredUsername` to the same value.

✅ This keeps DTaaS user-scoped routes aligned with OIDC identity claims.

▶️ RUN

Start the application:

```
docker compose up -d
```

Stop the application:

```
docker compose down
```

🌍 Use

The application will be accessible at: http://localhost from web browser. Sign in using the new user credentials in Dex configuration file (`config/dex-config.yaml`).

All the functionality of DTaaS should be available through the single page client now.

📚 DOCUMENTATION

See https://into-cps-association.github.io/DTaaS/development/index.html for complete documentation.

🧩 DEX COMPANION IN DETAIL

The companion service proxies all Dex endpoints and injects a `profile` claim into `/dex/userinfo` when `preferred_username` is present. This keeps the setup self-contained (no GitLab connector) while matching the DTaaS client expectation for username extraction.

ℹ️ Scope note for local/passwordDB mode:

- Supported: `openid profile email groups offline_access`
- Not supported by Dex local passwordDB: `read_user read_repository api`

🔗 REFERENCES

Image sources:

Traefik logo: https://www.laub-home.de/wiki/

Dex IdP: https://dexidp.io/

🧑‍💻 Developer Guide

Instructions for developing and testing the Dex companion proxy.

🧩 ARCHITECTURE

The Dex companion proxy (`companion/src/`) sits between the DTaaS client and the Dex identity provider. It forwards all HTTP requests to Dex and injects a `profile` claim into `/dex/userinfo` responses when `preferred_username` is present.

Module	Purpose
`config.py`	Environment variables and constants
`http_utils.py`	URL handling and HTTP connections
`profile.py`	Profile claim construction and injection
`handler.py`	HTTP request handler (proxy class)
`__main__.py`	Server entry point

PROJECT STRUCTURE

```
companion/
├── src/                # Source package (mounted in container)
│   ├── __init__.py
│   ├── __main__.py     # Entry point
│   ├── config.py       # Environment config and constants
│   ├── handler.py      # HTTP request handler
│   ├── http_utils.py   # URL and connection utilities
│   └── profile.py      # Profile claim injection
└── test/               # Test package
    ├── __init__.py
    ├── conftest.py     # Shared fixtures
    ├── test_handler.py  # Handler method tests
    ├── test_http_utils.py   # URL/connection tests
    ├── test_integration.py  # Full proxy round-trip tests
    └── test_profile.py  # Profile claim tests
```

PREREQUISITES

- Python 3.12+
- pip

SETUP

Install test dependencies:

```
pip install pytest pytest-cov
```

Install linting tools:

```
pip install flake8 pylint ruff
```

RUNNING TESTS

From the `deploy/workspace/dex/localhost/` directory:

```
pytest -v --cov=companion/src --cov-report=xml \
  --cov-report=term-missing companion/test
```

LINTING

From the `deploy/workspace/dex/localhost/` directory:

```
# flake8
flake8 --count --max-complexity=10 companion

# pylint
pylint --fail-under=9 --recursive=y companion

# ruff format check
ruff format --check companion
```

DOCKER

The `docker-compose.yml` mounts only `companion/src/` into the dex-companion container at `/app/companion/src/`. The container runs `python -m companion.src` from `/app`.

To test locally without Docker:

```
cd deploy/workspace/dex/localhost
DEX_UPSTREAM=http://localhost:5556 \
COMPANION_BIND=127.0.0.1 \
COMPANION_PORT=5557 \
python -m companion.src
```

4.3.2 Secure Server

Install

🏭 ↔️ 👥 Digital Twin as a Service DTaaS

🎉 Thank you for downloading **Digital Twin as a Service**.

This guide explains how to deploy the application for secure multi-user deployments.

❓ PREREQUISITES

✅ Docker Engine v27 or later

✅ Sufficient system resources (at least 2GB RAM per workspace instance)

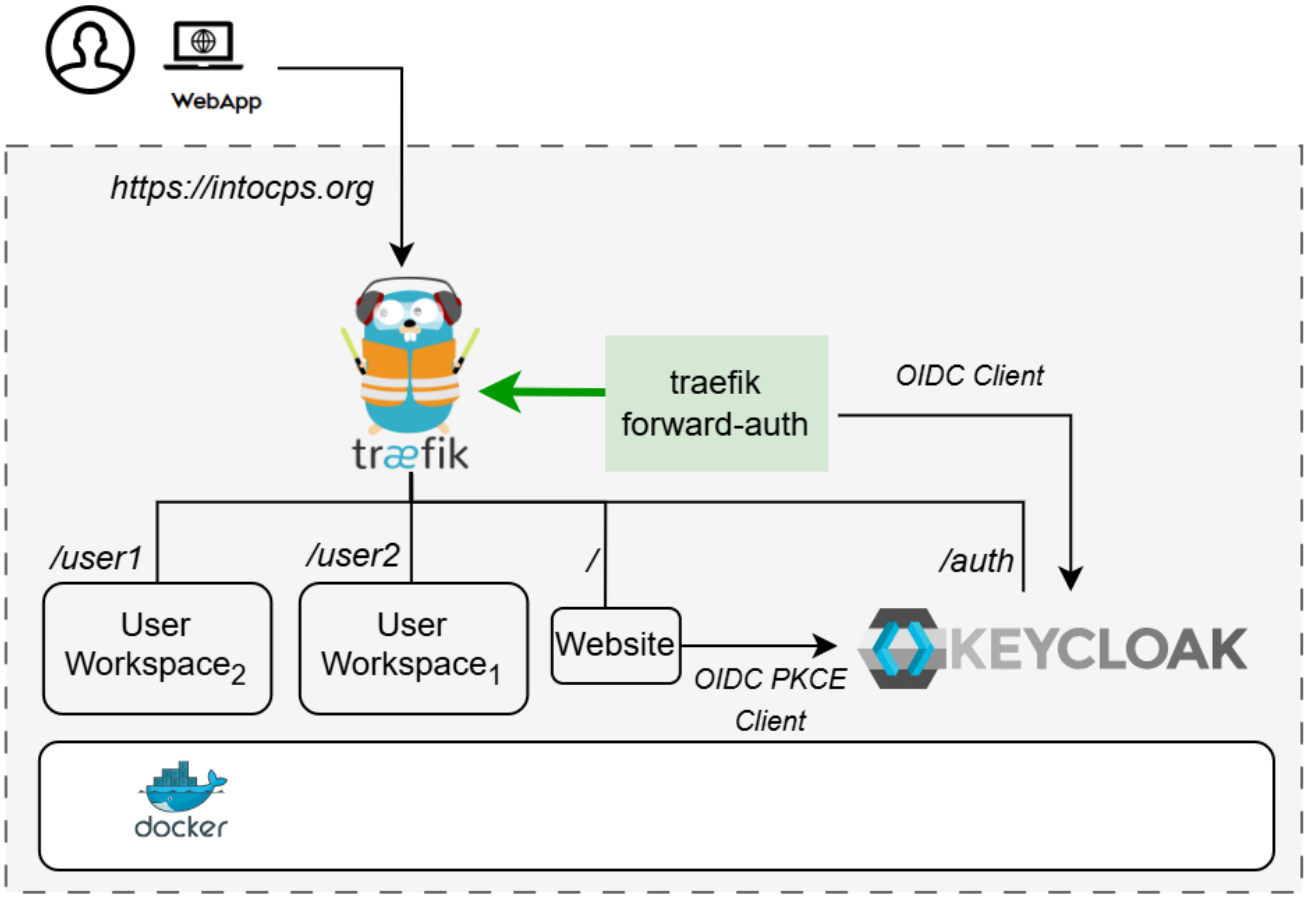
✅ Port 80 and 443 are available on the host

✅ Valid TLS certificates

✅ Domain name pointing to the server

🗒️ DESIGN

An illustration of the installation setup is shown here.



Note: The text starting with / at the beginning indicates the URL route at which a certain service is available. For example, user workspace is available at https://intocps.org/user1

📁 USER DIRECTORIES

All the deployment options require user directories for storing workspace files. These need to be created for `USERNAME1` and `USERNAME2` set in `.env` file.

```
# create required files
cp -R files/template files/<USERNAME1>
cp -R files/template files/<USERNAME2>
# set file permissions for use inside the container
sudo chown -R 1000:100 files
```

⚙️ CONFIGURATION

Follow the pre-installation steps in `configuration.md` for creating valid configuration.

▶️ Start the application:

```
docker compose up -d
```

🌵 Temporary Issues

The following issues in application startup are expected behaviour. This problem will be resolved during post-installation.

👉 `traefik-forward-auth` service will be restarting at this stage. 👉 Visiting `https://intocps.org` shows `HTTP ERROR 500`.

Now complete the post-installation steps in `configuration.md`. Restart `traefik-forward-auth`

```
docker compose up -d --force-recreate traefik-forward-auth
```

The application will be accessible at https://intocps.org from web browser. Login using the user credentials set in **Keycloak**.

Stop the demo:

```
docker compose down
```

To stop and remove volumes:

```
docker compose down -v
```

🔧 CUSTOMIZATION

Adding More Users

Create user account for USERNAME3 in **Keycloak**.

Create the user's workspace directory:

```
cp -r files/template files/<USERNAME3>
sudo chown -R 1000:100 files/<USERNAME3>
```

Add a new service in `docker-compose.yml`:

```
  user3:
    image: intocps/workspace:main-967bc10
    restart: unless-stopped
    environment:
      - MAIN_USER=${USERNAME3:-user3}
    volumes:
      - "./files/common:/workspace/common"
      - "./files/${USERNAME3:-user3}:/workspace"
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.u3.rule=Host(`${SERVER_DNS:-localhost}`) && PathPrefix(`/${USERNAME3:-user3}`)"
      - "traefik.http.routers.u3.tls=true"
      - "traefik.http.routers.u3.middlewares=traefik-forward-auth"
    networks:
      - users
```

Add the desired `USERNAME3` variable in `.env`:

```
# Username Configuration
# These usernames will be used as path prefixes for user workspaces
# Example: http://localhost/user1, http://localhost/user2
USERNAME1=user1
USERNAME2=user2
USERNAME3=user3 # <--- replace "user3" with your desired username
```

Add Forward Auth config for user3 in `forward-auth-conf`:

```
rule.user3_access.action=auth
rule.user3_access.rule=PathPrefix(`/user3`)
rule.user3_access.whitelist = user3@localhost
```

🐛 TROUBLESHOOTING

Certificate Issues

Problem: "NET::ERR_CERT_INVALID" in browser

Solutions:

- Verify certificate files exist in `./certs/` directory
- Check certificate file permissions
- For self-signed certs, add security exception in browser

OAuth2 Issues

Problem: Redirect loop after OAuth2 login

Solutions:

- Verify OAuth2 callback URL matches `https://intocps.org/_oauth`
- Check `SERVER_DNS` environment variable is set correctly
- Ensure `COOKIE_DOMAIN` matches the domain
- Verify OAuth2 application is approved and active

Service Access Issues

Problem: Cannot access workspace after authentication

Solutions:

- Check service health: `docker compose ps`
- View logs: `docker logs`
- Verify Traefik routes: `docker compose logs traefik`
- Test OAuth2 service: `docker compose logs traefik-forward-auth`

Port Conflicts

Problem: Ports 80 or 443 already in use

Solutions:

- Check for other services: `sudo netstat -tlnp | grep -E ':(80|443)'`
- Stop conflicting services
- Or modify port mappings in compose file (not recommended for production)

📚 ADDITIONAL RESOURCES

- Traefik Documentation
- Traefik Forward Auth
- Let's Encrypt Documentation
- Docker Compose Documentation
- OAuth 2.0 Specification

⚙️ DTaaS Configuration

This document outlines the configuration needed for the docker compose file. The configuration can be divided into pre and post-install parts. The pre-install configuration tasks must be completed before bringing up the docker compose services, while the post-install configuration tasks must be completed after bringing up the docker compose services.

Pre-install Configuration Tasks:

- Environment
- Domain
- TLS Certificates
- Usernames
- Forward Auth

Post-install Configuration Tasks:

- Keycloak Integration
- Web Client

🌍 ENVIRONMENT

The compose commands used in the setup guides sets the environment with an environment file. An example of this file can be found at `.env.example`.

Create a copy of this example file without the example suffix:

```
cp .env.example .env
```

🌐 DOMAIN

Decide whether the deployment is local or remote.

From now on whenever `<DOMAIN_NAME>` or `intocps.org` appears in this guide, replace it with the domain name of the remote machine. Ensure that the remote machine has a domain name and that it is accessible from the internet.

Open `.env` and replace the current value of the `SERVER_DNS` variable with the domain name:

```
# Server Configuration
# Replace with your domain name
SERVER_DNS=<DOMAIN_NAME>
```

🔒 TLS CERTIFICATES

Ensure that valid TLS certificates are present on the machine and that they are properly located. The `fullchain.pem` and `privkey.pem` secrets should be located in the `certs/` directory.

There are multiple ways to set up TLS certificates. If hosting on a webserver, Certbot from Let's Encrypt may be used:

```
# Install certbot
sudo apt-get update
sudo apt-get install certbot

# Generate certificates
sudo certbot certonly --standalone -d <DOMAIN_NAME>

# Copy certificates to the project
sudo cp /etc/letsencrypt/live/<DOMAIN_NAME>/fullchain.pem ./certs/
sudo cp /etc/letsencrypt/live/<DOMAIN_NAME>/privkey.pem ./certs/
sudo chown $USER:$USER ./certs/*.pem
chmod 644 ./certs/fullchain.pem
chmod 600 ./certs/privkey.pem
```

👥 USERNAMES

The usernames of the main users for the workspaces can be changed in the environment variable file `.env`. Change the default values (`user1` and `user2`) to the desired usernames:

```
# Username Configuration
# These usernames will be used as path prefixes for user workspaces
# Example: https://intocps.org/user1, https://intocps.org/user2
USERNAME1=user1
USERNAME2=user2
```

NOTE: These usernames must match the names of the keycloak users used in the forward auth.

🚦 TRAEFIK FORWARD AUTH CONFIGURATION

The `config/forward-auth-conf.example` file contains example configuration for the forward-auth service.

Create a copy of this example file without the example suffix:

```
cp config/forward-auth-conf.example config/forward-auth-conf
```

Then update the traefik forward auth configuration file with the usernames and emails of the Keycloak users that correspond to `user1` and `user2` respectively.

```
rule.user1_access.action=auth
rule.user1_access.rule=PathPrefix(`/<USERNAME_USER1>`)
rule.user1_access.whitelist = <EMAIL_USER1>

rule.user2_access.action=auth
rule.user2_access.rule=PathPrefix(`/<USERNAME_USER2>`)
rule.user2_access.whitelist = <EMAIL_USER2>
```

NOTE: Ensure that the usernames set in the Usernames configuration step are the same as those set in the Traefik Forward Auth configuration file.

🎯 KEYCLOAK INTEGRATION

The default configuration for `docker-compose.yml` now uses **Keycloak** for authentication via OIDC (OpenID Connect). Keycloak provides a robust, enterprise-grade identity and access management solution. The `traefik-forward-auth` and the DTaaS `client` docker services use **Keycloak** for authentication and authorisation. An OAuth2 application must be configured for each, using the integrated **Keycloak** service.

For detailed Keycloak setup instructions, see keycloak-setup.md

Configure Environment Variables

1. **For Keycloak (default)**, edit `.env` and fill in the Keycloak credentials:

```
# Keycloak Admin Credentials
KEYCLOAK_ADMIN=admin
KEYCLOAK_ADMIN_PASSWORD=changeme

# Keycloak Realm
KEYCLOAK_REALM=dtaas

# Keycloak Client Credentials (obtain from Keycloak after creating client)
KEYCLOAK_CLIENT_ID=dtaas-workspace
KEYCLOAK_CLIENT_SECRET=your_client_secret_here

# Keycloak Issuer URL
KEYCLOAK_ISSUER_URL=https://<DOMAIN_NAME>/auth/realms/<REALM>

# Secret key for encrypting OAuth session data
# Generate a random string (at least 16 characters)
# for example, using the following command
# openssl rand -base64 32
OAUTH_SECRET=your-oauth-secret
```

🖥️ DTAAS WEB CLIENT CONFIG

The DTaaS Web Client can be configured with a small javascript file, an example of which can be found at `config/client.js.example`.

Create a copy of this example file without the example suffix:

```
cp config/client.js.example config/client.js
```

Then, edit the new DTaaS Web Client config file, updating the following values:

🔑🖥️ Client OAuth2 Setup

The DTaaS web client is a React SPA that authenticates via Keycloak using the **Authorization Code flow with PKCE**. Follow the Create OAuth2 Client for DTaaS Client Service instructions in keycloak-setup.md to create the public PKCE client in Keycloak, then update `config/client.js`:

```
REACT_APP_CLIENT_ID: 'dtaas-client',          // Client ID set in Keycloak
REACT_APP_AUTH_AUTHORITY: 'https://<DOMAIN_NAME>/auth/realms/dtaas',
REACT_APP_REDIRECT_URI: 'https://<DOMAIN_NAME>/Library',
REACT_APP_LOGOUT_REDIRECT_URI: 'https://<DOMAIN_NAME>/',
REACT_APP_GITLAB_SCOPES: 'openid profile',    // OIDC scopes requested from Keycloak
```

`openid` and `profile` are standard OIDC scopes provided by Keycloak by default. `openid` is required for OIDC authentication and issues the ID token. `profile` triggers the `profile` claim mapper configured on the Keycloak client, which returns a URL of the form `https://<DOMAIN_NAME>/<username>` set as a user attribute in Keycloak. The DTaaS web client extracts the username from the last path segment of this URL. The variable is named `REACT_APP_GITLAB_SCOPES` for legacy reasons; it now carries the Keycloak OIDC scopes.

Replace `<DOMAIN_NAME>` with the value set in the Domain section and `dtaas` with the Keycloak realm name if a different one was chosen.

📚 ADDITIONAL RESOURCES

- Let's Encrypt Documentation
- Docker Compose Documentation
- OAuth 2.0 Specification

🔑 Keycloak Setup Guide for DTaaS

This guide explains how to configure Keycloak for authentication in the DTaaS workspace deployment.

🌟 KEY BENEFITS

✅ **Standards-Based**: Uses OIDC/OAuth2 standards

✅ **Enterprise-Ready**: Supports SSO, MFA, user federation

✅ **Minimal Changes**: Environment-variable based configuration

🔍 OVERVIEW

The configuration uses:

- **Keycloak** as the identity provider (IdP) with OIDC support
- **Traefik Forward Auth** to protect routes using OIDC
- **Traefik** as the reverse proxy

📐 ARCHITECTURE

```
User Request → Traefik → Forward Auth → Keycloak (OIDC)
                  ↓
          Protected Service
```

🚀 QUICK START

In this guide, either `<DOMAIN_NAME>` OR `intocps.org` are used as placeholders for server hostname of the installation.

1. Configure Environment Variables

Keycloak-specific environment variables are:

Variable	Purpose	Example
`KEYCLOAK_ADMIN`	Admin username	`admin`
`KEYCLOAK_ADMIN_PASSWORD`	Admin password	`changeme`
`KEYCLOAK_REALM`	Realm name	`dtaas`
`KEYCLOAK_CLIENT_ID`	OIDC client ID	`dtaas-workspace`
`KEYCLOAK_CLIENT_SECRET`	OIDC client secret	`<from-Keycloak>`
`KEYCLOAK_ISSUER_URL`	OIDC issuer URL	`https://intocps.org/auth/realms/dtaas`

Edit Keycloak-configuration in `.env`:

```
# Keycloak Admin Credentials (for initial setup)
KEYCLOAK_ADMIN=admin
KEYCLOAK_ADMIN_PASSWORD=changeme

# Keycloak Realm
KEYCLOAK_REALM=dtaas

# Keycloak Client Configuration (will be created in post-install step)
KEYCLOAK_CLIENT_ID=dtaas-workspace
KEYCLOAK_CLIENT_SECRET=<generated-secret>
```

2. Configure Keycloak

The following instructions are part of post-install step.

Access Keycloak Admin Console

1. Navigate to `https://intocps.org/auth`
2. Click **Administration Console**
3. Login with credentials from the `.env` file (default: `admin` / `changeme`)

Create a Realm

1. In the top-left dropdown (currently showing "Master"), click **Create Realm**
2. **Realm name**: `dtaas` (or match the `KEYCLOAK_REALM` in `.env`)
3. Click **Create**
4. Click on **Realm Settings** -> **User Profile**
5. Click on **Create attribute** with
6. name: profile
7. display name: profile
8. Permission: **who can edit** ✅ on admin
9. Click **Create**

Create Users

1. In the left sidebar, click **Users**
2. Click **Create new user**
3. Fill in user details:
4. **Username**: `user1` (or desired username)
5. **Email**: user's email
6. **First name** / **Last name**: required
7. **Email verified**: OFF
8. **Profile**: `https://intocps.org/auth/realms/dtaas/<USERNAME>` The profile URL is created by adding `/<USERNAME>` to the `<REACT_APP_AUTH_AUTHORITY>` URL used in the `client.js`
9. Click **Create**
10. Set password:
11. Go to the **Credentials** tab
12. Click **Set password**
13. Enter a password
14. **Temporary**: OFF (so users do not have to change it on first login)
15. Click **Save**
16. Repeat for additional users (e.g., `user2`)

Create OAuth2 Client for Traefik Forward-Auth Service

1. In the left sidebar, click **Clients**
2. Click **Create client**
3. Configure the client:
4. **Client type**: OpenID Connect
5. **Client ID**: `dtaas-workspace` (match `KEYCLOAK_CLIENT_ID` in `.env`)
6. Click **Next**
7. Capability config:
8. Client authentication: ON
9. Authorization: OFF
10. Authentication flow: enable **Standard flow**
11. Click **Next**
12. Login settings:
13. **Root URL**: `https://intocps.org`
14. **Valid redirect URIs**:
    - `https://intocps.org/_oauth/*`
    - `https://intocps.org/*`
15. **Valid post logout redirect URIs**: `https://intocps.org/*`
16. **Web origins**: `https://intocps.org`
17. Click **Save**
18. Get the client secret:
19. Go to the **Credentials** tab
20. Copy the **Client secret** value
21. Update `KEYCLOAK_CLIENT_SECRET` in the `.env` file

Create OAuth2 Client for DTaaS Client Service

The DTaaS web client is a React single-page application (SPA) that uses the **Authorization Code flow with PKCE** (Proof Key for Code Exchange). This requires a **public** client (no client secret) with PKCE enforced.

1. In the left sidebar, click **Clients**
2. Click **Create client**
3. Configure the client:
4. **Client type**: OpenID Connect
5. **Client ID**: `dtaas-client`
6. Click **Next**
7. Capability config:
8. **Client authentication**: OFF (public client — no secret needed)
9. **Authorization**: OFF
10. **Authentication flow**: enable **Standard flow** only
11. Click **Next**
12. Login settings:
13. **Root URL**: `https://intocps.org`
14. **Valid redirect URIs**: `https://intocps.org/*`
15. **Valid post logout redirect URIs**: `https://intocps.org/*`
16. **Web origins**: `https://intocps.org`
17. Click **Save**
18. Enforce PKCE:
19. Go to the **Advanced** tab of the client
20. Under **Advanced Settings**, set **Proof Key for Code Exchange Code Challenge Method** to `S256`
21. Click **Save**
22. Verify scopes:
23. Go to the **Client Scopes** tab of the client
24. Confirm that `profile` is listed as an assigned scope

3. Restart Services

After configuring Keycloak, restart the services to apply the new client secret:

```
docker compose down
docker compose up -d
```

4. Test Authentication

1. Navigate to `https://intocps.org`
2. The browser should redirect to the Keycloak login page
3. Log in with one of the created users
4. The browser should redirect back to the DTaaS landing page

🏭 PRODUCTION CONSIDERATIONS

To use an external Keycloak instance (recommended for production):

1. Update `KEYCLOAK_ISSUER_URL` in `.env`:

```
KEYCLOAK_ISSUER_URL=https://keycloak.intocps.org/auth/realms/dtaas
```

Update client redirect URIs in Keycloak to use the production domain

1. Secure Credentials

- Change the default Keycloak admin password
- Use strong client secrets
- Store secrets securely (Docker secrets or external secret managers)
- Rotate secrets regularly

2. Database Backend

For production, configure Keycloak with a proper database (PostgreSQL, MySQL):

```
keycloak:
  environment:
    - KC_DB=postgres
    - KC_DB_URL=jdbc:postgresql://postgres:5432/keycloak
    - KC_DB_USERNAME=keycloak
    - KC_DB_PASSWORD=secure_password
```

🔧 TROUBLESHOOTING

Cannot Access Keycloak Admin Console

- Ensure the Keycloak service is running: `docker compose ps`
- Check Keycloak logs: `docker compose logs keycloak`
- Verify port 80/443 is accessible

Authentication Loop/Redirect Issues

- Verify `KEYCLOAK_ISSUER_URL` matches the realm name
- Ensure redirect URIs in the Keycloak client include `/_oauth/*`
- Confirm `COOKIE_DOMAIN` matches the domain
- Clear browser cookies and retry

"Invalid Client" Error

- Verify `KEYCLOAK_CLIENT_ID` matches the client ID in Keycloak
- Ensure `KEYCLOAK_CLIENT_SECRET` is correct
- Confirm client authentication is enabled for the client

Forward Auth Not Working

- Check traefik-forward-auth logs: `docker compose logs traefik-forward-auth`
- Verify environment variables are set correctly
- Ensure Keycloak is reachable from the traefik-forward-auth container

⚙ ADVANCED CONFIGURATION

Custom Claims and Scopes

To access custom user attributes:

1. In Keycloak, create client scopes with mappers
2. Assign scopes to the client
3. Configure traefik-forward-auth to request additional scopes

Role-Based Access Control (RBAC)

RBAC is supported in Keycloak but not implemented in the traefik-forward-auth service by default.

Single Sign-On (SSO)

Keycloak supports SSO across multiple applications. Configure additional clients for other services as needed.

REFERENCES

- Keycloak Documentation
- Traefik Forward Auth
- OIDC Specification

Workspace with Traefik, OAuth2, and TLS

This guide explains how to deploy the workspace container with Traefik reverse proxy, OAuth2 authentication, and TLS/HTTPS support for secure multi-user deployments.

❓ PREREQUISITES

✅ Docker Engine v27 or later

✅ Docker Compose v2.x

✅ Sufficient system resources (at least 1GB RAM per workspace instance)

✅ Valid TLS certificates (production) or self-signed certs (testing)

✅ Domain name pointing to the server (production) or localhost (testing)

🗒️ DESIGN

```
User Request → Traefik → Forward Auth → Keycloak (OIDC)
                  ↓
          User Workspace
```

The `docker-compose.yml` file provides a production-ready setup with:

- **Traefik** reverse proxy with TLS termination (ports 80, 443)
- **Automatic HTTP to HTTPS redirect**
- **OAuth2 authentication** via traefik-forward-auth
- **Multiple workspace instances** (user1, user2) behind authentication
- **Secure communication** with TLS certificates
- **Workspace containers** (for each user) using the workspace image
- **Docker networks** defined in `docker-compose.yml` to isolate frontend, auth, and user workspaces
- **Two Docker networks**: `dtaas-frontend` and `dtaas-users`

⚙️ INITIAL CONFIGURATION

Follow the steps in `configuration.md` for creating suitable configuration.

CREATE WORKSPACE FILES

All the deployment options require user directories for storing workspace files. These need to be created for `USERNAME1` and `USERNAME2` set in `.env` file.

```
# create required files
cp -R files/template files/<USERNAME1>
cp -R files/template files/<USERNAME2>
# set file permissions for use inside the container
sudo chown -R 1000:100 files
```

🚀 START SERVICES

To start all services with TLS:

```
docker compose up -d
```

This will:

1. Start Traefik reverse proxy with TLS on ports 80 (HTTP → HTTPS redirect) and 443 (HTTPS)
2. Start traefik-forward-auth service for OAuth2 authentication
3. Start workspace instances for user1 and user2, protected by authentication

🕵️ ACCESSING WORKSPACES

Once all services are running, access the workspaces through Traefik with HTTPS:

User Workspace (workspace)

- **VNC Desktop**: `https://intocps.org/user1/tools/vnc?path=user1%2Ftools%2Fvnc%2Fwebsockify`
- **VS Code**: `https://intocps.org/user1/tools/vscode`
- **Jupyter Notebook**: `https://intocps.org/user1`
- **Jupyter Lab**: `https://intocps.org/user1/lab`

Service Discovery

The workspace provides a `/services` endpoint that returns a JSON list of available services. This enables dynamic service discovery for frontend applications.

Example: Get service list for user1

```
curl https://intocps.org/user1/services
```

Response:

```
{
  "desktop": {
    "name": "Desktop",
    "description": "Virtual Desktop Environment",
    "endpoint": "tools/vnc?path=user1%2Ftools%2Fvnc%2Fwebsockify"
  },
  "vscode": {
    "name": "VS Code",
    "description": "VS Code IDE",
    "endpoint": "tools/vscode"
  },
  "notebook": {
    "name": "Jupyter Notebook",
    "description": "Jupyter Notebook",
    "endpoint": ""
  },
  "lab": {
    "name": "Jupyter Lab",
    "description": "Jupyter Lab IDE",
    "endpoint": "lab"
  }
}
```

The endpoint values are dynamically populated with the user's username from the `MAIN_USER` environment variable. This variable corresponds to `USERNAME1` of `.env` file.

🔒 AUTHENTICATION FLOW

1. User attempts to access a workspace URL
2. Traefik forwards the request to traefik-forward-auth
3. If not authenticated, user is redirected to OAuth2 provider
4. User logs in with OAuth2 provider
5. Provider redirects back with authorisation code
6. traefik-forward-auth exchanges code for token and creates session
7. User is redirected to original URL and gains access

🛑 STOPPING SERVICES

To stop all services:

```
docker compose down
```

To stop and remove volumes:

```
docker compose down -v
```

🔧 CUSTOMIZATION

Adding More Users

To add additional workspace instances, add a new service in `docker-compose.yml`:

```
  user3:
    image: intocps/workspace:main-967bc10
    restart: unless-stopped
    environment:
      - MAIN_USER=${USERNAME3:-user3}
    volumes:
      - "./files/common:/workspace/common"
      - "./files/${USERNAME3:-user3}:/workspace"
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.u3.rule=Host(`${SERVER_DNS:-localhost}`) && PathPrefix(`/${USERNAME3:-user3}`)"
      - "traefik.http.routers.u3.tls=true"
      - "traefik.http.routers.u3.middlewares=traefik-forward-auth"
    networks:
      - users
```

Add the desired `USERNAME3` variable in `.env`:

```
# Username Configuration
# These usernames will be used as path prefixes for user workspaces
# Example: http://localhost/user1, http://localhost/user2
USERNAME1=user1
USERNAME2=user2
USERNAME3=user3 # <--- replace "user3" with your desired username
```

Add Forward Auth config for user3 in `forward-auth-conf`:

```
rule.user3_access.action=auth
rule.user3_access.rule=PathPrefix(`/user3`)
rule.user3_access.whitelist = user3@localhost
```

Ensure that the username and email correspond to the workspaces user.

Do not forget to create the user's directory:

```
cp -r files/template files/<USERNAME3>
sudo chown -R 1000:100 files/<USERNAME3>
```

Bring up the user workspace.

```
docker compose up -d --force-recreate user3
```

🐛 TROUBLESHOOTING

Certificate Issues

Problem: "NET::ERR_CERT_INVALID" in browser

Solutions:

- Verify certificate files exist in `./certs/` directory
- Check certificate file permissions
- Ensure `dynamic/tls.yml` correctly references certificate paths
- For self-signed certs, add security exception in browser

OAuth2 Issues

Problem: Redirect loop after OAuth2 login

Solutions:

- Verify OAuth2 callback URL matches `https://intocps.org/_oauth`
- Check `SERVER_DNS` environment variable is set correctly
- Ensure `COOKIE_DOMAIN` matches the domain
- Verify OAuth2 application is approved and active

Port Conflicts

Problem: Ports 80 or 443 already in use

Solutions:

- Check for other services: `sudo netstat -tlnp | grep -E ':(80|443)'`
- Stop conflicting services
- Or modify port mappings in compose file (not recommended for production)

📚 ADDITIONAL RESOURCES

- Traefik Documentation
- Traefik Forward Auth
- Let's Encrypt Documentation
- Docker Compose Documentation
- OAuth 2.0 Specification

4.4 DTaaS Services CLI

The DTaaS Services CLI manages platform services from a generated project structure.

This page now includes both CLI and manual compose operations. Canonical repository copy: `deploy/services/cli.md`.

Source package:

- `deploy/services/cli`

4.4.1 Managed Services

- InfluxDB
- Grafana
- RabbitMQ (with MQTT plugin)
- MongoDB
- ThingsBoard (with PostgreSQL)
- GitLab

4.4.2 Installation

Install the wheel package:

```
pip install dtaas_services-0.3.0-py3-none-any.whl
```

Verify:

```
dtaas-services --help
```

4.4.3 Quick Start

1. Generate a services project:

```
dtaas-services generate-project
```

1. Edit generated config files:
2. `config/services.env`
3. `config/credentials.csv`
4. Prepare certificates/permissions:

```
dtaas-services setup
```

1. Start services:

```
dtaas-services start
```

4.4.4 Manual Compose Operations

After generating a services project, services may be operated manually with compose files:

- `compose.services.yml`
- `compose.thingsboard.yml`
- `compose.gitlab.yml`

Start manually:

```
docker compose -f compose.services.yml --env-file config/services.env up -d
docker compose -f compose.thingsboard.yml --env-file config/services.env up -d
docker compose -f compose.gitlab.yml --env-file config/services.env up -d
```

Stop manually:

```
docker compose -f compose.services.yml --env-file config/services.env down
docker compose -f compose.thingsboard.yml --env-file config/services.env down
docker compose -f compose.gitlab.yml --env-file config/services.env down
```

4.4.5 Core Commands

Project and setup

```
dtaas-services generate-project [--path <dir>]
dtaas-services setup
```

Service lifecycle

```
dtaas-services start [-s influxdb,rabbitmq]
dtaas-services stop [-s influxdb]
dtaas-services restart [-s mongodb]
dtaas-services status [-s gitlab]
dtaas-services remove [-s service1,service2] [-v]
```

User operations

```
dtaas-services user add
dtaas-services user add -s rabbitmq
dtaas-services user reset-password -s thingsboard
dtaas-services user reset-password -s gitlab
```

4.4.6 GitLab and ThingsBoard Notes

• GitLab install and post-install setup are supported in the CLI.

• ThingsBoard installation depends on PostgreSQL readiness.

• For GitLab integration and runner setup, see:

- `deploy/services/cli/GITLAB_INTEGRATION.md`
- `deploy/services/runner/GITLAB-RUNNER.md`

4.4.7 Troubleshooting

Permissions (Linux/macOS)

```
sudo -E env PATH="$PATH" dtaas-services setup
```

Docker connectivity

```
docker ps
```

Source of truth

For full and latest command details, use:

- `deploy/services/cli/README.md`
- `deploy/services/cli.md`

4.5 GitLab

4.5.1 GitLab Server

DTaaS currently supports local GitLab installation using either:

1. `deploy/dtaas/docker/secure-server_with_integrated-gitlab`
2. `deploy/services/cli` GitLab workflow

Option A: Integrated DTaaS Package

Use `deploy/dtaas/docker/secure-server_with_integrated-gitlab` when GitLab should run behind Traefik at `https://<SERVER_DNS>/gitlab`.

1. Create runtime config files from examples.
2. Start package services:

```
cd deploy/dtaas/docker/secure-server_with_integrated-gitlab
docker compose --env-file config/.env up -d
```

1. Wait for GitLab health to become `healthy`.
2. Retrieve root password from `config/gitlab/initial_root_password`.
3. Continue with integration guide.

Option B: Services CLI GitLab Workflow

Use `deploy/services/cli` if GitLab is managed as part of the platform services project.

1. Generate/open a services project.
2. Configure `config/services.env`.
3. Install GitLab using CLI:

```
dtaas-services install -s gitlab
```

1. Continue with integration guide.

Notes

- DTaaS client auth authority must match the actual GitLab endpoint.
- For integrated package deployments, use the package-local `config/` files.
- For services-cli deployments, use the generated services project files.

Related

- Integration guide
- Runner setup (Linux)
- Runner setup (Windows)
- DTaaS integrated package config
- Workspace secure server config

4.5.2 GitLab Integration Guide

This guide summarizes how to integrate GitLab OAuth with DTaaS in current package layouts.

Integration Paths

There are two primary integration paths:

1. **Integrated package**: `deploy/dtaas/docker/secure-server_with_integrated-gitlab`
2. **Services CLI path**: `deploy/services/cli` + `deploy/services/cli/GITLAB_INTEGRATION.md`

A. Integrated Package (`secure-server_with_integrated-gitlab`)

1. Start DTaaS + GitLab:

```
cd deploy/dtaas/docker/secure-server_with_integrated-gitlab
docker compose --env-file config/.env up -d
```

1. Wait until GitLab is healthy.
2. Create users in GitLab matching `USERNAME1` / `USERNAME2` .
3. Create OAuth applications in GitLab:
4. DTaaS Client Authorization
5. DTaaS Server Authorization
6. Update:
7. `config/client.js`
8. `config/.env`
9. Reload services:

```
docker compose --env-file config/.env up -d --force-recreate client traefik-forward-auth
```

B. Services CLI GitLab Integration

Use the services project and follow the authoritative guide:

- `deploy/services/cli/GITLAB_INTEGRATION.md`

Typical flow:

1. Generate services project: `dtaas-services generate-project`
2. Configure `config/services.env`
3. Install GitLab: `dtaas-services install -s gitlab`
4. Apply OAuth values to DTaaS package config
5. Restart DTaaS `client` and `traefik-forward-auth`

Post-Setup Checks

- `https://<host>/gitlab` is reachable (integrated package)
- DTaaS login redirects to expected OAuth provider
- Workspace routes (`/user1` , `/user2`) are protected

Related Guides

- DTaaS integrated package config
- Workspace secure server config

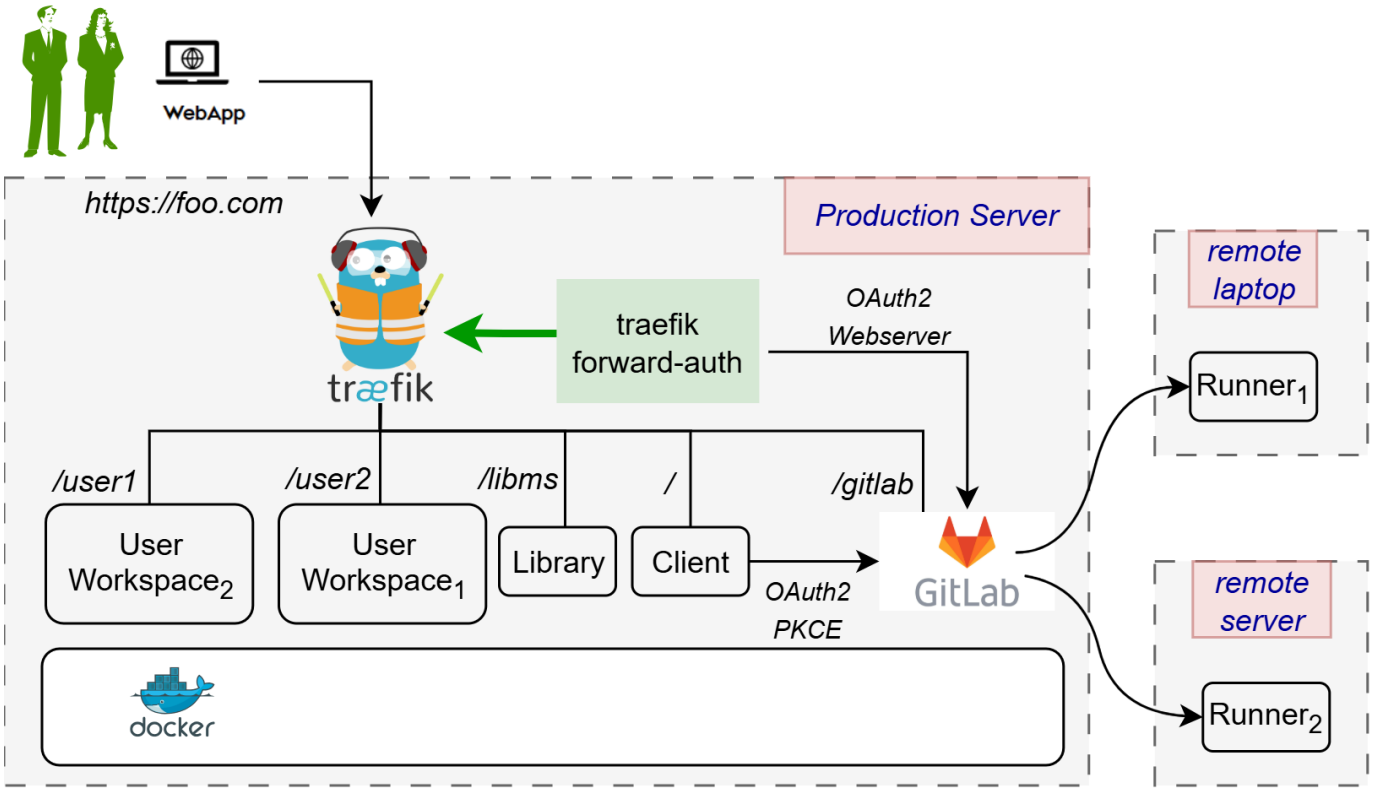
4.5.3 Runner

GitLab Runner Integration

For Windows-specific setup guidance, see `runner-windows.md`.

This document outlines the steps needed to create a `gitlab-runner` that will be responsible for the execution of Digital Twins. Many such runners can be installed and linked with the integrated GitLab.

An illustration of the intended installation setup is shown below.



There are two installation scenarios:

1. **Localhost Installation** - The integrated runner is used locally with a GitLab instance hosted at `https://localhost/gitlab`.
2. **Server Installation** - The integrated runner is used with a GitLab instance hosted on a production server. This server may be a remote server and not necessarily the host, and may have TLS enabled with a self-signed certificate.

Following the steps below sets up the integrated runner which can be used to execute digital twins from the Digital Twins Preview Page.

PREREQUISITES

A GitLab Runner picks up CI/CD jobs by communicating with a GitLab instance. For an explanation of how to set up a GitLab instance that integrates with a DTaaS platform, refer to the GitLab instance document and the GitLab integration guide.

The rest of this document assumes a running DTaaS platform with a GitLab instance is in place.

RUNNER SCOPES

A GitLab Runner can be configured for three different scopes:

Runner Scope	Description
Instance Runner	Available to all groups and projects in a GitLab instance.
Group Runner	Available to all projects and subgroups in a group.
Project Runner	Associated with one specific project.

Creating **instance runners** is recommended as they are the most straightforward, but any type will work. More about these three types can be found on the official GitLab documentation page.

OBTAINING A REGISTRATION TOKEN

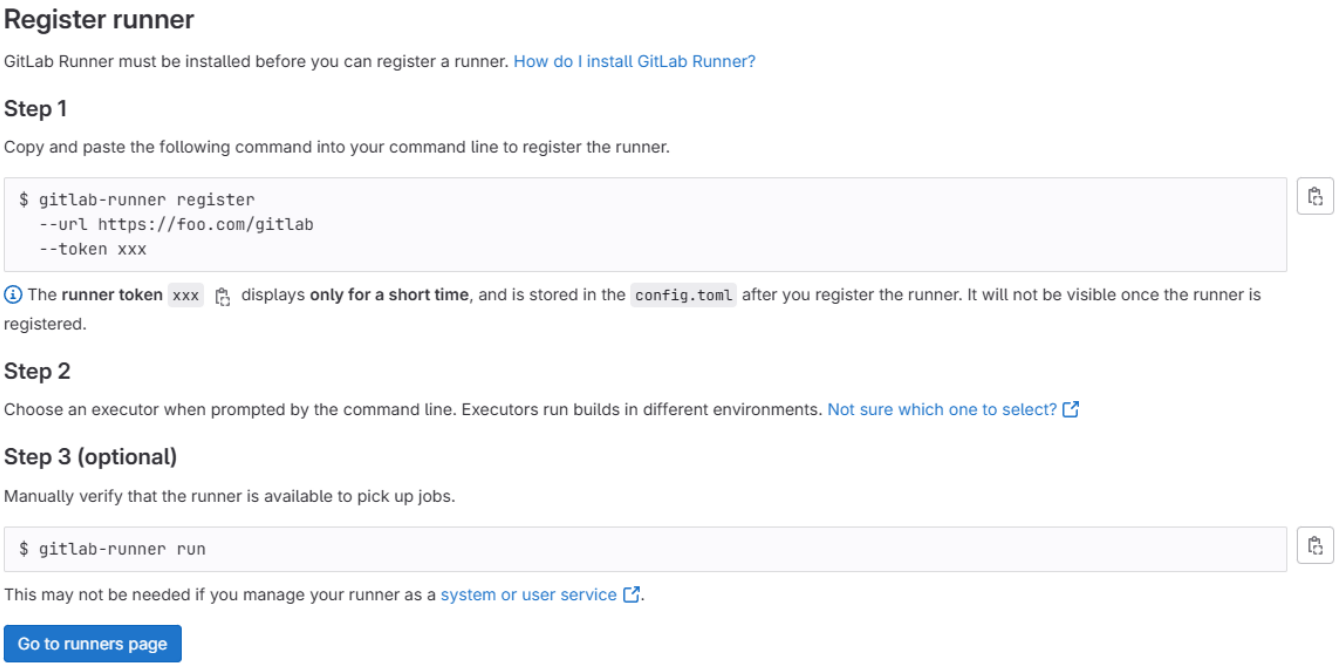
First, obtain the token necessary to register the runner for the GitLab instance. Open the GitLab instance (remote or local) and depending on the chosen runner scope, follow the steps given below:

Runner Scope	Steps
Instance Runner	1. On the **Admin** dashboard, navigate to **CI/CD** > **Runners**. 2. Select **New instance runner**.
Group Runner	1. On the **DTaaS** group page, navigate to **Settings** > **CI/CD** > **Runners**. 2. Ensure the **Enable shared runners for this group** option is enabled. 3. On the **DTaaS** group page, navigate to **Build** > **Runners**. 4. Select **New group runner**.
Project Runner	1. On the **DTaaS** group page, select the project named after the GitLab username. 2. Navigate to **Settings** > **CI/CD** > **Runners**. 3. Select **New project runner**.

For any chosen scope, a page to create a runner will be displayed:

1. Under **Platform**, select the Linux operating system.
2. Under **Tags**, add a `linux` tag.
3. Select **Create runner**.

The following screen should then appear:



Be sure to save the generated runner authentication token.

CONFIGURING THE RUNNER

Depending on the installation scenario, the runner setup reads certain configuration settings:

1. **Localhost Installation (legacy compose files)** uses `deploy/docker/.env.local`.
2. **Server Installation (legacy compose files)** uses `deploy/docker/.env.server`.
3. **Integrated package installation** uses `deploy/dtaas/docker/secure-server_with_integrated-gitlab/config/.env`.

The runner compose templates under `deploy/services/runner` were originally written for `deploy/docker/*` layouts. For package-based installs, update certificate mount paths in `compose.runner.local.yml` and `compose.runner.server.yml` to match the active package directory.

The runner must be registered with the GitLab instance so that they may communicate with each other. `deploy/services/runner/runner-config.toml` has the following template:

```
[[runners]]
  name = "dtaas-runner-1"
  url = "https://intocps.org/gitlab/" # Edit this
  token = "xxx" # Edit this
  executor = "docker"
  [runners.docker]
    tls_verify = false
    image = "ruby:2.7"
    privileged = false
    disable_entrypoint_overwrite = false
    oom_kill_disable = false
    volumes = ["/cache"]
    network_mode = "host" # Disable this in secure contexts
```

1. Set the `url` variable to the URL of the GitLab instance.
2. Set the `token` variable to the runner registration token obtained earlier.
3. For the server installation scenario, remove the line `network_mode = "host"`.

A list of advanced configuration options is provided on the GitLab documentation page.

START THE GITLAB RUNNER

The following commands may be used to start and stop the `gitlab-runner` container respectively, depending on the installation scenario:

1. Go to the DTaaS home directory (`DTaaS_DIR`) and execute one of the following commands.
2. Localhost Installation

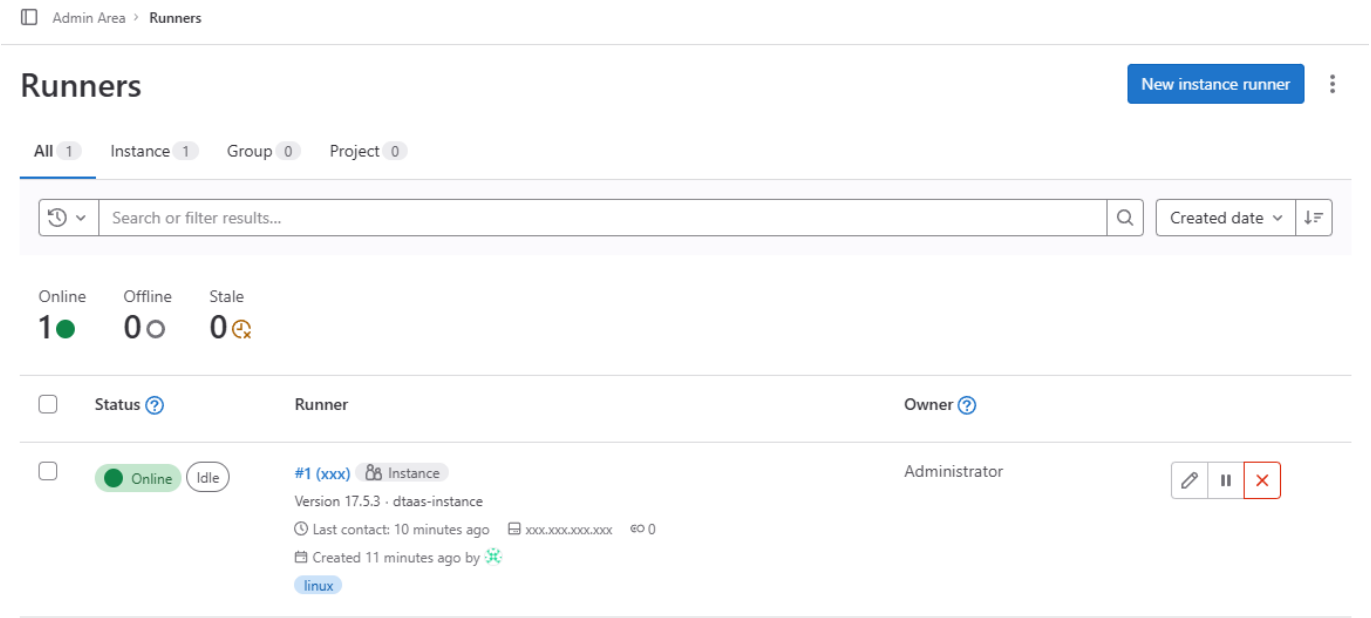
```
docker compose -f deploy/services/runner/compose.runner.local.yml \
  --env-file deploy/docker/.env.local up -d
docker compose -f deploy/services/runner/compose.runner.local.yml \
  --env-file deploy/docker/.env.local down
```

3. Server Installation

```
docker compose -f deploy/services/runner/compose.runner.server.yml \
  --env-file deploy/docker/.env.server up -d
docker compose -f deploy/services/runner/compose.runner.server.yml \
  --env-file deploy/docker/.env.server down
```

For package-based integrated installations, use the same compose files but set `--env-file` to `deploy/dtaas/docker/secure-server_with_integrated-gitlab/config/.env` and adjust the certificate volume mounts in the compose file accordingly.

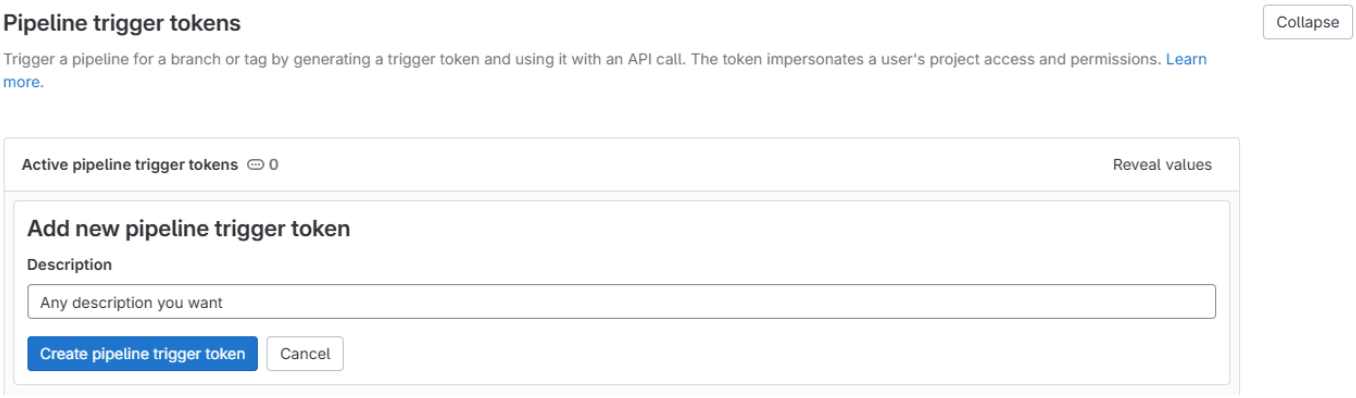
Once the container starts, the runner within it will run automatically. Whether the runner is operational can be verified by navigating to the page where the runner was created. For example, an Instance Runner would look like this:



A GitLab runner is now ready to accept jobs for the GitLab instance.

PIPELINE TRIGGER TOKEN

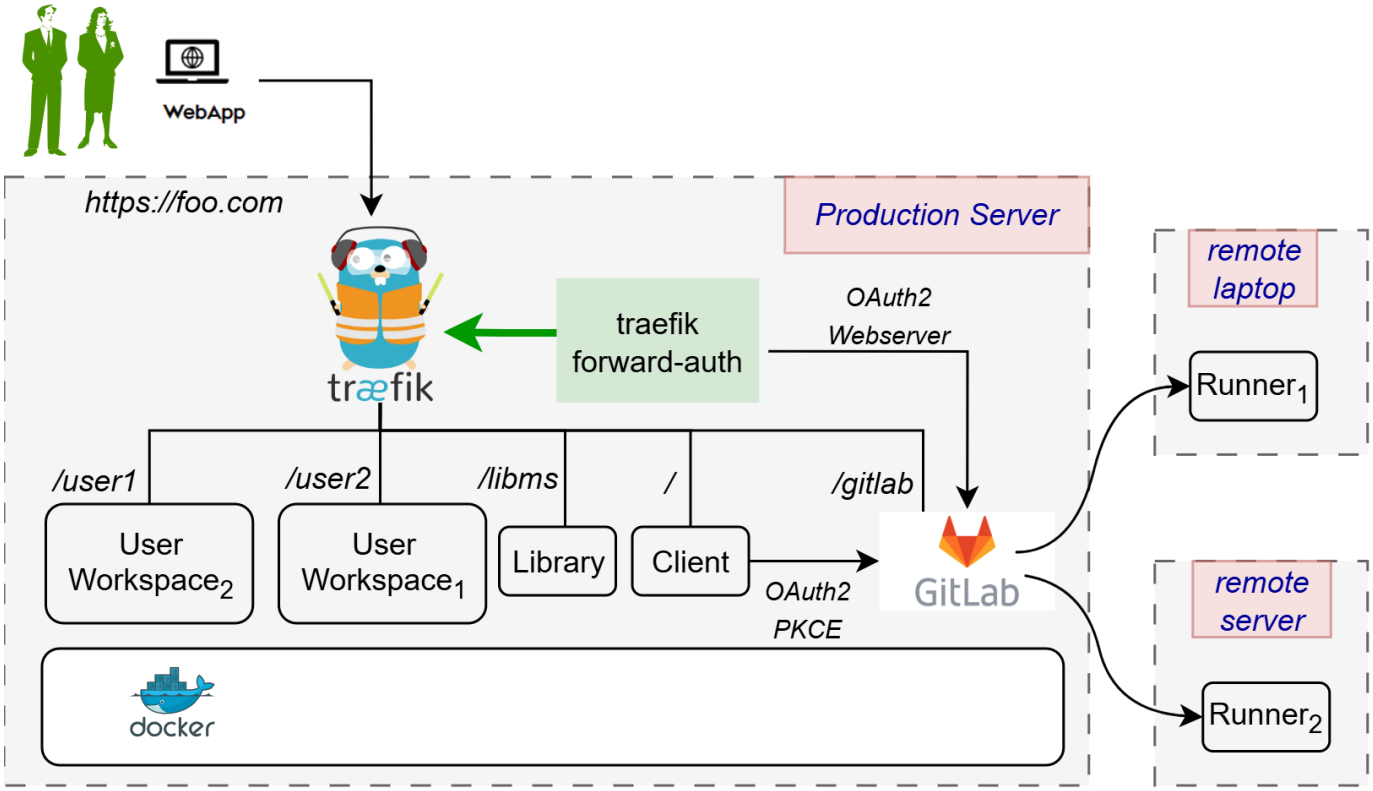
The Digital Twins Preview Page uses the GitLab API which requires a Pipeline Trigger Token. Go to the project in the **DTaaS** group and navigate to **Settings** > **CI/CD** > **Pipeline trigger tokens**. Add a new token with any description as desired.



The Digital Twins Preview Page can now be used to manage and execute digital twins.

Setting up GitLab Runners with Docker on Windows for DTaaS

This guide documents how to properly set up and configure GitLab runners with Docker on Windows for the DTaaS platform. An illustration of the intended installation setup is shown below.



There are two installation scenarios:

STEP-BY-STEP SETUP PROCESS

1. Install GitLab Runner

Download and install GitLab Runner for Windows from GitLab's official download page.

```
# Navigate to your download directory
cd C:\path\to\download\folder

# Run the GitLab Runner
.\gitlab-runner.exe install

# Start the service
.\gitlab-runner.exe start
```

2. Getting a token

To obtain the GitLab token, first navigate to the project page (e.g. https://dtaas-digitaltwin.com/gitlab/dtaas/USERNAME), then do the following:

1. settings -> CI/CD -> Runners
2. Now press **New Project Runner**
3. Add tag `linux`
4. Leave the rest as is, and press **Create Runner**
5. The token is now displayed. Save it.

The runner token is now available.

3. Register Your Runner for DTaaS

For the DTaaS project, register the runner using the specific GitLab instance URL and token:

```
# Register the runner for DTaaS
.\gitlab-runner.exe register --url "https://intocps.org/gitlab" --token ""

# When prompted, enter:
# - Name: [Your machine name or any preferred name]
# - Executor: docker
# - Default Docker image: ruby:2.7
# - Tags: linux
```

This configuration is designed for the DTaaS digital twins which require a Linux environment to run properly. The pipelines use shell scripts with commands like `chmod +x`, which need a Linux-compatible environment.

4. Configure Your config.toml

The most important part is properly configuring the `config.toml` file, which is typically located at `C:\Users\YourUsername\.gitlab-runner\config.toml` or in the directory where the gitlab-runner executable was downloaded and run.

DTaaS Configuration for Windows Hosts

Here is the recommended configuration for running DTaaS Digital Twins on Windows:

```
concurrent = 1
check_interval = 0
connection_max_age = "15m0s"
shutdown_timeout = 0

[session_server]
  session_timeout = 1800

[[runners]]
  name = "Some name"
  url = "https://dtaas-digitaltwin.com/gitlab"
  id = 3
  token = "YOUR_RUNNER_TOKEN"
  token_obtained_at = 2025-03-17T19:50:25Z
  token_expires_at = 0001-01-01T00:00:00Z
  executor = "docker"
  [runners.custom_build_dir]
    enabled = false
  [runners.cache]
    MaxUploadedArchiveSize = 0
    [runners.cache.s3]
    [runners.cache.gcs]
    [runners.cache.azure]
  [runners.feature_flags]
    FF_NETWORK_PER_BUILD = false
  [runners.docker]
    tls_verify = false
    image = "ruby:2.7"
    privileged = false
    disable_entrypoint_overwrite = false
    oom_kill_disable = false
    disable_cache = false
    volumes = ["/cache"]
    shm_size = 0
    network_mtu = 0
  tags = ["linux"]
```

6. Restart the Runner

After making these configuration changes, restart the runner:

```
.\gitlab-runner.exe stop
.\gitlab-runner.exe start
```

VERIFYING YOUR SETUP

Run a verification check to ensure the runner is properly configured:

```
.\gitlab-runner.exe verify
```

A successful verification will show something like:

```
Verifying runner... is valid       runner=YourRunnerToken
```

UNDERSTANDING THE DTAAS PROJECT SETUP

The DTaaS project uses a specific structure for running digital twins:

1. Digital twins are contained in the `digital_twins` directory
2. Each digital twin has lifecycle scripts (create, execute, terminate, clean)
3. The GitLab CI/CD pipeline triggers these scripts based on user actions
4. The scripts need a Linux environment to execute properly

COMMON ERRORS

When running GitLab CI/CD pipelines on Windows with Docker, the following errors may be encountered:

```
ERROR: Failed to remove network for build
ERROR: Job failed: invalid volume specification: "c:\\cache"
```

These errors are typically caused by:

1. Incorrect Docker executor configuration
2. Windows-style path specification not being compatible with Docker
3. Mismatch between the runner's executor type and the pipeline requirements

CONCLUSION

By following this guide, it should be possible to properly set up GitLab runners with Docker on Windows for the DTaaS project and avoid the common configuration errors. The most crucial aspects are using the Docker (Linux) executor and properly formatting the volume paths.

Remember that for the DTaaS digital twins, using the standard Docker executor is required, even when running on a Windows host, since the scripts are designed to run in a Linux environment.

4.6 Independent Packages

4.6.1 Independent Packages

The DTaaS development team publishes reusable packages which are then put together to form the complete DTaaS application.

The packages are published on github, npmjs, and docker hub repositories.

The packages on github are published more frequently but are not user tested. The packages on npmjs and docker hub are published at least once per release. The regular users are encouraged to use the packages from npm and docker hub.

A brief explanation of the packages is given below.

Package Name	Description	Documentation for	Availability
dtaas-web	React web application	Useful only for DevOps features. The workspace features will not be available in standalone package.	docker hub and github
libms	Library microservice	npm package	npmjs and github
		container image	docker hub and github
runner	REST API wrapper for multiple scripts/ programs	npm package	npmjs and github
workspace	User workspace container image	workspace localhost and workspace secure server	docker hub and `intocps/workspace:*` images referenced by deployment packages

Workspace Packages

The workspace deployments are maintained as package directories in:

- `deploy/workspace/dex/localhost`
- `deploy/workspace/keycloak/production`

These package directories include deployment templates, runtime configuration examples, and compose definitions for workspace-focused installations.

4.6.2 Workspace Deployment Scenarios

Workspace deployments are focused on user workspace access patterns and identity providers.

Available Workspace Scenarios

Scenario	Purpose	Source Directory
Dex localhost	Single-user local workspace deployment	`deploy/workspace/dex/localhost`
Keycloak secure server	Multi-user secure server deployment with OIDC	`deploy/workspace/keycloak/production`

Notes

- Use workspace scenarios when the primary focus is workspace auth and per-user route access.
- For DTaaS package deployments that include the full DTaaS web platform, use `../dtaas/localhost/install.md`, `../dtaas/server/install.md`, or `../dtaas/secure-server-gitlab/install.md`.

4.6.3 Library Microservice

Host Library Microservice

The **lib microservice** is a simplified file manager that serves files over GraphQL and HTTP API.

It has two features:

- Provide a listing of directory contents.
- Upload and download files

This document provides instructions for installing the npm package of the library microservice and running the same as a standalone service.

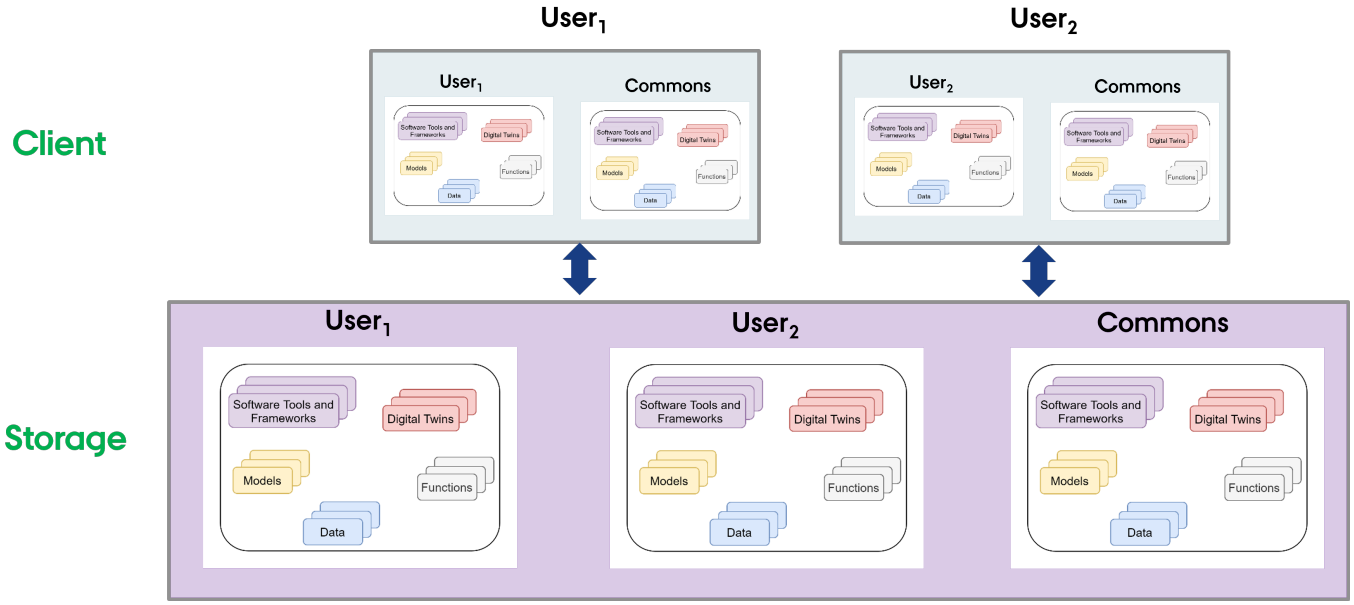
SETUP THE FILE SYSTEM

Outside the DTaaS Platform

The package can be used independently of the DTaaS. In this use case, no specific file structure is required. Any valid file directory is sufficient.

Inside the DTaaS Platform

The users of the DTaaS expect the following file system structure for their reusable assets.



A skeleton file structure is available in the DTaaS codebase. This can be copied to create a file system for users.

INSTALL

The npm package is available in GitHub packages registry and on npmjs. **Prefer the package on npmjs over GitHub**.

Set the registry and install the package with the one of the two following commands

npmjs

```
sudo npm install -g @into-cps-association/libms  # requires no login
```

GitHub

```
# requires login
sudo npm config set @into-cps-association:registry https://npm.pkg.github.com
```

The *github package registry* asks for username and password. The username is the GitHub username and the password is the GitHub personal access token. In order for the npm to download the package, the personal access token needs to have *read:packages* scope.

🚀 USE

Display help.

```
$libms -h
Usage: libms [options]

The lib microservice is a file server. It supports file transfer
over GraphQL and HTTP protocols.

Options:
  -c, --config <file>  provide the config file (default libms.yaml)
  -H, --http <file>    enable the HTTP server with the specified config
  -h, --help           display help for libms
```

Both the options are not mandatory.

See configuration for explanation of configuration conventions. The config is saved `libms.yaml` file by convention. If `-c` is not specified The **libms** looks for `libms.yaml` file in the working directory from which it is run. To run **libms** without explicitly specifying the configuration file, run

```
$libms
```

To run **libms** with a custom config file,

```
$libms -c FILE-PATH
$libms --config FILE-PATH
```

If the environment file is named something other than `libms.yaml`, for example as `libms-config.yaml`, run

```
$libms -c "config/libms-config.yaml"
```

Press `Ctl+C` to halt the application. To run the microservice in the background, use

```
$nohup libms [-c FILE-PATH] & disown
```

The lib microservice is now running and ready to serve files.

Protocol Support

The **libms** supports GraphQL protocol by default. This microservice can also serve files in a browser with files transferred over HTTP protocol.

This option needs to be enabled with `-H http.json` flag. A sample http config provided here can be used.

```
$nohup libms [-H http.json] & disown
```

The regular file upload and download options become available.

SERVICE ENDPOINTS

The GraphQL URL: `localhost:PORT/lib`

The HTTP URL: `localhost:PORT/lib/files`

The service API documentation is available on user page.

Host Library Microservice

The **lib microservice** is a simplified file manager that serves files over GraphQL and HTTP API.

It has two features:

- Provide a listing of directory contents.
- Transfer a file to the user.

This document provides instructions for running a docker container to provide a standalone library microservice.

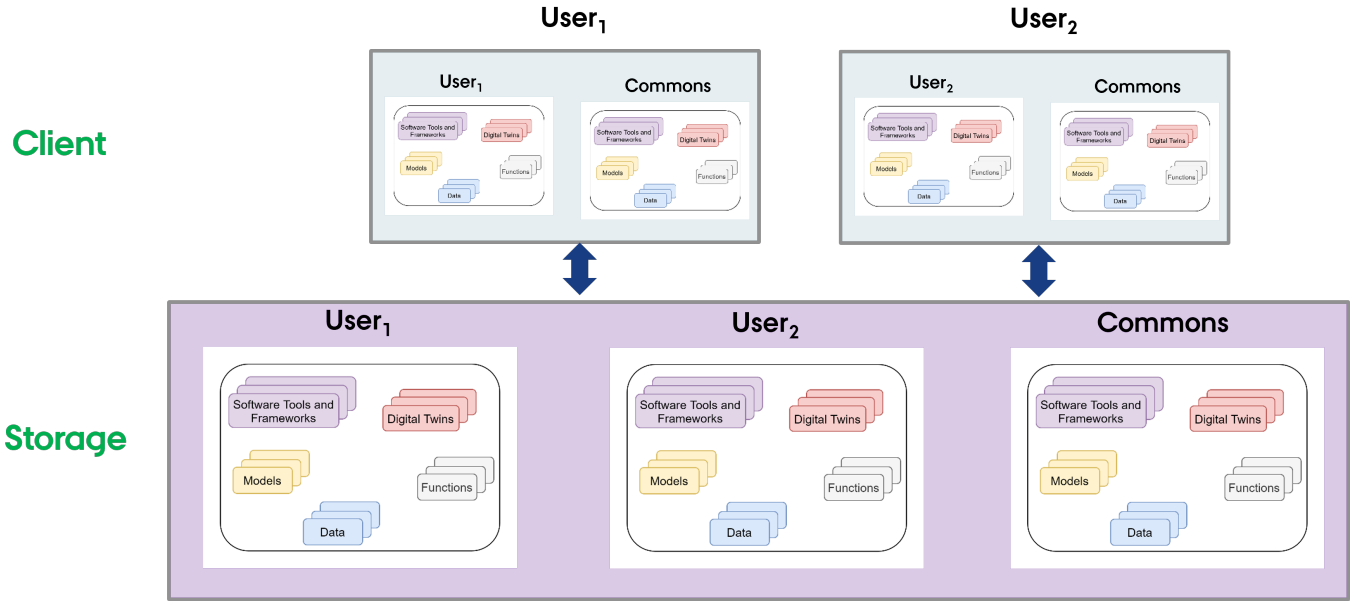
SETUP THE FILE SYSTEM

Outside the DTaaS Platform

The package can be used independently of the DTaaS. In this use case, no specific file structure is required. A valid file directory named `files` is sufficient and should be placed in the directory from which `compose.lib.yml` will be run.

Inside the DTaaS Platform

The users of DTaaS expect the following file system structure for their reusable assets.



A skeleton file structure is available in the DTaaS codebase. This can be copied to create a file system for users. The directory containing the file structure should be named `files` and placed in the directory from which `compose.lib.yml` will be run.

🚀 USE

Use the docker compose file to start the service.

```
# To bring up the container
docker compose -f compose.lib.yml up -d
# To bring down the container
docker compose -f compose.lib.yml down
```

SERVICE ENDPOINTS

The GraphQL URL: `localhost:4001/lib`

The HTTP URL: `localhost:4001/lib/files`

The service API documentation is available on user page.

4.7 Guides

4.7.1 💻 Install DTaaS on Localhost (GUI)

The installation instructions provided in this document are ideal for running DTaaS on localhost via a Graphical User Interface (GUI). This installation is ideal for single users intending to use DTaaS on their own computers.

Two installation scenarios are available:

Scenario	Auth Provider	External Account Required
DTaaS Localhost	GitLab OAuth	Yes (gitlab.com)
Workspace Localhost	Dex (local)	No

📋 Requirements

- Docker Desktop or Docker Engine with Compose plugin
- Portainer Community Edition (setup below)

Clone Codebase

```
git clone https://github.com/INTO-CPS-Association/DTaaS.git
cd DTaaS
```

> **Tip**
>
> The guide uses Linux-style paths such as `/Users/username/DTaaS`. On Windows use an equivalent path, for example `C:\DTaaS`.

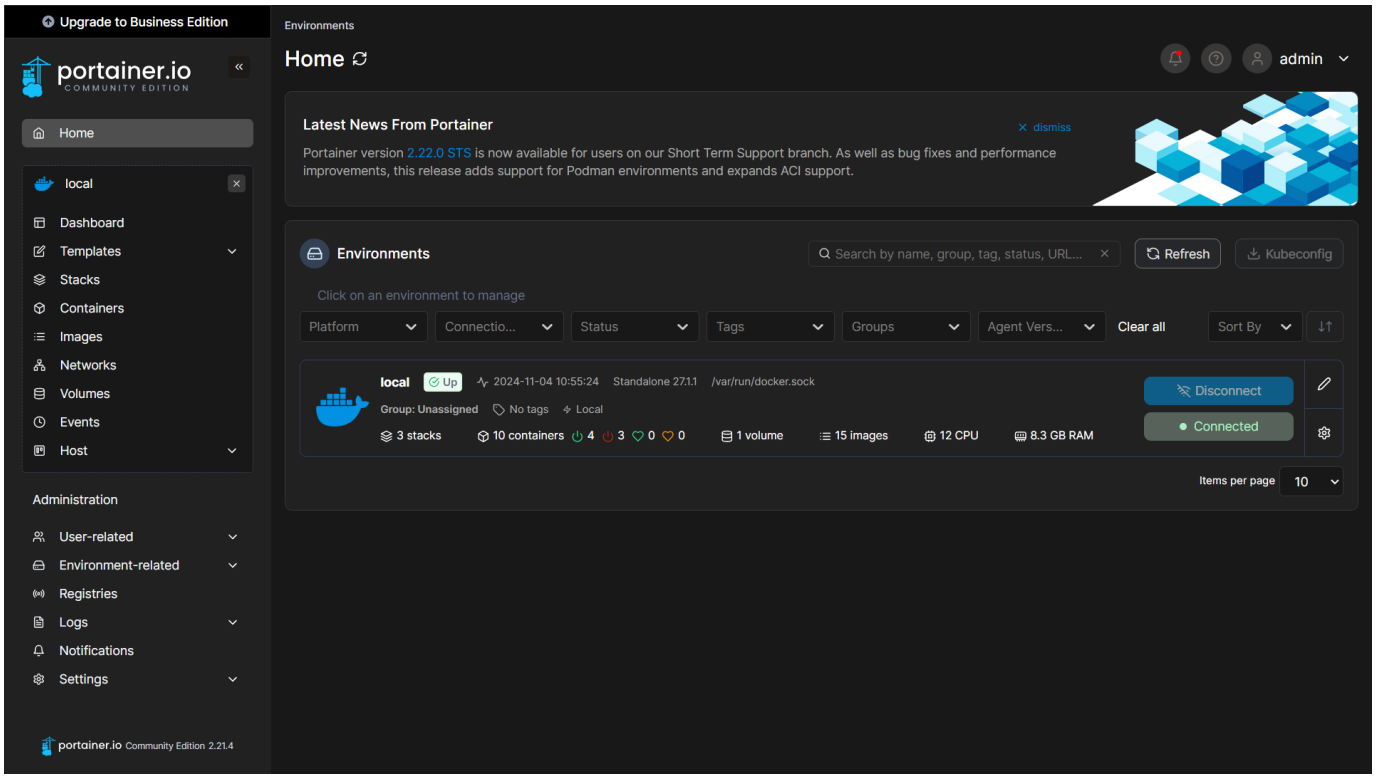
Starting Portainer

Portainer Community Edition provides a graphical interface for managing Docker containers at `https://localhost:9443`.

Follow the official Portainer CE documentation or run the commands below:

```
docker volume create portainer_data
docker run -d -p 8000:8000 -p 9443:9443 --name portainer --restart=always \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v portainer_data:/data portainer/portainer-ce:2.21.4
```

Open `https://localhost:9443` and complete the Initial Setup to create an administrator account.

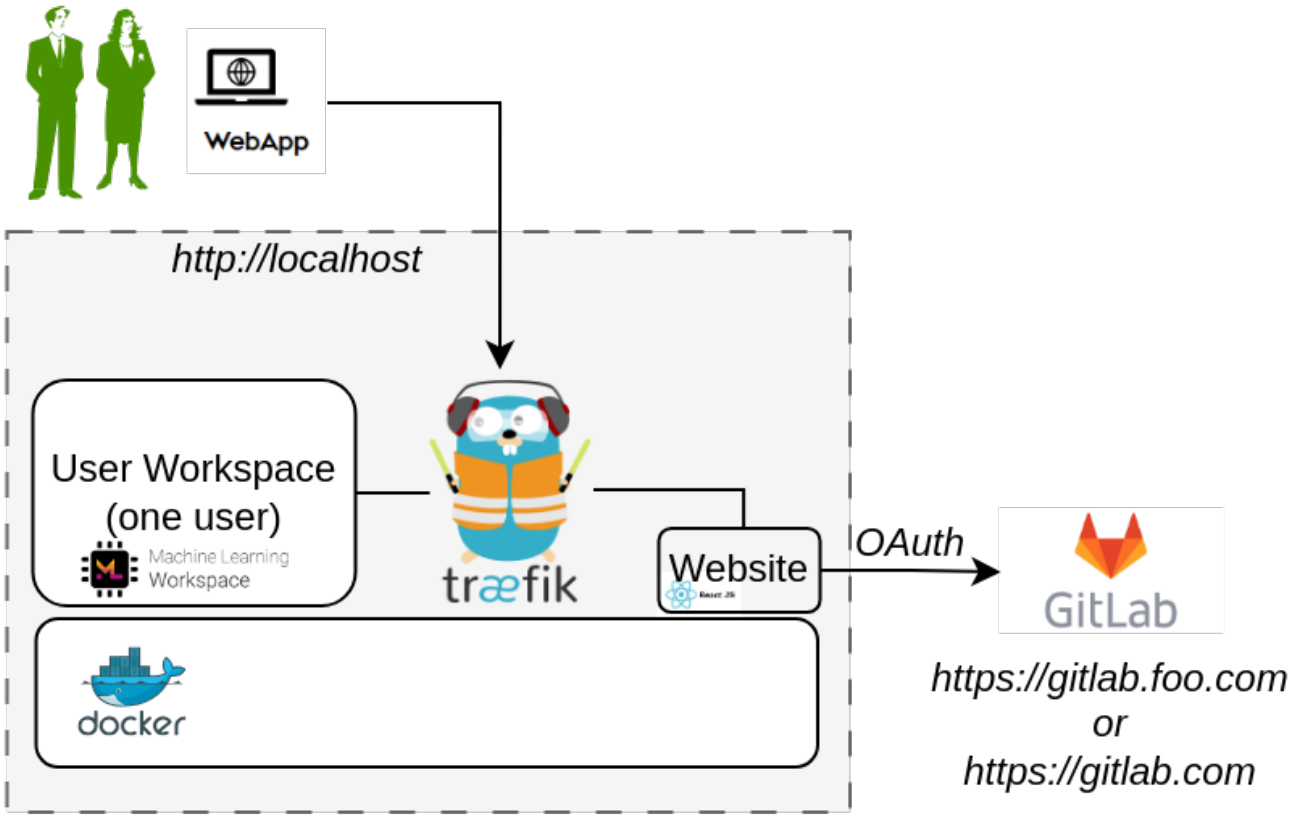


> **Tip**
>
> To restart Portainer later, run `docker start portainer`.

---

1. DTaaS Localhost

This scenario uses **GitLab OAuth** for authentication. A GitLab account on https://gitlab.com is required.



1.1 CONFIGURATION

1.2 Create Configuration Files

Navigate to `deploy/dtaas/docker/localhost` and copy the example files:

```
cp config/.env.example config/.env
cp config/client.js.example config/client.js
```

1.3 Environment Variables

Edit `config/.env` :

Variable	Example	Description
`USERNAME`	`user1`	Workspace path prefix and folder name
`COMPOSE_PROJECT_NAME`	`dtaas`	Docker Compose project name

1.4 Client Configuration

Edit `config/client.js` and set the OAuth application credentials from the GitLab account:

Variable	Example	Description
`REACT_APP_CLIENT_ID`	*(OAuth app id)*	GitLab OAuth application ID
`REACT_APP_AUTH_AUTHORITY`	`https://gitlab.com/`	OAuth authority URL

Tip

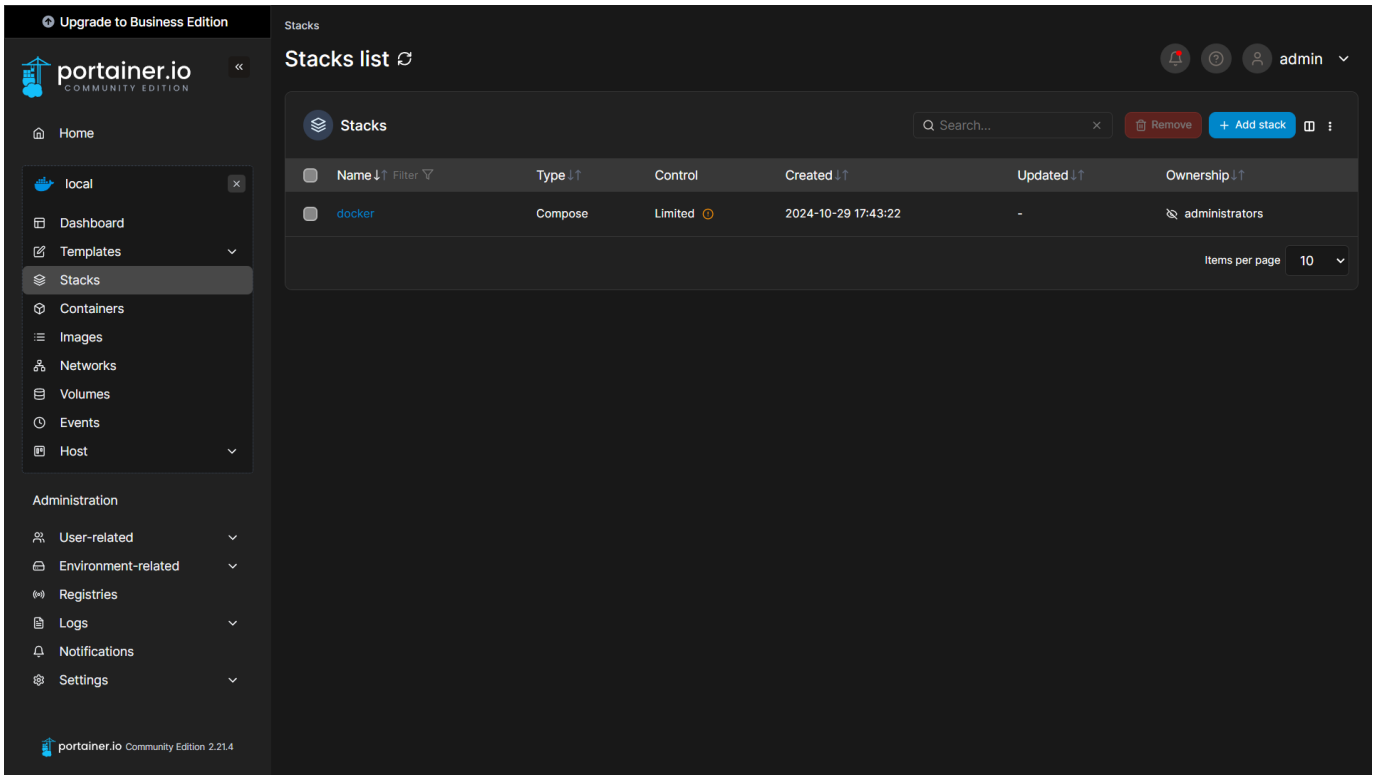
See the client auth docs for details on creating a GitLab OAuth application.

1.5 Create User Workspace

Create a workspace directory for the user set in `USERNAME`:

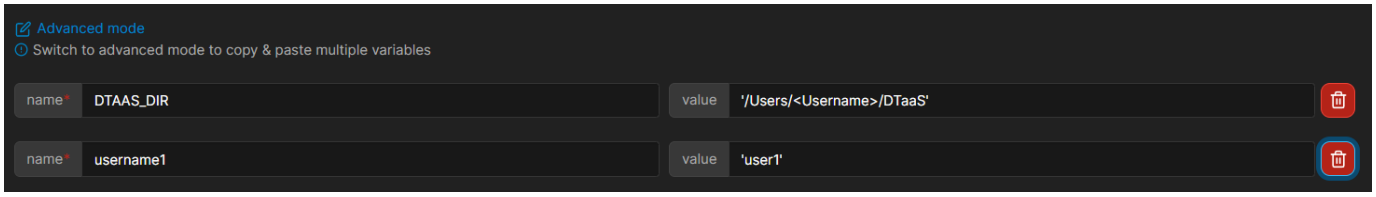
```
cp -R files/template files/<USERNAME>
sudo chown -R 1000:100 files/*
```

1.6 CREATE THE PORTAINER STACK



1. Navigate to **Stacks** and click **Add Stack**.
2. Name the stack, for example `dtaas-localhost`.
3. Select the **Upload** build method.
4. Upload the compose file at `deploy/dtaas/docker/localhost/docker-compose.yml`.
5. Load the environment file `deploy/dtaas/docker/localhost/config/.env`.

> **Tip**
>
> If the `.env` file is not visible, select **All Files** in the file explorer dialog.



Click **Deploy the stack**.

1.7 USE

Open http://localhost in a web browser and sign in with the configured GitLab credentials.

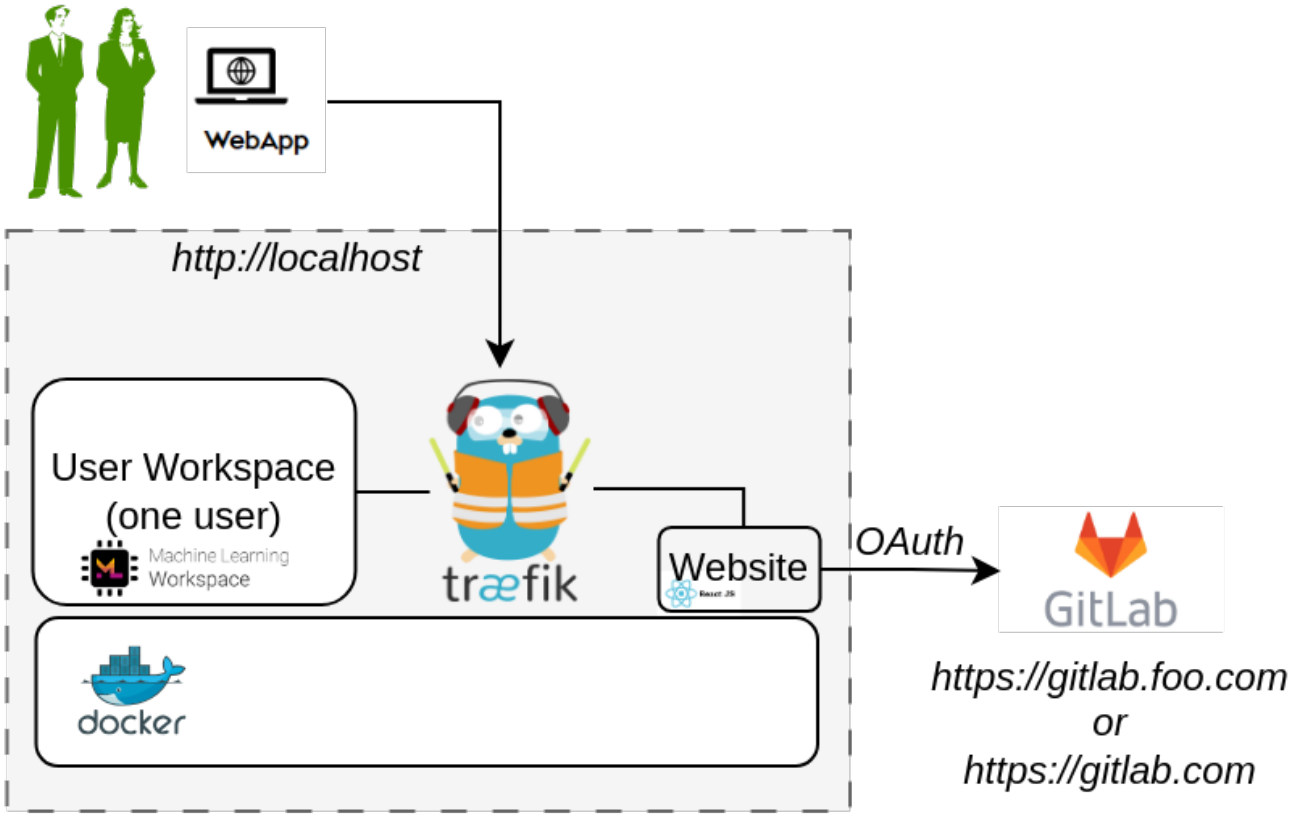
1.8 LIMITATIONS

The library microservice and backend forward-auth are not included in this scenario.

---

2. Workspace Localhost

This scenario uses **Dex** as a local identity provider. No external account is required; default credentials are provided.



2.1 CONFIGURATION

Navigate to `deploy/workspace/dex/localhost` and copy the example files:

```
cp .env.example .env
cp config/dex-config.yaml.example config/dex-config.yaml
```

2.2 Environment Variables

Edit `.env` :

Variable	Example	Description
`COMPOSE_PROJECT_NAME`	`dtaas`	Docker Compose project name
`DEFAULT_USER`	`user`	Default user login profile for Dex

> **Tip**
>
> Local login users may be customised in `deploy/workspace/dex/localhost/config/dex-config.yaml`.

2.3 CREATE THE PORTAINER STACK

1. Navigate to **Stacks** and click **Add Stack**.
2. Name the stack, for example `workspace-localhost`.
3. Select the **Upload** build method.
4. Upload the compose file at `deploy/workspace/dex/localhost/docker-compose.yml`.
5. Load the environment file `deploy/workspace/dex/localhost/.env`.

   Click **Deploy the stack**.

2.4 USE

Open http://localhost in a web browser. Sign in using the default Dex credentials:

- **Email:** `user@intocps.org`
- **Password:** `user`

2.5 LIMITATIONS

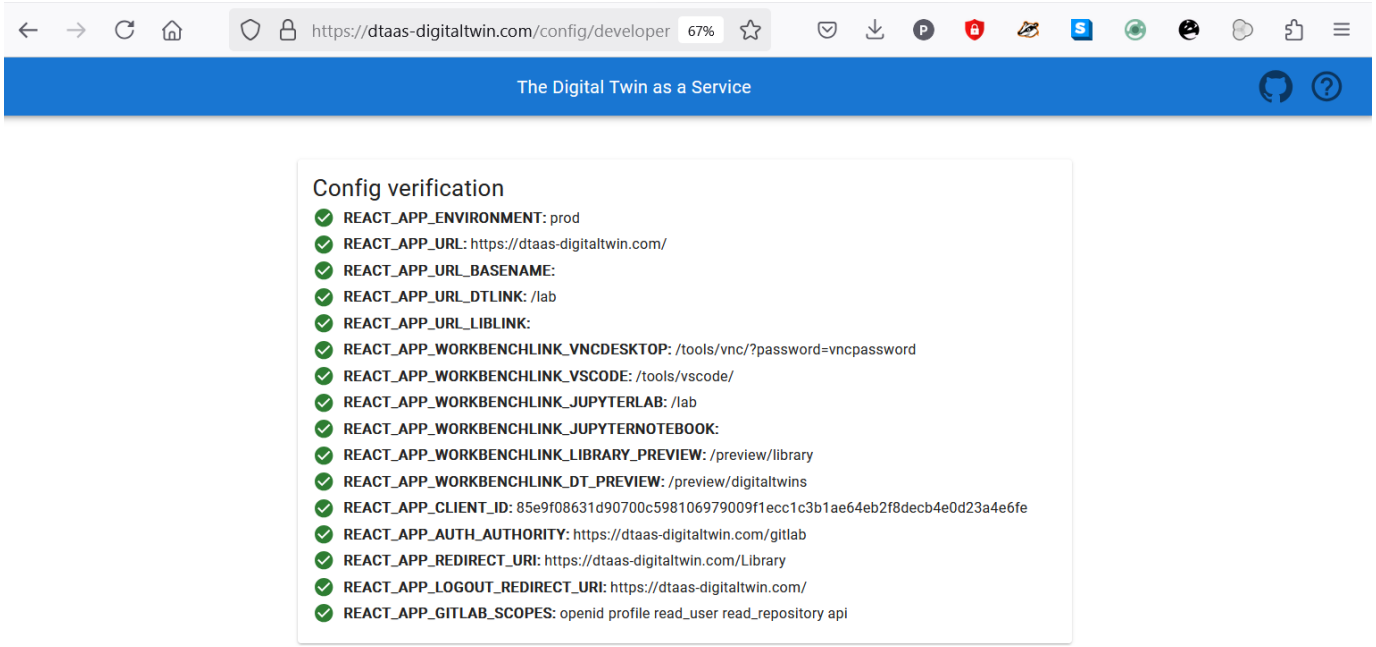
- The library microservice is not included in this scenario.
- DevOps features are not available.
- See custom user instructions for changing the default credentials.

References

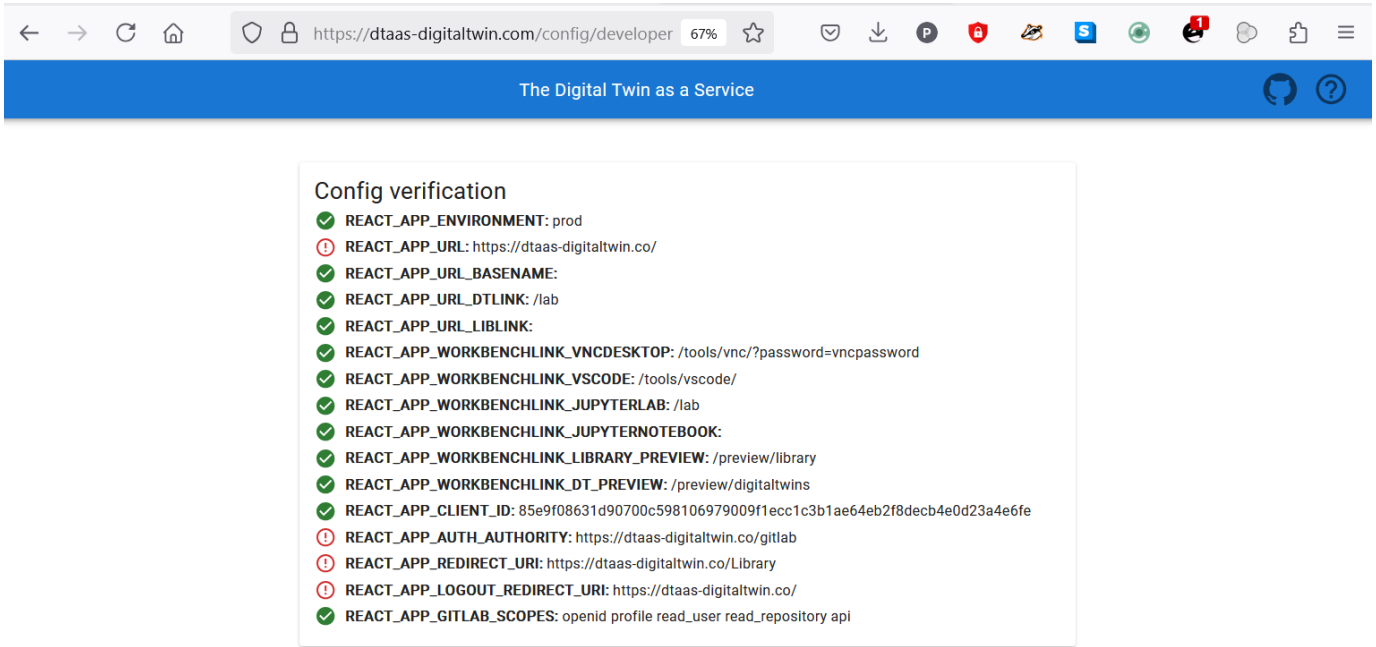
Image sources: Traefik logo, reactjs, gitlab

4.7.2 Check Client Configuration

One of the common errors is the incorrect configuration of the react website. There is now a helpful checklist to verify the client configuration. A correct configuration shows the following result.



In case of incorrect client website URL, the following result will be shown.



4.7.3 Add User

This page provides steps for adding a user to a DTaaS installation. The username **alice** is used here to illustrate the steps involved in adding a user account.

The following steps should be performed:

1. Add user to GitLab instance:

A new account for the new user should be added on the GitLab instance. The username and email of the new account should be noted.

2. Create User Workspace:

The DTaaS CLI should be used to bring up the workspaces for new users. This brings up the containers without backend authorisation.

3. Add backend authorisation for the user:

- Navigate to the secure server package directory

```
cd <DTaaS>/deploy/dtaas/docker/server
```

- Add three lines to `config/conf.server`

```
rule.onlyu3.action=auth
rule.onlyu3.rule=PathPrefix(`/alice`)
rule.onlyu3.whitelist = alice@intocps.org
```

4. Restart the docker container responsible for backend authorisation:

```
docker compose --env-file config/.env up \
  -d --force-recreate traefik-forward-auth
```

5. The new users are now added to the DTaaS instance with authorisation enabled.

4.7.4 Remove User

This page provides steps for removing a user from a DTaaS installation. The username **alice** is used here to illustrate the steps involved in removing a user account.

The following steps should be performed:

1. Remove an existing user with the DTaaS CLI.

2. Remove backend authorisation for the user:

- Navigate to the secure server package directory

```
cd <DTaaS>/deploy/dtaas/docker/server
```

- Remove these three lines from `config/conf.server`

```
rule.onlyu3.action=auth
rule.onlyu3.rule=PathPrefix(`/alice`)
rule.onlyu3.whitelist = alice@intocps.org
```

- Run the command for these changes to take effect:

```
docker compose --env-file config/.env up \
  -d --force-recreate traefik-forward-auth
```

The extra users now have no backend authorisation.

3. Remove users from GitLab instance (optional):

The GitLab docs provide additional guidance.

4. The user account is now deleted.

Caveat

The two base users that the DTaaS platform was installed with cannot be deleted. Only the extra users that have been added to the software can be deleted.

4.7.5 Make Common Assets Read Only

Why

In some cases it may be necessary to restrict the access rights of some users to the common assets. In order to make the common area read only, the install script section performing the creation of user workspaces must be changed.

> **Note**
>
> These step needs to be performed before installation of the application.

How

To make the common assets read-only for a user, the following changes need to be made to the secure-server `docker-compose.yml` file.

```
...
user1:
  ....
  volumes:
    - ./files/common:/workspace/common:ro
  ....

user2:
  ....
  volumes:
    - ./files/common:/workspace/common:ro
  ....
```

Note the `:ro` at the end of the line. This suffix makes the common assets read only.

To apply the same kind of read only restriction for new users as well, make a similar change to any additional `userX` service blocks added in `deploy/dtaas/docker/secure-server/docker-compose.yml` .

4.7.6 Renewing LetsEncrypt Certificates

LetsEncrypt certificates expire every three months and must be renewed. This guide covers renewal for current DTaaS package layouts.

Prerequisites

- Administrative access to the host
- `certbot` installed
- Access to Docker commands

Step 1: Renew certificates

```
sudo certbot renew --dry-run
sudo certbot renew
```

Step 2: Deploy to DTaaS server package

For the server package (`deploy/dtaas/docker/server`):

```
sudo cp /etc/letsencrypt/live/your-domain.com/fullchain.pem \
  /path/to/DTaaS/deploy/dtaas/docker/server/certs/fullchain.pem
sudo cp /etc/letsencrypt/live/your-domain.com/privkey.pem \
  /path/to/DTaaS/deploy/dtaas/docker/server/certs/privkey.pem
```

For the integrated package (`deploy/dtaas/docker/secure-server_with_integrated-gitlab`):

```
sudo cp /etc/letsencrypt/live/your-domain.com/fullchain.pem \
  /path/to/DTaaS/deploy/dtaas/docker/secure-server_with_integrated-gitlab/certs/fullchain.pem
sudo cp /etc/letsencrypt/live/your-domain.com/privkey.pem \
  /path/to/DTaaS/deploy/dtaas/docker/secure-server_with_integrated-gitlab/certs/privkey.pem
```

Step 3: Restart Traefik in DTaaS package

Server package:

```
cd /path/to/DTaaS/deploy/dtaas/docker/server
docker compose --env-file config/.env up -d --force-recreate traefik
```

Integrated package:

```
cd /path/to/DTaaS/deploy/dtaas/docker/secure-server_with_integrated-gitlab
docker compose --env-file config/.env up -d --force-recreate traefik
```

Step 4: Deploy to services project (if used)

If platform services are run from a generated `dtaas-services` project, copy the renewed certificates to that project's `certs/` directory.

```
sudo cp /etc/letsencrypt/live/your-domain.com/fullchain.pem \
  /path/to/services-project/certs/your-domain.com/fullchain.pem
sudo cp /etc/letsencrypt/live/your-domain.com/privkey.pem \
  /path/to/services-project/certs/your-domain.com/privkey.pem
```

Step 5: Restart affected services project containers

```
cd /path/to/services-project
docker compose -f compose.services.yml --env-file config/services.env up -d --force-recreate
```

If ThingsBoard or GitLab compose stacks are used in the same project:

```
docker compose -f compose.thingsboard.yml --env-file config/services.env up -d --force-recreate
docker compose -f compose.gitlab.yml --env-file config/services.env up -d --force-recreate
```

`deploy/dtaas/docker/secure-server` remains available as a compatibility alias for the same external GitLab server layout.

Verification

```
curl -I https://your-domain.com
```

Check logs if needed:

```
docker logs traefik
```

5. Frequently Asked Questions

5.1 Abreviations

Term	Full Form
DT	Digital Twin
DTaaS	Digital Twin as a Service
PT	Physical Twin

5.2 General Questions

What is the DTaaS platform?

The DTaaS platform is a software platform on which digital twins can be created and executed. The features page provides an overview of the capabilities available in DTaaS.

What is the scope and current capabilities of the DTaaS platform?

1. DTaaS is a web-based interface that allows invocation of various tools related to work to be performed with one or more DTs.
2. DTaaS permits users to run DTs in their private workspaces. These user workspaces are based on the Ubuntu 20.04 Operating system.
3. DTaaS can help create reusable DT assets only if DT asset authoring tools can operate in the Ubuntu 20.04 xfce desktop environment.
4. DTs are executables from the DTaaS platform perspective. Users are not constrained to work with DTs in a specific manner. DTaaS suggests creation of DTs from reusable assets and provides a suggestive structure for DTs. The examples provide more insight into the DTaaS workflow. However, this suggested workflow is not mandatory.
5. DTs can be run as services with REST API from within user workspaces, which can facilitate service-level DT composition.

What can not be done inside the DTaaS platform?

1. DTaaS as such does not help install DTs obtained from external sources.
2. The current user interface of the DTaaS web application is heavily reliant on the use of Jupyter Lab and Notebook. The **Digital Twins** page has Create / Execute / Analyze sections, but all point to Jupyter Lab web interface. The functionality of these pages is still under development.

Is there any fundamental difference between commercial solutions like Ansys Twin Builder and the DTaaS platform?

Commercial DT platforms like *Ansys Twin Builder* provide tight integration between models, simulation and sensors. This leads to fewer choices in DT design and implementation. In addition, there is a limitation of vendor lockin. On the other hand, DTaaS enables users to separate DT into reusable assets and combine these assets in a flexible way.

Is licensed software such as Matlab provided?

Proprietary and commercially licensed software is not available by default on the software platform. However, users have private workspaces based on a Linux xfce Desktop environment. Users can install proprietary and commercially licensed software in their workspaces. A screencast demonstrates using Matlab Simulink within the DTaaS platform. Licensed software installed by one user is not available to other users.

5.3 Digital Twin Assets

Can the DTaaS platform be used to create new DT assets?

The core feature of DTaaS software is to help users create DTs from assets already available in the library. Create Library Assets However, it is possible for users to take advantage of services available in their workspace to install asset authoring tools in their own workspace. These authoring tools can then be used to create and publish new assets. User workspaces are private and are not shared with other users. Thus, any licensed software tools installed in a workspace are only available to that user.

5.4 Digital Twin Models

Can the DTaaS platform create new DT models?

DTaaS is not a model creation tool. Model creation tools can be placed inside DTaaS to create new models. The DTaaS platform itself does not create digital twin models but can help users create digital twin models. Linux desktop/terminal tools can be run inside DTaaS. Thus, models can be created inside DTaaS and executed using tools that run on Linux. Windows-only tools cannot run in DTaaS.

How can the DTaaS platform help to design geometric model? Does it support 3D modelling and simulation?

DTaaS by itself does not produce any models. DTaaS only provides a platform and an ecosystem of services to facilitate digital twins to be run as services. Since each user has a Linux OS at their disposal, they can also run digital twins that have a graphical interface. In summary, the DTaaS platform is neither a modelling nor simulation tool. If such tools are required, they must be brought onto the platform. For example, if Matlab is needed, the user must bring the licensed Matlab software.

Can the DTaaS platform support only the information models (or behavioural models) or some other kind of models?

The DTaaS platform as such is agnostic to the kind of models used. DTaaS can run all kinds of models. This includes behavioural and data models. As long as the models and matching solvers can run in Linux OS, they can be used in DTaaS. In some cases, models and solvers (tools) are bundled together to form monolithic DTs. The DTaaS platform does not restrict running such DTs either. DTaaS does not provide dedicated solvers. However, if a solver can be installed in the workspace, the platform need not provide one.

Does it support XML-based representation and ontology representation?

Currently No. **The team is seeking users who require this capability. If concrete requirements and an example are available, a way of realising it in DTaaS can be discussed**.

5.5 Communication Between Physical Twin and Digital Twin

How can the DTaaS platform control the physical entity? Which technologies it uses for controlling the physical world?

At a very abstract level, there is a communication from physical entity to digital entity and back to physical entity. How this communication should happen is decided by the person designing the digital entity. The DTaaS can provide communication services that facilitate this communication with relative ease. InfluxDB, RabbitMQ and Mosquitto services hosted on DTaaS may be used for two communication between digital and physical entities.

How is a physical entity such as shape, size, weight, structure, or chemical attributes measured using DTaaS? Is any specific technology used in this case?

The real measurements are done at physical twin which are then communicated to the digital twin. Any digital twin platform like DTaaS can only facilitate this communication of these measurements from physical twin. The DTaaS provides InfluxDB, RabbitMQ and Mosquitto services for this purpose. These three are probably most widely used services for digital twin communication. Nevertheless, DTaaS permits the use of other communication technologies and services hosted elsewhere on the Internet.

How can real-time data differ from static data and what is the procedure to identify dynamic data? Is there any UI or specific tool used here?

The DTaaS platform cannot understand the static or dynamic nature of data. It can facilitate storing names, units and any other text description of interesting quantities (weight of batter, voltage output etc). It can also store the data being sent by the physical twin. The distinction between static and dynamic data needs to be made by the user. Only metadata of the data can reveal such more information about the nature of data. A tool can probably help in very specific cases, but metadata is required. If there is a human being making this distinction, then the need for metadata goes down but does not completely go away. In some of the DT platforms supported by manufacturers, there is a tight integration between data and model. In such cases, the tool itself manages the metadata. The DTaaS is a generic platform which can support execution of digital twins. If a tool can be executed on a Linux desktop / commandline, the tool can be supported within DTaaS. The tool (e.g., Matlab) itself can manage the metadata requirements.

5.6 Digital Twin DevOps Automation

Can a DT execute forever?

The web UI imposes a 10-minute timeout. The users can manually terminate an ongoing execution. The best choice would be for a DT to execute terminate script which consequently concludes the execution and returns the logs.

Is the DT execution really scalable?

This capacity of DT execution infrastructure is dependent on GitLab and the available compute power available to runners of GitLab. GitLab itself does not impose limits on the maximum number of runners.

If many GitLab runners are attached to a GitLab repository, which one is used?

This is indeterminate. One cannot rely on the order and location of execution for a DT.

5.7 Data Management

Can the DTaaS platform collect data directly from sensors?

Yes via platform services.

Does DTaaS support data collection from different sources like hardware, software and network? Is there any user interface or any tracking instruments used for data collection?

The DTaaS platform provides InfluxDB, PostgreSQL, RabbitMQ, MQTT, MongoDB and ThingsBoard services. Both the physical twin and digital twin can utilise these protocols for communication. The IoT (time-series) data can be collected using InfluxDB and MQTT broker services. There is a user interface for InfluxDB which can be used to analyze the data collected. Users can also manually upload their data files into the DTaaS.

Is the DTaaS platform able to transmit data to cloud in real time?

Yes via platform services.

Which transmission protocol does the DTaaS platform allow?

InfluxDB, RabbitMQ, MQTT and anything else that can be used from Cloud service providers.

Does the DTaaS platform support multisource information and combined multi sensor input data? Can it provide analysis and decision-supporting inferences?

Information from multiple sources can be stored. The existing InfluxDB services hosted on DTaaS already has a dedicated Influx / Flux query language for doing sensor fusion, analysis and inferences.

Which kinds of visualisation technologies can the DTaaS platform support (e.g. graphical, geometry, image, VR/AR representation)?

Graphical, geometric and images. If specific licensed software is needed for the visualisation, the license for it must be provided. DTaaS does not support AR/VR.

5.8 Platform Native Services on the DTaaS Platform

Is the DTaaS platform able to detect the anomalies about-to-fail components and prescribe solutions?

This is the job of a digital twin. If a ready-to-use digital twin exists for this purpose, DTaaS enables others to use that solution. It is possible to perform anomaly detection using the platform services such as Grafana, ThingsBoard and InfluxDB.

5.9 Comparison with other DT Platforms

All the DT platforms seem to provide different features. Is there a comparison chart?

Here is a qualitative comparison of different DT integration platforms:

Legend: high performance (**H**), mid performance (**M**) and low performance (**L**)

DT Platforms	License	DT Development Process	Connectivity	Security	Processing power, performance and Scalability	Data S
Microsoft Azure DT	Commercial Cloud	H	H	H	M	H
AWS IOT Greengrass	Open source commercial	H	H	H	M	H
Eclipse Ditto	Open source	M	H	M	H	H
Asset Administration Shell	Open source	H	H	L	H	M
PTC Thingworx	Commercial	H	H	H	H	H
GE Predix	Commercial	M	H	H	M	L
The DTaaS Platform	Open source	H	H	L	L	M

```
Adopted by Tanusree Roy from Table 4 and 5 of the following paper.

Ref: Naseri, F., Gil, S., Barbu, C., Cetkin, E., Yarimca, G., Jensen, A. C.,
... & Gomes, C. (2023). Digital twin of electric vehicle battery systems:
Comprehensive review of the use cases, requirements, and platforms.
Renewable and Sustainable Energy Reviews, 179, 113280.
```

All the comparisons between DT platforms seems so confusing. Why?

The fundamental confusion comes from the fact that different DT platforms (Azure DT, GE Predix) provide different kind of DT capabilities. One can run all kinds of models natively in GE Predix. In fact, models can be run even next to (on) PTs using GE Predix. However, this cannot natively be done in Azure DT service. The user must perform the work of integrating with other Azure services or third-party services to obtain the kind of capabilities that GE Predix natively provides in one interface. The conclusion is that an appropriate platform should be selected based on specific requirements.

5.10 GDPR Concerns

Does the platform adhere to GDPR compliance standards? If so, how?

The DTaaS platform does not store any personal information of users. It only stores username to identify users and these usernames do not contain enough information to deduce the true identify of users.

Which security measures are deployed? How is data encrypted (if exists)?

The default installation requires a HTTPS terminating reverse proxy server from user to the DTaaS platform installation. The administrators of DTaaS platform can also install HTTPS certificates into the application. The codebase can generate HTTPS application and the users also have the option of installing their own certificates obtained from certification agencies such as LetsEncrypt.

What security measures does the cloud provider offer?

The DTaaS platform can be installed inside a corporate server hosted behind network firewalls so that only permitted user groups have access to the network and physical access to the server.

How is user access controlled and authenticated?

There is a two-level authorisation mechanism in place in each default installation of the DTaaS. The first-level is HTTP basic authorisation over secure HTTPS connection. The second-level is the OAuth 2.0 PKCE authorisation flow for each user. The OAuth 2.0 authorisation is provided by a GitLab instance. The DTaaS does not store the account and authorisation information of users.

Does the platform manage personal data? How is data classified and tagged based on the sensitivity? Who has access to the critical data?

The platform does not store personal data of users.

How are identities and roles managed within the platform?

There are two roles for users on the platform. One is the administrator and the other one is user. The user roles are managed by the administrator.

6. Developer

6.1 Guide

6.1.1 Contributors Guide

This guide provides an overview of the contribution workflow for the Digital Twin as a Service (DTaaS) project. Contributors are encouraged to review the Code of Conduct to ensure a respectful and collaborative community environment.

The following sections outline the contribution process, from opening an issue to creating a pull request (PR), conducting reviews, and merging the PR.

Project Goals

The DTaaS platform aims to facilitate the creation, management, and execution of Digital Twins (DTs) through a service-oriented architecture that promotes reusability of DT assets [1]. This documentation assists development team members in understanding the DTaaS project software design and development processes.

Additional resources include:

- Developer Slides
- Video Presentation
- Research Paper

💻 Development Environment

A devcontainer configuration is provided in `.devcontainer/devcontainer.json` for the project. This configuration offers a dockerized development environment and represents the recommended approach for establishing a consistent development setup. DevContainer provides the most straightforward method for initializing the development environment.

🏗️ Development Workflow

A structured development workflow is employed to manage collaboration among multiple developers. Each contributor should adhere to the following procedure:

1. Create a fork of the main repository into a personal GitHub account.
2. Configure Qlty, Codecov, and SonarQube for the forked repository. Note that Codecov does not require a secret token for public repositories.
3. Utilize Node.js 24 and Python 3.12 development environments.
4. Follow the Fork, Branch, PR workflow methodology.
5. Develop within the fork and create a PR from the working branch to the `feature/distributed-demo` branch. The PR triggers all GitHub Actions, Qlty, SonarQube, and Codecov checks.
6. Address all issues identified during the automated checks.
7. Upon successful verification, submit a PR to the `feature/distributed-demo` branch of the upstream DTaaS repository.
8. The PR undergoes review and is merged by either project administrators or maintainers.

Each PR should represent a meaningful contribution that satisfies either a well-defined user story or improves code quality.

✨ Coding Agents and Editors

The project makes extensive use of coding agents. The primary usage scenarios include:

👍 Co-development assistance

👍 Code review support

👍 Draft pull requests for prototyping ideas

This project is structured as a monorepo and includes copilot instructions that inform GitHub Copilot about the project structure and software development conventions.

Modern Code IDEs such as VS Code and Cursor provide native integration with coding agents, requiring no additional configuration for LLM-driven development workflows. Contributors should disclose code generated by coding agents, particularly when such code embeds significant programming logic.

The following practices are strictly prohibited:

👎 Contributing unknown or unreviewed code

👎 Failing to understand the implications of generated code

Summary: Contributors must fully understand their contributions and should not delegate critical thinking to coding agents.

👁️ Code Quality

Project code quality is assessed through the following mechanisms:

- **Static Analysis**: Linting issues are identified by Qlty. Installation of Qlty CLI is recommended. Execute the command `qlty check --no-fail --sample 5 --no-formatters` to identify code quality issues. Note that Qlty only analyzes files that differ from the default branch (`feature/distributed-demo`).
- **Security Scans**: Code quality and security issues are identified by SonarQube.
- **Test Coverage**: Coverage reports are collected by Codecov.
- **Continuous Integration**: All GitHub Actions must complete successfully.

QLTY

Qlty performs static analysis, linting, and style checks to ensure optimal code quality for project contributions.

While newly introduced issues appear directly on the PR page, specific issues can be addressed by visiting the issues or code section of the Qlty dashboard.

Code contributions should not introduce new quality issues. If such issues arise, they should be resolved immediately using the appropriate suggestions from Qlty. In exceptional cases, an ignore flag may be added, though this approach should be employed sparingly.

CODECOV

Codecov maintains test coverage metrics for the entire project. For detailed information about testing practices and workflows, refer to the testing documentation.

GITHUB ACTIONS

The project defines multiple GitHub Actions. All pull requests and direct commits must achieve successful status on all GitHub Actions.

References

[1] Talasila, Prasad, et al. "Composable digital twins on Digital Twin as a Service platform." Simulation 101.3 (2025): 287-311.

6.2 System

6.2.1 System Overview

The Digital Twin as a Service (DTaaS) platform is designed to support the complete digital twin (DT) lifecycle, enabling users to create, configure, execute, and share digital twins through reusable assets[1]. The platform architecture reflects established principles for realising digital twins in practice[2], while also supporting advanced use cases such as runtime verification of autonomous systems[3].

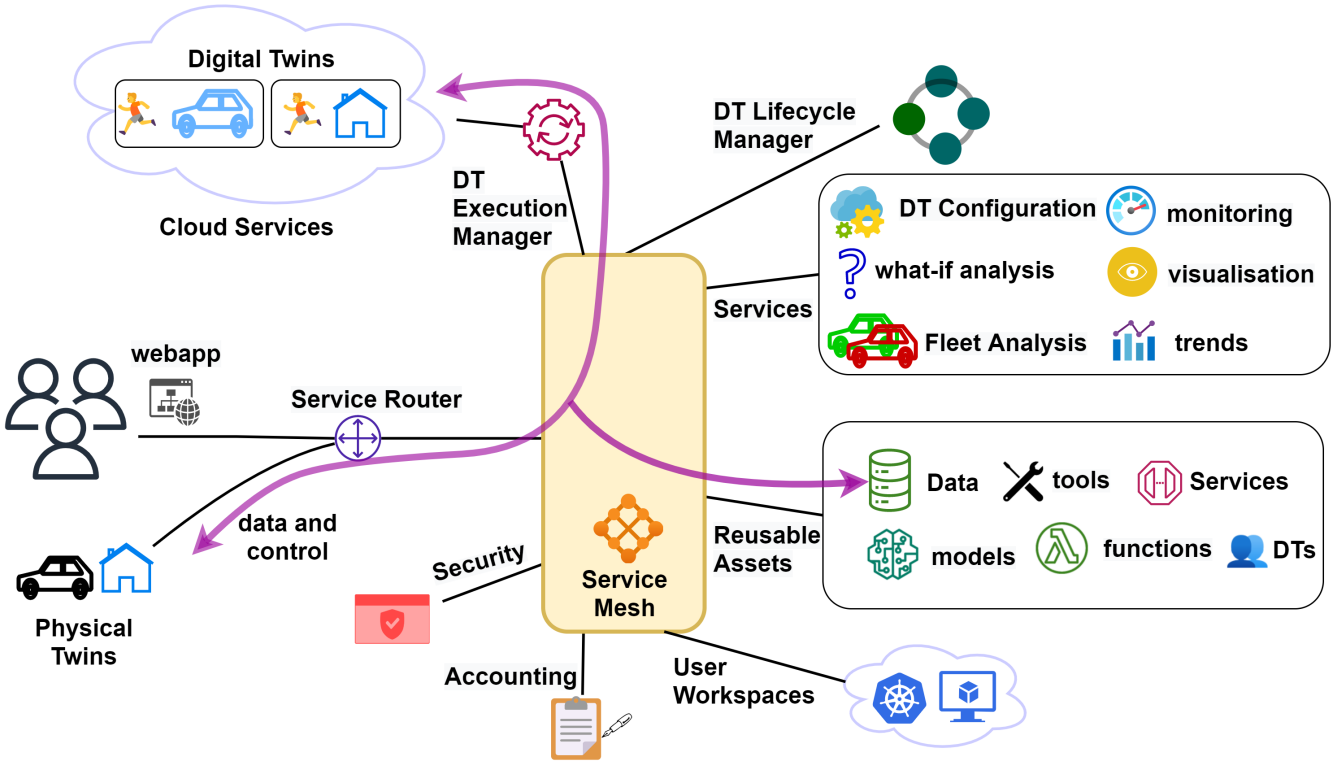
User Requirements

The platform provides the following core capabilities:

1. **Author** – create different assets of the DT on the platform itself. This step requires use of some software frameworks and tools whose sole purpose is to author DT assets.
2. **Consolidate** – consolidate the list of available DT assets and authoring tools so that user can navigate the library of reusable assets. This functionality requires support for discovery of available assets.
3. **Configure** – support selection and configuration of DTs. This functionality also requires support for validation of a given configuration.
4. **Execute** – provision computing infrastructure on demand to support execution of a DT.
5. **Explore** – interact with a DT and explore the results stored both inside and outside the platform. Exploration may lead to analytical insights.
6. **Save** – save the state of a DT that is already in the execution phase. This functionality is required for on-demand saving and re-spawning of DTs.
7. **Services** – integrate DTs with on-platform or external services with which users can interact with.
8. **Share** – share a DT with other users of their organisation.

System Architecture

The figure shows the system architecture of the the DTaaS software platform.



SYSTEM COMPONENTS

Users interact with the software platform through a web application. The service router serves as the single point of entry for direct access to platform services and is responsible for controlling user access to the microservice components. The service mesh enables discovery of microservices, load balancing, and authorization functionalities.

In addition, there are microservices for catering to managing DT reusable assets, platform services, DT lifecycle manager, DT execution manager, accouting and security. The microservices are complementary and composable; they fulfil core requirements of the system.

The microservices responsible for satisfying the user requirements are:

1. **The security microservice** implements role-based access control (RBAC) in the platform.
2. **The accounting microservice** is responsible for keeping track of the live status of platform, DT asset and infrastructure usage. Any licensing, usage restrictions need to be enforced by the accounting microservice. Accounting is a pre-requisite to commercialisation of the platform. Due to significant use of external infrastructure and resources via the platform, the accounting microservice needs to interface with accounting systems of the external services.
3. **User Workspaces** are virtual environments in which users can perform lifecycle operations on DTs. These virtual environments are either docker containers or virtual machines which provide desktop interface to users.
4. **Reusable Assets** are assets / parts from which DTs are created. Further explation is available on the assets page
5. **Services** are dedicated services available to all the DTs and users of the DTaaS platform. Services build upon DTs and provide user interfaces to users.
6. **DT Execution Manager** provides virtual and isolated execution environments for DTs. The execution manager is also responsible for dynamic resource provisioning of cloud resources.
7. **DT Lifecycle Manager** manages the lifecycle operations on all DTs. It also directs *DT Execution Manager* to perform execute, save and terminate operations on DTs.

For a more detailed view, refer to the C4 architectural diagram.

A mapping of the architectural components to related pages in the documentation is available in the table.

System Component	Doc Page(s)
Service Router	Traefik Gateway
Web Application	React Webapplication
Reusable Assets	Library Microservice
Digital Twins and DevOps	Integrated GitLab
Platform Services	Third-party Services (MQTT, InfluxDB, RabbitMQ, Grafana, MongoDB, PostgreSQL, and ThingsBoard)
DT Lifecycle Manager	Not available yet
Security	GitLab client OAuth 2.0 and server OAuth 2.0
Digital Twins as Services	DT Runner
Accounting	Not available yet
Execution Manager	Not available yet

References

Font sources: fileformat

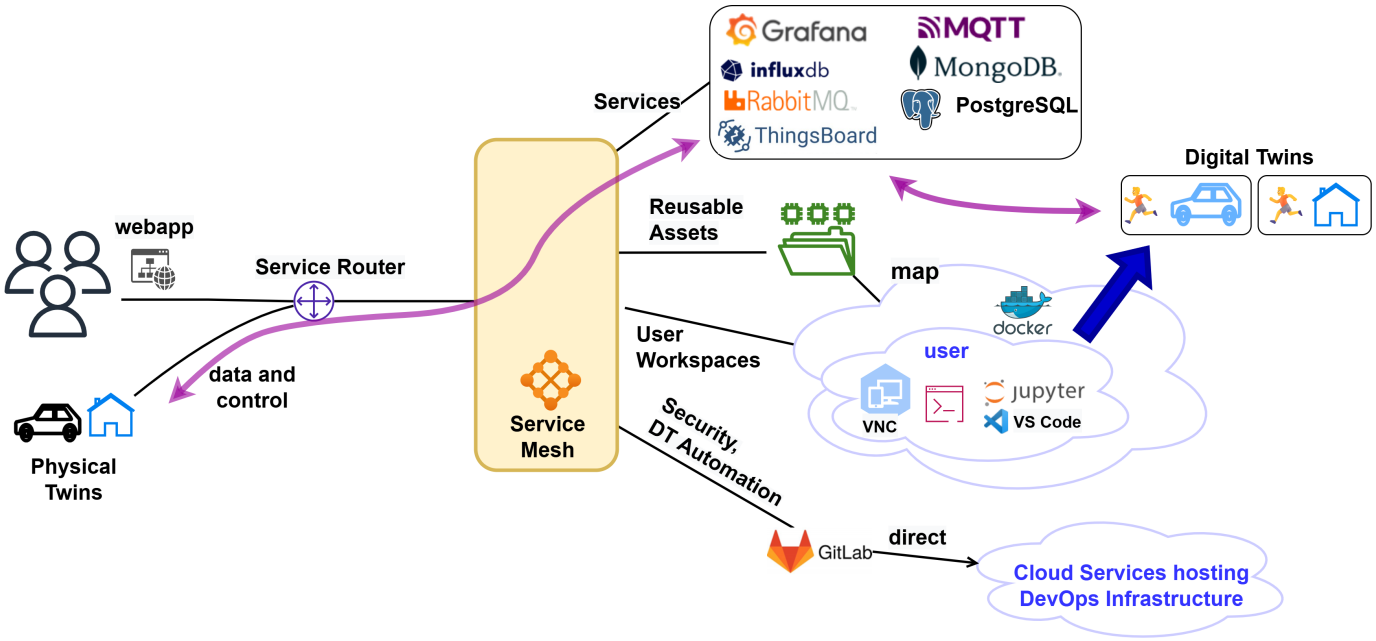
[1]: Talasila, Prasad, et al. "Composable digital twins on Digital Twin as a Service platform." Simulation 101.3 (2025): 287-311.

[2]: Talasila, Prasad, et al. "Realising digital twins." The engineering of digital twins. Cham: Springer International Publishing, 2024. 225-256.

[3]: Kristensen, Morten Haahr, et al. "Runtime Verification of Autonomous Systems Utilizing Digital Twins as a Service." 2024 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C). IEEE, 2024.

6.2.2 Current Status 📋

DTaaS is an active monorepo with production-oriented deployment scenarios and ongoing evolution of execution automation, reusable assets, and contributor tooling.



A C4 representation is also available at current-status-developer-c4.png.

Platform Snapshot

- React 19 client with typed backend abstraction over GitLab APIs.
- NestJS-based library and runner microservices.
- Python CLIs for DTaaS administration and platform services management.
- Scenario-based admin documentation for DTaaS and workspace deployments.

Security and Access Model 🔒

Two complementary authorization patterns are in active use:

- GitLab-centered OAuth flows for DTaaS package scenarios.
- Keycloak-centered OIDC flows for secure workspace scenarios.

Route protection and gateway mediation are managed through Traefik and forward-auth patterns.

Workspaces and Assets 👷

Workspace deployments provide user-isolated environments and shared asset access patterns. Asset management currently spans file-system workflows and evolving library microservice functionality.

Current Engineering Focus

- Improve scenario automation and configuration guidance.
- Strengthen test coverage and CI reliability across packages.
- Continue improving DevOps execution visibility in the client UI.
- Keep package publishing and release flows consistent across npm, Docker, and Poetry-based tools.

Contributions that improve reproducibility, documentation quality, and cross-component integration are especially valuable.

6.2.3 📚 DTaaS Research Overview

This document summarises the research behind the DTaaS platform. It is written for code contributors who need to understand the design rationale, architectural decisions, and intended evolution of the system.

1. 💡 Introduction

A digital twin is a software representation of a physical asset that stays connected to its real-world counterpart through data exchange, enabling monitoring, analysis, and prediction throughout the asset lifecycle. Digital twin platforms provide shared infrastructure for creating, managing, and executing digital twins at scale.

DTaaS (Digital Twin as a Service) is an open-source platform that treats digital twins as compositions of reusable assets rather than monolithic applications. The platform organises assets into four categories: data, models, functions (methods and scripts), and services. Users compose these assets into digital twin configurations that can be executed, monitored, and evolved independently.

DT asset relationship

The platform targets operability: creating, configuring, executing, monitoring, and evolving twins in long-running environments. This focus distinguishes DTaaS from simulation-only tools or pure modelling frameworks.

2. 🎯 Requirements

Eight core requirements drive the platform design:

1. **Author** — create and edit digital twin assets in isolated workspaces
2. **Consolidate** — combine assets from different sources into coherent twin configurations
3. **Configure** — set parameters for execution without modifying asset source code
4. **Execute** — run digital twin configurations on demand or continuously
5. **Explore** — browse available assets, twins, and execution results
6. **Save** — persist configurations, results, and intermediate state
7. **What-if Analysis** — evaluate alternative scenarios by varying parameters
8. **Collaborate** — share assets and twins across users and organisations

These requirements apply across domains. Validated case studies include food fermentation process control and indoor climate management for firefighter training, as well as structural health monitoring of civil infrastructure.

3. 🏗️ System Architecture

System Architecture

The architecture decomposes the platform into four concern areas:

- **Gateway and authentication** — Traefik reverse proxy with OAuth 2.0 / OpenID Connect for route protection and user identity management
- **Execution management** — backend services that trigger, monitor, and stop digital twin runs via GitLab CI/CD pipelines
- **Reusable-asset handling** — library microservice backed by GitLab repositories, exposing asset discovery and retrieval APIs
- **Workspace isolation** — per-user JupyterLab containers with private file systems for authoring and running assets

System Architecture C4 Diagram

This decomposition maps directly to the service split in the current codebase: the React client handles user interaction, Traefik manages routing and TLS, the library microservice wraps GitLab APIs for asset management, and the execution service orchestrates pipeline triggers.

Implementation Status

4. 👤 User View

DTaaS user view

From a user perspective, DTaaS presents a single web interface that aggregates workspace operations, asset browsing, digital twin execution, and result exploration. Users interact with their own isolated workspace while sharing assets through the common library. The platform supports both class-level twin definitions (reusable templates) and instance-level twins (bound to specific physical assets).

DT class and instance

5. 🔄 Lifecycle-Driven Engineering

Lifecycle Phases on DTaaS

Lifecycle coverage is a first-class requirement, not an add-on. The platform explicitly supports create, manage, execute, and stop phases with iterative reconfiguration flows. This is why client and backend contracts include both file management and execution tracking concerns.

The lifecycle model acknowledges that digital twins evolve: models are updated as physical assets change, new data sources are integrated, and execution configurations are refined based on operational experience.

6. ⚙ DevOps and Federation

DevOps with GitLab

DTaaS integrates GitLab-backed DevOps automation as a core platform capability. Repository structure, pipeline triggers, and OAuth configuration are part of the platform contract, not just deployment details. The platform supports two execution modes: continuous (long-running services) and one-off (batch analysis jobs), both managed through GitLab CI/CD pipelines.

Federated DTaaS

The federation extension enables multiple DTaaS instances to collaborate by discovering and reusing assets across organisational boundaries. Federated digital twins can be organised as composites (assembled from assets on different instances), fleets (coordinated groups), or hierarchies (parent-child relationships).

User interactions in Federated DTaaS

7. 🌉 Application: Structural Health Monitoring

SHM DT Workflow

Structural health monitoring (SHM) of civil infrastructure demonstrates DTaaS as an integration substrate for specialized engineering workflows. In this application domain, digital twin assets include sensor data acquisition modules (accelerometers, distributed optical sensors), operational modal analysis functions, finite element model updating procedures, and fatigue-based remaining useful life estimation methods.

The SHM workflow composes these assets into a pipeline: sensor data flows through modal analysis to extract structural properties, which feed into model updating to calibrate a finite element model, which in turn drives damage prognosis and remaining useful life estimation. This demonstrates that DTaaS enables domain specialists to compose complex workflows from reusable building blocks without replacing their existing tools.

🧭 Guidance for Contributors

When implementing or reviewing DTaaS changes:

1. Map the change to one of the concern areas above (architecture, lifecycle, DevOps/federation, domain application, monitoring).
2. Verify the change preserves intended lifecycle and composability behavior.
3. Check whether the change shifts architecture boundaries (gateway/auth, execution manager, reusable-asset layer, workspace isolation).
4. Document any intentional departure from design assumptions.

Common anti-patterns to avoid:

- Hard-coding one modeling or tooling workflow where the design expects heterogeneity.
- Collapsing layered responsibilities into UI code for short-term convenience.
- Treating DevOps and federation behavior as optional integration noise.

References

The research behind DTaaS is documented in the following papers, located in the `.research-papers/latex/` directory of this repository:

1. **Composable Digital Twins on Digital Twin as a Service Platform** — Defines the core platform architecture, asset model, lifecycle phases, and system requirements. (`Talasila, Prasad, et al., Composable digital twins on Digital Twin as a Service platform." Simulation 101.3 (2025): 287-311.`)
2. **Realising Digital Twins** — Positions DTaaS within the broader digital twin landscape with a comparative survey of existing frameworks, cloud deployment strategies, and fleet management concepts. (`Talasila, Prasad, et al. "Realising digital twins." The engineering of digital twins. Cham: Springer International Publishing, 2024. 225-256.`)
3. **Towards Federated Digital Twin Platforms** — Extends DTaaS for multi-instance federation, DevOps integration, and cross-organizational asset reuse. (`Frasheri, Mirgita, Prasad Talasila, and Vanessa Scherma. "Towards federated digital twin platforms." arXiv preprint arXiv:2505.04324 (2025).`)
4. **Structural Health Monitoring of Engineering Structures Using Digital Twins** — Demonstrates DTaaS for vibration-based SHM with modal analysis and model updating workflows. (`Talasila, P., et al. "Structural health monitoring of engineering structures using digital twins: A digital twin platform approach. "International Conference on Experimental Vibration Analysis for Civil Engineering Structures. Cham: Springer Nature Switzerland, 2025.`)

6.3 Technical Concepts

6.3.1 OAuth 2.0 and Keycloak Summary 🔒

DTaaS uses OAuth 2.0/OIDC patterns at two levels:

- Browser-side login for the web client.
- Gateway-side protection for backend routes and workspaces.

Two Deployment Patterns

1. GITLAB-CENTERED OAUTH

This pattern is common in DTaaS package scenarios that use GitLab as identity provider.

- The web client authenticates users through OAuth/OIDC settings in runtime config.
- Gateway authorization is handled by `traefik-forward-auth`.
- GitLab applications provide client ID and client secret values used by DTaaS components.

2. KEYCLOAK-CENTERED OIDC

This pattern is used in workspace secure-server deployments.

- Keycloak provides realm, users, and OIDC clients.
- `traefik-forward-auth` validates sessions/tokens and enforces route access.
- Workspace routes stay protected even when users access different sub-paths.

Control Plane vs Data Plane

- Control plane: provider setup, secret management, callback URI configuration.
- Data plane: request flow through Traefik and forward-auth before DTaaS.

Core Runtime Values

Typical secure-server OIDC variables include:

- `KEYCLOAK_ISSUER_URL`
- `KEYCLOAK_CLIENT_ID`
- `KEYCLOAK_CLIENT_SECRET`
- `OAUTH_SECRET`

GitLab-oriented scenarios use equivalent OAuth authority and client settings in scenario-specific configuration files.

Why This Matters for Developers

When debugging login failures, first identify which identity provider pattern is active. Most issues come from mismatch in one of these:

- Redirect/callback URL.
- Client ID and secret.
- Hostname and TLS assumptions.
- User whitelist/route policy in forward-auth config.

Start troubleshooting at configuration boundaries, then inspect gateway and provider logs.

6.3.2 OAuth 2.0 Summary

The Authentication Microservice (Auth MS) operates according to the OAuth 2.0 RFC specification. This document provides a brief summary of the OAuth 2.0 technology and its implementation within the DTaaS platform.

Entities

OAuth 2.0, as used for user identity verification, has 3 main entities:

- **The User:** This is the entity whose identity the platform is attempting to verify. In the DTaaS context, this is the same as the user of the DTaaS software.
- **The Client:** This is the entity that wishes to know/verify the identity of a user. In the DTaaS context, this is the Auth MS (initialised with a GitLab application). This should not be confused with the frontend website of DTaaS (referred to as Client in the previous section).
- **The OAuth 2.0 Identity Provider:** This is the entity that allows the client to know the identity of the user. In the DTaaS context, this is GitLab. Most commonly, users have an existing, protected account with this entity. The account is registered using a unique key, like an email ID or username and is usually password protected so that only that specific user can login using that account. After the user has logged in, they will be asked to approve sharing their profile information with the client. If they approve, the client will have access to the user's email id, username, and other profile information. This information can be used to know/verify the identity of the user.

Note: In general, the Authorization server (which requests user approval) and the Resource (User Identity) provider can be two different servers. However, in the DTaaS implementation, the GitLab instance handles both functions through different API endpoints. The underlying concepts remain the same. Therefore, this discussion focuses on the three main entities: the User, the OAuth 2.0 Client, and the GitLab instance.

THE OAUTH 2.0 CLIENT

Many platforms allow the initialization of an OAuth 2.0 client. For the DTaaS implementation, GitLab is used by creating an "application" within GitLab. However, it is not necessary to initialize a client using the same platform as the identity provider; these are separate concerns. The DTaaS OAuth 2.0 client is initialized by creating and configuring a GitLab instance-wide application. There are two main elements in this configuration:

- **Redirect URI**: This is the URI to which users are redirected after they approve sharing information with the client.
- **Scopes:** These define the types and levels of access that the client can have over the user's profile. For the DTaaS, only the "read user" scope is required, which permits access to the user's profile information for identity verification.

After the GitLab application is successfully created, a Client ID and Client Secret are generated. These credentials can be used in any application, effectively making that application an OAuth 2.0 Client. For this reason, the Client Secret must never be shared. The DTaaS Auth MS uses this Client ID and Client Secret, thereby functioning as an OAuth 2.0 Client application capable of following the OAuth 2.0 workflow to verify user identity.

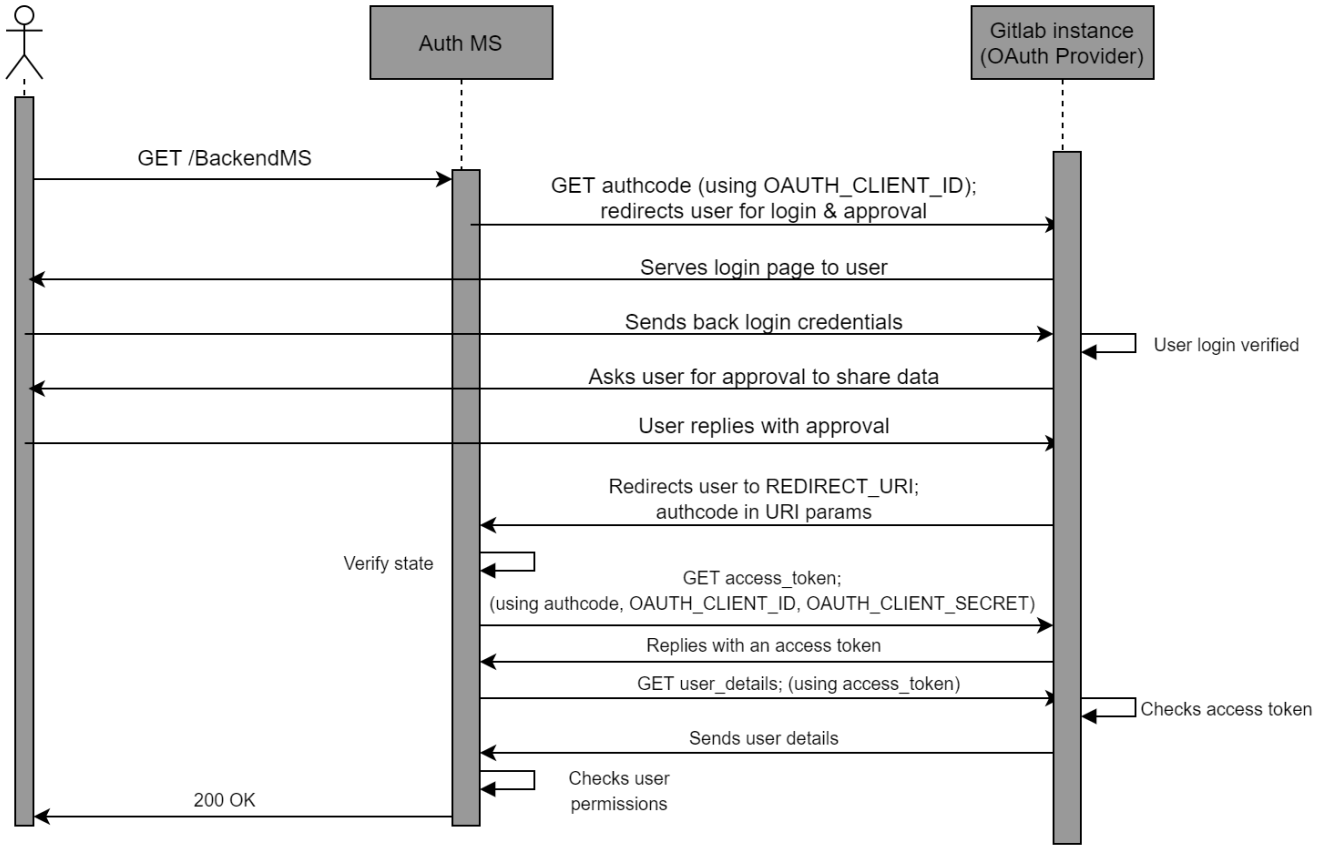
OAuth 2.0 Workflows

Two major OAuth 2.0 flows are employed in the DTaaS platform.

OAUTH 2.0 AUTHORIZATION CODE FLOW

This flow involves several steps and the exchange of an authorization code for access tokens to ensure secure authorization. This flow is used by the DTaaS Auth MS, which is responsible for securing all backend DTaaS services.

The OAuth 2.0 workflow is initiated by the Client (Auth MS) whenever user identity verification is required. The flow begins when the Auth MS sends an authorization request to GitLab. The Auth MS attempts to obtain an access token, which enables it to gather user information. Once user information is retrieved, the Auth MS can verify the user's identity and determine whether the user has permission to access the requested resource.



The requests made by the Auth MS to the OAuth 2.0 provider are shown in abbreviated form. A detailed explanation of the workflow specific to the DTaaS can be found in the Auth MS implementation documentation.

OAUTH 2.0 PKCE (PROOF KEY FOR CODE EXCHANGE) FLOW

PKCE is an extension to the OAuth 2.0 Authorization Code Flow designed to provide an additional layer of security, particularly for public clients that cannot securely store client secrets. PKCE mitigates certain attack vectors, such as authorization code interception.

The DTaaS client website login is implemented using the PKCE OAuth 2.0 flow. Further details about this flow are available in the Auth0 documentation.

6.3.3 System Design of DTaaS Authorization Microservice

DTaaS requires backend authorization to protect its backend services and user workspaces. This document details the system design of the DTaaS Auth Microservice which is responsible for the same.

Requirements

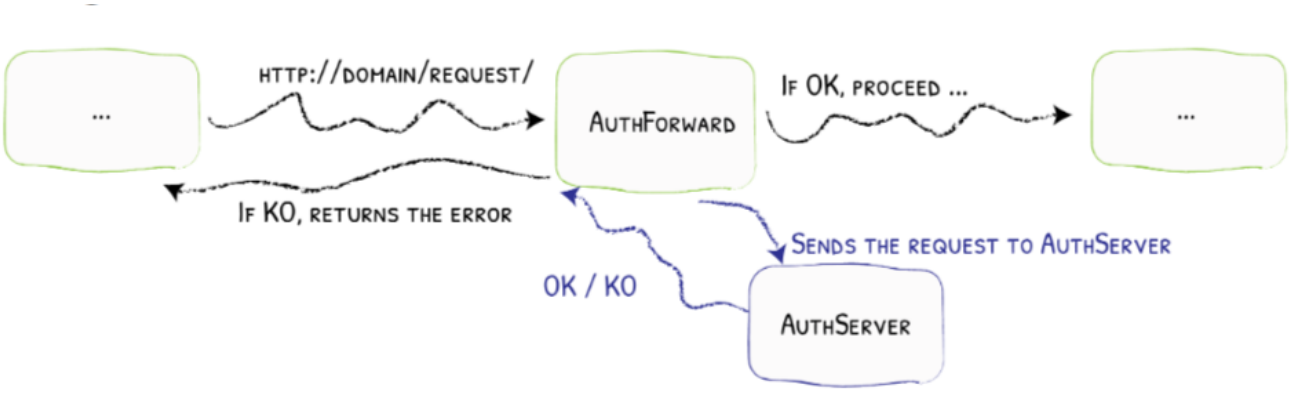
For this purpose, the Auth MS is required to handle only requests of the general form ”Is User X allowed to access /BackendMS/example?”.

If the user’s identity is correctly verified though the GitLab OAuth 2.0 provider AND this user is allowed to access the requested microservice/action, then the Auth MS should respond with a 200 (OK) code and let the request pass through the gateway to the required microservice/server.

If the user’s identity verification through GitLab OAuth 2.0 fails OR this user is not permitted to access the request resource, then the Auth MS should respond with a 40X (NOT OK) code, and restrict the request from going forward.

Forward Auth Middleware in Traefik

Traefik allows middlewares to be set for the routes configured into it. These middlewares intercept the route path requests, and perform analysis/modifications before sending the requests ahead to the services. Traefik has a ForwardAuth middleware that delegates authentication to an external service. If the external authentication server responds to the middleware with a 2XX response codes, the middleware acts as a proxy, letting the request pass through to the desired service. However, if the external server responds with any other response code, the request is dropped, and the response code returned by the external auth server is returned to the user

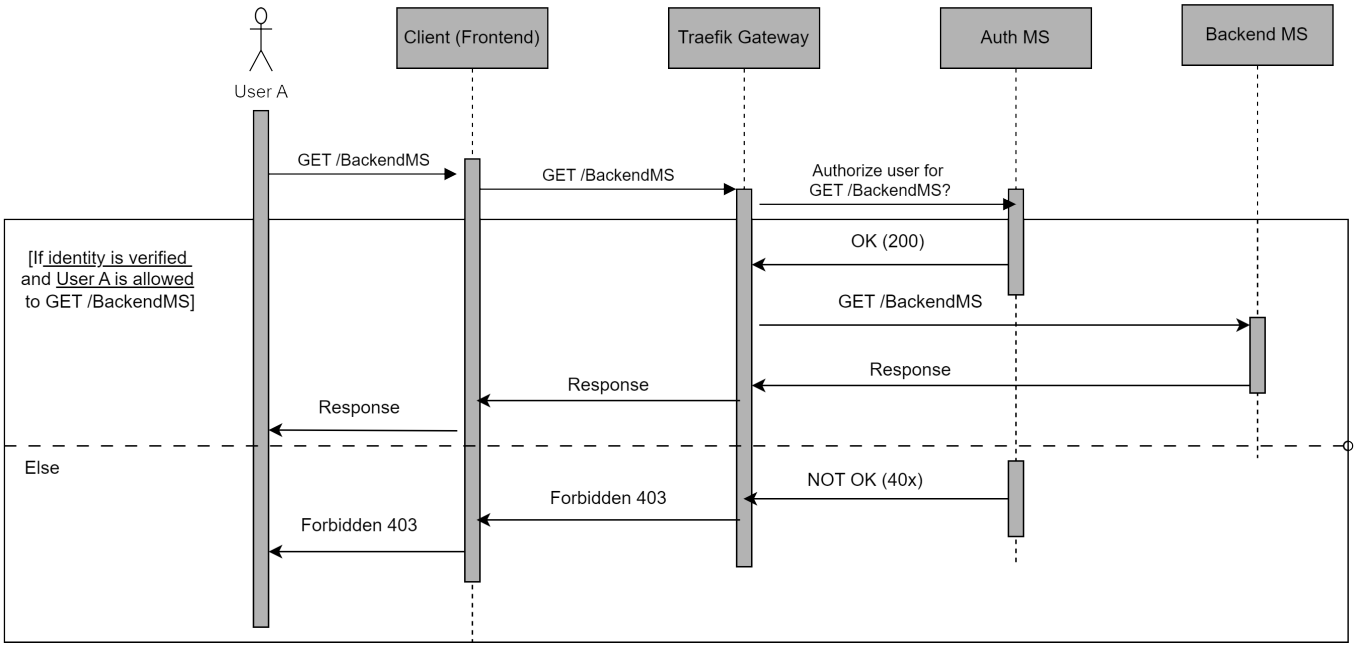


(source: Treafik documentation)

Thus, an Auth Microservice can be integrated into the existing gateway and DTaaS system structure easily by adding it as the external authentication server for ForwardAuth middlewares. These middlewares can be added on whichever routes/requests require authentication. For our specific purpose, this will be added to all routes since the platform imposes at least identity verification of users for any request through the gateway

Auth MS Design

The integrated Auth MS should thus work as described in the sequence diagram.



- Any request made by the user is made on the React website, i.e. the frontend of the DTaaS platform.
- This request then goes through the Traefik gateway. Here it should be interrupted by the respective ForwardAuth middleware.
- The middleware asks the Auth MS if this request for the given user should be allowed.
- The Auth MS, i.e. the Auth server verifies the identity of the user using OAuth 2.0 with GitLab, and checks if this user should be allowed to make this request.
- If the user is verified and allowed to make the request, the Auth server responds with a 200 OK to Traefik Gateway (more specifically to the middleware in Traefik)
- Traefik then forwards this request to the respective service. A response by the service, if any, will be passed through the chain back to the user.
- However, If the user is not verified or not allowed to make this request, the Auth server responds with a 40x to Traefik gateway.
- Traefik will then drop the request and respond to the Client informing that the request was forbidden. It will also pass the Auth servers response code

6.3.4 Auth Microservice

This document details the workflow and implementation of the DTaaS Auth Microservice. Please go through the System Design and the summary of the OAuth 2.0 technology to be able to understand the content here better.

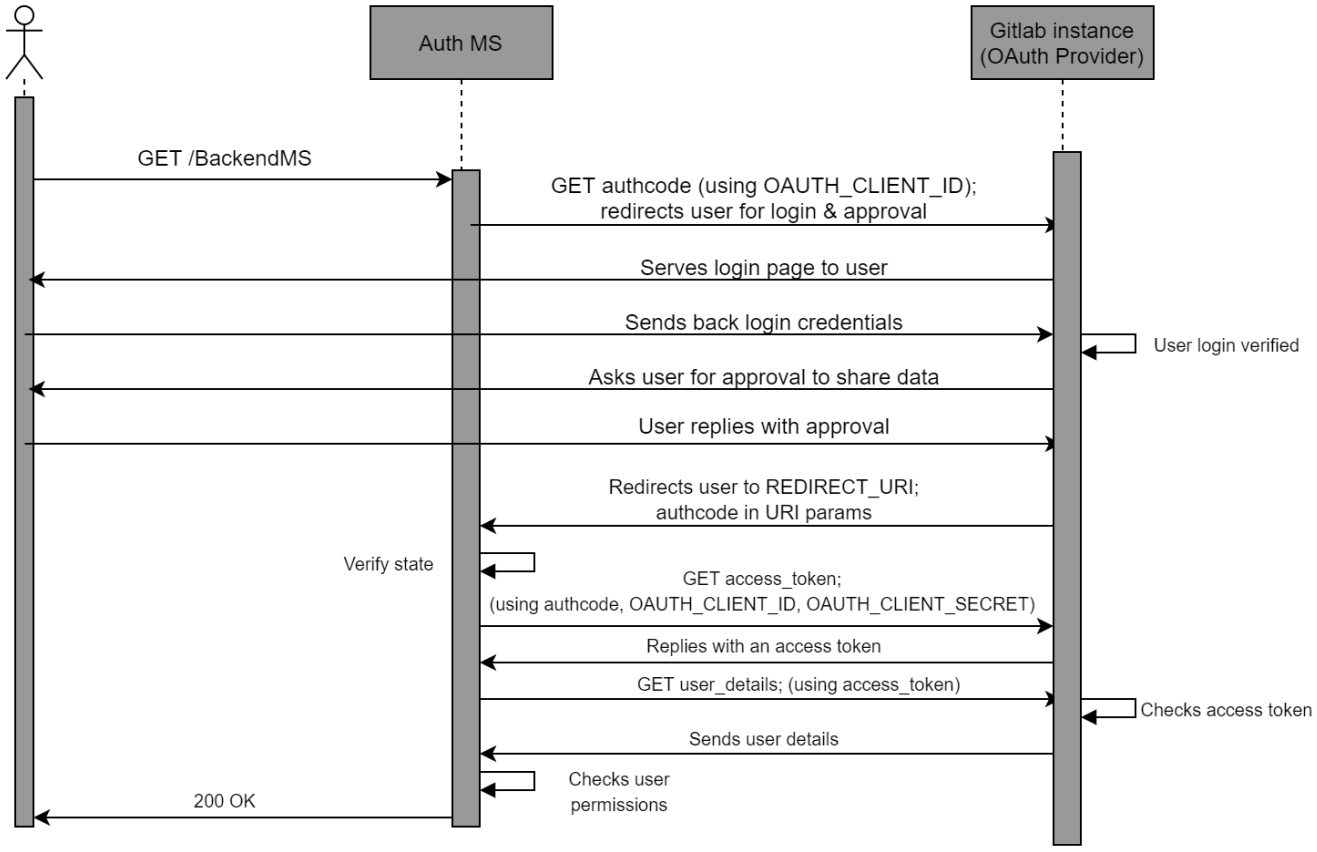
Workflow

USER IDENTITY USING OAUTH 2.0

The following constants are defined for the purposes of this discussion:

- CLIENT ID: The OAuth 2.0 Client ID of the Auth MS
- CLIENT SECRET: The OAuth 2.0 Client Secret of Auth MS
- REDIRECT URI: The URI where the user is redirected to after the user has approved sharing of information with the client.
- STATE: A random string used as an identifier for the specific "GET authcode" request (Figure 3.3)
- AUTHCODE: The one-use-only Authorization code returned by the OAuth 2.0 provider (GitLab instance) in response to "GET authcode" after user approval.

Additionally, suppose DTaaS uses a dedicated gitlab instance hosted at the URL https://gitlab.intocps.org (instead of https://intocps.org)



A successful OAuth 2.0 workflow (Figure 3.3) has the following steps:

- The user requests a resource, say *GET/BackendMS*
- The Auth MS intercepts this request, and starts the OAuth 2.0 process.
- The Auth MS sends a authorization request to the GitLab instance.

This is written in shorthand as *GET/authcode*. The actual request (a user redirect) looks like:

```
https ://gitlab.intocps.org/oauth/
authorize?
response_type=code&
client_id=OAUTH_CLIENT_ID&
redirect_uri=REDIRECT_URI&
scope=read_user&state = STATE
```

Here the gitlab.intocps.org/oauth/authorize is the specific endpoint of the GitLab instance that handles authorisation code requests.

The query parameters in the request include the expected response type, which is fixed as ”code”, meaning that an Authorization code is expected. Other query parameters are the client id, the redirect uri, the scope which is set to read user for our purpose, and the state (the random string to identify the specific request).

- The OAuth 2.0 provider redirects the user to the login page. Here the user logs into their protected account with their username/email ID and password.
- The OAuth 2.0 provider then asks the user to approve/deny sharing the requested information with the Auth MS. The user should approve this for successful authentication.
- After approval, the user is redirected by the GitLab instance to the REDIRECT URI. This URI has the following form:

```
REDIRECT_URI?code=AUTHCODE&state=STATE
```

The REDIRECT URI is as defined previously, during the OAuth 2.0 Client initialisation, i.e. the same as the one provided in the ”GET authcode” request by the Auth MS. The query parameters are provided by the GitLab instance. These include the AUTHCODE which is the authoriation code that the Auth MS had requested, and the STATE which is the same random string in the ”GET authcode” request.

- The Auth MS retrieves these query parameters. It verifies that the STATE is the same as the random string it provided during the "GET authcode" request. This confirms that the AUTHCODE it has received is in response to the specific request it had made.
- The Auth MS uses this one-use-only AUTHCODE to exchange it for a general access token. This access token wouldn’t be one-use-only, although it would expire after a specified duration of time. To perform this exchange, the Auth MS makes another request to the GitLab instance. This request is written in shorthand as *GET/access_token* in the sequence diagram. The true form of the request is:

```
POST https://gitlab.intocps.org/oauth/token,
parameters = 'client_id=OAUTH_CLIENT_ID&
client_secret=OAUTH_CLIENT_SECRET&
code=AUTHCODE&
grant_type=authorization_code&
redirect_uri=REDIRECT_URI'
```

The request to get a token by exchanging an authorization code, is actually a POST request (for most OAuth 2.0 providers). The https://gitlab.intocps.org/oauth/token API endpoint handles the token exchange requests. The parameters sent with the POST request are the client ID, the client secret, the AUTHCODE and the redirect uri. The grant type parameter is always set to the string ”authorisation code”, which conveys that an authentication code is being exchanged for an access token.

- The GitLab instance exchanges a valid AUTHCODE for an Access Token. This is sent as a response to the Auth MS. An example response is of the following form:

```
{
  "access_token": "d8aed28aa506f9dd350e54",
  "token_type": "bearer",
  "expires_in": 7200,
  "refresh_token":"825f3bffb2544b976633a1",
  "created_at": 1607635748
}
```

The access token field provides the string that can be used as an access token in the headers of requests tryng to access user information. The token type field is usually ”bearer”, the expires in field specifies the time in seconds for which the access token will be valid, and the

created at field is the Epoch timestamp at which the token was created. The refresh token field has a string that can be used to refresh the access token, increasing its lifetime. However, the refresh token field is not used. If an access token expires, the Auth MS simply requests a new one. TOKEN is the access token string returned in the response.

- The Auth MS has finally obtained an access token that it can use to retrieve the user's information. Note that if the Auth MS already had an existing valid access token for information about this user, the steps above wouldn't be necessary, and thus wouldn't be performed by the Auth MS. The steps till now in the sequence diagram are simply to get a valid access token for the user information.
- The Auth MS makes a final request to the GitLab instance, shorthanded as *GET user_details* in the sequence diagram. The actual request is of the form:

```
GET https ://gitlab.intocps.org/api/v4/user
"Authorization": Bearer <TOKEN>
```

Here, https://gitlab.intocps.org/api/v4/user is the API endpoint that responds with user information. An authorization header is required on the request, with a valid access token. The required header is added here, and TOKEN is the access token that the Auth MS holds.

- The GitLab instance verifies the access token, and if it is valid, responds with the required user information. This includes username, email ID, etc. An example response looks like:

```
{
  "id": 8,
  "username": "UserX",
  "name": "XX",
  "state": "active",
  "web_url": "http://gitlab.intocps.org/UserX",
  "created_at":"2023-12-03 T10:47:21.970 Z",
  "bio": "",
  "location": "",
  "public_email": null,
  "skype": "",
  "linkedin": "",
  "twitter": "",
  "organization": "",
  "job_title": "",
  "work_information": null,
  "followers": 0,
  "following": 0,
  "is_followed ": false,
  "local_time": null,
  "last_sign_in_at": "2023-12-13 T12:46:21.223 Z",
  "confirmed_at": "2023-12-03 T10:47:21.542 Z ",
  "last_activity_on": "2023-12-13",
  "email": "UserX@localhost",
  "projects_limit": 100000,
}
```

The important fields from this response are the ”email”, ”username” keys. These keys are unique to a user, and thus provide an identity to the user.

- The Auth MS retrieves the values of candidate key fields like ”email”, ”username” from the response. Thus, the Auth MS now knows the identity of the user.

CHECKING USER PERMISSIONS - AUTHORIZATION

An important feature of the Auth MS is to implement access policies for DTaaS resources. There may be requirements that certain resources and/or microservices in DTaaS should only be accessible to certain users. For example, it may be required that /BackendMS/user1 should only be accessible to the user who has username user1. Another example may be that /BackendMS/group3 should only be available to users who have an email ID in the domain @gmail.com. The Auth MS should be able to impose these restrictions and make certain services selectively available to certain users. There are two steps to doing this:

- Firstly, the user's identity should be known and trusted. The Auth MS should know the identity of a user and believe that the user is who they claim to be. This has been achieved in the previous section
- Secondly, this identity should be analysed against certain rules or against a database of allowed users, to determine whether this user should be allowed to access the requested resource.

The second step requires, for every service, either a set of rules that define which users should be allowed access to the service, or a database of user identities that are allowed to access the service. This database and/or set of rules should use the user identities, in this case the email ID or username, to decide whether the user should be allowed or not. This means that the rules should be built based on the

kind of username/ email ID the user has, say maybe using some RegEx. In the case of a database, the database should have the user identity as a key. For any service, a simple lookup can determine whether the key exists in the database or not, and the user can be allowed or denied access based on that.

In the sequence diagram, the Auth MS has a self-request marked as ”Checks user permissions” after receiving the user identity from the GitLab instance. This is when the Auth MS compares the identity of the user to the rules and/or database it has for the requested service. Based on this, if the given identity has access to the requested resource, the Auth MS responds with a 200 OK. This finally marks a succcessful authentication, and the user can now access the requested resource. Note: Again, the Auth MS and user do not communicate directly. All requests/responses of the Auth MS are with the Traefik gateway, not the User directly. Infact, the Auth MS is the external server used by the ForwardAuth middleware of the specific route, and communicates with this middleware. If the authentication is successful, The gateway forwards the request to the specific resource when the 200 OK is recieved, else it drops the request and returns the error code to the user.

Implementation

TRAEFIK-FORWARD-AUTH

The implementation approach is setting up and configuring the open source thomseddon/traefik-forward-auth for the specific DTaaS use case. This would serve as the Auth microservice.

The traefik-forward-auth software is available as a docker.io image. This works as a docker container. Thus there are no dependency management issues. Additionally, it can be added as a middleware server to traefik routers. Thus, it needs at least Traefik to work along with it properly. It also needs active services that it will be controlling access to. Traefik, the traefikforward-auth service and any services are thus, treated as a stack of docker containers. The main setup needed for this system is configuring the compose.yml file.

There are three main steps of configuring the Auth MS properly.

- The traefik-forward-auth service needs to be configured carefully. Firstly, the environment variables are set for the specific case. Since GitLab is used, the generic-oauth provider configuration is employed. Some important variables that are required are the OAuth 2.0 Client ID, Client Secret, Scope. The API endpoints for getting an AUTHCODE, exchanging the code for an access token and getting user information are also necessary

Additionally, it is necessary to create a router that handles the REDIRECT URI path. This router should have a middleware which is set to traefik-forward-auth itself. This is so that after approval, when the user is taken to REDIRECT URI, this can be handled by the gateway and passed to the Auth service for token exchange. The ForwardAuth middleware is added here, which is a necessary part of the design as discussed previously. A load balancer is also added for the service. A conf file must also be added as a volume, for selective authorization rules (discussed later). This is according to the suggested configuration. Thus, the following is added to the docker services:

```
traefik-forward-auth:
image: thomseddon/traefik-forward-auth:latest
volumes:
  - <filepath>/conf:/conf
environment:
  - DEFAULT_PROVIDER = generic - oauth
  - PROVIDERS_GENERIC_OAUTH_AUTH_URL=https://gitlab.intocps.org/oauth/authorize
  - PROVIDERS_GENERIC_OAUTH_TOKEN_URL=https://gitlab.intocps.org/oauth/token
  - PROVIDERS_GENERIC_OAUTH_USER_URL=https://gitlab.intocps.org/api/v4/user
  - PROVIDERS_GENERIC_OAUTH_CLIENT_ID=OAUTH_CLIENT_ID
  - PROVIDERS_GENERIC_OAUTH_CLIENT_SECRET=OAUTH_CLIENT_SECRET
  - PROVIDERS_GENERIC_OAUTH_SCOPE = read_user
  - SECRET = a - random - string
  # INSECURE_COOKIE is required if
  # not using a https entrypoint
  - INSECURE_COOKIE = true
labels:
  - "traefik.enable=true"
  - "traefik.http.routers.redirect.entryPoints=web"
  - "traefik.http.routers.redirect.rule=PathPrefix(/_oauth)"
  - "traefik.http.routers.redirect.middlewares=traefik-forward-auth"
  - "traefik.http.middlewares.traefik-forward-auth.forwardauth.address=http://traefik-forward-auth:4181"
  - "traefik.http.middlewares.traefik-forward-auth.forwardauth.authResponseHeaders=X-Forwarded-User"
  - "traefik.http.services.traefik-forward-auth.loadbalancer.server.port=4181"
```

- The traefik-forward-auth service should be added to the backend services as a middleware.

To do this, the docker-compose configurations of the services need to be updated by adding the following lines:

```
      - "traefik.http.routers.<service-router>.rule=Path(/<path>)"
      - "traefik.http.routers.<service-router>.middlewares=traefik-forward-auth"
```

This creates a router that maps to the required route, and adds the auth middleware to the required route.

- Finally, user permissions must be set on user identities by creating rules in the conf file. Each rule has a name (an identifier for the rule), and an associated route for which the rule will be invoked. The rule also has an action property, which can be either ”auth” or ”allow”. If action is set to ”allow”, any requests on this route are allowed to bypass even the OAuth 2.0 identification. If the action is set to ”auth”, requests on this route will require User identity OAuth 2.0 and the system will follow the sequence diagram. For rules with action=”auth”, the user information is retrieved. The identity used for a user is the user's email ID. For "auth" rules, two types of User restrictions/permissions can be configured on this identity:
- Whitelist - This would be a list of user identities (email IDs in the DTaaS context) that are allowed to access the corresponding route.
- Domain - This would be a domain (example: gmail.com), and only email IDs (user identities) of that domain (example: johndoe@gmail.com) would be allowed access to the corresponding route.

Configuring any of these two properties of an "auth" rule enables selective access control for certain users to certain resources. Not configuring any of these properties for an ”auth” rule means that the OAuth 2.0 process is carried out and the user identity is retrieved, but all known user identities (i.e. all users that successfully complete the OAuth 2.0) are allowed to access the resource.

DTaaS currently uses only the whitelist type of rules.

These rules can be used in 3 different ways described below. The exact format of lines to be added to the conf file are also shown.

- No Auth - Serves the Path(‘/public‘) route. A rule with action=”allow” should be imposed on this.

```
rule.noauth.action=allow
rule.noauth.rule=Path(`/public`)
```

- User specific: Serves the Path(‘/user1‘) route. A rule that only allows ”user1@localhost” identity should be imposed on this

```
rule.onlyu1.action=auth
rule.onlyu1.rule=Path(`/user1`)
rule.onlyu1.whitelist=user1@localhost
```

- Common Auth - Serves the Path(‘/common‘) route. A rule that requires OAuth 2.0, i.e. with action=”allow”, but allows all valid and known user identities should be imposed on this.

```
rule.all.action = auth
rule.all.rule = Path(`/common`)
```

6.3.5 GitLab CI/CD Infrastructure

Given that files in the Library are stored in a Git repository, the approach employed was that of GitLab's parent-child pipelines. In this context, a parent pipeline initiates the execution of another pipeline within the same project, the latter of which is known as the child pipeline. More about pipelines can be found in GitLab's documentation on CI/CD Pipelines.

CI/CD Pipelines

Continuous Integration (CI) and Continuous Deployment (CD) represent two key components of the DevOps methodology.

CI involves frequent integration of code changes into a common repository. Each integration triggers automated builds and tests that permit the detection of issues at an early stage. This practice ensures that the changes made to the code are checked fast enough, reducing the possibilities of integration problems and hence ensuring high-quality software.

CD automates the process of release, ensuring that code changes are automatically tested and prepared for deployment. Teams using CD can deploy updates rapidly and reliably, improving the responsiveness and quality of software. Performed together, CI/CD automates the whole delivery pipeline for software, increasing efficiency and reducing errors. They entirely eliminate, or significantly reduce, the manual human input required for a code change to be moved from a commit to a production environment. The entire process of compilation, testing (including unit, integration, and regression testing), deployment, and infrastructure provisioning is included.

CI/CD practices are explained in more detail in this article by GitLab.

A CI/CD pipeline is a series of automated processes that manage CI and CD of software. They are configured to run automatically, with no need for manual intervention once activated.

GitLab is a single application for the entire DevOps lifecycle, which means it performs all of the basics required for CI/CD in one environment. The documentation provided by GitLab was instrumental in enabling a comprehensive understanding of the CI/CD pipelines.

Pipelines are composed of a number of essential components. *Jobs* delineate the specific tasks to be accomplished, while *stages* define the sequence in which jobs are executed. In this way, stages ensure that each step takes place in the right order and make the pipeline more efficient and consistent. In the event that all jobs within a stage are successfully completed, the pipeline will automatically proceed to the subsequent stage. However, if any of the jobs fail, the flow is interrupted without proceeding.

When a pipeline is initiated, the jobs that have been defined within it are then distributed among the available runners.

GitLab runners are agents within the GitLab Runner application that execute the jobs in accordance with their configuration and the available resources. They can be configured to operate on a variety of platforms, including virtual machines, containers, and physical servers. They can also be managed locally or in a cloud environment.

This GitLab parent-child pipeline setup is used to trigger execution of digital twins stored in a user's GitLab repository.

> **Note**
>
> The recommended practice is to modified these pipelines via the Digital Twins Preview Page.

Parent Pipeline

The parent pipeline was configured as a top-level element. There is a single stage called `triggers`, which is responsible for triggering other child pipelines.

In the `.gitlab-ci.yml` file, triggers are managed for DTs inside the user repository. Each trigger is connected with one distinct DT and becomes active when the corresponding value of the `DTName` variable is provided by the API call. The `RunnerTag` variable is used to specify a custom runner tag that will execute each job in the DT's pipeline.

Below is an explanation of the keywords used in the CI/CD pipeline configuration:

- **Image**: Specifies the Docker image, such as `fedora:41`, providing the environment for the pipeline execution.
- **Stages**: Defines phases in the pipeline, such as triggers, organizing tasks sequentially.
- **Trigger**: Initiates another pipeline or job, incorporating configurations from an external file.
- **Include**: Imports configurations from another file for modular pipeline setups.
- **Rules**: Sets conditions for job execution based on variables or states.
- **If**: A condition within rules specifying when a job should run based on the value of a variable.
- **When**: Specifies the timing of job execution, such as `always`.
- **Variables**: Defines dynamic variables, like `RunnerTag`, used in the pipeline.

Here is an example of such a YAML file that registers a trigger for a DT named `mass-spring-damper`:

```yaml
image: fedora:41

stages:
  - triggers

trigger_mass-spring-damper:
  stage: triggers
  trigger:
    include: digital_twins/mass-spring-damper/.gitlab-ci.yml
  rules:
    - if: '$DTName == "mass-spring-damper"'
      when: always
  variables:
    RunnerTag: $RunnerTag
```

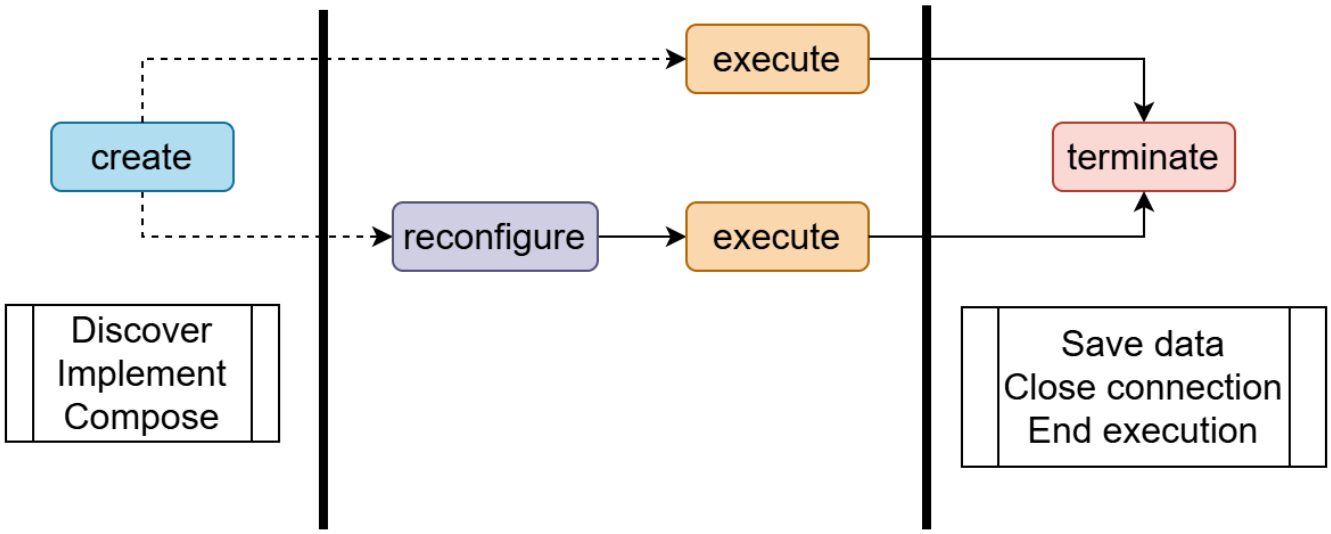
Digital Twin Structure

The `digital_twins` folder contains DTs that have been pre-built by one or more users. The intention is that they should be sufficiently flexible to be reconfigured as required for specific use cases.

The following is an example of such a configuration. The dtaas/user1 repository on gitlab.com contains the `digital_twins` directory with a `hello_world` example. Its file structure looks like this:

```
hello_world/
├── lifecycle/
│   ├── clean
│   ├── create
│   ├── execute
│   └── terminate
├── .gitlab-ci.yml
└── description.md
```

The `lifecycle` directory here contains four files - `clean`, `create`, `execute` and `terminate`, which are simple BASH scripts. These correspond to stages in a digital twin's lifecycle.



Child Pipelines

To automate the lifecycle of DT, a child pipeline has been incorporated into its corresponding folder. Regardless of the image provided in the parent pipeline, each child pipeline will use its own specified image specified in its YAML configuration or Ruby's default image.

The following are the explanations of the keywords used within the CI/CD child pipeline based on GitLab's CI/CD YAML syntax reference:

1. `Stage` It defines the steps that happen in a pipeline sequentially, for example, create, execute and clean, to make sure that tasks occur in a specific order.
2. `Script` It lists commands to be run at each step; for example, changing directories, modifying permissions, or running lifecycle scripts.
3. `Tags` It specifies which runner should run the jobs, thereby providing an additional control over where and how the jobs are run.

With the DT `mass-spring-damper` serving as a point of reference, the stages in question are designed to facilitate the creation, execution, and termination of the DT simulation, as well as the cleaning and restoration of the environment to ensure its readiness for future executions. Here is an example of a configuration that defines `create`, `execute` and `clean` as part of the child pipeline:

```
image: ubuntu:20.04

stages:
    - create
    - execute
    - clean

create_mass-spring-damper:
    stage: create
    script:
        - cd digital_twins/mass-spring-damper
        - chmod +x lifecycle/create
        - lifecycle/create
    tags:
        - $RunnerTag

execute_mass-spring-damper:
    stage: execute
    script:
        - cd digital_twins/mass-spring-damper
        - chmod +x lifecycle/execute
        - lifecycle/execute
    tags:
        - $RunnerTag

clean_mass-spring-damper:
    stage: clean
    script:
        - cd digital_twins/mass-spring-damper
        - chmod +x lifecycle/terminate
        - chmod +x lifecycle/clean
        - lifecycle/terminate
    tags:
        - $RunnerTag
```

Direct API Calls

A GitLab DevOps pipeline can be triggered via an API call using a pipeline trigger token which is created on the GitLab instance, with the following values:

1. `<trigger_token>` : The user GitLab trigger token.
2. `<digital_twin_name>` : The name of the DT (e.g. mass-spring-damper).
3. `<runner_tag>` : The specific tag of the GitLab runner that the user wants to use.
4. `<project_id>` : The ID of the GitLab project, displayed in the project overview page.

The example given below sets the `DTName` variable to the desired DT name, the `RunnerTag` variable to the specified GitLab Runner tag, and ensures the call will be executed in the `main` branch:

```
curl --request POST \
  --form "token=<trigger_token>" \
  --form ref=main \
  --form "variables[DTName]=<digital_twin_name>" \
  --form "variables[RunnerTag]=<runner_tag>" \
  "https://intocps.org/gitlab/api/v4/projects/<project_id>/trigger/pipeline"
```

Reference

Vanessa Scherma, Design and implementation of an integrated DevOps framework for Digital Twins as a Service software platform, Master's Degree Thesis, Politecnico Di Torino, 2024.

6.3.6 Testing

? Fundamental Concepts in Software Testing

DEFINITION OF SOFTWARE TESTING

Software testing is a procedure to investigate the quality of a software product across different scenarios. It can also be defined as the process of verifying and validating that a software program or application works as expected and meets the business and technical requirements that guided its design and development.

IMPORTANCE OF SOFTWARE TESTING

Software testing is essential for identifying defects and errors introduced during different development phases. Testing also ensures that the product under test works as expected across expected scenarios — a stronger test suite results in greater confidence in the product being built. One important benefit of software testing is that it enables developers to make incremental changes to source code while ensuring that current changes do not break the functionality of previously existing code.

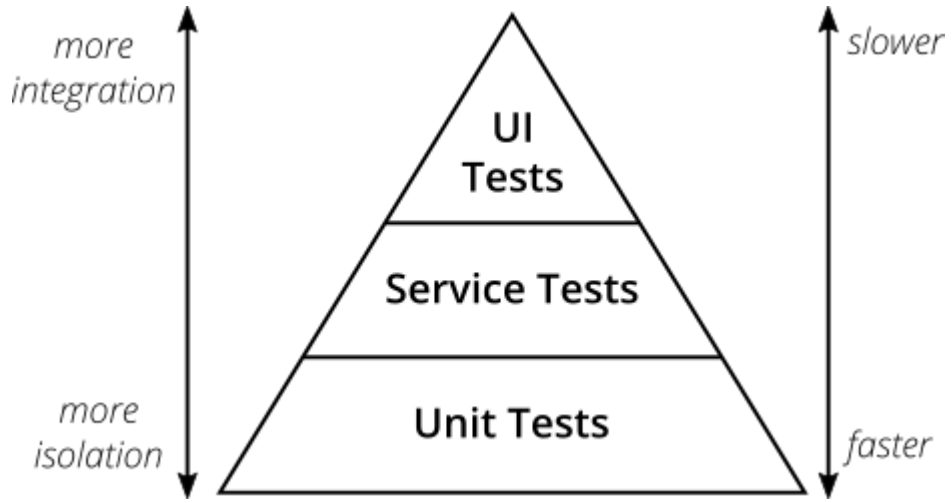
TEST DRIVEN DEVELOPMENT (TDD)

Test Driven Development (TDD) is a software development process that relies on the repetition of a very short development cycle: first, the developer writes an (initially failing) automated test case that defines a desired improvement or new function, then produces the minimum amount of code to pass that test, and finally refactors the new code to acceptable standards. The goal of TDD can be viewed as specification rather than validation. In other words, TDD provides a methodology for thinking through requirements or design before writing functional code.

BEHAVIOUR DRIVEN DEVELOPMENT (BDD)

Behaviour Driven Development (BDD) is a software development process that emerged from TDD. It includes the practice of writing tests first, but focuses on tests that describe behavior rather than tests that verify a unit of implementation. This approach provides software development and management teams with shared tools and a shared process for collaborating on software development. BDD is largely facilitated through the use of a simple domain-specific language (DSL) employing natural language constructs (e.g., English-like sentences) that can express behavior and expected outcomes. Mocha and Cucumber testing libraries are built around the concepts of BDD.

Testing Workflow



(Ref: Ham Vocke, The Practical Test Pyramid)

The DTaaS project follows a testing workflow in accordance with the test pyramid diagram shown above, starting with isolated tests and moving towards complete integration for any new feature changes. The different types of tests, in the order that they should be performed, are explained below:

UNIT TESTS

Unit testing is a level of software testing where individual units/ components of a software are tested. The objective of Unit Testing is to isolate a section of code and verify its correctness.

Ideally, each test case is independent from the others. Substitutes such as method stubs, mock objects, and spies can be used to assist testing a module in isolation.

Benefits of Unit Testing

- Unit testing increases confidence in changing and maintaining code. When good unit tests are written and executed every time code is changed, defects introduced due to the change can be promptly identified.
- When code is designed to be less interdependent to facilitate unit testing, the unintended impact of changes to any code is reduced.
- The cost, in terms of time, effort, and money, of fixing a defect detected during unit testing is lower compared to defects detected at higher levels.

Unit Tests in DTaaS

Each component of the DTaaS project uses a unique technology stack; therefore, the packages used for unit tests differ across components. The `test/` directory of each component contains information about the specific unit test packages employed.

INTEGRATION TESTS

Integration testing is the phase in software testing in which individual software modules are combined and tested as a group. The DTaaS project uses an integration server for software development as well as integration testing.

The existing integration tests are performed at the component level. Integration tests between components have not yet been implemented; this task has been deferred to future development.

END-TO-END TESTS

Testing code changes through the end-user interface of the software is essential to verify that the code produces the desired effect for users. End-to-End tests in DTaaS require a functional setup.

End-to-end testing capabilities in DTaaS are currently limited; further development of this testing layer has been deferred to future work.

FEATURE TESTS

A software feature can be defined as changes made to the system to add new functionality or modify existing functionality. Each feature is characterized by being useful, intuitive, and effective. It is important to test new features upon implementation and to ensure that they do not break the functionality of existing features. Feature tests are therefore essential for maintaining software quality.

The DTaaS project does not currently include feature tests. Cucumber is planned for use in implementing feature tests in future development.

References

1. Arthur Hicken, Shift left approach to software testing
2. Justin Searls and Kevin Buchanan, Contributing Tests wiki.
3. This wiki has good explanation of TDD and test doubles.

6.4 Codebase

6.4.1 React Client 💻

The DTaaS web client is a React + TypeScript single page application. It provides user-facing workflows for DT composition, asset browsing, configuration, and DevOps-driven execution.

Current Stack

- React 19
- React Router 7
- Redux Toolkit 2
- Material UI 7
- Gitbeaker REST client for GitLab APIs

Current Source Layout

```
client/
├── src/
│   ├── index.tsx
│   ├── AppProvider.tsx
│   ├── routes.tsx
│   ├── components/
│   ├── database/
│   ├── model/
│   │   └── backend/
│   │       ├── interfaces/
│   │       ├── gitlab/
│   │       ├── util/
│   │       └── ...
│   ├── page/
│   ├── route/
│   ├── store/
│   ├── util/
│   └── utils/
├── test/
│   ├── __mocks__/
│   ├── unit/
│   ├── integration/
│   └── e2e/
├── config/
├── public/
└── package.json
```

Backend Integration Layer

The backend model layer in `src/model/backend/` encapsulates GitLab-backed operations.

Key classes:

- `GitlabAPI` : low-level GitLab REST wrapper.
- `GitlabInstance` : project and trigger-token context.
- `DigitalTwin` : DT-level operations and execution history integration.
- `LibraryAsset` : reusable asset access patterns.

Important capability:

- Batched file updates through `commitMultipleActions` , enabling one commit for multiple create/update/delete operations.

Build and Test Commands

Run from `client/`:

```
yarn syntax
yarn test:unit
yarn test:int
yarn test:e2e
yarn build
```

Notes for Contributors

- Keep GitLab-specific logic in backend implementation modules.
- Keep route/view components focused on presentation and user workflows.
- Prefer typed interfaces in `model/backend/interfaces/` when extending API contracts.

6.4.2 DevOps Framework

This page consolidates the DevOps framework documentation for the current DTaaS codebase.

Purpose

The DevOps framework allows the DTaaS client to manage Digital Twin lifecycle actions through GitLab APIs and CI/CD pipelines.

Core capabilities include:

- Discovering DT definitions and reusable assets from repositories.
- Creating/updating DT files in Git repositories.
- Triggering pipelines for DT execution.
- Collecting pipeline/job status and logs for UI feedback.

Current Implementation Footprint

Key code is in `client/src/model/backend/`.

Main modules:

- `gitlab/backend.ts`: GitLab API wrapper.
- `gitlab/instance.ts`: backend session and project context.
- `digitalTwin.ts`: DT-level operations and execution tracking.
- `interfaces/backendInterfaces.ts`: backend contracts and API shapes.
- `util/`: execution history and file-management helpers.

API Integration Highlights

The GitLab backend implementation supports:

- Pipeline trigger and cancellation.
- Repository file create/edit/delete operations.
- Group and project discovery.
- Pipeline job listing and log retrieval.
- Pipeline status polling.
- Multi-file commit batching via `commitMultipleActions`.

The multi-action commit path reduces fragmentation and keeps DT creation or updates atomic from the user perspective.

Execution Flow

1. `GitlabInstance` resolves project IDs and trigger token.
2. The UI operation calls DT logic in `DigitalTwin`.
3. `DigitalTwin` uses backend APIs to prepare files and trigger pipeline execution.
4. Execution history is updated and surfaced to the UI.
5. Job logs and statuses are fetched and rendered for diagnostics.

Pipeline Contract

The framework assumes GitLab CI pipelines are prepared to consume DT selection and runner context as variables. This enables controlled execution across available runners and environments.

Practical Guidance

- Prefer backend interface methods over ad-hoc API calls in UI code.
- Use the batched commit method when updating multiple files.
- Keep execution history updates centralized in utilities for consistent status reporting.

6.4.3 Client DevOps Integration

This page consolidates the previous DevOps overview and implementation notes for the current React client codebase.

Scope

The DevOps integration in the client is responsible for:

- discovering Digital Twin and library assets from GitLab repositories,
- creating, updating, and deleting DT files,
- triggering and stopping CI/CD pipeline executions,
- collecting pipeline status and logs for user-facing execution views.

Current Module Map

Primary implementation lives in `client/src/model/backend/`:

- `gitlab/backend.ts`: `GitlabAPI` concrete `BackendAPI` implementation.
- `gitlab/instance.ts`: `GitlabInstance` project context and trigger-token holder.
- `digitalTwin.ts`: DT lifecycle operations, execution orchestration hooks.
- `DTAssets.ts`, `fileHandler.ts`, `libraryAsset.ts`, `libraryManager.ts`: file and asset management primitives.
- `util/digitalTwinPipelineExecution.ts`: start/stop execution paths.
- `util/digitalTwinExecutionHistory.ts`: execution history and log updates.
- `state/`: Redux state for execution history and status views.
- `interfaces/backendInterfaces.ts`: backend contracts and shared types.

Architectural Roles

The implementation keeps a clear split between context, API access, and DT domain behavior.

- `GitlabAPI` handles direct GitLab REST operations via `@gitbeaker/rest` (files, pipelines, groups/projects, jobs, logs, multi-action commits).
- `GitlabInstance` resolves `projectId` and `commonProjectId`, retrieves the trigger token, and exposes high-level execution data accessors.
- `DigitalTwin` composes DT asset/file operations with pipeline execution and execution-history updates.
- Supporting classes (`DTAssets`, `LibraryAsset`, `LibraryManager`, `FileHandler`) isolate low-level file and repository interactions.

This separation keeps UI code independent from GitLab implementation details.

Execution Flow

1. A session creates/initializes `GitlabInstance`.
2. The UI performs DT actions through `DigitalTwin` and backend utility modules.
3. Pipeline start/stop calls are routed through `GitlabInstance` and `GitlabAPI`.
4. Status polling and job log fetching update execution history state.
5. UI components render current and historical execution results.

File Operations and Commit Strategy

The client supports both per-file operations and batched commit updates.

- Single-file operations use `createRepositoryFile`, `editRepositoryFile`, and `removeRepositoryFile`.
- Multi-file updates should use `commitMultipleActions` to produce one coherent commit for DT scaffolding/reconfiguration workflows.

The batched path reduces commit fragmentation and keeps lifecycle updates atomic.

Key Contracts

The `BackendAPI` contract includes:

- pipeline trigger/cancel/status,
- pipeline job listing and job log retrieval,
- repository file CRUD and tree listing,
- group/project discovery,
- multi-action commit support (`commitMultipleActions`).

When extending DevOps behavior, update interface contracts first, then concrete GitLab implementation and call sites.

Contributor Guidance

- Keep GitLab-specific details inside `model/backend/gitlab/` .
- Keep route/page components focused on presentation and user interaction.
- Reuse utility/state modules for execution history instead of introducing ad-hoc polling in UI components.
- Add or update unit/integration tests when introducing backend contract changes.

6.4.4 Library Microservice

The Library Microservice exposes workspace files via two principal mechanisms: (1) a GraphQL API and (2) an HTTP file server accessible from web browsers. This service interfaces with the local file system and GitLab to provide uniform GitLab-compliant API access to files. The microservice serves as a critical component of the DTaaS platform, enabling both human users and digital twins to access reusable assets stored in the platform's library structure.

> **Warning**
>
> This microservice is still under heavy development. It is still not a satisfactory replacement for the file server currently in use.

Architecture and Design

The microservice is built using NestJS framework with Apollo GraphQL. The architecture employs the Strategy pattern via a factory service, enabling support for multiple file storage backends (local filesystem, GitLab). The GraphQL API provided by the library microservice shall be compliant with the GitLab GraphQL service.

Package Structure

```
servers/lib/
├── src/                          # Application source code
│   ├── main.ts                    # Application entry point
│   ├── bootstrap.ts               # NestJS bootstrap configuration
│   ├── app.module.ts              # Root application module
│   ├── schema.gql                 # Auto-generated GraphQL schema
│   ├── types.ts                   # GraphQL type definitions
│   ├── config/                    # Configuration module
│   │   ├── config.module.ts       # Configuration DI module
│   │   ├── config.service.ts      # Configuration service
│   │   ├── config.interface.ts    # Configuration interface
│   │   ├── config.model.ts        # Configuration model
│   │   └── util.ts                # Configuration utilities
│   ├── files/                     # Files module (core functionality)
│   │   ├── files.module.ts        # Files DI module
│   │   ├── files.resolver.ts      # GraphQL resolver
│   │   ├── files-service.factory.ts  # Backend factory
│   │   ├── interfaces/            # Service interfaces
│   │   │   └── files.service.interface.ts
│   │   ├── local/                 # Local filesystem backend
│   │   │   ├── local-files.module.ts
│   │   │   └── local-files.service.ts
│   │   └── git/                   # GitLab backend
│   │       ├── git-files.module.ts
│   │       └── git-files.service.ts
│   ├── enums/                     # Enumeration definitions
│   │   └── config-mode.enum.ts    # Backend mode enum
│   └── cloudcmd/                  # CloudCmd file browser integration
├── test/                          # Test suites
└── dist/                          # Compiled output
```

Key Components

GRAPHQL RESOLVER

The `FilesResolver` class exposes two GraphQL queries:

Query	Purpose
`listDirectory`	Returns directory contents (files and folders)
`readFile`	Returns file content as raw text

FILE SERVICE INTERFACE

The `IFilesService` interface defines the contract for all file backends:

```
interface IFilesService {
  listDirectory(path: string): Promise<Project>;
  readFile(path: string): Promise<Project>;
  getMode(): CONFIG_MODE;
  init(): Promise<any>;
}
```

BACKEND IMPLEMENTATIONS

Backend	Purpose
`LocalFilesService`	Accesses files from local filesystem
`GitFilesService`	Accesses files from GitLab repository

FACTORY PATTERN

The `FilesServiceFactory` creates the appropriate backend service based on configuration, enabling runtime selection of the storage backend.

UML Diagrams

CLASS DIAGRAM

```
classDiagram
    class FilesResolver {
    -filesService: IFilesService
    +listDirectory(path: string): Promise<Project>
    +readFile(path: string): Promise<Project>
    }

    class FilesServiceFactory {
    -configService: ConfigService
    -localFilesService: LocalFilesService
    +create(): IFilesService
    }

    class LocalFilesService {
    -configService: ConfigService
    -getFileStats(fullPath: string, file: string): Promise<Project>
    +listDirectory(path: string): Promise<Project>
    +readFile(path: string): Promise<Project>
    }

    class ConfigService {
    +get(propertyPath: string): any
    }

    class IFilesService{
    listDirectory(path: string): Promise<Project>
    readFile(path: string): Promise<Project>
    }

    IFilesService <|-- FilesResolver: uses
    IFilesService <|.. LocalFilesService: implements
    IFilesService <|-- FilesServiceFactory: creates
    ConfigService <|-- FilesServiceFactory: uses
    ConfigService <|-- LocalFilesService: uses
```

SEQUENCE DIAGRAM

```
sequenceDiagram
    actor Client
    actor Traefik

    box LightGreen Library Microservice
    participant FR as FilesResolver
    participant FSF as FilesServiceFactory
    participant CS as ConfigService
    participant IFS as IFilesService
    participant LFS as LocalFilesService
    end

    participant FS as Local File System DB

    Client ->> Traefik : HTTP request
    Traefik ->> FR : GraphQL query
    activate FR
```

```
    FR ->> FSF : create()
    activate FSF

    FSF ->> CS : getConfiguration("MODE")
    activate CS

    CS -->> FSF : return configuration value
    deactivate CS

    alt MODE = Local
    FSF ->> FR : return filesService (LFS)
    deactivate FSF

    FR ->> IFS : listDirectory(path) or readFile(path)
    activate IFS

    IFS ->> LFS : listDirectory(path) or readFile(path)
    activate LFS

    LFS ->> CS : getConfiguration("LOCAL_PATH")
    activate CS

    CS -->> LFS : return local path
    deactivate CS

    LFS ->> FS : Access filesystem
    alt Filesystem error
        FS -->> LFS : Filesystem error
        LFS ->> LFS : Throw new InternalServerErrorException
        LFS -->> IFS : Error
    else Successful file operation
        FS -->> LFS : Return filesystem data
        LFS ->> IFS : return Promise<Project>
    end
    deactivate LFS
    activate IFS
    end

    alt Error thrown
    IFS ->> FR : return Error
    deactivate IFS
    FR ->> Traefik : return Error
    Traefik ->> Client : HTTP error response
    else Successful operation
    IFS ->> FR : return Promise<Project>
    deactivate IFS
    FR ->> Traefik : return Promise<Project>
    Traefik ->> Client : HTTP response
    end

    deactivate FR
```

6.4.5 Runner Microservice

The Runner microservice provides script execution capabilities for digital twins within the DTaaS platform. This service accepts HTTP requests to execute pre-configured commands and returns execution status and logs to the caller.

Package Structure

```
servers/execution/runner/
├── src/                          # Application source code
│   ├── main.ts                    # Application entry point
│   ├── app.module.ts              # Root NestJS module
│   ├── app.controller.ts          # REST API controller
│   ├── execa-manager.service.ts   # Command execution manager
│   ├── execa-runner.ts            # Command runner implementation
│   ├── queue.service.ts           # Command queue service
│   ├── runner-factory.service.ts  # Runner instance factory
│   ├── validation.pipe.ts         # Request validation pipe
│   ├── config/                    # Configuration module
│   │   ├── commander.ts           # CLI argument parsing
│   │   ├── config.interface.ts    # Configuration interface
│   │   ├── configuration.service.ts  # Configuration service
│   │   └── util.ts                # Configuration utilities
│   ├── dto/                       # Data transfer objects
│   │   └── command.dto.ts         # Command request DTO
│   └── interfaces/                # Interface definitions
│       ├── command.interface.ts   # Command types
│       └── runner.interface.ts    # Runner interface
├── api/                          # API documentation
├── test/                         # Test suites
└── dist/                         # Compiled output
```

Architecture and Design

The microservice is built using NestJS framework and employs the `execa` library for command execution. The architecture implements a command queue pattern for managing execution requests and a factory pattern for creating runner instances.

CORE COMPONENTS

```
classDiagram
    class AppController {
        -manager: ExecaManager
        +getHistory(): ExecuteCommandDto[]
        +newCommand(dto): void
        +cmdStatus(): CommandStatus
    }

    class ExecaManager {
        -commandQueue: Queue
        -config: Config
        +newCommand(name): [boolean, Map]
        +checkStatus(): CommandStatus
        +checkHistory(): ExecuteCommandDto[]
    }

    class Queue {
        -queue: Command[]
        +enqueue(command): void
        +checkHistory(): ExecuteCommandDto[]
        +activeCommand(): Command
    }

    class RunnerFactory {
        +create(command): Runner
    }

    class ExecaRunner {
        -command: string
        -stdout: string
        -stderr: string
        +run(): Promise~boolean~
        +checkLogs(): Map
    }

    class Config {
        -configValues: ConfigValues
        +loadConfig(options): void
        +permitCommands(): string[]
        +getPort(): number
        +getLocation(): string
    }

    AppController --> ExecaManager : uses
```

```
    ExecaManager --> Queue : uses
    ExecaManager --> Config : uses
    ExecaManager --> RunnerFactory : uses
    RunnerFactory --> ExecaRunner : creates
```

Key Components

REST API CONTROLLER

The `AppController` exposes three endpoints:

Endpoint	Method	Purpose
`/`	POST	Submit a new command for execution
`/`	GET	Get status of the current command
`/history`	GET	Retrieve execution history

EXECUTION MANAGER

The `ExecaManager` service orchestrates command execution:

1. Validates command against permitted commands list
2. Creates runner instance via factory
3. Queues command for execution
4. Returns execution logs and status

COMMAND QUEUE

The `Queue` service maintains execution history and tracks the currently active command. Commands are stored with their execution status (valid/invalid).

RUNNER INTERFACE

The `Runner` interface defines the contract for command executors:

```
interface Runner {
  run(): Promise<boolean>;
  checkLogs(): Map<string, string>;
}
```

CONFIGURATION

The service reads configuration from a YAML file specifying:

Setting	Purpose
`port`	HTTP server port
`location`	Base directory for executable scripts
`commands`	Whitelist of permitted command names

Security Considerations

The runner implements a whitelist-based security model where only commands explicitly listed in the configuration file may be executed. This prevents arbitrary command execution attacks.

6.4.6 Command Line Interface

The Command Line Interface (CLI) provides administrators with programmatic control over the DTaaS platform. This administrative tool manages provisioning and deprovisioning of user accounts.

Package Structure

The CLI package is organized as follows:

```
cli/
├─── src/                          # Application source code
│    ├─── __init__.py              # Package initialization
│    ├─── cmd.py                   # Click command groups and handlers
│    └─── pkg/                     # Package modules
│         ├─── config.py           # Configuration management (TOML parsing)
│         ├─── constants.py        # Constant definitions
│         ├─── users.py            # User lifecycle operations
│         └─── utils.py            # File I/O and utilities
├─── tests/                        # Test suites
│    ├─── test_cli.py              # CLI integration tests
│    ├─── test_cmd.py              # Command handler tests
│    ├─── test_config.py           # Configuration tests
│    ├─── test_users.py            # User operations tests
│    ├─── test_utils.py            # Utility function tests
│    └─── data/                    # Test fixtures
├─── examples/                     # Example configuration files
├─── dtaas.toml                    # Default configuration file
├─── users.local.yml               # Local deployment template
├─── users.server.yml              # Server deployment template
├─── users.server.secure.yml       # Secure server template
└─── pyproject.toml                # Poetry project configuration
```

Architecture and Design

The CLI is implemented as a Python package using the Click framework for command parsing. The design separates configuration management, user operations, and utility functions into distinct classes and utility functions.

```
classDiagram
    class dtaas {
        +admin() group
    }

    class admin {
        +user() group
    }

    class user {
        +add() command
        +delete() command
    }

    class Config {
        -data: dict
        +get_config() tuple
        +get_from_config(key) tuple
        +get_common() tuple
        +get_users() tuple
        +get_path() tuple
        +get_server_dns() tuple
        +get_add_users_list() tuple
        +get_delete_users_list() tuple
        +get_resource_limits() tuple
    }

    class users {
        +add_users(config_obj) error
        +delete_user(config_obj) error
        +get_compose_config(username, server, path, resources) tuple
        +create_user_files(users, file_path) void
        +start_user_containers(users) error
        +stop_user_containers(users) error
    }

    class utils {
        +import_yaml(filename) tuple
        +export_yaml(data, filename) error
        +import_toml(filename) tuple
        +replace_all(obj, mapping) tuple
        +check_error(err) void
    }
```

```
    dtaas --> admin : contains
    admin --> user : contains
    user --> Config : uses
    user --> users : calls
    users --> utils : uses
    Config --> utils : uses
```

KEY MODULES

Module	Purpose
`cmd.py`	Defines Click command groups and command handlers
`config.py`	Configuration management via TOML file parsing
`users.py`	User lifecycle operations (add/delete)
`utils.py`	File I/O and template substitution utilities

CONFIGURATION MODULE

The `Config` class serves as the central configuration manager, reading from `dtaas.toml`. It provides accessor methods that return tuples of `(value, error)`, following a Go-style error handling pattern that enables explicit error propagation without exceptions.

Key configuration sections include:

- **common**: Server DNS, installation path, and resource limits
- **users**: Lists of users to add or delete

USER OPERATIONS MODULE

The user operations module manages the complete lifecycle of user workspaces:

1. **Workspace Creation**: Copies template directory structure for each user
2. **Container Configuration**: Generates Docker Compose service definitions with configurable resource limits (CPU, memory, shared memory, process limits)
3. **Container Orchestration**: Starts and stops user containers via Docker Compose commands

UTILITY FUNCTIONS

The utilities module provides:

- **YAML/TOML I/O**: Safe file reading and writing operations
- **Template Substitution**: Recursive placeholder replacement in nested data structures (strings, lists, dictionaries)
- **Error Checking**: Helper function for converting errors to exceptions

Sequence Diagram

The following diagram illustrates the user addition workflow:

```
sequenceDiagram
    actor Admin
    participant CLI as Click CLI
    participant Config as Config Class
    participant Users as users.py
    participant Utils as utils.py
    participant Docker as Docker Compose
    participant FS as File System

    Admin ->> CLI: dtaas admin user add
    activate CLI

    CLI ->> Config: Config()
    activate Config
    Config ->> Utils: import_toml("dtaas.toml")
    Utils -->> Config: config data
    deactivate Config

    CLI ->> Users: add_users(config_obj)
    activate Users

    Users ->> Utils: import_yaml("compose.users.yml")
```

```
    Utils -->> Users: compose dict

    Users ->> Config: get_add_users_list()
    Config -->> Users: user list

    Users ->> Config: get_server_dns()
    Config -->> Users: server DNS

    Users ->> Config: get_path()
    Config -->> Users: installation path

    Users ->> Config: get_resource_limits()
    Config -->> Users: resource limits

    loop For each user
        Users ->> FS: Copy template directory
        Users ->> Users: get_compose_config()
        Users ->> Users: Add service to compose
    end

    Users ->> Utils: export_yaml(compose)
    Utils ->> FS: Write compose.users.yml

    Users ->> Docker: docker compose up -d
    Docker -->> Users: Container started

    Users -->> CLI: Success
    deactivate Users

    CLI -->> Admin: "Users added successfully"
    deactivate CLI
```

Error Handling Pattern

The CLI employs a consistent error handling strategy throughout the codebase:

1. **Functions return errors as values**: Most functions return a tuple of `(result, error)` rather than raising exceptions
2. **Explicit error propagation**: Callers check for errors and propagate them up the call stack
3. **Click exceptions at boundaries**: At the CLI entry points, errors are converted to `click.ClickException` for user-friendly output

This pattern provides explicit control flow and facilitates testing by making error paths explicit and testable.

Resource Limits Configuration

The CLI supports configurable resource limits for user containers, enabling administrators to control resource consumption per user and reduce the possibility of a single user making excessive use of limited computing resources. The default resource limits are:

Parameter	Description	Example Value
`shm_size`	Shared memory size	`512m`
`cpus`	CPU core allocation	`2`
`mem_limit`	Memory limit	`2g`
`pids_limit`	Maximum process count	`100`

6.4.7 DTaaS Services Codebase

This page documents the implementation codebase for the `dtaas-services` CLI used to provision and operate DTaaS platform services.

Location and Purpose

Source package:

- `deploy/services/cli`

Primary responsibilities:

- generate a runnable services project structure,
- set up certificates and service permissions,
- start/stop/restart/remove/clean service containers,
- install post-start workflows for ThingsBoard and GitLab,
- create users and reset service passwords from CSV/env configuration.

Runtime Command Surface

CLI entrypoint is defined in `dtaas_services/cmd.py` and exposes:

- `generate-project`
- `setup`
- `install`
- `start`
- `stop`
- `restart`
- `status`
- `remove`
- `clean`
- `user add`
- `user reset-password`

Poetry script mapping (`pyproject.toml`):

```
[tool.poetry.scripts]
dtaas-services = "dtaas_services.cmd:services"
```

Package Layout

```
deploy/services/cli/
├── dtaas_services/
│   ├── cmd.py
│   ├── commands/
│   │   ├── setup_ops.py
│   │   ├── service_ops.py
│   │   ├── user_ops.py
│   │   └── utility.py
│   ├── pkg/
│   │   ├── config.py
│   │   ├── cert.py
│   │   ├── template.py
│   │   ├── utils.py
│   │   ├── password_store.py
│   │   ├── lib/
│   │   │   ├── manager.py
│   │   │   ├── docker_executor.py
│   │   │   ├── status.py
│   │   │   ├── cleanup.py
│   │   │   └── initialization.py
│   │   └── services/
│   │       ├── mongodb.py
│   │       ├── rabbitmq.py
│   │       ├── influxdb/
│   │       ├── postgres/
│   │       ├── thingsboard/
│   │       └── gitlab/
│   └── templates/
├── tests/
├── README.md
└── pyproject.toml
```

Layered Design

COMMAND LAYER

Files in `dtaas_services/commands/` define Click commands and high-level flows:

- `setup_ops.py` : project generation, TLS setup, service install flows.
- `service_ops.py` : lifecycle operations (`start` , `stop` , `status` , `remove` , `clean`).
- `user_ops.py` : user provisioning and password reset orchestration.

CORE LIBRARY LAYER

Files in `dtaas_services/pkg/` provide reusable logic:

- `config.py` : environment/config loading.
- `template.py` : generated project scaffolding and template copying.
- `cert.py` : certificate placement and normalization.
- `password_store.py` : current admin-password tracking for supported services.
- `lib/*` : compose command execution and service-state management.

SERVICE MODULES LAYER

`dtaas_services/pkg/services/` implements service-specific behavior:

- `gitlab/` : health checks, root password setup, PAT creation, OAuth app setup, and GitLab user provisioning.
- `thingsboard/` : setup, sysadmin/tenant workflows, credential-driven user setup.
- `influxdb/` , `rabbitmq.py` , `mongodb.py` , `postgres/` : permissions, readiness checks, and account/bootstrap routines.

Configuration Inputs

Generated projects are driven by:

- `config/services.env` : ports, hostnames, SSL behavior, service credentials.
- `config/credentials.csv` : user list (`username,password,email`).
- TLS certificates copied into generated `certs/` layout.

Critical GitLab-related environment variables include `HOSTNAME`, `GITLAB_PORT`, `SSL_VERIFY`, and `GITLAB_ROOT_NEW_PASSWORD`.

Typical Workflow

1. Generate project files:
2. `dtaas-services generate-project`
3. Edit configuration:
4. `config/services.env`
5. `config/credentials.csv`
6. Set up certs and permissions:
7. `dtaas-services setup`
8. Start selected services:
9. `dtaas-services start [-s ...]`
10. Run post-install setup where needed:
11. `dtaas-services install -s thingsboard`
12. `dtaas-services install -s gitlab`
13. Provision users:
14. `dtaas-services user add`

Testing and Quality

The package uses pytest with strict markers and test discovery rooted in `deploy/services/cli/tests`.

Common developer checks:

```
cd deploy/services/cli
poetry install
poetry run pytest
```

Contributor Notes

- Keep command modules thin and push business logic into `pkg/`.
- Add service-specific behavior in `pkg/services/<service>/` or peer modules, not in generic command handlers.
- Preserve CLI ergonomics: commands should return actionable messages and avoid partial side effects without clear status output.
- When adding new service operations, include tests under `tests/test_services` and command-level coverage under `tests/test_commands`.

6.4.8 Publish Packages

DTaaS is a monorepo with publishable artifacts across npm, Docker, and Python package channels.

Package Channels

- npm packages (public): `@into-cps-association/*`
- Docker images: GitHub Container Registry and Docker Hub
- Python packages: DTaaS CLI and platform-services CLI release flows

JavaScript/TypeScript Packages

Notable package roots:

- `servers/lib` -> `@into-cps-association/libms`
- `servers/execution/runner` -> `@into-cps-association/runner`
- `client` -> web client package metadata and build artifacts

Typical publish prerequisites:

1. Lint/syntax checks.
2. Build output generation.
3. Test execution.
4. Registry authentication.

Private Registry Workflow (Development)

Use a private registry (for example Verdaccio) when testing publish/unpublish behavior before public releases.

Typical local flow:

```
docker run -d --name verdaccio -p 4873:4873 verdaccio/verdaccio
npm adduser --registry http://localhost:4873
npm set registry http://localhost:4873/
```

Docker Artifacts

The repository includes dedicated Docker build configurations under `docker/` and deployment-level compose definitions under `deploy/`.

When changing runtime dependencies, validate image builds and scenario startup paths before release tagging.

Python Packages

Two Poetry-based CLIs are maintained:

- `cli` (`dtaas`)
- `deploy/services/cli` (`dtaas-services`)

Use Poetry-managed versioning and lockfile workflows for package reproducibility.

Release Guidance

- Keep version bumps intentional and scoped to changed packages.
- Prefer automated CI publishing where available.
- Validate package install and smoke-run behavior after publishing.

6.5 Contributions

6.5.1 Contributor Covenant Code of Conduct

Our Pledge

We as members, contributors, and leaders pledge to make participation in our community a harassment-free experience for everyone, regardless of age, body size, visible or invisible disability, ethnicity, sex characteristics, gender identity and expression, level of experience, education, socio-economic status, nationality, personal appearance, race, religion, or sexual identity and orientation.

We pledge to act and interact in ways that contribute to an open, welcoming, diverse, inclusive, and healthy community.

Our Standards

Examples of behavior that contributes to a positive environment for our community include:

- Demonstrating empathy and kindness toward other people
- Being respectful of differing opinions, viewpoints, and experiences
- Giving and gracefully accepting constructive feedback
- Accepting responsibility and apologizing to those affected by our mistakes, and learning from the experience
- Focusing on what is best not just for us as individuals, but for the overall community

Examples of unacceptable behavior include:

- The use of sexualized language or imagery, and sexual attention or advances of any kind
- Trolling, insulting or derogatory comments, and personal or political attacks
- Public or private harassment
- Publishing others' private information, such as a physical or email address, without their explicit permission
- Other conduct which could reasonably be considered inappropriate in a professional setting

Enforcement Responsibilities

Community leaders are responsible for clarifying and enforcing our standards of acceptable behavior and will take appropriate and fair corrective action in response to any behavior that they deem inappropriate, threatening, offensive, or harmful.

Community leaders have the right and responsibility to remove, edit, or reject comments, commits, code, wiki edits, issues, and other contributions that are not aligned to this Code of Conduct, and will communicate reasons for moderation decisions when appropriate.

Scope

This Code of Conduct applies within all community spaces, and also applies when an individual is officially representing the community in public spaces. Examples of representing our community include using an official e-mail address, posting via an official social media account, or acting as an appointed representative at an online or offline event.

Enforcement

Instances of abusive, harassing, or otherwise unacceptable behavior may be reported to the community leaders responsible for enforcement at Open new issue. All complaints will be reviewed and investigated promptly and fairly.

All community leaders are obligated to respect the privacy and security of the reporter of any incident.

Enforcement Guidelines

Community leaders will follow these Community Impact Guidelines in determining the consequences for any action they deem in violation of this Code of Conduct:

1. CORRECTION

Community Impact: Use of inappropriate language or other behavior deemed unprofessional or unwelcome in the community.

Consequence: A private, written warning from community leaders, providing clarity around the nature of the violation and an explanation of why the behavior was inappropriate. A public apology may be requested.

2. WARNING

Community Impact: A violation through a single incident or series of actions.

Consequence: A warning with consequences for continued behavior. No interaction with the people involved, including unsolicited interaction with those enforcing the Code of Conduct, for a specified period of time. This includes avoiding interactions in community spaces as well as external channels like social media. Violating these terms may lead to a temporary or permanent ban.

3. TEMPORARY BAN

Community Impact: A serious violation of community standards, including sustained inappropriate behavior.

Consequence: A temporary ban from any sort of interaction or public communication with the community for a specified period of time. No public or private interaction with the people involved, including unsolicited interaction with those enforcing the Code of Conduct, is allowed during this period. Violating these terms may lead to a permanent ban.

4. PERMANENT BAN

Community Impact: Demonstrating a pattern of violation of community standards, including sustained inappropriate behavior, harassment of an individual, or aggression toward or disparagement of classes of individuals.

Consequence: A permanent ban from any sort of public interaction within the community.

Attribution

This Code of Conduct is adapted from the Contributor Covenant, version 2.0, available at https://www.contributor-covenant.org/version/2/0/code_of_conduct.html.

Community Impact Guidelines were inspired by Mozilla's code of conduct enforcement ladder.

For answers to common questions about this code of conduct, see the FAQ at https://www.contributor-covenant.org/faq. Translations are available at https://www.contributor-covenant.org/translations.

6.5.2 Copilot Instructions 🤖

The authoritative instruction set for coding agents is maintained at:

- `.github/copilot-instructions.md`

Use that file as the source of truth for:

- Monorepo structure and directory-specific conventions.
- Coding style rules for TypeScript, Python, and NestJS services.
- Documentation and deployment workflow expectations.
- Testing and release-quality guidance.

Practical Usage

When contributing with coding agents:

1. Read `.github/copilot-instructions.md` before implementation.
2. Follow component-local conventions (`cli/`, `client/`, `servers/`, `deploy/`, `docs/`).
3. Validate impacted modules and documentation paths before opening a pull request.

6.5.3 Local Testing 🧪

This guide provides a practical local test loop for contributors working in the DTaaS monorepo.

Recommended Order

1. Run syntax and lint checks for the changed component.
2. Run fast local tests (unit or focused integration).
3. Run broader test suites before opening a pull request.
4. Build docs if markdown or navigation changed.

Client (React + TypeScript)

Run from `client/`:

```
yarn syntax
yarn test:unit
yarn test:int
yarn build
```

For end-to-end checks:

```
yarn test:e2e
```

Library Microservice (NestJS)

Run from `servers/lib/`:

```
yarn syntax
yarn build
yarn test:all
```

If you need API-level checks for cloudcmd integration, use `yarn test:http`.

Runner Microservice (NestJS)

Run from `servers/execution/runner/`:

```
yarn syntax
yarn build
yarn test
```

Use `yarn test:int` and `yarn test:e2e` for deeper coverage.

DTaaS CLI (Python)

Run from `cli/`:

```
poetry install
poetry run pytest
```

Platform Services CLI (Python)

Run from `deploy/services/cli/`:

```
poetry install
poetry run pytest
```

Documentation Validation

Run from repository root:

```
mkdocs build -f mkdocs-github.yml
```

If on Linux and full docs validation with optional PDF wiring is needed:

```
MKDOCS_ENABLE_PDF_EXPORT=1 mkdocs build -f mkdocs.yml
```

Practical Tip

When changing multiple components, validate each package in isolation first. This keeps failures local, shortens debug time, and avoids chasing unrelated test errors.

6.5.4 Docker Workflow for DTaaS

This document describes the building and use of different Docker files for development and installation of the DTaaS platform.

NOTE: A local Docker CE installation is a prerequisite for using Docker workflows.

Run

Follow the instructions in `docker/README.md` to spawn a localhost development instance of DTaaS. It is an end-to-end testing of the current codebase as it exists in the local git directory.

Publish Docker Images

Build and publish the docker images. This step is required only for the publication of images to Docker Hub.

🛑 This publishing step is managed only by project maintainers. Regular developers can skip this step.

The DTaaS development team publishes reusable packages which are then put together to form the complete DTaaS application.

The packages are published on github, npmjs, and docker hub repositories.

The packages on github are published more frequently but are not user tested. The packages on npmjs and docker hub are published at least once per release. The regular users are encouraged to use the packages from npm and docker.

A brief explanation of the packages is given below.

Package Name	Description	Availability
dtaas-web	React web application	docker hub and github
libms	Library microservice	npmjs and github
		docker hub and github
runner	REST API wrapper for multiple scripts/programs	npmjs and github

REACT WEBSITE

```
docker build -t intocps/dtaas-web:latest -f ./docker/client.built.dockerfile .
docker tag intocps/dtaas-web:latest intocps/dtaas-web:<version>
docker push intocps/dtaas-web:latest
docker push intocps/dtaas-web:<version>
```

To tag version **0.3.1** for example, use

```
docker tag intocps/dtaas-web:latest intocps/dtaas-web:0.3.1
```

To test the react website container on localhost, please use

```
docker run -d \
  -v ${PWD}/client/config/local.js:/dtaas/client/build/env.js \
  -p 4000:4000 intocps/dtaas-web:latest
```

LIBRARY MICROSERVICE

The Dockerfile of library microservice has `VERSION` argument. This argument helps pick the right package version from http://npmjs.com.

```
docker login -u <username> -p <password>
docker build -t intocps/libms:latest -f ./docker/libms.npm.dockerfile .
docker push intocps/libms:latest
docker build --build-arg="VERSION=<version>" \
  -t intocps/libms:<version> -f ./docker/libms.npm.dockerfile .
docker push intocps/libms:<version>
```

To tag version 0.3.1 for example, use

```
docker build --build-arg="VERSION=0.3.1" \
  -t intocps/libms:0.3.1 -f ./docker/libms.npm.dockerfile .
```

To test the library microservice on localhost, please use

```
docker run -d -v ${PWD}/files:/dtaas/libms/files \
  -p 4001:4001 intocps/libms:latest
```

6.5.5 Secrets for GitHub Action

The GitHub actions require the following secrets to be obtained from docker hub:

Secret Name	Explanation
`DOCKERHUB_SCOPE`	Username or organization name on docker hub
`DOCKERHUB_USERNAME`	Username on docker hub
`DOCKERHUB_TOKEN`	API token to publish images to docker hub, with `Read`, `Write` and `Delete` permissions
`PACKAGE_NAME`	The name with which libms npm package must be published

Similarly, GitHub actions require the following secrets to be obtained from npmjs:

Secret Name	Explanation
`NPM_TOKEN`	Token to publish npm packages to the default npm registry.
`PACKAGE_NAME`	The name with which libms npm package must be published
`NPM_RUNNER_PACKAGE_NAME`	The name with which runner npm package must be published

Remember to add these secrets to GitHub Secrets Setting of the fork.

6.5.6 Publish NPM packages

The DTaaS platform is developed as a monorepo with multiple npm packages.

Default npm registry

The default registry for npm packages is npmjs. The freely-accessible public packages are published to the **npmjs** registry. The publication step is manual for the runner.

```
npm login --registry="https://registry.npmjs.org"
cat ~/.npmrc  #shows the auth token for the registry
//registry.npmjs.org/:_authToken=xxxxxxxxxx
yarn publish --registry="https://registry.npmjs.org" \
  --no-git-tag-version --access public
```

At least one version of runner package is published to this registry for each release of the DTaaS platform.

The publication steps for library microservice and runner are automated via github actions.

GitHub npm registry

The GitHub actions of the project publish packages. The only limitation is that the users need an access token from GitHub.

Private Registry

SETUP PRIVATE NPM REGISTRY

Since publishing to npmjs is irrevocable and public, developers are encouraged to setup their own private npm registry for local development. A private npm registry will help with local publish and unpublish steps.

We recommend using verdaccio for this task. The following commands help you create a working private npm registry for development.

```
docker run -d --name verdaccio -p 4873:4873 verdaccio/verdaccio
npm adduser --registry http://localhost:4873 #create a user on the verdaccio registry
npm set registry http://localhost:4873/
yarn config set registry "http://localhost:4873"
yarn login --registry "http://localhost:4873" #login with the credentials for yarn utility
npm login #login with the credentials for npm utility
```

You can open `http://localhost:4873` in the browser, login with the user credentials to see the packages published.

Publish to private npm registry

To publish a package to the local registry, do:

```
yarn install
yarn build #the dist/ directory is needed for publishing step
yarn publish --no-git-tag-version #increments version in package.json, publishes to registry
yarn publish #increments version in package.json, publishes to registry and adds a git tag
```

The package version in package.json gets updated as well. You can open `http://localhost:4873` in the browser, login with the user credentials to see the packages published. Please see verdaccio docs for more information.

If there is a need to unpublish a package, ex: `@dtaas/runner@0.0.2`, do:

```
npm unpublish --registry http://localhost:4873/ \
  @dtaas/runner@0.0.2
```

To install / uninstall this utility for all users, do:

```
sudo npm install --registry http://localhost:4873 \
  -g @dtaas/runner
sudo npm list -g # should list @dtaas/runner in the packages
sudo npm remove --global @dtaas/runner
```

🚀 USE THE PACKAGES

The packages available in private npm registry can be used like the regular npm packages installed from npmjs.

For example, to use `@dtaas/runner@0.0.2` package, do:

```
sudo npm install --registry http://localhost:4873 \
  -g @dtaas/runner
runner # launch the digital twin runner
```

6.5.7 Vagrant Deployment

This page summarises the Vagrant-based deployment available in this repository. The source material for the setup is maintained in:

- `deploy/vagrant/README.md`

Purpose

The Vagrant setup provides:

- Cross-platform DTaaS installation using a virtual machine
- A reproducible development and evaluation environment

Host Requirements

Recommended host capacity:

- 16 GB RAM
- 8 vCPUs
- 50 GB available disk space

Required tooling:

- Vagrant
- vagrant-disksize plugin

Provisioning Modes

Two provisioning scripts are provided in `deploy/vagrant`:

- `user.sh` for user-oriented installation (default)
- `developer.sh` for extended developer tooling

Developer provisioning can be enabled in `deploy/vagrant/Vagrantfile`.

Basic Workflow

Run the following from the `deploy/vagrant` directory:

```
vagrant up
vagrant ssh
sudo bash /vagrant/route.sh
```

Configuration

The file `deploy/vagrant/config.json` controls machine configuration, including hostname, VM name, memory, CPUs, and disk size.

Related

- DTaaS installation scenarios: `../admin/overview.md`
- DTaaS package deployments: `../admin/dtaas/`
- Workspace deployments: `../admin/workspace/`
- Platform services: `../admin/services/cli.md`

7. Known Issues in the Software

If a bug is discovered, an issue should be opened

7.1 Third-Party Software

The explanation given below corresponds to bugs that may be encountered from third party software included in the DTaaS platform. Known issues are listed below.

7.2 GitLab

The GitLab OAuth 2.0 authorisation service does not have a way to sign out of a third-party application. Even if a user signs out of the DTaaS platform, GitLab still shows the user as signed in. The next time the sign in button is clicked on the DTaaS platform page, the login page is not displayed. Instead the user is directly taken to the **Library** page. Therefore, the browser window should be closed after use. Another way to overcome this limitation is to open the linked GitLab instance (ex: `https://intocps.org/gitlab`) and sign out from there. Thus the user needs to sign out of two places, namely the DTaaS platform and GitLab, in order to completely exit the DTaaS platform.

The same observation holds true for the Workspace installation as well.

8. Thanks

8.1 Funding Sources

This project has been funded by the following projects.

- Sustainable Water-based Cooling in Megacities (SWiM) - a project sponsored by the Grundfos Foundation
- Security for the Digital Twin as a Service Platform - a project sponsored by Thomas B. Thriges Foundation
- CP-SENS (Cyber-Physical Sensing for Machinery and Structures) - a Grand Solution project funded by the Innovation Fund Denmark under the grant agreement 2081-00006B.
- The DIGITbrain (Digital Twins for Manufacturing SMEs) - a European Union's Horizon 2020 research and innovation program under grant agreement No 952071.
- Digital Twins for Cyber-Physical Systems (DiT4CPS) - a project sponsored by the Grundfos Foundation.

8.2 Developers

Please see the list of contributors on code contributors

8.3 Example Contributors

Example Name	Contributors
Mass Spring Damper	Prasad Talasila
Water Tank Fault Injection	Henrik Ejersbo and Mirgita Frasheri
Water Tank Model Swap	Henrik Ejersbo and Mirgita Frasheri
Desktop Robotti with RabbitMQ	Mirgita Frasheri
Water Treatment Plant and OPC-UA	Lucia Royo and Alejandro Labarias
Three Water Tanks with DT Manager Framework	Santiago Gil Arboleda
Flex-Cell with Two Industrial Robots	Santiago Gil Arboleda
Incubator	Morten Haahr Kristensen
Firefighters in Emergency Environments	Lars Vosteen and Hannes Iven
Mass Spring Damper with NuRV Runtime Monitor	Alberto Bonizzi
Incubator with NuRV Runtime Monitor	Alberto Bonizzi and Morten Haahr Kristensen
Incubator with NuRV Runtime Monitor Service	Valdemar Tang
Water Tank Fault Injection with NuRV Runtime Monitor	Alberto Bonizzi
Incubator Co-Simulation with NuRV Runtime Monitor FMU	Morten Haahr Kristensen
Incubator with NuRV Runtime Monitor FMU as Service	Valdemar Tang and Morten Haahr Kristensen
Incubator with NuRV Runtime Monitor as Service	Morten Haahr Kristensen and Valdemar Tang

8.4 Documentation

1. Talasila, P., Gomes, C., Mikkelsen, P. H., Arboleda, S. G., Kamburjan, E., & Larsen, P. G. (2023). Digital Twin as a Service (DTaaS): A Platform for Digital Twin Developers and Users arXiv preprint arXiv:2305.07244.
2. Astitva Sehgal for developer and example documentation.
3. Tanusree Roy and Farshid Naseri for asking interesting questions that ended up in FAQs.

9. License

9.1 License

--- Start of Definition of INTO-CPS Association Public License ---

/*

- This file is part of the INTO-CPS Association.
- Copyright (c) 2017-CurrentYear, INTO-CPS Association (ICA),
- c/o Peter Gorm Larsen, Aarhus University, Department of Engineering,
- Finlandsgade 22, 8200 Aarhus N, Denmark.
- All rights reserved.
- THIS PROGRAM IS PROVIDED UNDER THE TERMS OF GPL VERSION 3 LICENSE OR
- THIS INTO-CPS ASSOCIATION PUBLIC LICENSE (ICAPL) VERSION 1.0.
- ANY USE, REPRODUCTION OR DISTRIBUTION OF THIS PROGRAM CONSTITUTES
- RECIPIENT'S ACCEPTANCE OF THE INTO-CPS ASSOCIATION PUBLIC LICENSE OR
- THE GPL VERSION 3, ACCORDING TO RECIPIENTS CHOICE.
- The INTO-CPS tool suite software and the INTO-CPS Association
- Public License (ICAPL) are obtained from the INTO-CPS Association, either
- from the above address, from the URLs: http://www.into-cps.org or
- in the INTO-CPS tool suite distribution.
- GNU version 3 is obtained from:
- http://www.gnu.org/copyleft/gpl.html.
- This program is distributed WITHOUT ANY WARRANTY; without
- even the implied warranty of MERCHANTABILITY or FITNESS
- FOR A PARTICULAR PURPOSE, EXCEPT AS EXPRESSLY SET FORTH
- IN THE BY RECIPIENT SELECTED SUBSIDIARY LICENSE CONDITIONS OF
- THE INTO-CPS ASSOCIATION PUBLIC LICENSE.
- See the full ICAPL conditions for more details.

*/

--- End of INTO-CPS Association Public License Header ---

The ICAPL is a public license for the INTO-CPS tool suite with three modes/alternatives (GPL, ICA-Internal-EPL, ICA-External-EPL) for use and redistribution, in source and/or binary/object-code form:

- GPL. Any party (member or non-member of the INTO-CPS Association) may use and redistribute INTO-CPS tool suite under GPL version 3.
- Silver Level members of the INTO-CPS Association may also use and redistribute the INTO-CPS tool suite under ICA-Internal-EPL conditions.
- Gold Level members of the INTO-CPS Association may also use and redistribute The INTO-CPS tool suite under ICA-Internal-EPL or ICA-External-EPL conditions.

Definitions of the INTO-CPS Association Public license modes:

- GPL = GPL version 3.
- ICA-Internal-EPL = These INTO-CPS Association Public license conditions together with Internally restricted EPL, i.e., EPL version 1.0 with the Additional Condition that use and redistribution by a member of the INTO-CPS Association is only allowed within the INTO-CPS Association member's own organization (i.e., its own legal entity), or for a member of the INTO-CPS Association paying a membership fee corresponding to the size of the organization including all its affiliates, use and redistribution is allowed within/between its affiliates.
- ICA-External-EPL = These INTO-CPS Association Public license conditions together with Externally restricted EPL, i.e., EPL version 1.0 with the Additional Condition that use and redistribution by a member of the INTO-CPS Association, or by a Licensed Third Party Distributor having a redistribution agreement with that member, to parties external to the INTO-CPS Association member’s own organization (i.e., its own legal entity) is only allowed in binary/object-code form, except the case of redistribution to other members the INTO-CPS Association to which source is also allowed to be distributed.

[This has the consequence that an external party who wishes to use the INTO-CPS Association in source form together with its own proprietary software in all cases must be a member of the INTO-CPS Association].

In all cases of usage and redistribution by recipients, the following conditions also apply:

a) Redistributions of source code must retain the above copyright notice, all definitions, and conditions. It is sufficient if the ICAPL Header is present in each source file, if the full ICAPL is available in a prominent and easily located place in the redistribution.

b) Redistributions in binary/object-code form must reproduce the above copyright notice, all definitions, and conditions. It is sufficient if the ICAPL Header and the location in the redistribution of the full ICAPL are present in the documentation and/or other materials provided with the redistribution, if the full ICAPL is available in a prominent and easily located place in the redistribution.

c) A recipient must clearly indicate its chosen usage mode of ICAPL, in accompanying documentation and in a text file ICA-USAGE-MODE.txt, provided with the distribution.

d) Contributor(s) making a Contribution to the INTO-CPS Association thereby also makes a Transfer of Contribution Copyright. In return, upon the effective date of the transfer, ICA grants the Contributor(s) a Contribution License of the Contribution. ICA has the right to accept or refuse Contributions.

Definitions:

"Subsidiary license conditions" means:

The additional license conditions depending on the by the recipient chosen mode of ICAPL, defined by GPL version 3.0 for GPL, and by EPL for ICA-Internal-EPL and ICA-External-EPL.

"ICAPL" means:

INTO-CPS Association Public License version 1.0, i.e., the license defined here (the text between "--- Start of Definition of INTO-CPS Association Public License ---" and "--- End of Definition of INTO-CPS Association Public License ---", or later versions thereof.

"ICAPL Header" means:

INTO-CPS Association Public License Header version 1.2, i.e., the text between "--- Start of Definition of INTO-CPS Association Public License ---" and "--- End of INTO-CPS Association Public License Header ---, or later versions thereof.

"Contribution" means:

a) in the case of the initial Contributor, the initial code and documentation distributed under ICAPL, and

b) in the case of each subsequent Contributor: i) changes to the INTO-CPS tool suite, and ii) additions to the INTO-CPS tool suite;

where such changes and/or additions to the INTO-CPS tool suite originate from and are distributed by that particular Contributor. A Contribution 'originates' from a Contributor if it was added to the INTO-CPS tool suite by such Contributor itself or anyone acting on such Contributor's behalf.

For Contributors licensing the INTO-CPS tool suite under ICA-Internal-EPL or ICA-External-EPL conditions, the following conditions also hold:

Contributions do not include additions to the distributed Program which: (i) are separate modules of software distributed in conjunction with the INTO-CPS tool suite under their own license agreement, (ii) are separate modules which are not derivative works of the INTO-CPS tool suite, and (iii) are separate modules of software distributed in conjunction with the INTO-CPS tool suite under their own license agreement where these separate modules are merged with (weaved together with) modules of The INTO-CPS tool suite to form new modules that are distributed as object code or source code under their own license agreement, as allowed under the Additional Condition of internal distribution according to ICA-Internal-EPL and/or Additional Condition for external distribution according to ICA-External-EPL.

"Transfer of Contribution Copyright" means that the Contributors of a Contribution transfer the ownership and the copyright of the Contribution to the INTO-CPS Association, the INTO-CPS Association Copyright owner, for inclusion in the INTO-CPS tool suite. The transfer takes place upon the effective date when the Contribution is made available on the INTO-CPS Association web site under ICAPL, by such Contributors themselves or anyone acting on such Contributors' behalf. The transfer is free of charge. If the Contributors or the INTO-CPS Association so wish, an optional Copyright transfer agreement can be signed between the INTO-CPS Association and the Contributors.

"Contribution License" means a license from the INTO-CPS Association to the Contributors of the Contribution, effective on the date of the Transfer of Contribution Copyright, where the INTO-CPS Association grants the Contributors a non-exclusive, world-wide, transferable, free of charge, perpetual license, including sublicensing rights, to use, have used, modify, have modified, reproduce and or have reproduced the contributed material, for business and other purposes, including but not limited to evaluation, development, testing, integration and merging with other software and distribution. The warranty and liability disclaimers of ICAPL apply to this license.

"Contributor" means any person or entity that distributes (part of) the INTO-CPS tool chain.

"The Program" means the Contributions distributed in accordance with ICAPL.

"The INTO-CPS tool chain" means the Contributions distributed in accordance with ICAPL.

"Recipient" means anyone who receives the INTO-CPS tool chain under ICAPL, including all Contributors.

"Licensed Third Party Distributor" means a reseller/distributor having signed a redistribution/resale agreement in accordance with ICAPL and the INTO-CPS Association Bylaws, with a Gold Level organizational member which is not an Affiliate of the reseller/distributor, for distributing a product containing part(s) of the INTO-CPS tool suite. The Licensed Third Party Distributor shall only be allowed further redistribution to other resellers if the Gold Level member is granting such a right to it in the redistribution/resale agreement between the Gold Level member and the Licensed Third Party Distributor.

"Affiliate" shall mean any legal entity, directly or indirectly, through one or more intermediaries, controlling or controlled by or under common control with any other legal entity, as the case may be. For purposes of this definition, the term "control" (including the terms "controlling," "controlled by" and "under common control with") means the possession, direct or indirect, of the power to direct or cause the direction of the management and policies of a legal entity, whether through the ownership of voting securities, by contract or otherwise.

NO WARRANTY

EXCEPT AS EXPRESSLY SET FORTH IN THE BY RECIPIENT SELECTED SUBSIDIARY LICENSE CONDITIONS OF ICAPL, THE INTO-CPS ASSOCIATION IS PROVIDED ON AN "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, EITHER EXPRESS OR IMPLIED INCLUDING, WITHOUT LIMITATION, ANY WARRANTIES OR CONDITIONS OF TITLE, NON-INFRINGEMENT, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. Each Recipient is solely responsible for determining the appropriateness of using and distributing the INTO-CPS tool suite and assumes all risks associated with its exercise of rights under ICAPL , including but not limited to the risks and costs of program errors, compliance with applicable laws, damage to or loss of data, programs or equipment, and unavailability or interruption of operations.

DISCLAIMER OF LIABILITY

EXCEPT AS EXPRESSLY SET FORTH IN THE BY RECIPIENT SELECTED SUBSIDIARY LICENSE CONDITIONS OF ICAPL, NEITHER RECIPIENT NOR ANY CONTRIBUTORS SHALL HAVE ANY LIABILITY FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING WITHOUT LIMITATION LOST PROFITS), HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OR DISTRIBUTION OF THE INTO-CPS TOOL SUITE OR THE EXERCISE OF ANY RIGHTS GRANTED HEREUNDER, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

A Contributor licensing the INTO-CPS tool suite under ICA-Internal-EPL or ICA-External-EPL may choose to distribute (parts of) the INTO-CPS tool suite in object code form under its own license agreement, provided that:

a) it complies with the terms and conditions of ICAPL; or for the case of redistribution of the INTO-CPS tool suite together with proprietary code it is a dual license where the INTO-CPS tool suite parts are distributed under ICAPL compatible conditions and the proprietary code is distributed under proprietary license conditions; and

b) its license agreement: i) effectively disclaims on behalf of all Contributors all warranties and conditions, express and implied, including warranties or conditions of title and non-infringement, and implied warranties or conditions of merchantability and fitness for a particular purpose; ii) effectively excludes on behalf of all Contributors all liability for damages, including direct, indirect, special, incidental and consequential damages, such as lost profits; iii) states that any provisions which differ from ICAPL are offered by that Contributor alone and not by any other party; and iv) states from where the source code for the INTO-CPS tool suite is available, and informs licensees how to obtain it in a reasonable manner on or through a medium customarily used for software exchange.

When the INTO-CPS tool suite is made available in source code form:

a) it must be made available under ICAPL; and

b) a copy of ICAPL must be included with each copy of the INTO-CPS tool suite.

c) a copy of the subsidiary license associated with the selected mode of ICAPL must be included with each copy of the INTO-CPS tool suite.

Contributors may not remove or alter any copyright notices contained within The INTO-CPS tool suite.

If there is a conflict between ICAPL and the subsidiary license conditions, ICAPL has priority.

This Agreement is governed by the laws of Denmark. The place of jurisdiction for all disagreements related to this Agreement, is Aarhus, Denmark.

The EPL 1.0 license definition has been obtained from: http://www.eclipse.org/legal/epl-v10.html. It is also reproduced in the INTO-CPS distribution.

The GPL Version 3 license definition has been obtained from http://www.gnu.org/copyleft/gpl.html. It is also reproduced in the INTO-CPS distribution.

--- End of Definition of INTO-CPS Association Public License ---

9.2 Third Party Software

The DTaaS platform uses a range of third-party software components. Each component remains subject to its upstream licence terms.

9.2.1 Deployment and Runtime Software

The core deployment stack currently references the following software:

Software Package	Usage	Licence
Docker CE	mandatory	Apache 2.0
Traefik	mandatory	MIT
Traefik Forward Auth	optional	MIT
GitLab CE	optional	MIT
GitLab Runner	optional	MIT
Dex	optional	Apache 2.0
Keycloak	optional	Apache 2.0
RabbitMQ	optional	MPL 2.0
MongoDB	optional	SSPL v1
Grafana	optional	AGPL v3
InfluxDB	optional	Apache 2.0 / MIT
PostgreSQL	optional	PostgreSQL Licence
ThingsBoard	optional	Apache 2.0

9.2.2 Development Environments

In addition to runtime software, the development and documentation workflows use the following tools.

Software Package	Usage	Licence
Node.js	mandatory	Node.js licence
npm	mandatory	Artistic Licence 2.0
Yarn	optional	BSD 2-Clause
Material for MkDocs	mandatory	MIT
JupyterLab	optional	BSD 3-Clause
MicroK8s	optional	Apache 2.0

9.2.3 Package Dependencies

Additional third-party dependencies for client, servers, CLI, and tooling are declared in:

- `client/package.json`
- `servers/**/package.json`
- `cli/pyproject.toml`
- `deploy/services/cli/pyproject.toml`
- `script/docs/mkdocs-requirements.txt`

Upstream dependency licences should be consulted in those manifests when detailed licence auditing is required.